STM32 嵌入式系统作业汇编

01-13 文件夹 **README** 合集

周贤中

2026-05-30

STM32 嵌入式系统作业汇编

01-13 文件夹 README 合集

作者：周贤中

2026-05-30

目录

1 STM32 GPIO 输入输出实验 **1**

1.1 统一作业说明 . . . 1
1.1.1 学生需要完成的核心任务 . . . 1
1.1.2 本次作业验收目标 . . . 1
1.1.3 本次必须提交的内容 . . . 1
1.1.4 报告必须回答的问题 . . . 1
1.2 作业要求 . . . 1
1.3 STM32CubeMX 配置步骤 . . . 2
1.3.1 新建工程 . . . 2
1.3.2 配置系统时钟（RCC） . . . 2
1.3.3 配置调试接口（SYS） . . . 3
1.3.4 配置 GPIO 引脚 . . . 5
1.3.5 配置工程输出 . . . 6
1.3.6 生成代码 . . . 7
1.3.7 添加用户代码 . . . 7
1.4 核心代码解析 . . . 7
1.4.1 GPIO 初始化（MX_GPIO_Init） . . . 7
1.4.2 主循环逻辑 . . . 8
1.5 不同 GPIO 配置对结果的影响分析 . . . 8
1.5.1 输入配置 . . . 8
1.5.2 输出配置 . . . 8
1.5.3 输出速度配置 . . . 9
1.6 硬件连接说明 . . . 9
1.7 实验报告要求 . . . 9
1.7.1 报告建议结构 . . . 9
1.7.2 图片与流程图要求 . . . 10
1.7.3 提交规范 . . . 10
1.7.4 最低验收标准 . . . 10

2 LED 闪烁实验 **11**

2.1 统一作业说明 . . . 11
2.1.1 学生需要完成的核心任务 . . . 11
2.1.2 本次作业验收目标 . . . 11
2.1.3 本次必须提交的内容 . . . 11
2.1.4 报告必须回答的问题 . . . 11
2.2 实验目的 . . . 11
2.3 基础知识 . . . 12
2.3.1 RCC（Reset and Clock Control）时钟系统 . . . 12
2.3.2 HAL_Delay() 延时函数详解 . . . 14
2.3.3 本实验中 RCC 和 HAL_Delay 的配合 . . . 17
2.4 实验要求 . . . 17
2.4.1 硬件要求 . . . 17
2.4.2 软件要求 . . . 17

2.4.3 功能要求 . . . 17
2.5 实验步骤 . . . 18
2.5.1 使用 STM32CubeMX 配置工程 . . . 18
2.5.2 编写 LED 闪烁代码 . . . 20
2.5.3 编译工程 . . . 20
2.5.4 下载程序 . . . 20
2.5.5 观察现象 . . . 21
2.6 代码详解 . . . 21
2.6.1 主要函数说明 . . . 21
2.6.2 LED 闪烁原理 . . . 22
2.6.3 GPIO 工作模式说明 . . . 23
2.7 思考题 . . . 23
2.8 讨论与扩展 . . . 24
2.8.1 讨论 1：延时方式的优缺点 . . . 24
2.8.2 讨论 2：LED 驱动电路 . . . 24
2.8.3 讨论 3：功耗优化 . . . 24
2.8.4 扩展任务 1：呼吸灯效果 . . . 24
2.8.5 扩展任务 2：多 LED 流水灯 . . . 25
2.8.6 扩展任务 3：按键控制 LED . . . 25
2.9 实验报告要求 . . . 25
2.9.1 报告建议结构 . . . 25
2.9.2 图片与流程图要求 . . . 25
2.9.3 提交规范 . . . 25
2.9.4 最低验收标准 . . . 25
2.10 评分标准 . . . 26
2.11 参考资料 . . . 26
2.12 常见问题 FAQ . . . 26

3 STM32 中断实验 27
3.1 统一作业说明 . . . 27
3.1.1 学生需要完成的核心任务 . . . 27
3.1.2 本次作业验收目标 . . . 27
3.1.3 本次必须提交的内容 . . . 27
3.1.4 报告必须回答的问题 . . . 27
3.2 实验目的 . . . 28
3.3 实验原理 . . . 28
3.3.1 什么是中断? . . . 28
3.3.2 STM32 的 EXTI（外部中断/事件控制器） . . . 28
3.3.3 中断处理流程 . . . 28
3.3.4 按键去抖动原理 . . . 29
3.4 硬件连接 . . . 29
3.4.1 使用的 IO 口 . . . 29
3.4.2 连接方式 . . . 29
3.5 STM32CubeMX 详细配置步骤 . . . 29
3.5.1 创建新工程 . . . 29
3.5.2 时钟配置（RCC） . . . 30
3.5.3 GPIO 配置 . . . 31

3.5.4 NVIC 配置（关键步骤） . . . 32
3.5.5 项目设置 . . . 33
3.5.6 生成代码 . . . 33
3.5.7 验证生成的代码 . . . 34
3.5.8 常见配置问题 . . . 34
3.5.9 使用现有.ioc 文件 . . . 34
3.6 代码详细解析 . . . 34
3.6.1 GPIO 初始化代码（gpio.c） . . . 34
3.6.2 主程序代码（main.c） . . . 36
3.6.3 中断服务函数（stm32f1xx_it.c） . . . 37
3.7 中断机制深入解析 . . . 38
3.7.1 中断优先级 . . . 38
3.7.2 中断标志位 . . . 38
3.7.3 中断延迟 . . . 38
3.7.4 中断安全性 . . . 38
3.8 实验步骤 . . . 39
3.8.1 硬件准备 . . . 39
3.8.2 软件准备 . . . 39
3.8.3 实验操作 . . . 39
3.8.4 预期现象 . . . 39
3.9 调试技巧 . . . 40
3.9.1 LED 不翻转 . . . 40
3.9.2 LED 频繁闪烁 . . . 40
3.9.3 使用调试器 . . . 40
3.10 课后作业 . . . 40
3.10.1 说明 . . . 40
3.10.2 课后作业题 . . . 41
3.11 实验报告提交要求 . . . 41
3.11.1 报告建议结构 . . . 41
3.11.2 图片与流程图要求 . . . 41
3.11.3 提交规范 . . . 41
3.11.4 最低验收标准 . . . 42
3.12 参考资料 . . . 42
3.13 常见问题 FAQ . . . 42

4 STM32 USB 虚拟串口通信实验 43
4.1 统一作业说明 . . . 43
4.1.1 学生需要完成的核心任务 . . . 43
4.1.2 本次作业验收目标 . . . 43
4.1.3 本次必须提交的内容 . . . 43
4.1.4 报告必须回答的问题 . . . 43
4.2 实验目的 . . . 44
4.3 实验环境 . . . 44
4.3.1 硬件环境 . . . 44
4.3.2 软件环境 . . . 44
4.4 STM32CubeMX 配置步骤 . . . 44
4.4.1 新建工程并选择芯片 . . . 44

4.4.2 配置调试接口（SYS） . . . 45
4.4.3 配置高速外部时钟（RCC） . . . 45
4.4.4 启用 USB 外设（Connectivity） . . . 46
4.4.5 配置 USB 中间件（Middleware） . . . 46
4.4.6 配置时钟树（Clock Configuration） . . . 47
4.4.7 配置工程并生成代码 . . . 47
4.4.8 配置工具链为 CMake . . . 48
4.5 硬件连接要点 . . . 48
4.6 电脑端驱动与串口助手准备 . . . 48
4.7 本工程实现的实验内容 . . . 49
4.8 核心代码说明 . . . 49
4.8.1 main.c 中新增的头文件、变量与函数声明 . . . 49
4.8.2 main.c 中的发送函数 USB_SendString . . . 50
4.8.3 main.c 中的命令解析函数 USB_ProcessCommand . . . 50
4.8.4 main.c 中的初始化阶段代码 . . . 52
4.8.5 main.c 中的主循环核心代码 . . . 52
4.8.6 usbd_cdc_if.h 中新增的共享变量声明 . . . 53
4.8.7 usbd_cdc_if.c 中新增的共享变量定义 . . . 54
4.8.8 usbd_cdc_if.c 中的接收回调函数 . . . 54
4.8.9 代码结构的整体工作流程 . . . 55
4.9 当前示例支持的命令 . . . 55
4.9.1 help . . . 55
4.9.2 ping . . . 55
4.9.3 status . . . 55
4.9.4 dice . . . 55
4.9.5 echo . . . 55
4.10 编程技巧总结 . . . 56
4.11 实验操作步骤 . . . 56
4.11.1 编译工程 . . . 56
4.11.2 烧录程序 . . . 56
4.11.3 连接电脑 . . . 56
4.11.4 查看 COM 口 . . . 56
4.11.5 打开串口调试助手 . . . 56
4.11.6 输入测试命令 . . . 56
4.11.7 记录实验结果 . . . 57
4.12 实验现象与结果分析 . . . 57
4.13 常见问题排查 . . . 58
4.14 课后思考题 . . . 58
4.15 实验报告提交要求 . . . 58
4.15.1 报告建议结构 . . . 58
4.15.2 图片与流程图要求 . . . 59
4.15.3 提交规范 . . . 59
4.15.4 最低验收标准 . . . 59
4.16 可进一步扩展的方向 . . . 59
4.17 总结 . . . 59

5 STM32 UART 串口通信实验 60
5.1 实验概述 . . . 60
5.2 硬件需求 . . . 60
5.3 STM32CubeMX 配置步骤 . . . 60
5.3.1 新建工程 . . . 60
5.3.2 配置时钟源（RCC） . . . 61
5.3.3 配置调试接口（SYS） . . . 61
5.3.4 配置 USART1 . . . 62
5.3.5 配置 GPIO（PC13 LED） . . . 62
5.3.6 配置时钟树（Clock Configuration） . . . 63
5.3.7 配置工程输出 . . . 64
5.3.8 生成代码 . . . 64
5.4 程序结构 . . . 64
5.4.1 文件职责速览 . . . 65
5.5 核心代码详解 . . . 65
5.5.1 程序流程图 . . . 65
5.5.2 代码逐段解析 . . . 65
5.6 API 函数详解 . . . 67
5.6.1 HAL_UART_Receive . . . 67
5.6.2 HAL_UART_Transmit . . . 67
5.6.3 HAL_GPIO_WritePin . . . 67
5.6.4 HAL_Delay . . . 67
5.7 使用说明 . . . 67
5.7.1 编译与烧录 . . . 67
5.7.2 串口助手配置 . . . 68
5.7.3 预期现象 . . . 69
5.7.4 调试技巧 . . . 69
5.8 常见问题 . . . 69
5.8.1 Q1: 为什么不用中断或 DMA 接收 UART? . . . 69
5.8.2 Q2: 延时 1 秒会影响命令响应吗? . . . 69
5.8.3 Q3: PC13 开漏输出为什么这样控制 LED? . . . 69
5.8.4 Q4: 如何使用虚拟串口（Virtual COM Port)? . . . 70
5.9 扩展练习 . . . 70
5.10 参考资料 . . . 70

6 STM32 UART 中断方式控制 LED 71
6.1 实验概述 . . . 71
6.2 中断方式 vs 轮询方式 . . . 71
6.3 硬件需求 . . . 71
6.4 STM32CubeMX 配置步骤 . . . 72
6.4.1 新建工程 . . . 72
6.4.2 配置时钟源（RCC） . . . 72
6.4.3 配置调试接口（SYS） . . . 73
6.4.4 配置 USART1 . . . 73
6.4.5 使能 USART1 中断（NVIC）* 关键步骤 . . . 73
6.4.6 配置 GPIO（PC13 LED） . . . 74
6.4.7 配置时钟树（Clock Configuration） . . . 75

6.4.8 生成代码 . . . 75
6.5 程序结构 . . . 75
6.5.1 关键文件说明 . . . 76
6.6 核心代码详解 . . . 76
6.6.1 程序流程图 . . . 76
6.6.2 代码逐段解析 . . . 77
6.7 API 函数详解 . . . 78
6.7.1 HAL_UART_Receive_IT . . . 78
6.7.2 HAL_UART_RxCpltCallback . . . 79
6.7.3 中断数据流总结 . . . 79
6.8 中断方式的优势总结 . . . 79
6.8.1 响应速度 . . . 79
6.8.2 CPU 利用率 . . . 79
6.8.3 程序扩展性 . . . 79
6.8.4 功耗 . . . 79
6.9 使用说明 . . . 80
6.9.1 编译与烧录 . . . 80
6.9.2 串口助手配置 . . . 80
6.9.3 预期现象 . . . 80
6.9.4 调试技巧 . . . 80
6.10 常见问题 . . . 80
6.10.1 Q1: 为什么回调函数中还要用轮询发送 HAL_UART_Transmit，不也用中断发送吗? . . . 80
6.10.2 Q2: HAL_UART_Receive_IT 可以一次接收多个字节吗? . . . 81
6.10.3 Q3: 多个 UART 同时使用中断模式，回调怎么区分? . . . 81
6.10.4 Q4: 中断接收会不会丢失数据? . . . 81
6.11 扩展练习 . . . 81
6.12 参考资料 . . . 81

7 STM32 I2C OLED 显示实验 82
7.1 实验概述 . . . 82
7.2 硬件需求 . . . 82
7.3 STM32CubeMX 配置步骤 . . . 83
7.3.1 新建工程 . . . 83
7.3.2 配置时钟源（RCC） . . . 83
7.3.3 配置调试接口（SYS） . . . 84
7.3.4 配置 I2C1 . . . 84
7.3.5 配置时钟树（Clock Configuration） . . . 85
7.3.6 配置工程输出 . . . 86
7.3.7 生成代码 . . . 86
7.4 程序结构 . . . 87
7.4.1 文件职责速览 . . . 87
7.5 核心代码详解 . . . 88
7.5.1 程序流程图 . . . 88
7.5.2 I2C 协议基础 . . . 88
7.5.3 I2C 地址说明 . . . 88
7.5.4 代码逐段解析 . . . 89

7.6 API 函数详解 . . . 92
7.6.1 HAL_I2C_Master_Transmit . . . 92
7.6.2 HAL_I2C_Mem_Write . . . 93
7.6.3 HAL_I2C_Init . . . 93
7.7 使用说明 . . . 93
7.7.1 编译与烧录 . . . 93
7.7.2 预期现象 . . . 94
7.7.3 调试技巧 . . . 94
7.8 常见问题 . . . 94
7.8.1 Q1: 为什么我的 OLED 地址是 0x7B 但代码用的是 0x78? . . . 94
7.8.2 Q2: 为什么不能使用硬件滚动? . . . 94
7.8.3 Q3: 字体如何修改? . . . 94
7.8.4 Q4: 如何显示中文? . . . 94
7.8.5 Q5: I2C 速率能否提高到 400kHz? . . . 95
7.9 扩展练习 . . . 95
7.10 作业要求 . . . 95
7.10.1 基础要求（必做） . . . 95
7.10.2 进阶要求（选做，加分） . . . 96
7.10.3 实验报告要求 . . . 96
7.11 参考资料 . . . 97

8 STM32 SPI 主从回环通信实验 98
8.1 实验概述 . . . 98
8.2 硬件需求 . . . 98
8.3 SPI 协议基础 . . . 98
8.3.1 四线制信号 . . . 99
8.3.2 时钟极性与相位（SPI Mode） . . . 99
8.3.3 通信过程（全双工） . . . 99
8.3.4 SPI 与 I2C 的对比 . . . 99
8.4 STM32CubeMX 配置步骤 . . . 100
8.4.1 新建工程 . . . 100
8.4.2 配置调试接口（SYS） . . . 100
8.4.3 配置时钟源（RCC） . . . 100
8.4.4 配置 SPI1（主机） . . . 101
8.4.5 配置 SPI2（从机） . . . 102
8.4.6 配置时钟树（Clock Configuration） . . . 102
8.4.7 配置工程输出 . . . 103
8.4.8 生成代码 . . . 103
8.5 程序结构 . . . 104
8.5.1 文件职责速览 . . . 104
8.6 核心代码详解 . . . 105
8.6.1 程序流程图 . . . 105
8.6.2 全局变量与回调函数 . . . 105
8.6.3 中断配置 . . . 105
8.6.4 主循环回环测试逻辑 . . . 106
8.6.5 数据传输详解 . . . 106
8.6.6 SPI1 初始化（主机） . . . 108

8.6.7 SPI2 初始化（从机） . . . 108
8.6.8 HAL_SPI_MspInit – GPIO 引脚配置 . . . 108
8.7 API 函数详解 . . . 109
8.7.1 HAL_SPI_TransmitReceive（阻塞模式） . . . 109
8.7.2 HAL_SPI_TransmitReceive_IT（中断模式） . . . 110
8.7.3 HAL_SPI_TxRxCpltCallback（回调函数） . . . 110
8.7.4 HAL_SPI_Init . . . 110
8.8 使用说明 . . . 110
8.8.1 编译与烧录 . . . 110
8.8.2 调试与验证 . . . 111
8.8.3 调试技巧 . . . 111
8.9 常见问题 . . . 112
8.9.1 Q1: 为什么从机也要准备发送数据? 它不是在”接收”吗? . . . 112
8.9.2 Q2: 为什么要用硬件 NSS 而不是软件 NSS? . . . 112
8.9.3 Q3: 为什么 SPI1 用阻塞模式，SPI2 用中断模式? . . . 112
8.9.4 Q4: 如果我把 CPOL/CPHA 设反了会怎样? . . . 112
8.9.5 Q5: SPI1 和 SPI2 可以放在不同芯片上吗? . . . 112
8.10 扩展练习 . . . 113
8.11 作业要求 . . . 113
8.11.1 基础要求（必做） . . . 113
8.11.2 进阶要求（选做，加分） . . . 114
8.11.3 实验报告要求 . . . 114
8.12 参考资料 . . . 115

9 STM32 ADC 内置温度传感器采集与虚拟串口传输实验 116
9.1 统一作业说明 . . . 116
9.1.1 学生需要完成的核心任务 . . . 116
9.1.2 本次作业验收目标 . . . 116
9.1.3 本次必须提交的内容 . . . 116
9.1.4 报告必须回答的问题 . . . 117
9.2 实验目的 . . . 117
9.3 实验原理 . . . 117
9.3.1 STM32F103 内置温度传感器 . . . 117
9.3.2 VREFINT 内部参考电压与 ADC 校正原理 . . . 118
9.3.3 ADC 量化原理 . . . 118
9.3.4 USB CDC 虚拟串口原理 . . . 119
9.4 实验环境 . . . 119
9.4.1 硬件环境 . . . 119
9.4.2 软件环境 . . . 119
9.5 STM32CubeMX 配置步骤 . . . 119
9.5.1 新建工程并选择芯片 . . . 119
9.5.2 配置调试接口（SYS） . . . 119
9.5.3 配置高速外部时钟（RCC） . . . 120
9.5.4 配置 ADC1（温度传感器 + VREFINT 双通道） . . . 120
9.5.5 启用 USB 外设 . . . 122
9.5.6 配置 USB CDC 中间件 . . . 122
9.5.7 配置时钟树 . . . 123

9.5.8 配置工程并生成代码 . . . 123
9.6 硬件连接要点 . . . 124
9.7 核心代码说明 . . . 124
9.7.1 main.c 中新增的头文件 . . . 124
9.7.2 main.c 中的初始化阶段代码 . . . 125
9.7.3 main.c 中的主循环核心代码 . . . 125
9.7.4 温度计算原理详解 . . . 126
9.7.5 CDC_Transmit_FS 发送函数 . . . 127
9.7.6 ADC1 初始化代码（由 STM32CubeMX 生成） . . . 128
9.7.7 系统时钟配置代码（由 STM32CubeMX 生成） . . . 128
9.7.8 代码结构的整体工作流程 . . . 128
9.8 编程技巧总结 . . . 129
9.9 实验操作步骤 . . . 129
9.9.1 编译工程 . . . 129
9.9.2 烧录程序 . . . 130
9.9.3 连接电脑 . . . 130
9.9.4 查看 COM 口 . . . 130
9.9.5 打开串口调试助手 . . . 130
9.9.6 观察温度输出 . . . 131
9.9.7 记录实验结果 . . . 131
9.10 实验现象与结果分析 . . . 131
9.11 常见问题排查 . . . 132
9.12 课后思考题 . . . 133
9.13 实验报告提交要求 . . . 133
9.13.1 报告建议结构 . . . 133
9.13.2 图片与流程图要求 . . . 134
9.13.3 提交规范 . . . 134
9.13.4 最低验收标准 . . . 134
9.14 可进一步扩展的方向 . . . 134
9.15 总结 . . . 135
9.16 参考 . . . 135

10 STM32 ADC 温度传感器 DMA 采集与虚拟串口高速传输实验 136
10.1 统一作业说明 . . . 136
10.1.1 学生需要完成的核心任务 . . . 136
10.1.2 本次作业验收目标 . . . 136
10.1.3 本次必须提交的内容 . . . 136
10.1.4 报告必须回答的问题 . . . 137
10.2 实验目的 . . . 137
10.3 实验原理 . . . 137
10.3.1 STM32F103 内置温度传感器 . . . 137
10.3.2 ADC 连续转换模式 . . . 138
10.3.3 DMA 循环模式原理 . . . 138
10.3.4 HAL ADC DMA 回调机制 . . . 139
10.3.5 USB CDC 虚拟串口原理 . . . 139
10.3.6 newlib-nano 与浮点格式化限制 . . . 139
10.3.7 数据吞吐能力估算 . . . 140

10.4 实验环境 . . . 140
10.4.1 硬件环境 . . . 140
10.4.2 软件环境 . . . 140
10.4.3 Python 环境配置 . . . 140
10.5 STM32CubeMX 配置步骤 . . . 141
10.5.1 从作业 9 工程另存为新工程 . . . 141
10.5.2 检查调试接口（SYS） . . . 141
10.5.3 检查高速外部时钟（RCC） . . . 141
10.5.4 配置 ADC1（温度传感器通道） . . . 141
10.5.5 配置 DMA（ADC1 数据传输） . . . 143
10.5.6 启用 USB 外设并配置 CDC 中间件 . . . 144
10.5.7 配置时钟树 . . . 144
10.5.8 配置工程并生成代码 . . . 144
10.6 硬件连接要点 . . . 145
10.7 核心代码说明 . . . 145
10.7.1 代码整体结构 . . . 145
10.7.2 main.c 中新增的头文件和变量声明 . . . 145
10.7.3 DMA 传输完成回调函数 . . . 146
10.7.4 main() 中的初始化与启动代码 . . . 146
10.7.5 ADC1 初始化代码（由 STM32CubeMX 生成） . . . 147
10.7.6 DMA 初始化代码（由 STM32CubeMX 生成） . . . 148
10.7.7 DMA 中断处理函数 . . . 148
10.7.8 Python 客户端代码 . . . 148
10.7.9 代码结构的整体工作流程 . . . 149
10.8 编程技巧总结 . . . 149
10.9 实验操作步骤 . . . 150
10.9.1 编译工程 . . . 150
10.9.2 烧录程序 . . . 150
10.9.3 连接电脑 . . . 150
10.9.4 查看 COM 口 . . . 150
10.9.5 用串口调试助手验证单片机输出 . . . 150
10.9.6 安装 Python 依赖 . . . 151
10.9.7 运行 Python 客户端 . . . 151
10.9.8 记录实验结果 . . . 152
10.10实验现象与结果分析 . . . 152
10.10.1 预期现象 . . . 152
10.10.2 结果分析 . . . 152
10.11常见问题排查 . . . 152
10.12课后思考题 . . . 153
10.13实验报告提交要求 . . . 153
10.13.1 报告建议结构 . . . 153
10.13.2 图片与流程图要求 . . . 154
10.13.3 提交规范 . . . 154
10.13.4 最低验收标准 . . . 154
10.14可进一步扩展的方向 . . . 154
10.15总结 . . . 155
10.16参考 . . . 155

11 STM32 通用定时器 – LED 呼吸灯实验 **156**
11.1 统一作业说明 . . . 156
11.1.1 学生需要完成的核心任务 . . . 156
11.1.2 本次作业验收目标 . . . 156
11.1.3 本次必须提交的内容 . . . 156
11.1.4 报告必须回答的问题 . . . 156
11.2 关键说明 . . . 157
11.3 实验目的 . . . 157
11.4 实验环境 . . . 157
11.4.1 硬件环境 . . . 157
11.4.2 软件环境 . . . 157
11.5 STM32CubeMX 配置步骤 . . . 157
11.5.1 新建工程并选择芯片 . . . 158
11.5.2 配置调试接口（SYS） . . . 158
11.5.3 配置高速外部时钟（RCC） . . . 158
11.5.4 配置时钟树（Clock Configuration） . . . 159
11.5.5 配置 PC13 为 GPIO 输出 . . . 159
11.5.6 配置通用定时器参数 . . . 160
11.5.7 使能定时器中断 . . . 161
11.5.8 工程设置与生成代码 . . . 161
11.6 硬件连接要点 . . . 162
11.7 本工程实现的实验内容 . . . 162
11.8 核心代码说明 . . . 162
11.8.1 软件 PWM 呼吸灯原理 . . . 163
11.8.2 main.c 中新增的变量定义 . . . 163
11.8.3 main.c 中的定时器中断回调函数 . . . 164
11.8.4 main.c 中的主函数 . . . 165
11.8.5 代码结构整体工作流程 . . . 166
11.9 编程技巧总结 . . . 166
11.10实验操作步骤 . . . 166
11.10.1 配置 CubeMX 并生成代码 . . . 166
11.10.2 添加应用代码 . . . 166
11.10.3 编译工程 . . . 166
11.10.4 烧录程序 . . . 166
11.10.5 观察现象 . . . 167
11.11实验现象与结果分析 . . . 167
11.12常见问题排查 . . . 167
11.13课后思考题 . . . 168
11.14实验报告提交要求 . . . 168
11.14.1 报告建议结构 . . . 168
11.14.2 图片与流程图要求 . . . 168
11.14.3 提交规范 . . . 168
11.14.4 最低验收标准 . . . 169
11.15可进一步扩展的方向 . . . 169
11.16总结 . . . 169

12 STM32 通用定时器 – 三通道硬件 PWM 交替呼吸灯实验 **170**
12.1 统一作业说明 . . . 170
12.1.1 学生需要完成的核心任务 . . . 170
12.1.2 本次作业验收目标 . . . 170
12.1.3 本次必须提交的内容 . . . 170
12.1.4 报告必须回答的问题 . . . 170
12.2 关键说明 . . . 171
12.3 实验目的 . . . 171
12.4 实验环境 . . . 171
12.4.1 硬件环境 . . . 171
12.4.2 软件环境 . . . 171
12.5 STM32CubeMX 配置步骤 . . . 171
12.5.1 新建工程并选择芯片 . . . 172
12.5.2 配置调试接口（SYS） . . . 172
12.5.3 配置高速外部时钟（RCC） . . . 172
12.5.4 配置时钟树（Clock Configuration） . . . 173
12.5.5 配置 TIM3 PWM 输出通道 . . . 173
12.5.6 工程设置与生成代码 . . . 174
12.6 硬件连接要点 . . . 175
12.7 本工程实现的实验内容 . . . 175
12.8 核心代码说明 . . . 176
12.8.1 硬件 PWM 交替呼吸灯原理 . . . 176
12.8.2 main.c 中新增的变量定义 . . . 176
12.8.3 main.c 中的 PWM 启动代码 . . . 177
12.8.4 main.c 中的主循环 . . . 177
12.8.5 为什么不需要中断 . . . 178
12.9 编程技巧总结 . . . 178
12.10实验操作步骤 . . . 178
12.10.1 配置 CubeMX 并生成代码 . . . 178
12.10.2 添加应用代码 . . . 178
12.10.3 编译工程 . . . 179
12.10.4 烧录程序 . . . 179
12.10.5 观察现象 . . . 179
12.11实验现象与结果分析 . . . 179
12.12常见问题排查 . . . 180
12.13课后思考题 . . . 180
12.14实验报告提交要求 . . . 180
12.14.1 报告建议结构 . . . 180
12.14.2 图片与流程图要求 . . . 181
12.14.3 提交规范 . . . 181
12.14.4 最低验收标准 . . . 181
12.15可进一步扩展的方向 . . . 181
12.16总结 . . . 181

13 STM32 高级定时器 – 互补 PWM 输出实验 **182**
13.1 统一作业说明 . . . 182
13.1.1 学生需要完成的核心任务 . . . 182

13.1.2 本次作业验收目标 . . . 182
13.1.3 本次必须提交的内容 . . . 182
13.1.4 报告必须回答的问题 . . . 182
13.2 关键说明 . . . 183
13.3 实验目的 . . . 183
13.4 实验环境 . . . 183
13.4.1 硬件环境 . . . 183
13.4.2 软件环境 . . . 183
13.5 STM32CubeMX 配置步骤 . . . 183
13.5.1 新建工程并选择芯片 . . . 184
13.5.2 配置调试接口（SYS） . . . 184
13.5.3 配置高速外部时钟（RCC） . . . 184
13.5.4 配置时钟树（Clock Configuration） . . . 185
13.5.5 配置 TIM1 基本参数 . . . 185
13.5.6 配置互补 PWM 输出通道（CH2 + CH2N） . . . 186
13.5.7 配置死区时间与刹车功能 . . . 187
13.5.8 配置 BKIN 刹车输入引脚（GPIO） . . . 188
13.5.9 工程设置与生成代码 . . . 188
13.6 硬件连接要点 . . . 189
13.7 本工程实现的实验内容 . . . 190
13.8 核心代码说明 . . . 190
13.8.1 互补 PWM 硬件原理 . . . 190
13.8.2 main.c 中启动互补 PWM 的代码 . . . 190
13.8.3 CubeMX 生成的 TIM1 初始化代码 . . . 191
13.8.4 互补 PWM 波形示意图 . . . 191
13.9 编程技巧总结 . . . 192
13.10实验操作步骤 . . . 192
13.10.1 配置 CubeMX 并生成代码 . . . 192
13.10.2 添加应用代码 . . . 192
13.10.3 编译工程 . . . 192
13.10.4 烧录程序 . . . 192
13.10.5 逻辑分析仪验证 . . . 192
13.11实验现象与结果分析 . . . 193
13.12常见问题排查 . . . 193
13.13课后思考题 . . . 194
13.14实验报告提交要求 . . . 194
13.14.1 报告建议结构 . . . 194
13.14.2 图片与流程图要求 . . . 194
13.14.3 提交规范 . . . 194
13.14.4 最低验收标准 . . . 195
13.15可进一步扩展的方向 . . . 195
13.16总结 . . . 195

1 STM32 GPIO 输入输出实验

1.1 统一作业说明

1.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成本实验所需的最小工程配置，并确保 `.ioc` 文件可再次打开和重新生成代码。
2. 在 HAL 库框架下完成 GPIO 输入与输出程序编写，不得仅提交伪代码或截图。
3. 成功编译、下载并在开发板上验证实验现象，记录实际现象与理论分析是否一致。
4. 在实验报告中说明配置步骤、关键代码、运行结果、问题排查过程与个人分析。
5. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告，最终提交 PDF。

1.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6
输入信号	读取 PB12 按键电平
输出信号	使用 PC13 驱动 LED
必做功能	按键状态变化能够正确映射到 LED 状态
验收方式	现场演示或结果照片/截图，能够说明按下与松开时 LED 的变化

1.1.3 本次必须提交的内容

1. 可编译的完整工程目录。
2. 一份 PDF 格式实验报告。
3. 关键结果图片，至少包含硬件连接或板载现象图片、CubeMX 配置截图、关键代码截图各 1 张。
4. 如教师有要求，可额外提交演示视频；若无特别要求，图片即可。

1.1.4 报告必须回答的问题

1. 本实验的输入端口、输出端口分别是什么，电气连接逻辑是什么。
2. PB12 采用上拉输入的原因是什么。
3. PC13 采用推挽输出、低速输出的原因是什么。
4. 程序运行现象是否与硬件极性一致，如不一致应如何解释。

1.2 作业要求

使用 STM32CubeMX 生成工程框架，在 `main.c` 中编写用户代码，通过 HAL 库实现：

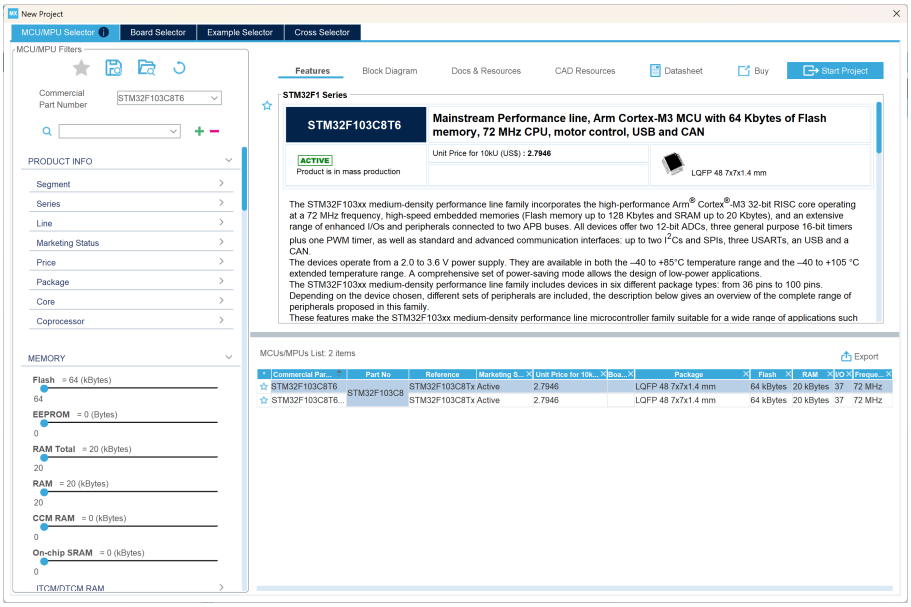
- 输入：读取按键（PB12）的电平状态
- 输出：将读取到的电平状态直接驱动 LED（PC13）

即按键按下时 LED 亮，松开时 LED 灭（或相反，取决于硬件连接极性）。

1.3 STM32CubeMX 配置步骤

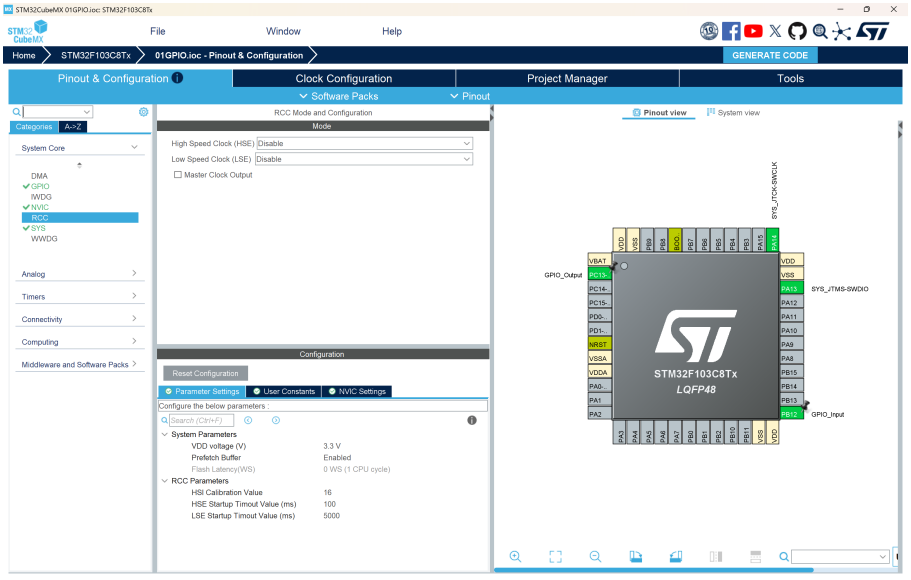
1.3.1 新建工程

1. 打开 STM32CubeMX，点击 **File → New Project**
2. 在芯片选择界面搜索 `STM32F103C8T6`，选中后点击 **Start Project**

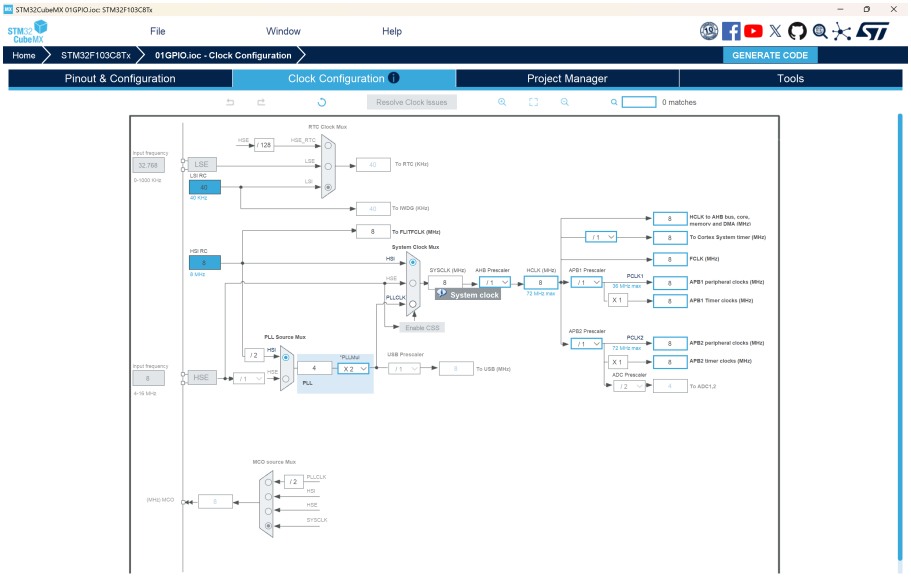


1.3.2 配置系统时钟（RCC）

1. 左侧 **Pinout & Configuration → System Core → RCC**
2. **High Speed Clock (HSE)** 选择 `Disable`（使用内部 HSI，8 MHz）

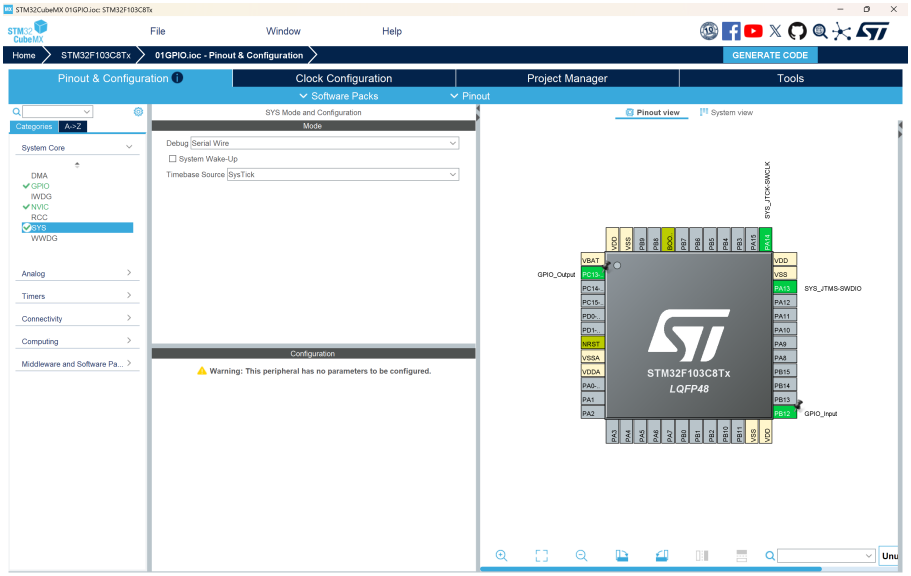


3. 切换到 **Clock Configuration** 标签页，确认 SYSCLK 来源为 HSI，频率为 8 MHz



1.3.3 配置调试接口（SYS）

1. 左侧 **Pinout & Configuration → System Core → SYS**
2. **Debug** 下拉菜单选择 `Serial Wire`



为什么必须选择 **Serial Wire**：

STM32F103C8T6 支持两种调试接口：

选项	引脚占用	说明
`No Debug`	无	禁用调试接口，PA13/PA14 作为普通 GPIO
`Serial Wire`	PA13 (SWDIO)、PA14 (SWCLK)	SWD 调试接口，ST-Link 使用此协议
`JTAG (4 pins)`	PA13/PA14/PA15/PB3	完整 JTAG，占用引脚多

选项	引脚占用	说明
JTAG (5 pins)	上述 + PB4	完整 JTAG + TRST

选择 No Debug 后烧录程序会发生什么：

CubeMX 生成的初始化代码会将 PA13/PA14 重映射为普通 GPIO，**SWD** 接口被禁用。烧录该程序后：

- ST-Link 无法再通过 SWD 连接芯片
- 后续无法使用 ST-Link 烧录新程序或调试
- 表现为 ST-Link 报错：No target connected 或 Cannot connect to target

这会让 **STM32** 变砖吗?

不会永久变砖，但需要通过串口 **ISP**（**UART** 引导加载程序）恢复：

1. 将 BOOT0 引脚拉高（接 VCC），BOOT1 拉低（接 GND）
2. 芯片上电后进入内置 Bootloader 模式
3. 使用 **STM32CubeProgrammer** 通过 UART（PA9/PA10）连接芯片
4. 烧录一个正确配置了 Serial Wire 的新程序
5. 将 BOOT0 重新拉低，复位芯片，SWD 恢复正常

BOOT0 / BOOT1 引脚详解：

STM32F103 上电或复位时采样 BOOT0 和 BOOT1 电平，决定启动来源：

BOOT0	BOOT1	启动模式	说明
0（GND）	任意	主闪存（Flash）启动	正常运行用户程序，默认模式
1（VCC）	0（GND）	系统存储器启动	运行出厂 Bootloader，支持串口 ISP 烧录
1（VCC）	1（VCC）	SRAM 启动	从 SRAM 运行代码，极少使用

Blue Pill 板上有两个跳线帽，标注 BOOT0 和 BOOT1，拨到 0 侧接 GND，拨到 1 侧接 VCC。

进入串口 ISP 模式步骤: 1. **BOOT0** 跳线拨到 1,BOOT1 保持 0 2. 按 RESET 或重新上电,芯片进入系统 Bootloader 3. USB-TTL 模块连接: PA9 → RX, PA10 → TX, GND → GND 4. STM32CubeProgrammer 选接口 UART，选对应串口，点击 Connect 5. 烧录完成后 **BOOT0** 拨回 0，按 RESET 正常启动

> 结论：选择 No Debug 不会物理损坏芯片，但会导致 SWD 调试接口失效，必须通过串口 Bootloader 才能恢复，操作繁琐。务必选择 Serial Wire。

利用 **Reset** 按键配合 **ST-Link** 恢复 **SWD** 的方法：

如果 SWD 已被禁用，ST-Link 无法正常连接，可以尝试以下方法，利用芯片复位瞬间 SWD 尚未被用户代码覆盖的时间窗口强行连接：

1. 打开 **STM32CubeProgrammer**，连接方式选择 ST-LINK，Mode 选择 Under Reset
2. 用手按住开发板上的 **RESET** 按键不松开（保持芯片处于复位状态）

3. 点击 STM32CubeProgrammer 的 **Connect** 按钮
4. 在 Connect 的同时松开 **RESET** 按键，让芯片开始运行
5. 芯片刚上电的极短时间内，SWD 引脚尚处于默认复用状态，STM32CubeProgrammer 可以在用户代码将其重映射为 GPIO 之前抢先建立连接
6. 连接成功后，立即烧录一个正确配置了 `Serial Wire` 的新程序

   此方法成功率取决于时序配合，需要多试几次。若多次失败，则改用串口 ISP（BOOT0 拉高）方式恢复。

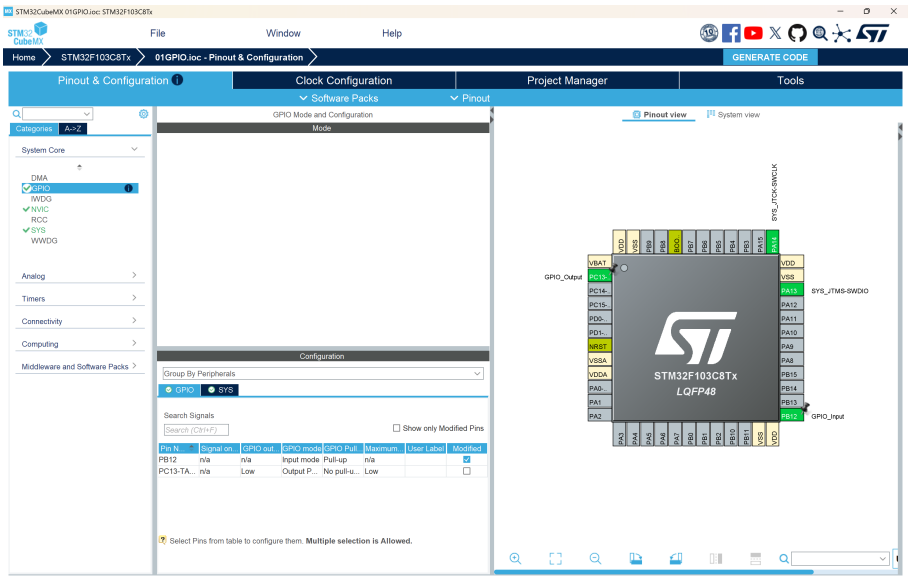
1.3.4 配置 GPIO 引脚

1.3.4.1 PC13（LED 输出）：

1. 在芯片引脚图上点击 **PC13**，选择 `GPIO_Output`
2. 左侧 **System Core → GPIO**，点击 PC13 行进行详细配置：
   - GPIO output level：`Low`
   - GPIO mode：`Output Push Pull`（推挽输出）
   - GPIO Pull-up/Pull-down：`No pull-up and no pull-down`
   - Maximum output speed：`Low`
   - User Label：`LED`（可选）

1.3.4.2 PB12（按键输入）：

1. 在芯片引脚图上点击 **PB12**，选择 `GPIO_Input`
2. 点击 PB12 行进行详细配置：
   - GPIO mode：`Input mode`
   - GPIO Pull-up/Pull-down：`Pull-up`（上拉）
   - User Label：`KEY`（可选）



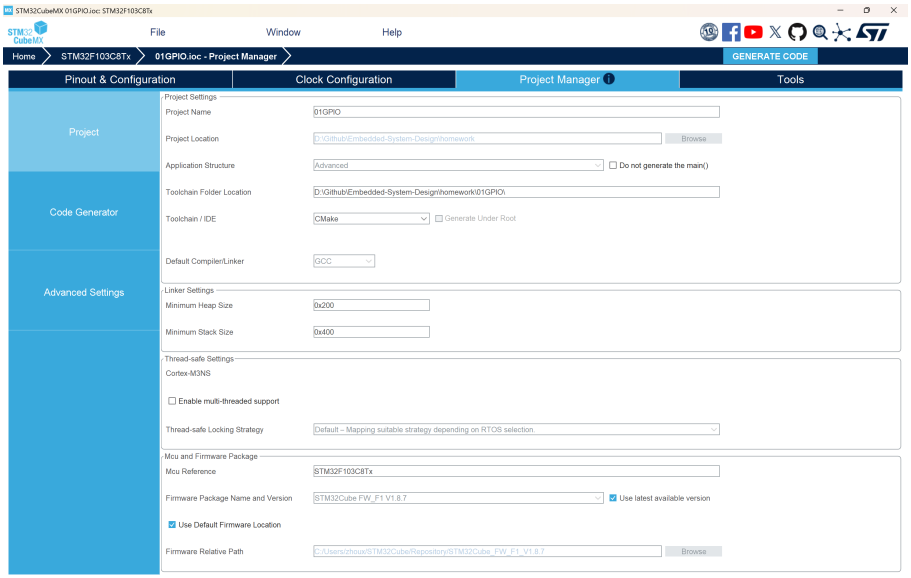
1.3.5 配置工程输出

1. 切换到 **Project Manager** 标签页
2. **Project** 子页：

   - Project Name：填写工程名（如 `01GPIO`）
   - Project Location：选择保存路径
   - Toolchain/IDE：根据开发环境选择，各选项区别如下：

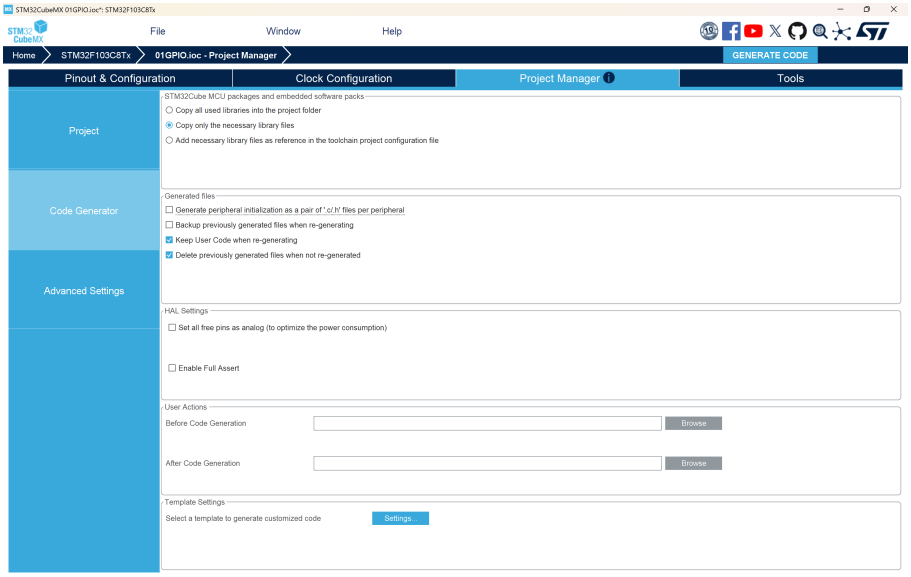
选项	说明	适用场景
`STM32CubeIDE`	ST 官方 IDE，基于 Eclipse，开箱即用	使用 STM32CubeIDE 开发
`MDK-ARM`	Keil MDK，商业 IDE，调试功能强	使用 Keil 开发
`EWARM`	IAR Embedded Workbench，商业 IDE	使用 IAR 开发
`Makefile`	生成 Makefile，命令行构建	Linux 环境或自定义工具链
`CMake`	生成 CMakeLists.txt，跨平台构建系统	VSCode + STM32 插件

本实验选择 `CMake`，原因是使用 VSCode 配合 **STM32 VS Code Extension** 插件 `STM32CubeIDE for VS Code` 进行开发。该插件原生支持 CMake 工程，可在 VSCode 中直接完成编译、烧录和调试，无需安装 STM32CubeIDE 或 Keil。



3. **Code Generator** 子页：

   - 勾选 **Generate peripheral initialization as a pair of '.c/.h' files per peripheral**（可选）



1.3.6 生成代码

点击右上角 **GENERATE CODE**，CubeMX 自动生成包含 `MX_GPIO_Init` 的工程框架。

1.3.7 添加用户代码

在生成的 `Core/Src/main.c` 中，找到 `while (1)` 循环内的 `USER CODE BEGIN 3` 注释区域，添加：

```
GPIO_PinState key_state = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_12);
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, key_state);
```

1.4 核心代码解析

1.4.1 GPIO 初始化（MX_GPIO_Init）

```
// 使能 GPIO 时钟
__HAL_RCC_GPIOC_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();

// 配置 PC13 为推挽输出
GPIO_InitStruct.Pin   = GPIO_PIN_13;
GPIO_InitStruct.Mode  = GPIO_MODE_OUTPUT_PP;  // 推挽输出
GPIO_InitStruct.Pull  = GPIO_NOPULL;          // 无上下拉
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

// 配置 PB12 为上拉输入
GPIO_InitStruct.Pin  = GPIO_PIN_12;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;       // 输入模式
GPIO_InitStruct.Pull = GPIO_PULLUP;           // 内部上拉
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

关键点： - 使用 GPIO 外设前必须先使能对应端口的时钟，否则寄存器写入无效。- GPIO_InitTypeDef 结构体复用时，只需重新赋值需要改变的字段，再调用 HAL_GPIO_Init。

1.4.2 主循环逻辑

```
while (1)
{
    GPIO_PinState key_state = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_12);
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, key_state);
}
```

- HAL_GPIO_ReadPin 返回 GPIO_PIN_SET（1）或 GPIO_PIN_RESET（0）。
- 将读取值直接写入输出引脚，实现输入到输出的直通映射。
- 由于 PB12 配置了内部上拉，未按键时读到高电平（SET），按下后拉低读到低电平（RESET）。

1.5 不同 GPIO 配置对结果的影响分析

1.5.1 输入配置

配置	宏定义	未接外部信号时的引脚状态	典型应用场景
上拉输入	GPIO_PULLUP	默认高电平（1）	按键低电平有效（按下接 GND）
下拉输入	GPIO_PULLDOWN	默认低电平（0）	按键高电平有效（按下接 VCC）
浮空输入	GPIO_NOPULL	不确定，易受干扰	外部已有确定驱动时使用

本实验使用上拉输入（GPIO_PULLUP）： - 未按键：PB12 通过内部上拉电阻接 VCC → 读到 GPIO_PIN_SET → PC13 输出高电平 - 按键按下（接 GND）：PB12 被拉低 → 读到 GPIO_PIN_RESET → PC13 输出低电平

若改为下拉输入（GPIO_PULLDOWN），逻辑相反：- 未按键：默认低电平 → LED 状态翻转 - 需要按键接 VCC 才能触发

若改为浮空输入（GPIO_NOPULL）：- 引脚悬空时电平不确定，LED 会随机闪烁，不可靠

1.5.2 输出配置

配置	宏定义	特性	典型应用场景
推挽输出	GPIO_MODE_OUTPUT_PP	可主动输出高/低电平，驱动能力强	直接驱动 LED、继电器等
开漏输出	GPIO_MODE_OUTPUT_OD	只能主动拉低，高电平需外部上拉	I2C 总线、电平转换

本实验使用推挽输出（GPIO_MODE_OUTPUT_PP）： - 输出高电平时，内部 P-MOS 导通，引脚直接驱动到 VCC - 输出低电平时，内部 N-MOS 导通，引脚直接拉到 GND - 驱动能力强，适合直接点亮 LED

若改为开漏输出（GPIO_MODE_OUTPUT_OD）: - 写高电平时引脚处于高阻态，需外部上拉电阻才能输出高电平 - 若无外部上拉，LED 将无法被点亮（引脚悬空）

1.5.3 输出速度配置

配置	宏定义	最大翻转频率	说明
低速	GPIO_SPEED_FREQ_LOW	~2 MHz	功耗低，EMI 小，适合低频信号
中速	GPIO_SPEED_FREQ_MEDIUM	~10 MHz	平衡选择
高速	GPIO_SPEED_FREQ_HIGH	~50 MHz	适合高频通信（SPI、高速 PWM）

本实验 LED 控制对速度无要求，使用低速即可，可减少电磁干扰。

1.6 硬件连接说明

引脚	功能	配置
PC13	LED 输出	推挽输出，低电平点亮（板载 LED 通常低有效）
PB12	按键输入	上拉输入，按下接 GND

1.7 实验报告要求

1.7.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：用 1 段文字说明本次实验要实现的功能。
3. 实验环境：开发板、调试器、软件版本、编译方式。
4. CubeMX 配置过程：至少包含 RCC、SYS、GPIO 的配置说明与截图。
5. 程序设计与代码说明：给出关键代码，并解释 HAL_GPIO_ReadPin 与 HAL_GPIO_WritePin 的作用。
6. 程序流程图：说明主循环中“读按键-写 LED”的执行流程。
7. 实验结果：展示按键按下/松开时的现象图片或结果描述，并说明是否符合预期。
8. 问题分析：记录遇到的问题、排查方法和最终解决办法。
9. 总结：简要总结本实验掌握的知识点。
10. 附录：可附核心源码、额外截图、参考资料。

1.7.2 图片与流程图要求

1. 所有截图或照片都要有图号和图题。
2. 结果图片必须能反映实验是否成功，不能只放代码截图。
3. 程序流程图可以手绘后拍照，也可以使用 Visio、draw.io、Mermaid、TikZ 绘制。
4. 图片应在正文中被引用并配文字说明，不能只在末尾堆图。

1.7.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 1-GPIO 输入输出实验.pdf`。
2. 报告中所有代码截图应保证清晰可读，必要时可直接贴文本代码。
3. 若实验失败，也必须提交报告，并明确写出失败现象、原因分析与改进思路。

1.7.4 最低验收标准

1. 能正确下载程序并运行。
2. 能清楚说明 PB12 和 PC13 的配置理由。
3. 能给出实际实验结果而不是只贴理论说明。
4. 能独立解释输入、输出、上拉、电平极性之间的关系。

2 LED 闪烁实验

2.1 统一作业说明

2.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 配置 GPIO、RCC、SYS 等基础外设，生成可编译工程。
2. 在 HAL 框架下完成 LED 周期闪烁程序，并能够说明闪烁频率的计算过程。
3. 成功编译、下载并验证 LED 以 1 Hz 频率闪烁，记录实验现象。
4. 在实验报告中展示关键配置、核心代码、实验结果、问题排查和思考题答案。
5. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

2.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6
控制对象	PC13 LED
必做功能	LED 亮 500 ms、灭 500 ms，循环运行
理论要求	能解释 RCC 时钟配置和 `HAL_Delay()` 的工作原理
验收方式	现场演示或结果图片/视频，能够证明闪烁周期正确

2.1.3 本次必须提交的内容

1. 可编译的完整工程目录。
2. 一份 PDF 格式实验报告。
3. CubeMX 关键配置截图、主循环代码截图、实验结果图片各至少 1 张。
4. 思考题和讨论题的书面回答。

2.1.4 报告必须回答的问题

1. 为什么本实验要先正确配置系统时钟。
2. `HAL_Delay(500)` 对应的闪烁频率是多少，如何计算。
3. 如果系统时钟不正确，LED 闪烁频率为什么会偏差。
4. 阻塞式延时在嵌入式系统中的优点和局限是什么。

2.2 实验目的

1. 掌握 STM32 GPIO 的基本配置方法
2. 学习使用 STM32CubeMX 进行工程配置
3. 理解 HAL 库的 GPIO 操作函数

4. 掌握延时函数的使用方法
5. 完成一个简单的 LED 闪烁程序

2.3 基础知识

在开始实验之前，我们需要理解两个核心概念：RCC 时钟系统和 HAL_Delay 延时函数。这两个知识点是本次实验的理论基础。

2.3.1 RCC（Reset and Clock Control）时钟系统

2.3.1.1 什么是 RCC?

RCC 是 STM32 的复位和时钟控制模块，负责管理整个芯片的时钟系统。可以把它理解为 STM32 的”心脏起搏器”，为所有外设提供工作节拍。

为什么需要时钟？ - 数字电路需要时钟信号来同步各种操作 - 没有时钟，CPU 无法执行指令，外设无法工作 - 不同的外设可能需要不同频率的时钟

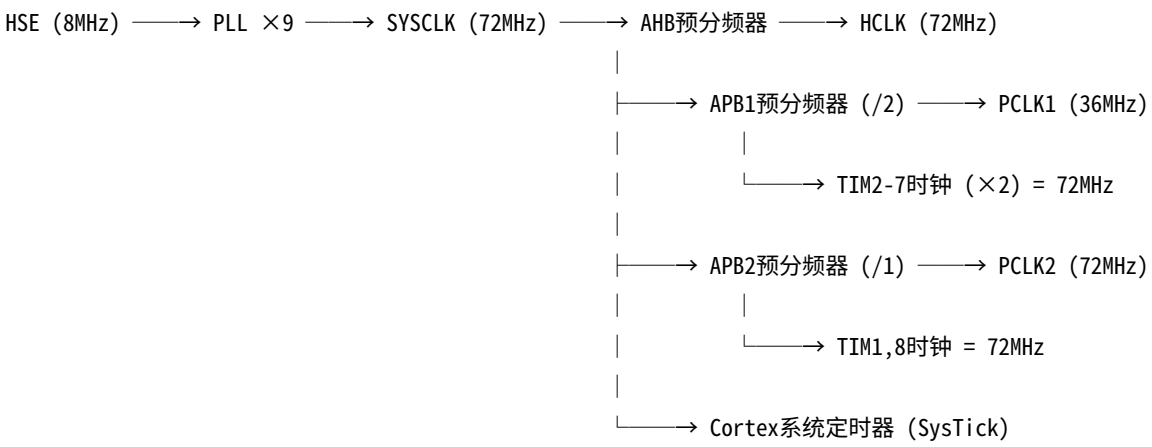
2.3.1.2 STM32F103 的时钟源

STM32F103 有多个时钟源可供选择：

1. **HSI（High Speed Internal）**：内部高速时钟
   - 频率：8MHz
   - 优点：无需外部元件，成本低
   - 缺点：精度较低（约 1% 误差），温度漂移大
   - 适用场景：对时钟精度要求不高的应用
2. **HSE（High Speed External）**：外部高速时钟
   - 频率：通常为 8MHz（由外部晶振决定）
   - 优点：精度高（通常 <50ppm），稳定性好
   - 缺点：需要外部晶振，增加成本
   - 适用场景：需要精确时钟的应用（如 USB、CAN 通信）
   - 本实验使用 **HSE**
3. **LSI（Low Speed Internal）**：内部低速时钟
   - 频率：40kHz
   - 用途：独立看门狗、RTC（实时时钟）
4. **LSE（Low Speed External）**：外部低速时钟
   - 频率：32.768kHz
   - 用途：RTC，功耗敏感应用

2.3.1.3 STM32F103 时钟树结构

STM32 的时钟系统是一个复杂的树状结构，从时钟源到各个外设经过多级分频和选择：



关键时钟说明：

- **SYSCLK**（系统时钟）：CPU 的工作时钟，本实验配置为 72MHz
- **HCLK**（**AHB** 时钟）：AHB 总线、内核、内存、DMA 的时钟
- **PCLK1**（**APB1** 时钟）：低速外设时钟（最高 36MHz），包括 TIM2-7、USART2-5、I2C、SPI2 等
- **PCLK2**（**APB2** 时钟）：高速外设时钟（最高 72MHz），包括 TIM1/8、USART1、SPI1、ADC、GPIO 等

2.3.1.4 PLL（锁相环）倍频

STM32F103 的最高工作频率是 72MHz，但外部晶振通常只有 8MHz，如何达到 72MHz？答案是使用 PLL 倍频。

PLL 工作原理：

```
输出频率 = 输入频率 × 倍频系数
72MHz = 8MHz × 9
```

在本实验中：- 输入：HSE = 8MHz - 倍频系数：9 - 输出：SYSCLK = 72MHz

为什么要用 **72MHz**？ - 更高的时钟频率意味着更快的运算速度 - 72MHz 是 STM32F103 的最高安全工作频率 - 更高的频率也意味着更高的功耗

2.3.1.5 外设时钟使能

在 STM32 中，为了降低功耗，所有外设的时钟默认是关闭的。使用外设前必须先使能其时钟。

例如，使用 GPIOC 之前必须执行：

```
__HAL_RCC_GPIOC_CLK_ENABLE();
```

这个宏展开后实际上是操作 RCC 寄存器：

```
// 等价于
RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;
```

为什么要这样设计？ - 节省功耗：未使用的外设不消耗时钟功耗 - 灵活性：可以根据需要动态开关外设时钟 - 低功耗模式：可以选择性关闭部分外设时钟

2.3.2 HAL_Delay() 延时函数详解

2.3.2.1 HAL_Delay() 函数原型

```
void HAL_Delay(uint32_t Delay);
```

- 参数：延时时间，单位为毫秒（ms）
- 返回值：无
- 功能：阻塞式延时，延时期间 CPU 一直等待

2.3.2.2 实现原理：基于 SysTick 定时器

HAL_Delay() 的实现依赖于 ARM Cortex-M3 内核的 SysTick 定时器。

SysTick 定时器简介：- SysTick 是 ARM Cortex-M 内核自带的 24 位递减定时器 - 专门用于提供系统时基（System Tick）- 每个 Cortex-M 内核都有一个 SysTick - 不占用 STM32 的通用定时器资源

SysTick 工作原理：

1. 设置重装载值（RELOAD）
2. 启动定时器，开始递减计数
3. 计数到0时产生中断
4. 自动重装载，继续计数

2.3.2.3 HAL_Delay() 的实现流程

第一步：**HAL_Init()** 初始化 **SysTick**

在 HAL_Init() 函数中，会调用 HAL_InitTick() 配置 SysTick：

```
HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
  // 配置 SysTick 为 1ms 中断一次
  // HCLK = 72MHz，需要 72000 个时钟周期产生 1ms
  HAL_SYSTICK_Config(SystemCoreClock / 1000U);

  // 设置 SysTick 中断优先级
  HAL_NVIC_SetPriority(SysTick_IRQn, TickPriority, 0U);

  return HAL_OK;
}
```

关键计算：

```
SysTick重装载值 = SystemCoreClock / 1000
                = 72,000,000 Hz / 1000
                = 72,000
```

即每72,000个时钟周期产生一次中断（1ms）

第二步：**SysTick** 中断服务函数

每 1ms，SysTick 会产生一次中断，执行中断服务函数：

```
void SysTick_Handler(void)
{
  HAL_IncTick();  // 增加系统滴答计数
}

void HAL_IncTick(void)
{
  uwTick += uwTickFreq;  // uwTick 是全局变量，记录系统运行的毫秒数
}
```

第三步：**HAL_Delay()** 延时实现

```
void HAL_Delay(uint32_t Delay)
{
  uint32_t tickstart = HAL_GetTick();  // 获取当前时间戳
  uint32_t wait = Delay;

  // 等待指定的时间
  while((HAL_GetTick() - tickstart) < wait)
  {
    // 空循环等待
  }
}

uint32_t HAL_GetTick(void)
{
  return uwTick;  // 返回当前系统滴答数（毫秒）
}
```

工作流程图：

```
HAL_Delay(500) 调用
    ↓
记录起始时间 tickstart = HAL_GetTick()  // 假设为1000ms
    ↓
进入while循环
    ↓
每次循环检查：HAL_GetTick() - tickstart < 500 ?
    ↓
SysTick每1ms中断一次，uwTick++
    ↓
当 uwTick = 1500 时，(1500 - 1000) = 500，退出循环
    ↓
延时结束
```

2.3.2.4 HAL_Delay() 的特点

优点：1. 使用简单：一行代码实现延时 2. 精度较高：基于硬件定时器，精度为 1ms 3. 不受编译优化影响：不像软件延时会被编译器优化 4. 时间准确：只要系统时钟配置正确，延时就准确

缺点：1. 阻塞式延时：延时期间 CPU 无法执行其他任务 2. 浪费 **CPU** 资源：CPU 在 while 循环中空转 3. 不适合多任务：无法同时处理多个事件 4. 功耗较高：CPU 一直运行，无法进入低功耗模式

2.3.2.5 延时精度分析

理论精度：1ms（SysTick 中断周期）

实际精度：

```
HAL_Delay(500);  // 实际延时：500ms ± 1ms
```

误差来源：1. SysTick 中断响应延迟（几个时钟周期）2. 函数调用开销 3. while 循环判断时间

精度测试：

```
// 使用示波器测量 GPIO 翻转周期
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
HAL_Delay(500);
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
// 测量高电平持续时间，应该非常接近 500ms
```

2.3.2.6 HAL_Delay() 与系统时钟的关系

HAL_Delay() 的准确性完全依赖于系统时钟配置：

情况 **1**：时钟配置正确（**72MHz**）

```
SystemCoreClock = 72,000,000 Hz
SysTick 重装载值 = 72,000
实际中断周期 = 72,000 / 72,000,000 = 1ms [OK]
HAL_Delay(500) = 500ms [OK]
```

情况 **2**：时钟配置错误（**8MHz**，未使能 **PLL**）

```
SystemCoreClock = 8,000,000 Hz
SysTick 重装载值 = 8,000 (按8MHz计算)
但实际 SystemCoreClock = 72,000,000 Hz
实际中断周期 = 8,000 / 72,000,000 = 0.111ms [X]
HAL_Delay(500) = 500 × 0.111ms = 55.5ms [X] (延时变短)
```

情况 **3**：时钟配置错误（使用 **HSI** 而非 **HSE**）

```
如果误用 HSI (8MHz) 但代码按 HSE+PLL (72MHz) 配置
可能导致时钟配置失败，系统运行在默认的 HSI (8MHz)
HAL_Delay(500)实际延时 = 500 × (8/72) = 55.5ms
```

结论：正确配置 RCC 时钟是 HAL_Delay() 准确工作的前提！

2.3.3 本实验中 RCC 和 HAL_Delay 的配合

在本实验中，RCC 和 HAL_Delay() 的关系如下:

```
1. RCC配置HSE（8MHz）和PLL（×9）
   ↓
2. 系统时钟SYSCLK = 72MHz
   ↓
3. HAL_Init()根据SystemCoreClock配置SysTick
   ↓
4. SysTick每1ms产生中断，uwTick++
   ↓
5. HAL_Delay(500)通过检查uwTick实现500ms延时
   ↓
6. LED每500ms翻转一次，实现1Hz闪烁
```

关键点:- RCC 提供稳定的 72MHz 系统时钟 - SysTick 基于 72MHz 时钟产生精确的 1ms 时基 - HAL_Delay() 基于 1ms 时基实现毫秒级延时 - LED 闪烁频率的准确性最终取决于 RCC 时钟配置

2.4 实验要求

2.4.1 硬件要求

- STM32F103C8T6 开发板
- LED 连接到 PC13 引脚
- USB 转串口下载器（ST-Link 或其他）

2.4.2 软件要求

- STM32CubeMX（用于配置工程）
- CMake 构建工具
- ARM GCC 工具链
- 支持 CMake 的 IDE（如 CLion、VSCode 等）

2.4.3 功能要求

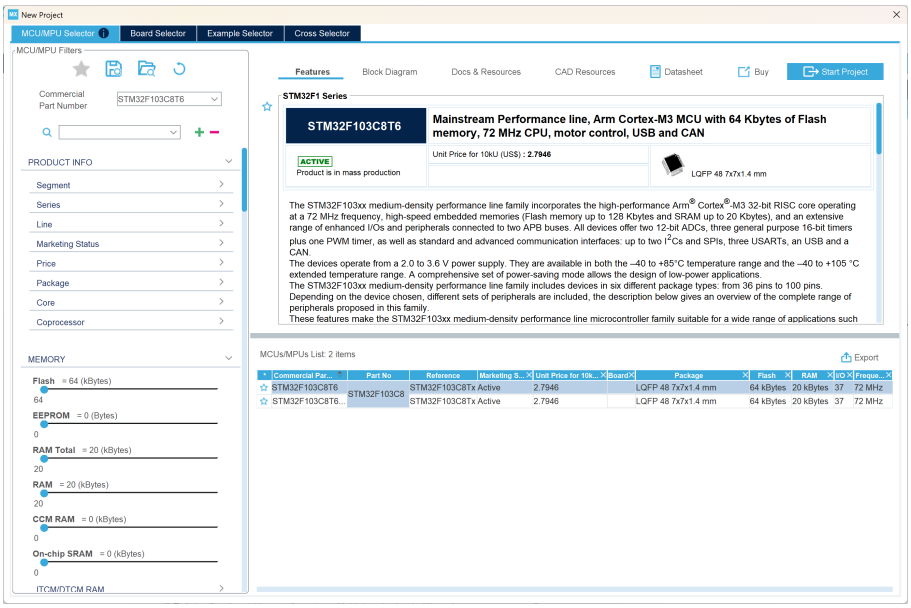
1. LED 灯以 1Hz 频率闪烁（每秒闪烁 1 次）
2. 使用 HAL 库的延时函数实现
3. 使用 GPIO 翻转函数控制 LED

2.5 实验步骤

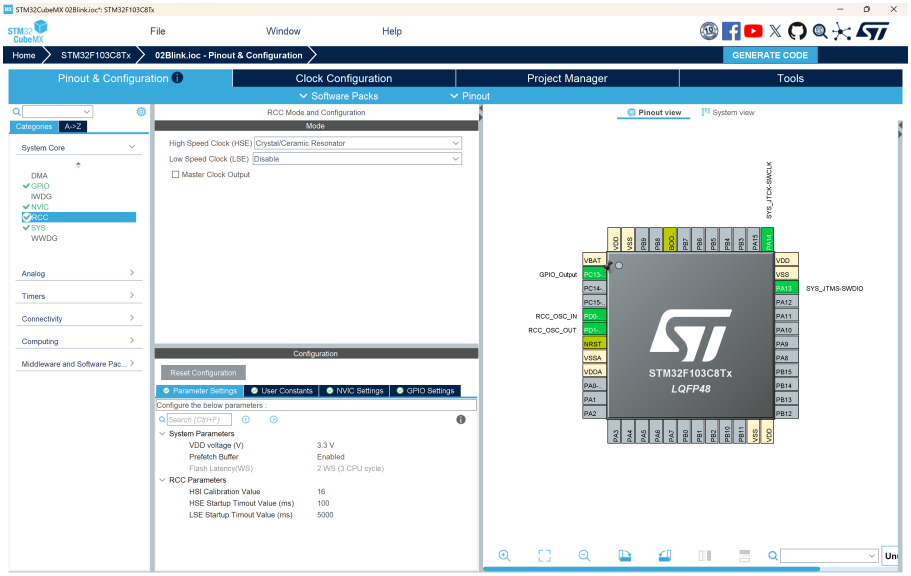
2.5.1 使用 STM32CubeMX 配置工程

1. 创建新工程

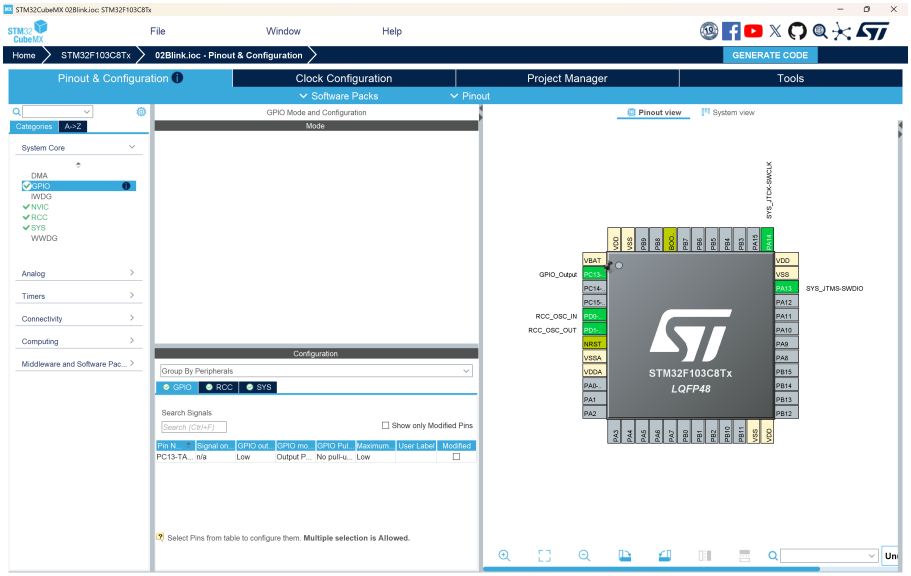
   - 打开 STM32CubeMX
   - 选择芯片型号：STM32F103C8T6
   - 点击”Start Project”



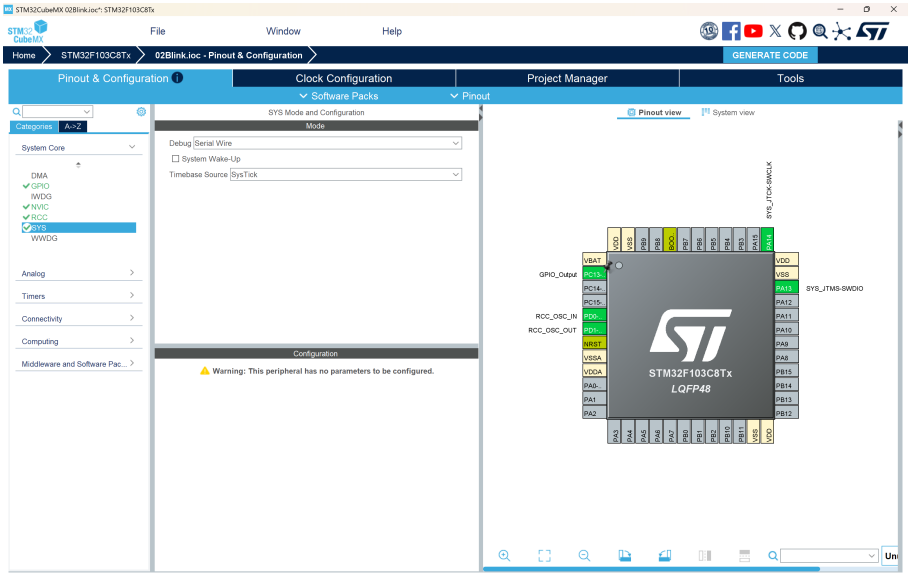
2. 配置 **RCC**（时钟源） - 在左侧 Pinout & Configuration 中找到 System Core → RCC - 将 HSE（High Speed External）设置为” Crystal/Ceramic Resonator” - 这样使用外部 8MHz 晶振作为时钟源



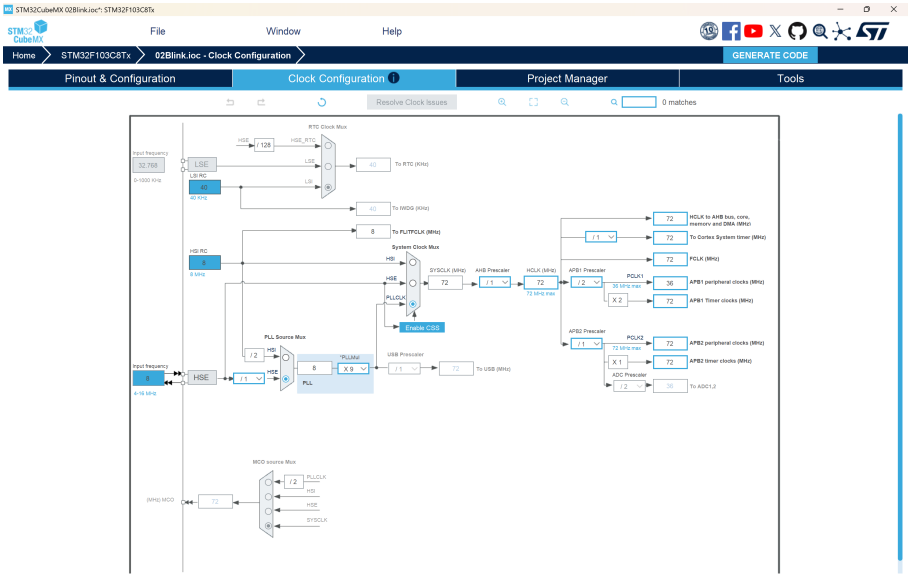
3. 配置 **GPIO** - 在芯片引脚图中找到 PC13 引脚 - 点击 PC13，选择”GPIO_Output” - 在左侧 System Core → GPIO 中可以看到 PC13 的配置 - 默认配置即可： - GPIO output level: Low - GPIO mode: Output Push Pull - GPIO Pull-up/Pull-down: No pull-up and no pull-down - Maximum output speed: Low



4. 配置 **Debug** 模式 - 在 System Core → SYS 中配置 Debug 为 Serial Wire - 这样可以使用 ST-Link 进行调试和下载

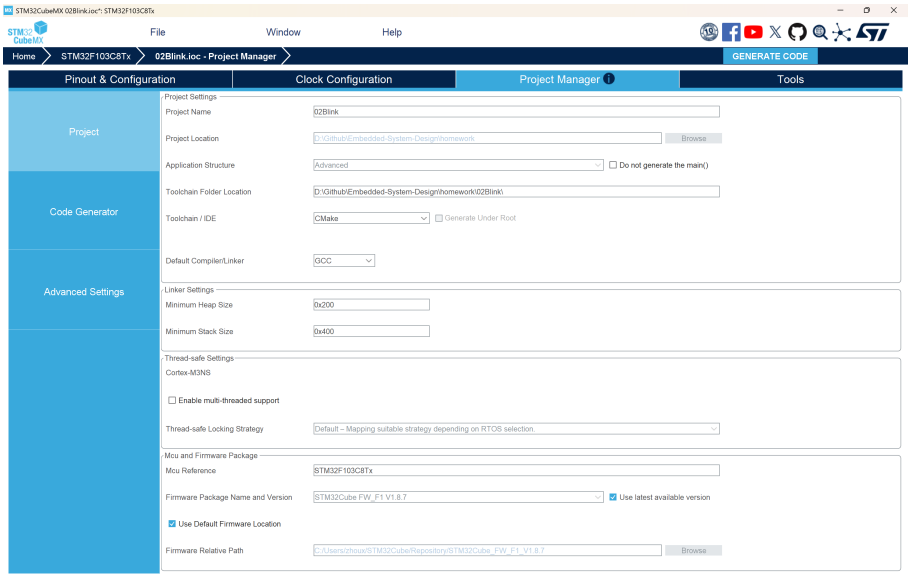


5. 配置时钟树 - 切换到 Clock Configuration 标签页 - 设置 HCLK 为 72MHz（STM32F103 的最高频率）- 系统会自动计算 PLL 倍频系数



6. 生成代码 - 切换到 Project

Manager 标签页 - 设置 Project Name: 02Blink - 设置 Toolchain/IDE: CMake - 点击”GENERATE CODE”



2.5.2 编写 LED 闪烁代码

打开生成的 `Core/Src/main.c` 文件，在主循环中添加 LED 控制代码：

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
  HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);  // 翻转 PC13 引脚电平
  HAL_Delay(500);                          // 延时 500ms
}
/* USER CODE END 3 */
```

注意：代码必须写在 `USER CODE BEGIN` 和 `USER CODE END` 之间，这样重新生成代码时不会被覆盖。

2.5.3 编译工程

使用 CMake 编译工程：

```
# 配置 CMake
cmake -B build -G "Ninja"

# 编译
cmake --build build
```

编译成功后会在 `build` 目录下生成 `02Blink.elf`、`02Blink.hex` 和 `02Blink.bin` 文件。

2.5.4 下载程序

使用 ST-Link 或其他下载工具将程序下载到开发板：

```
# 使用 OpenOCD 下载（示例）
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg -c "program build/02Blink.elf verify reset exit"
```

或使用 STM32CubeProgrammer 图形界面下载。

2.5.5 观察现象

程序运行后，PC13 引脚连接的 LED 应该以 1 秒闪烁 1 次的频率闪烁（亮 500ms，灭 500ms）。

2.6 代码详解

2.6.1 主要函数说明

2.6.1.1 HAL_Init()

```
HAL_Init();
```

- 功能：初始化 HAL 库
- 作用：配置 SysTick 定时器、初始化 Flash 接口等
- 必须在使用 HAL 库函数前调用

2.6.1.2 SystemClock_Config()

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  // 配置 HSE 和 PLL
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;  // 8MHz * 9 = 72MHz
  HAL_RCC_OscConfig(&RCC_OscInitStruct);

  // 配置系统时钟
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;     // HCLK = 72MHz
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;      // PCLK1 = 36MHz
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;      // PCLK2 = 72MHz
  HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2);
}
```

- 功能：配置系统时钟
- 时钟源：外部 8MHz 晶振
- 系统时钟：72MHz（通过 PLL 倍频）

2.6.1.3 MX_GPIO_Init()

```
void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  // 使能 GPIOC 时钟
  __HAL_RCC_GPIOC_CLK_ENABLE();

  // 设置 PC13 初始电平为低
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);

  // 配置 PC13 为推挽输出
  GPIO_InitStruct.Pin = GPIO_PIN_13;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;    // 推挽输出
  GPIO_InitStruct.Pull = GPIO_NOPULL;            // 无上下拉
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;   // 低速
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
}
```

- 功能：初始化 GPIO
- 配置 PC13 为推挽输出模式
- 初始电平为低

2.6.1.4 HAL_GPIO_TogglePin()

```
HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
```

- 功能：翻转 GPIO 引脚电平
- 参数 1：GPIO 端口（GPIOC）
- 参数 2：GPIO 引脚（GPIO_PIN_13）
- 作用：如果当前是高电平则变为低电平，反之亦然

2.6.1.5 HAL_Delay()

```
HAL_Delay(500);
```

- 功能：毫秒级延时
- 参数：延时时间（单位：毫秒）
- 实现原理：基于 SysTick 定时器
- 注意：这是阻塞式延时，延时期间 CPU 无法执行其他任务

2.6.2 LED 闪烁原理

```
while (1)
{
  HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);  // 翻转电平
  HAL_Delay(500);                          // 延时 500ms
}
```

工作流程：1. 第 1 次循环：PC13 从低电平变为高电平 → LED 点亮 → 延时 500ms 2. 第 2 次循环：PC13 从高电平变为低电平 → LED 熄灭 → 延时 500ms 3. 第 3 次循环：重复步骤 1 4. 以此类推...

闪烁频率计算：- 一个完整周期 = 亮 500ms + 灭 500ms = 1000ms = 1 秒 - 频率 = 1/周期 = 1/1s = 1Hz - 即每秒闪烁 1 次

2.6.3 GPIO 工作模式说明

STM32 的 GPIO 有多种工作模式：

1. 输入模式
   - 浮空输入（GPIO_MODE_INPUT）
   - 上拉输入（GPIO_MODE_INPUT + GPIO_PULLUP）
   - 下拉输入（GPIO_MODE_INPUT + GPIO_PULLDOWN）
   - 模拟输入（GPIO_MODE_ANALOG）
2. 输出模式
   - 推挽输出（GPIO_MODE_OUTPUT_PP）：本实验使用
   - 开漏输出（GPIO_MODE_OUTPUT_OD）
3. 复用功能模式
   - 复用推挽（GPIO_MODE_AF_PP）
   - 复用开漏（GPIO_MODE_AF_OD）

推挽输出特点：- 可以输出高电平和低电平 - 驱动能力强 - 适合驱动 LED、蜂鸣器等负载

2.7 思考题

1. 如果要让 **LED** 闪烁频率变为 **2Hz**（每秒闪烁 **2** 次），应该如何修改代码？
2. **HAL_Delay()** 函数的延时精度如何？它适合用在什么场合？不适合用在什么场合？
3. 除了使用 **HAL_GPIO_TogglePin()** 函数，还可以用哪些 **HAL** 库函数控制 **LED**？请写出至少两种方法。
4. 如果将 **GPIO** 配置为开漏输出模式（**GPIO_MODE_OUTPUT_OD**），**LED** 还能正常工作吗？为什么？
5. 在主循环中使用 **HAL_Delay()** 会导致什么问题？如果需要同时控制多个 **LED** 以不同频率闪烁，应该如何实现？
6. **STM32F103C8T6** 的 **PC13** 引脚有什么特殊性？它的驱动能力如何？
7. 如果 **LED** 不亮或不闪烁，应该从哪些方面排查问题？请列出至少 **5** 个排查步骤。
8. 计算一下：如果系统时钟配置错误，**HCLK** 只有 **8MHz** 而不是 **72MHz**，**HAL_Delay(500)** 实际延时多长时间？

2.8 讨论与扩展

2.8.1 讨论 1：延时方式的优缺点

请讨论以下几种延时方式的优缺点，并说明各自的适用场景：

1. 软件延时（空循环）

```
for(int i = 0; i < 1000000; i++);
```

2. **HAL_Delay()** 延时（本实验使用）

```
HAL_Delay(500);
```

3. 定时器中断延时

```
// 使用 TIM2 定时器产生 1ms 中断
```

4. **SysTick** 中断延时

```
// 在 SysTick 中断中计数
```

2.8.2 讨论 2：LED 驱动电路

请分析以下两种 LED 连接方式的区别：

1. 共阳极连接：LED 正极接 VCC，负极接 GPIO
   - GPIO 输出低电平时 LED 亮
   - GPIO 输出高电平时 LED 灭
2. 共阴极连接：LED 正极接 GPIO，负极接 GND
   - GPIO 输出高电平时 LED 亮
   - GPIO 输出低电平时 LED 灭

思考：- 你的开发板使用的是哪种连接方式？- 如何通过实验判断 LED 的连接方式？- 两种方式各有什么优缺点？

2.8.3 讨论 3：功耗优化

如果这是一个电池供电的设备，需要尽可能降低功耗，请讨论：

1. 在延时期间，CPU 一直在执行 HAL_Delay() 函数，这样功耗高吗？
2. 如何让 CPU 在延时期间进入低功耗模式？
3. 使用定时器中断方式是否能降低功耗？为什么？
4. STM32 有哪些低功耗模式？各有什么特点？

2.8.4 扩展任务 1：呼吸灯效果

修改程序，实现 LED 呼吸灯效果（亮度逐渐增加再逐渐减小）。

提示：- 需要使用 PWM（脉宽调制）- 配置定时器输出 PWM 信号 - 通过改变占空比控制 LED 亮度

2.8.5 扩展任务 2：多 LED 流水灯

如果有 8 个 LED 连接到 PA0-PA7，编写程序实现流水灯效果。

要求：- LED 依次点亮，每个 LED 亮 200ms - 实现循环流动效果 - 使用数组和循环简化代码

2.8.6 扩展任务 3：按键控制 LED

添加一个按键（连接到 PB0），实现以下功能：- 按键按下时 LED 闪烁 - 按键松开时 LED 常亮或常灭 - 需要考虑按键消抖

2.9 实验报告要求

2.9.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述 LED 闪烁实验要完成的功能和验收指标。
3. 实验原理：说明 GPIO 输出、系统时钟、`HAL_Delay()` 和闪烁频率之间的关系。
4. CubeMX 配置：至少展示 RCC、SYS、GPIO、Clock Configuration 的截图与解释。
5. 程序设计：展示主循环关键代码，并说明为什么要放在 `USER CODE` 区域。
6. 程序流程图：描述“翻转 LED -> 延时 500 ms -> 重复”的控制逻辑。
7. 实验结果：展示 LED 闪烁照片、录像截图或逻辑分析结果，并说明如何判断频率正确。
8. 思考题解答：必须完整回答本作业 8 个思考题。
9. 讨论与扩展：至少完成 3 个讨论题；若完成扩展任务，应单独给出结果与分析。
10. 问题分析与总结：记录调试过程、问题原因、解决方法和个人收获。

2.9.2 图片与流程图要求

1. 报告中至少包含 3 类图片：配置截图、代码或构建结果截图、实验现象图片。
2. 每张图片都应有图号、图题和正文引用说明。
3. 程序流程图必须单独成图，不能只用文字代替。
4. 若使用视频验证，可在报告中给出关键截图并补充说明。

2.9.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 2-LED 闪烁实验.pdf`。
2. 若扩展任务未完成，应在报告中明确说明，不影响基础任务评分。
3. 若实验失败，也必须提交完整分析，不能空交。

2.9.4 最低验收标准

1. LED 能稳定闪烁，周期与设计一致。
2. 能解释 1 Hz 闪烁与 500 ms 延时的关系。
3. 能说明 `HAL_Delay()` 的使用前提和局限。
4. 报告包含真实实验结果，不得只有理论文字。

2.10 评分标准

项目	分值	说明
程序功能	40 分	LED 能否正常闪烁，频率是否正确
代码规范	15 分	代码注释、命名规范、结构清晰
思考题	20 分	回答的完整性和正确性
讨论题	15 分	分析的深度和广度
实验报告	10 分	报告格式、内容完整性
扩展任务	加分项	每完成一个扩展任务加 5-10 分

2.11 参考资料

1. STM32F103C8T6 数据手册
2. STM32F1 系列参考手册
3. STM32 HAL 库用户手册
4. STM32CubeMX 用户指南

2.12 常见问题 FAQ

Q1：编译时提示找不到 **arm-none-eabi-gcc**?

A：需要安装 ARM GCC 工具链，并将其添加到系统 PATH 环境变量中。

Q2：下载程序后 **LED** 不闪烁?

A：检查以下几点：- 硬件连接是否正确 - 程序是否成功下载 - 时钟配置是否正确 - GPIO 配置是否正确 - LED 极性是否接反

Q3：**LED** 闪烁频率不对?

A：检查 HAL_Delay() 的参数和系统时钟配置。

Q4：重新用 **CubeMX** 生成代码后，我的代码丢失了?

A：代码必须写在 `USER CODE BEGIN` 和 `USER CODE END` 之间，否则会被覆盖。

Q5：如何调试程序?

A：可以使用以下方法：- 使用 ST-Link 的 SWD 调试功能 - 在代码中添加断点 - 使用串口打印调试信息 - 使用逻辑分析仪观察 GPIO 波形

作业提交截止时间：请在课程安排的时间内完成并提交实验报告。

联系方式：如有问题请在课程讨论区提问或发送邮件至教师邮箱。

3 STM32 中断实验

3.1 统一作业说明

3.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成 GPIO 外部中断、NVIC、时钟等配置，并保留可重新生成代码的 `.ioc` 文件。
2. 基于 HAL 库完成 PB12 外部中断控制 PC13 LED 的程序编写。
3. 实现基础的软件去抖动逻辑，并能够解释其工作原理。
4. 成功编译、下载并验证中断触发、LED 翻转和去抖动效果。
5. 在实验报告中完整说明配置步骤、关键代码、实验现象、调试过程和课后作业答案。
6. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

3.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6
输入对象	PB12 外部中断
输出对象	PC13 LED
必做功能	PB12 下降沿触发中断，LED 翻转，200 ms 内重复触发不重复响应
理论要求	能解释 EXTI、NVIC、中断回调和软件去抖动
验收方式	现场演示或结果图片/截图，能够证明中断响应和去抖动有效

3.1.3 本次必须提交的内容

1. 可编译的完整工程目录。
2. 一份 PDF 格式实验报告。
3. CubeMX 配置截图、关键中断代码截图、实验现象图片各至少 1 张。
4. 课后作业题的完整书面回答。

3.1.4 报告必须回答的问题

1. 为什么 PB12 要配置为下降沿触发和上拉输入。
2. EXTI12、EXTI15_10_IRQn 与 `HAL_GPIO_EXTI_Callback()` 之间是什么关系。
3. 为什么中断服务程序中不宜执行复杂业务逻辑。
4. 去抖动阈值设置过大或过小分别会带来什么问题。

3.2 实验目的

1. 掌握 STM32 外部中断（EXTI）的配置和使用方法
2. 理解中断的工作原理和中断优先级机制
3. 学习使用 HAL 库进行中断编程
4. 掌握按键去抖动的软件实现方法
5. 理解中断服务函数和回调函数的关系

3.3 实验原理

3.3.1 什么是中断?

中断是一种异步事件处理机制。当某个事件发生时（如按键按下、定时器溢出等），CPU 会暂停当前正在执行的程序，转而去执行与该事件相关的中断服务程序（ISR），处理完成后再返回原来的程序继续执行。

中断的优势： - 实时响应：能够快速响应外部事件 - 提高效率：CPU 不需要一直轮询检查事件是否发生 - 降低功耗：CPU 可以进入低功耗模式，等待中断唤醒

3.3.2 STM32 的 EXTI（外部中断/事件控制器）

STM32F103C8T6 的 EXTI 模块具有以下特点： - 支持 19 个外部中断/事件线（EXTI0-EXTI18） - 每个 GPIO 引脚都可以配置为外部中断源 - 支持上升沿、下降沿或双边沿触发 - 可配置中断优先级

EXTI 线与 **GPIO** 的对应关系： - EXTI0 对应所有 GPIO 的 Pin0（PA0, PB0, PC0…） - EXTI1 对应所有 GPIO 的 Pin1（PA1, PB1, PC1…） - …以此类推 - EXTI12 对应所有 GPIO 的 Pin12（PA12, PB12, PC12…）

中断向量分组： - EXTI0-EXTI4：每个线有独立的中断向量 - EXTI5-EXTI9：共享 EXTI9_5_IRQn 中断向量 - EXTI10-EXTI15：共享 EXTI15_10_IRQn 中断向量

本实验使用 PB12，对应 EXTI12，属于 EXTI15_10 中断线。

3.3.3 中断处理流程

外部事件发生（PB12下降沿）
↓
硬件检测到触发条件
↓
EXTI12中断标志位置位
↓
NVIC判断中断优先级
↓
CPU保存当前状态，跳转到中断向量表
↓
执行EXTI15_10_IRQHandler()
↓
调用HAL_GPIO_EXTI_IRQHandler()

↓
清除中断标志位
↓
调用HAL_GPIO_EXTI_Callback()（用户回调函数）
↓
执行用户代码（LED翻转）
↓
返回主程序

3.3.4 按键去抖动原理

机械按键在按下和释放时，由于机械弹性作用，会产生抖动信号，导致多次触发中断。

抖动特点： - 持续时间：通常 5-10ms - 表现形式：在按下瞬间产生多个脉冲

软件去抖动方法：使用时间戳记录上次中断时间，只有当两次中断间隔超过设定阈值（如 200ms）时，才认为是有效按键。

3.4 硬件连接

3.4.1 使用的 IO 口

引脚	功能	配置	说明
PC13	LED 输出	推挽输出	低电平点亮 LED
PB12	按键输入	上拉输入 + 外部中断	下降沿触发

3.4.2 连接方式

方式一：使用按键 - 按键一端连接 PB12 - 按键另一端连接 GND - 按下按键时，PB12 被拉低，产生下降沿

方式二：使用杜邦线 - 用杜邦线短接 PB12 和 GND - 每次接触产生下降沿触发中断

LED 连接： - PC13 通常板载连接 LED - 低电平点亮，高电平熄灭

3.5 STM32CubeMX 详细配置步骤

3.5.1 创建新工程

1. 启动 **STM32CubeMX**
   - 打开 STM32CubeMX 软件
   - 选择”File” → “New Project”
2. 选择芯片型号
   - 在”Part Number” 搜索框中输入：STM32F103C8

- 在列表中选择”STM32F103C8Tx”
- 点击”Start Project”

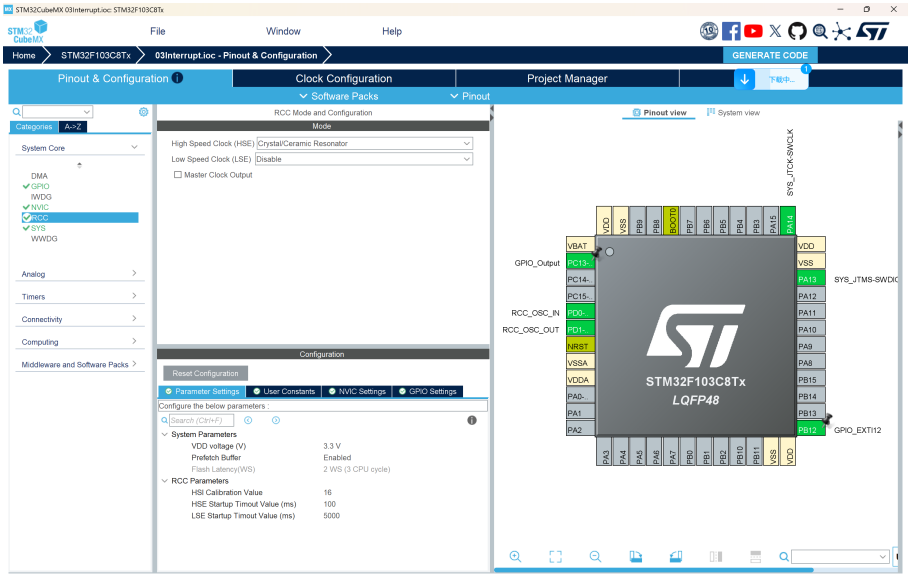
3.5.2 时钟配置（RCC）

1. 配置外部时钟源

   - 在左侧 Pinout 视图中，找到”System Core” → “RCC”
   - 或在芯片引脚图中找到 OSC_IN 和 OSC_OUT 引脚

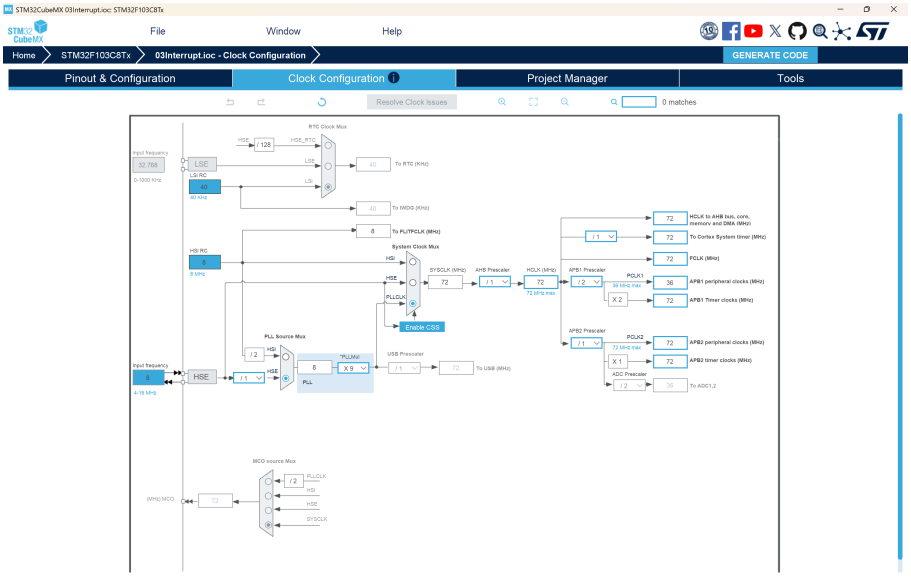
2. 设置 **HSE**（高速外部时钟）

   - HSE (High Speed Clock)：选择”Crystal/Ceramic Resonator”（晶振/陶瓷谐振器）
   - 这会自动配置 PD0-OSC_IN 和 PD1-OSC_OUT 引脚



3. 配置系统时钟树

   - 点击顶部的”Clock Configuration” 标签页
   - 在”HSE” 输入框中输入：8（MHz）
   - 勾选”PLL Source Mux” 选择”HSE”
   - 在”PLLMUL”（PLL 倍频系数）选择”x9”
   - 在”System Clock Mux” 选择”PLLCLK”
   - 最终”HCLK” 应显示为 72MHz
   - 如果有警告，点击”Resolve Clock Issues” 自动解决



3.5.3 GPIO 配置

3.5.3.1 配置 PC13（LED 输出）

1. 在引脚图中找到 **PC13**
   - 在芯片引脚图中找到 PC13 引脚
   - 点击 PC13 引脚
2. 设置为 **GPIO** 输出
   - 在弹出菜单中选择”GPIO_Output”
   - 引脚会变成绿色，表示已配置为输出
3. 配置 **GPIO** 参数
   - 在左侧”System Core” → “GPIO” 中
   - 找到 PC13 配置项
   - 设置如下:
     - GPIO output level: Low（初始输出低电平）
     - GPIO mode: Output Push Pull（推挽输出）
     - GPIO Pull-up/Pull-down: No pull-up and no pull-down（无上下拉）
     - Maximum output speed: Low（低速）
     - User Label: 可选，输入”LED” 作为标签

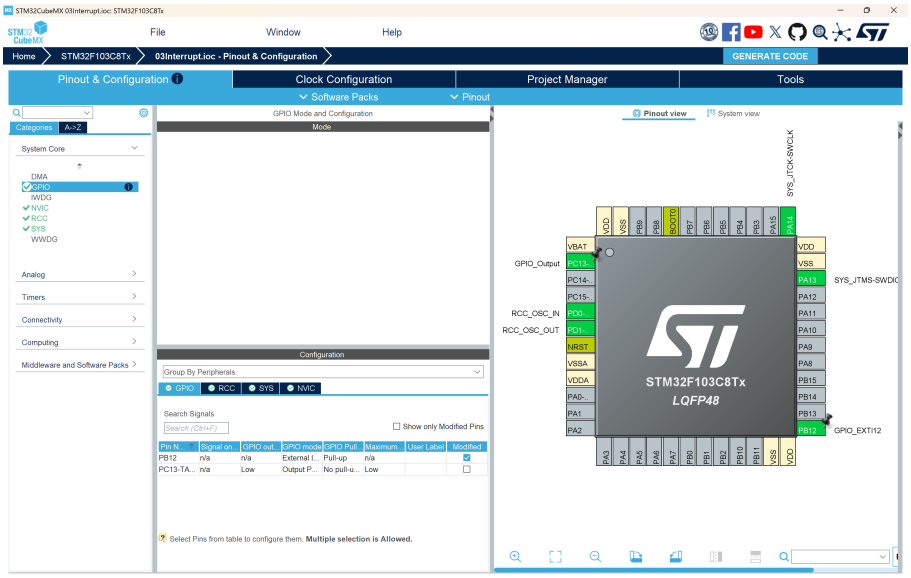
3.5.3.2 配置 PB12（外部中断输入）

1. 在引脚图中找到 **PB12**
   - 在芯片引脚图中找到 PB12 引脚
   - 点击 PB12 引脚
2. 设置为外部中断模式
   - 在弹出菜单中选择”GPIO_EXTI12”

- 引脚会变成黄色，表示已配置为外部中断

3. 配置 **GPIO** 参数

- 在左侧”System Core” → “GPIO” 中
- 找到 PB12 配置项
- 设置如下：
  - GPIO mode：External Interrupt Mode with Falling edge trigger detection（下降沿触发外部中断模式）
  - GPIO Pull-up/Pull-down：Pull-up（上拉）
  - User Label：可选，输入”BUTTON” 作为标签



重要说明： - 必须选择”External Interrupt Mode” 而不是”External Event Mode” - 触发方式选择”Falling edge”（下降沿），因为按键按下时 PB12 从高电平变为低电平 - 必须配置上拉电阻，保证默认状态为高电平

3.5.4 NVIC 配置（关键步骤）

1. 进入 **NVIC** 配置页面

- 在左侧”System Core” → “NVIC”
- 或点击顶部”Configuration” 按钮，然后选择”NVIC”

2. 使能 **EXTI** 中断

- 在”NVIC” 标签页中，找到”EXTI line[15:10] interrupts”
- 勾选**”Enabled”** 复选框（这一步非常重要!）
- 设置优先级：
  - Preemption Priority：0（抢占优先级，0 为最高）
  - Sub Priority：0（子优先级）

3. **Code generation** 选项

- 切换到”Code generation” 标签页
- 确认”EXTI line[15:10] interrupts” 已勾选
- 勾选”Generate IRQ handler”（生成中断处理函数）

注意事项： - 如果不在 NVIC 中使能中断，生成的代码将不包含 `HAL_NVIC_SetPriority()` 和 `HAL_NVIC_EnableIRQ()` - 本工程已在 gpio.c 的 USER CODE 区域手动添加了 NVIC 配置，即使 CubeMX 未生成也能正常工作 - 但建议在 CubeMX 中正确配置，保持配置文件与代码一致

3.5.5 项目设置

1. 进入 **Project Manager**
   - 点击顶部"Project Manager" 标签页
2. **Project** 设置
   - Project Name: 输入"03Interrupt"
   - Project Location: 选择保存路径
   - Toolchain/IDE: 推荐选择适合 VS Code 工作流的工程类型（如 Makefile）
     - 推荐: VS Code + STM32CubeIDE 插件（用于配置/生成/调试辅助）
     - 不推荐: STM32CubeIDE 独立软件作为主开发环境
     - Makefile
     - Keil MDK-ARM
     - IAR EWARM
3. **Code Generator** 设置
   - 点击左侧"Code Generator"
   - 勾选"Copy only the necessary library files"（只复制必要的库文件）
   - 勾选"Generate peripheral initialization as a pair of '.c/.h' files per peripheral"（每个外设生成独立的.c/.h 文件）

3.5.6 生成代码

1. 检查配置
   - 返回"Pinout & Configuration" 检查所有配置
   - 确认 PC13 为 GPIO_Output
   - 确认 PB12 为 GPIO_EXTI12
   - 确认 NVIC 中 EXTI15_10 已使能
2. 生成代码
   - 点击顶部工具栏的"GENERATE CODE" 按钮（齿轮图标）
   - 或选择"Project" → "Generate Code"
   - 等待代码生成完成
3. 打开工程
   - 生成完成后，点击"Open Project"
   - 或手动打开生成的工程文件

3.5.7 验证生成的代码

生成代码后，检查以下文件：

1. **gpio.c**
   - 确认 PC13 配置为 GPIO_MODE_OUTPUT_PP
   - 确认 PB12 配置为 GPIO_MODE_IT_FALLING
   - 如果 NVIC 配置正确，应该包含 HAL_NVIC_SetPriority() 和 HAL_NVIC_EnableIRQ()
2. **stm32f1xx_it.c**
   - 如果 NVIC 配置正确，应该包含 EXTI15_10_IRQHandler() 函数框架
3. **main.c**
   - 确认调用了 MX_GPIO_Init()

3.5.8 常见配置问题

问题 **1**：生成代码后没有 **NVIC** 配置 - 原因：忘记在 NVIC 页面勾选”Enabled” - 解决：重新打开.ioc 文件，在 NVIC 中使能 EXTI15_10，重新生成代码 - 或者：手动在 gpio.c 的 USER CODE 区域添加 NVIC 配置（本工程已添加）

问题 **2**：中断不触发 - 检查 PB12 是否配置为 GPIO_EXTI12 而不是普通 GPIO_Input - 检查触发方式是否为 Falling edge - 检查是否配置了上拉电阻

问题 **3**：重新生成代码后用户代码丢失 - 确保用户代码写在`/* USER CODE BEGIN */`和`/* USER CODE END */`之间 - CubeMX 只会保留这些区域的代码

3.5.9 使用现有.ioc 文件

如果直接使用本仓库的 03Interrupt.ioc 文件：

1. 用 STM32CubeMX 打开 03Interrupt.ioc
2. 查看所有配置是否正确
3. 如需修改，直接修改后重新生成代码
4. 点击”GENERATE CODE” 生成代码

3.6 代码详细解析

3.6.1 GPIO 初始化代码（gpio.c）

```
void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO 时钟使能 */
  __HAL_RCC_GPIOC_CLK_ENABLE();  // 使能 GPIOC 时钟（PC13）
  __HAL_RCC_GPIOD_CLK_ENABLE();  // 使能 GPIOD 时钟（外部晶振）
  __HAL_RCC_GPIOB_CLK_ENABLE();  // 使能 GPIOB 时钟（PB12）
  __HAL_RCC_GPIOA_CLK_ENABLE();  // 使能 GPIOA 时钟
```

```
  /* 配置 PC13 初始输出电平为低电平（LED 熄灭） */
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);

  /* 配置 PC13 为推挽输出 */
  GPIO_InitStruct.Pin = GPIO_PIN_13;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;  // 推挽输出模式
  GPIO_InitStruct.Pull = GPIO_NOPULL;          // 无上下拉
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW; // 低速
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /* 配置 PB12 为外部中断模式 */
  GPIO_InitStruct.Pin = GPIO_PIN_12;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;  // 下降沿触发中断
  GPIO_InitStruct.Pull = GPIO_PULLUP;           // 上拉电阻
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  /* USER CODE BEGIN 2 */
  /* EXTI 中断初始化 - 如果 STM32CubeMX 的 NVIC 中没有使能，需要手动添加 */
  HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);  // 设置中断优先级
  HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);          // 使能 EXTI15_10 中断
  /* USER CODE END 2 */
}
```

代码解析：

1. **GPIO_MODE_IT_FALLING**：配置为下降沿触发的外部中断模式
   - IT = Interrupt（中断）
   - FALLING = 下降沿触发
   - 当 PB12 从高电平变为低电平时触发中断
2. **GPIO_PULLUP**：使能内部上拉电阻
   - 默认状态下 PB12 为高电平
   - 按键按下时被拉低到 GND（低电平）
   - 产生下降沿触发中断
3. **USER CODE 区域的 NVIC 配置**：
   - 这部分代码放在`/* USER CODE BEGIN 2 */`和`/* USER CODE END 2 */`之间
   - 即使 STM32CubeMX 重新生成代码，这部分也不会被覆盖
   - 如果在 CubeMX 的 NVIC 页面正确配置并使能了 EXTI15_10，CubeMX 会自动生成这两行代码
   - 如果忘记在 CubeMX 中使能，手动添加在 USER CODE 区域也可以正常工作
4. **HAL_NVIC_SetPriority()**：设置中断优先级
   - 参数 1：中断线编号（EXTI15_10_IRQn）
   - 参数 2：抢占优先级（0，最高）
   - 参数 3：子优先级（0）
5. **HAL_NVIC_EnableIRQ()**：使能 NVIC 中断
   - 只有使能后，中断才能被响应
   - 这是中断能够工作的关键步骤

重要提示： - 如果你在 STM32CubeMX 的 NVIC 配置页面正确勾选了”EXTI line[15:10] interrupts” 的”Enabled”，CubeMX 会自动生成 NVIC 配置代码（但不在 USER CODE 区域）- 本工程为了防止重新生成代码时丢失 NVIC 配置，将其放在了 USER CODE 区域 - 两种方式都可以，但建议在 CubeMX 中正确配置，保持配置文件与代码的一致性

3.6.2 主程序代码（main.c）

3.6.2.1 全局变量定义

```
/* USER CODE BEGIN PV */
// 用于记录上一次中断触发的时间，用于软件去抖动
uint32_t last_interrupt_time = 0;
// 去抖动时间阈值，单位：毫秒
#define DEBOUNCE_TIME_MS 200
/* USER CODE END PV */
```

代码解析：- last_interrupt_time：记录上次中断的时间戳（毫秒）- DEBOUNCE_TIME_MS：去抖动时间阈值，200ms 内的重复触发会被忽略

3.6.2.2 中断回调函数

```
/* USER CODE BEGIN 4 */

/**
  * @brief  GPIO 外部中断回调函数
  * @param  GPIO_Pin: 触发中断的 GPIO 引脚编号
  * @retval None
  * @note   当外部中断触发时，HAL 库会自动调用此函数
  *         这里实现了软件去抖动和 LED 状态翻转功能
  */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  // 检查是否是 PB12 引脚触发的中断
  if(GPIO_Pin == GPIO_PIN_12)
  {
    // 获取当前系统时间（毫秒）
    uint32_t current_time = HAL_GetTick();

    // 软件去抖动：检查距离上次中断是否超过阈值时间
    // 如果时间间隔小于 DEBOUNCE_TIME_MS，则忽略本次中断（认为是抖动）
    if((current_time - last_interrupt_time) > DEBOUNCE_TIME_MS)
    {
      // 更新上次中断时间
      last_interrupt_time = current_time;

      // 翻转 LED 状态（PC13）
      HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
    }
  }
}

/* USER CODE END 4 */
```

代码解析：

1. **HAL_GPIO_EXTI_Callback()**：HAL 库定义的弱函数
   - 用户可以重写此函数实现自己的中断处理逻辑
   - 参数 GPIO_Pin 指示哪个引脚触发了中断
   - 由 HAL_GPIO_EXTI_IRQHandler() 自动调用
2. **if(GPIO_Pin == GPIO_PIN_12)**：判断中断源

- 因为 EXTI15_10 共享一个中断向量
- 需要判断具体是哪个引脚触发的中断

3. **HAL_GetTick()**：获取系统时间

- 返回自系统启动以来的毫秒数
- 基于 SysTick 定时器，每 1ms 递增
- 用于实现软件去抖动的时间判断

4. 去抖动逻辑：

```
if((current_time - last_interrupt_time) > DEBOUNCE_TIME_MS)
```

- 计算两次中断的时间间隔
- 只有间隔大于 200ms 才认为是有效按键
- 小于 200ms 的触发被认为是抖动，直接忽略

5. **HAL_GPIO_TogglePin()**：翻转 GPIO 状态

- 如果当前是高电平，则变为低电平
- 如果当前是低电平，则变为高电平
- 实现 LED 的开关切换

3.6.3 中断服务函数（stm32f1xx_it.c）

```
/**
  * @brief  EXTI 线 [15:10] 中断处理函数
  * @param  None
  * @retval None
  * @note   PB12 对应 EXTI12，属于 EXTI15_10 中断线
  *         当 PB12 检测到下降沿时，会触发此中断
  */
void EXTI15_10_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI15_10_IRQn 0 */

  /* USER CODE END EXTI15_10_IRQn 0 */
  // HAL 库中断处理函数，会自动清除中断标志位并调用回调函数
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_12);
  /* USER CODE BEGIN EXTI15_10_IRQn 1 */

  /* USER CODE END EXTI15_10_IRQn 1 */
}
```

代码解析：

1. **EXTI15_10_IRQHandler()**：中断服务函数（ISR）

- 函数名必须与启动文件中的中断向量表名称一致
- 当 EXTI10-EXTI15 任一线触发中断时，都会执行此函数
- 这是真正的硬件中断入口

2. **HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_12)**：HAL 库中断处理

- 检查 GPIO_PIN_12 的中断标志位
- 如果标志位置位，则清除标志位

- 调用用户定义的 HAL_GPIO_EXTI_Callback() 回调函数

3. 为什么需要两层函数?

- ISR（EXTI15_10_IRQHandler）：硬件层，由中断向量表调用
- Callback（HAL_GPIO_EXTI_Callback）：应用层，用户实现具体逻辑
- 这种设计使代码结构更清晰，便于移植和维护

3.7 中断机制深入解析

3.7.1 中断优先级

STM32 使用 NVIC（嵌套向量中断控制器）管理中断，支持中断嵌套。

优先级分组： - STM32F103 有 4 位优先级配置（0-15）- 可分为抢占优先级和子优先级 - 本实验使用默认分组

优先级规则： 1. 抢占优先级高的中断可以打断抢占优先级低的中断 2. 抢占优先级相同时，子优先级高的先响应 3. 两者都相同时，中断向量表中位置靠前的先响应

3.7.2 中断标志位

EXTI_PR 寄存器（**Pending Register**）： - 每个 EXTI 线对应一位 - 当触发条件满足时，硬件自动置位 - 必须在 ISR 中清除，否则会重复触发 - HAL_GPIO_EXTI_IRQHandler() 会自动清除

清除方法:

```
// HAL 库内部实现
__HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_12);
```

3.7.3 中断延迟

从事件发生到执行用户代码的时间包括：1. 硬件检测时间：几个时钟周期 2. NVIC 响应时间：12 个时钟周期 3. 进入 ISR 时间：保存现场 4. HAL 库处理时间：几十个时钟周期

总延迟通常在 1-2 微秒级别。

3.7.4 中断安全性

注意事项： 1. 中断函数应尽量短小精悍 2. 避免在中断中使用延时函数 3. 访问共享变量时注意数据一致性 4. 避免在中断中调用复杂的库函数

本实验的安全性： - HAL_GetTick()：读取全局变量，安全 - HAL_GPIO_TogglePin()：原子操作，安全 - 去抖动逻辑简单，执行时间短

3.8 实验步骤

3.8.1 硬件准备

1. STM32F103C8T6 最小系统板
2. ST-Link 调试器
3. 杜邦线若干
4. 面包板（可选）
5. 按键开关（可选）

3.8.2 软件准备

1. STM32CubeMX（配置工具）
2. VS Code + STM32CubeIDE 插件（推荐开发环境），或 Keil MDK（可选）
3. ST-Link 驱动程序

3.8.3 实验操作

1. 打开工程
    - 使用 STM32CubeMX 打开 03Interrupt.ioc
    - 查看配置是否正确
    - 如需修改，重新生成代码
2. 编译工程
    - 打开 VS Code 工作区（或对应 IDE），导入工程
    - 编译代码，检查是否有错误
3. 下载程序
    - 连接 ST-Link 到电脑和开发板
    - 下载程序到 STM32
4. 测试功能
    - 观察 LED 初始状态
    - 用杜邦线短接 PB12 和 GND
    - 观察 LED 状态是否翻转
    - 多次快速触发，验证去抖动功能

3.8.4 预期现象

1. 程序运行后，LED 处于某个状态（亮或灭）
2. 每次 PB12 接地，LED 状态翻转一次
3. 快速多次触发（200ms 内），只响应一次
4. 间隔超过 200ms 的触发，每次都响应

3.9 调试技巧

3.9.1 LED 不翻转

可能原因： 1. 中断未使能：检查 NVIC 配置 2. GPIO 配置错误：检查 PB12 是否配置为 EXTI 模式 3. 中断函数未执行：在回调函数中设置断点调试 4. 硬件连接问题：检查 PB12 是否正确接地

调试方法：

```
// 在回调函数中添加计数器
static uint32_t interrupt_count = 0;
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  if(GPIO_Pin == GPIO_PIN_12)
  {
    interrupt_count++;  // 设置断点观察此变量
    // ...
  }
}
```

3.9.2 LED 频繁闪烁

可能原因： 1. 去抖动时间太短 2. 硬件接触不良 3. 没有上拉电阻

解决方法： - 增加 DEBOUNCE_TIME_MS 的值 - 检查硬件连接 - 确认 GPIO 配置了上拉

3.9.3 使用调试器

1. 在关键位置设置断点
2. 单步执行，观察变量变化
3. 查看寄存器状态（EXTI_PR、NVIC 等）

3.10 课后作业

3.10.1 说明

1. 本部分为学生课后作业，不属于课堂必做实验。
2. 原“扩展实验”和“思考题”已合并为统一作业，减少重复内容。
3. 涉及额外硬件器件的内容（如 RC 硬件去抖动，需要电阻、电容）不作为学生必做实操。
4. 对于额外硬件相关内容，统一作为课后问答题进行原理分析即可。

3.10.2 课后作业题

1. 为什么 **PB12** 要配置上拉电阻？如果配置为下拉会怎样？

   提示：考虑按键未按下时的默认状态和触发边沿。

2. 如果去掉软件去抖动代码，会出现什么现象？为什么？

   提示：思考机械按键的物理特性。

3. 如果系统运行超过 **49.7** 天，**HAL_GetTick()** 时间戳溢出后，去抖动逻辑是否还能正常工作？请说明原因。

   提示：分析 `(current_time - last_interrupt_time)` 在无符号整数下的行为。

4. 如果同时有 **PB10**、**PB11**、**PB12** 都配置外部中断，它们是否共享中断服务函数？如何区分中断来源？

   提示：查看 EXTI 中断向量分组和回调函数参数。

5. 对比轮询与中断两种方式实现按键控制 **LED**，在 **CPU** 占用、响应速度和功耗方面各有什么优缺点？

   提示：结合本实验场景进行分析。

3.11 实验报告提交要求

3.11.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述本实验需要完成的中断功能与验收条件。
3. 实验原理：说明 EXTI、NVIC、中断服务函数、回调函数与去抖动原理。
4. CubeMX 配置：至少展示 RCC、GPIO、NVIC 的关键配置截图与解释。
5. 程序设计：展示 `MX_GPIO_Init`、`EXTI15_10_IRQHandler`、`HAL_GPIO_EXTI_Callback` 等关键代码并解释作用。
6. 程序流程图：描述“检测下降沿 -> 进入中断 -> 去抖判断 -> 翻转 LED -> 返回主程序”的流程。
7. 实验结果：展示 LED 初始状态、中断触发结果、快速重复触发时的去抖效果。
8. 调试与问题分析：记录未触发、误触发、频繁触发等问题的定位过程与解决方法。
9. 课后作业答案：必须完整回答本 README 中的 5 个课后作业题。
10. 总结：概括本实验对中断编程和嵌入式调试的收获。

3.11.2 图片与流程图要求

1. 报告中至少包含 3 类图片：CubeMX 配置截图、代码或调试截图、实验现象图片。
2. 每张图片必须标注图号和图题，并在正文中解释该图说明了什么。
3. 程序流程图必须单独绘制，不能仅用伪代码代替。
4. 若使用示波器、逻辑分析仪或串口辅助调试，可将波形图作为加分材料。

3.11.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 3-STM32 中断实验.pdf`。
2. 如实验未完全成功，仍需提交报告，并写明失败现象、原因分析与后续改进方案。
3. 所有课后问答题均应独立作答，不能只写“见上文分析”。

3.11.4 最低验收标准

1. PB12 触发后 LED 能正确翻转。
2. 快速重复触发时能体现去抖效果。
3. 能清楚区分轮询方式和中断方式的差异。
4. 报告包含真实调试过程与实验结果。

3.12 参考资料

1. STM32F103C8T6 数据手册
2. STM32F1xx HAL 库用户手册
3. Cortex-M3 权威指南
4. STM32 中文参考手册

3.13 常见问题 FAQ

Q1: 为什么我的 **LED** 一直闪烁不停?

A: 可能是接触不良或干扰导致的误触发，增加去抖动时间或检查硬件连接。

Q2: 中断函数中可以使用 **printf** 吗?

A: 不建议。printf 执行时间长，会阻塞中断，影响系统实时性。如需调试，使用全局变量或标志位。

Q3: 如何验证中断确实被触发了?

A: 在回调函数中翻转另一个 LED，或使用逻辑分析仪/示波器观察 GPIO 波形。

Q4: 多个引脚共享中断向量，会不会冲突?

A: 不会。HAL 库会根据 GPIO_Pin 参数区分，只要在回调函数中正确判断即可。

Q5: 中断优先级设置为 **0** 是最高还是最低?

A: 在 STM32 中，数值越小优先级越高，0 是最高优先级。

实验完成标志： - [ ] 能够正确配置外部中断 - [ ] 理解中断处理流程 - [ ] 掌握软件去抖动方法 - [ ] 能够独立调试中断问题 - [ ] 完成课后作业题

作业提交要求： 1. 仅提交一份电子文档。2. 电子文档需包含详细实验报告。3. 电子文档需包含课程作业答案。4. 电子文档需包含核心代码解析。5. 电子文档需包含实验现象照片。

祝实验顺利!

4 STM32 USB 虚拟串口通信实验

4.1 统一作业说明

4.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成 USB Device CDC、时钟树、调试接口等配置，并保留 `.ioc` 文件。
2. 基于 HAL 与 USB CDC 中间件实现虚拟串口的收发功能，不得只完成单向发送。
3. 在主循环与接收回调配合下，实现至少 `help`、`ping`、`status`、`dice`、`echo <text>` 这些命令的处理。
4. 成功编译、下载并在电脑端串口调试助手中验证枚举、发送、接收和命令交互。
5. 在实验报告中说明 USB 时钟配置、代码结构、实验现象、问题排查和课后思考。
6. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

4.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6 或课程指定的带 USB 开发板
通信方式	USB CDC 虚拟串口
必做功能	电脑枚举识别 COM 口，STM32 能主动发送并响应命令
理论要求	能解释 48 MHz USB 时钟、CDC 接收回调、主循环命令处理
验收方式	现场演示或结果截图，能展示串口助手中的收发记录

4.1.3 本次必须提交的内容

1. 可编译的完整工程目录。
2. 一份 PDF 格式实验报告。
3. 设备管理器识别截图、串口调试助手交互截图、关键代码截图各至少 1 张。
4. 课后思考题的书面回答。

4.1.4 报告必须回答的问题

1. 为什么 USB 外设时钟必须精确到 48 MHz。
2. 为什么不建议在 `CDC_Receive_FS` 回调中直接处理复杂业务。
3. 当前“单缓冲区 + 标志位”的接收方案有什么优点和局限。
4. 主循环与接收回调之间为什么要考虑共享数据保护。

4.2 实验目的

本实验基于 STM32 的 USB CDC 类实现虚拟串口通信，要求学生完成从 CubeMX 配置、代码生成、程序编写到上位机验证的完整实验流程。通过本实验，你应掌握以下内容：

1. 理解 USB 虚拟串口（Virtual COM Port, VCP）的基本原理。
2. 掌握 STM32CubeMX 中 USB Device CDC 的配置方法。
3. 学会在 STM32 工程中使用 CDC_Transmit_FS 完成数据发送。
4. 学会在接收回调函数中获取上位机发来的数据，并在主循环中完成命令处理。
5. 能够配合电脑端串口调试助手完成实验验证、现象记录与结果分析。

4.3 实验环境

4.3.1 硬件环境

1. 一块带 USB 接口的 STM32 开发板。
2. 一根可正常传输数据的 USB 数据线。
3. ST-Link 下载器或板载调试器。

4.3.2 软件环境

1. STM32CubeMX。
2. VS Code 或其他支持该工程的 STM32 开发环境。
3. ARM GCC / CMake 工具链或等效编译环境。
4. 串口调试助手。
5. ST-Link 驱动。

4.4 STM32CubeMX 配置步骤

下面的步骤必须保留在实验报告中，因为它们是本实验成功的前提条件。你不仅要会“照着做”，还要理解每一步为什么这样设置。

4.4.1 新建工程并选择芯片

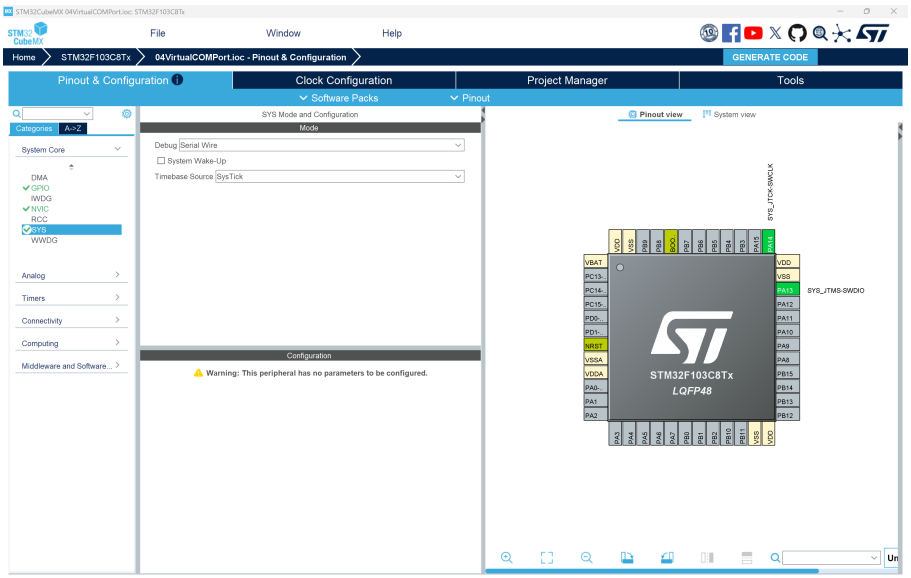
打开 STM32CubeMX，点击 ACCESS TO MCU SELECTOR，根据你使用的开发板或芯片型号选择对应的 STM32 器件。

4.4.2 配置调试接口（SYS）

在 Pinout & Configuration 页面中，找到 SYS，将 Debug 设置为 Serial Wire。

这样做的原因是：

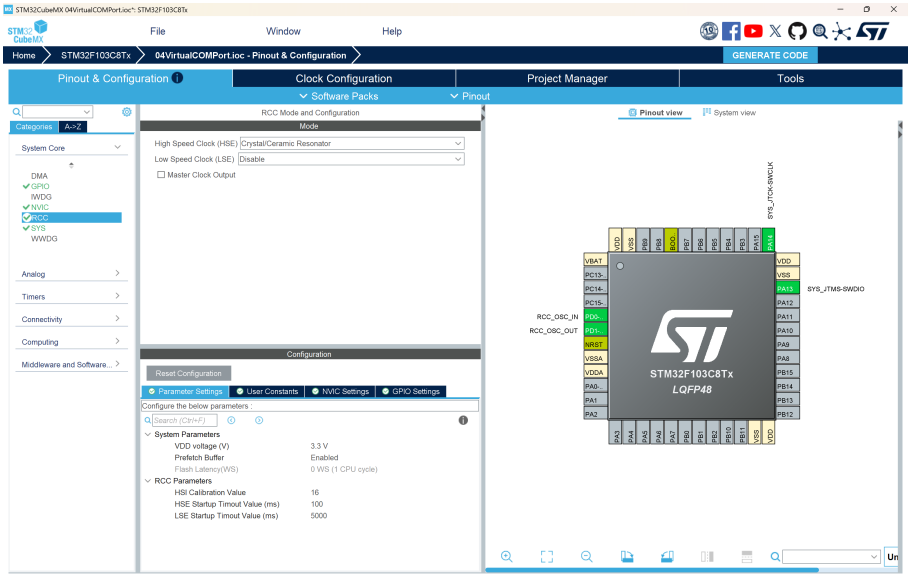
1. 便于后续通过 ST-Link 下载和调试程序。
2. 如果未正确配置调试接口，后续可能出现芯片无法正常调试的问题。



4.4.3 配置高速外部时钟（RCC）

在 RCC 配置项中，将 High Speed Clock (HSE) 设置为 Crystal/Ceramic Resonator。

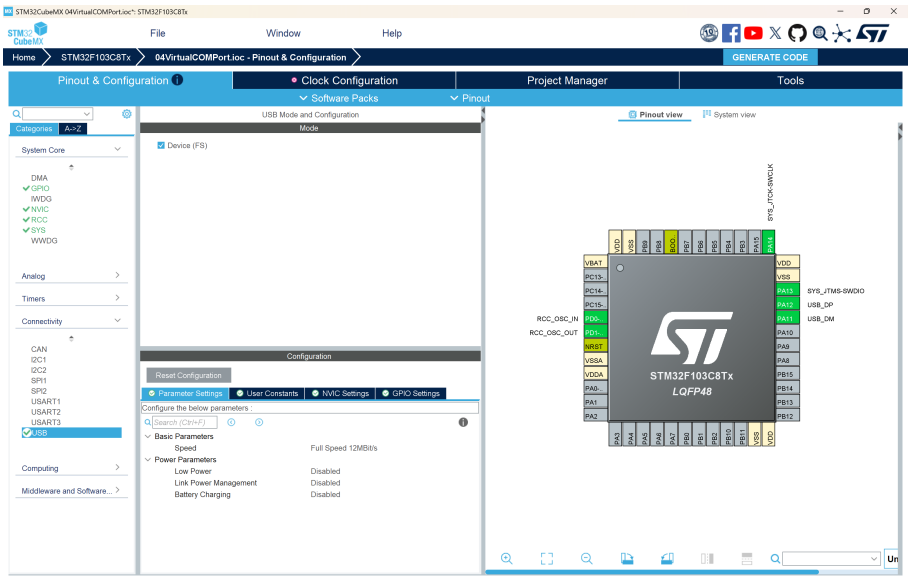
这样做的目的是为 PLL 和 USB 提供稳定时钟来源。



4.4.4 启用 USB 外设（Connectivity）

在左侧 Connectivity 菜单下找到 USB，部分芯片上显示为 USB_OTG_FS。将其配置为 Device_Only 或 Device (FS)。

此步骤的作用是启用芯片的 USB 设备功能，使 STM32 作为一个 USB 外设被电脑识别。

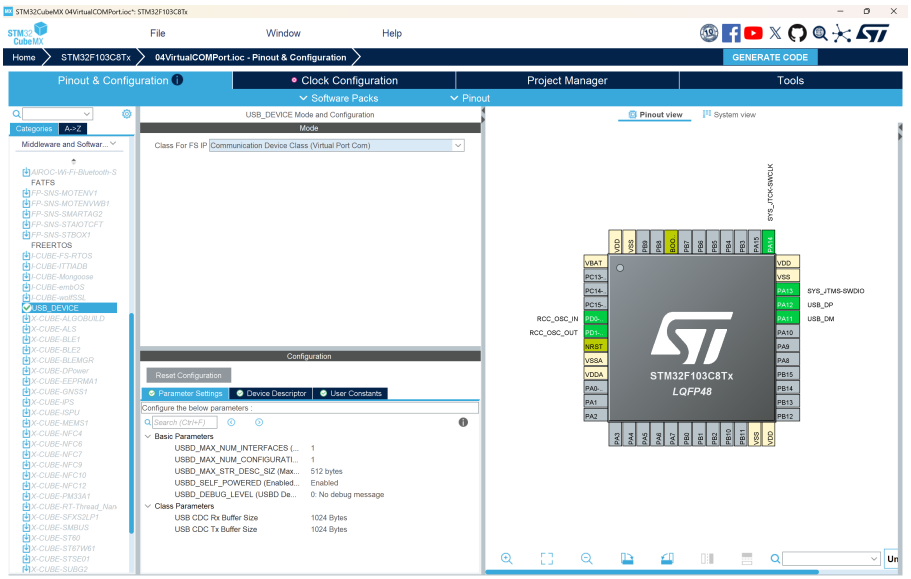


4.4.5 配置 USB 中间件（Middleware）

在 Middleware and Software Packs 中打开 USB_DEVICE，将 Class for FS IP 设置为 Communication Device Class (Virtual Port Com)。

这一步的意义在于：

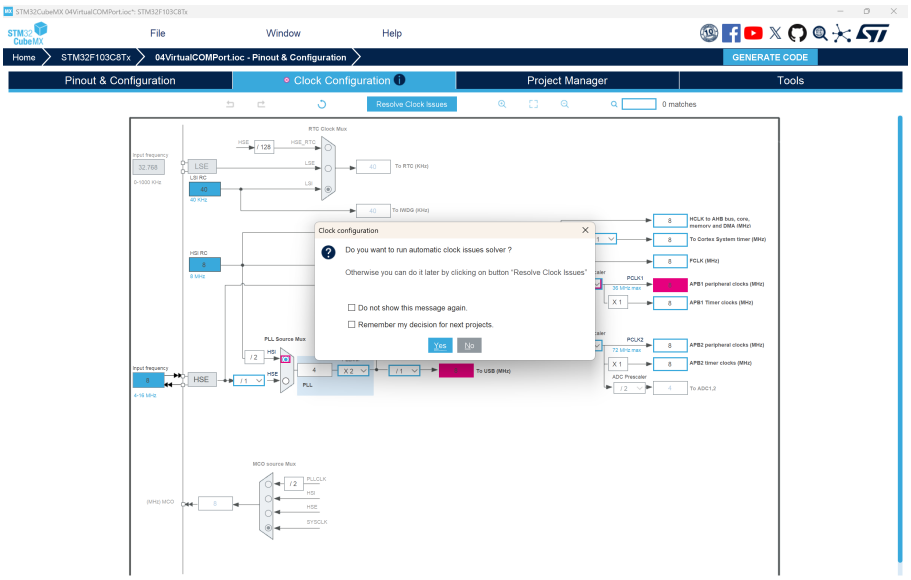
1. 告诉 CubeMX 当前 USB 设备要工作在 CDC 类。
2. 生成 USB 虚拟串口所需的描述符、接口文件和中间件代码。



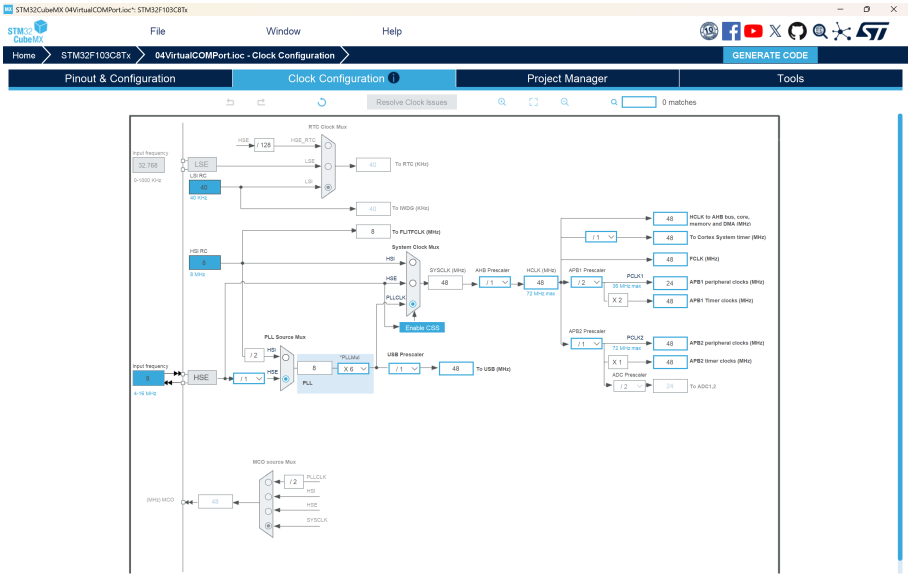
4.4.6 配置时钟树（Clock Configuration）

这是 USB 虚拟串口实验最关键的一步。USB 外设通常要求精确的 48MHz 时钟。如果这里配置错误，电脑可能无法识别设备，或者识别后通信异常。

进入 Clock Configuration 页面后，如果弹出自动配置提示，可以选择自动配置。



随后检查 PLL 与 USB 时钟分配结果，确保 USB 时钟精确为 48MHz。



4.4.7 配置工程并生成代码

在 Project Manager 页面中：

1. 设置工程名称与保存路径。
2. 选择对应的开发环境或工具链。
3. 在 Code Generator 选项中勾选将外设初始化代码分离为独立的 c/h 文件，便于管理。
4. 点击 GENERATE CODE 生成工程。

4.4.8 配置工具链为 CMake

如果你希望在 VS Code 环境中继续使用当前工程，建议将工具链选择为 CMake，这样与本仓库的工程结构更加一致，也更方便后续在 VS Code 中编译和维护。

4.5 硬件连接要点

1. USB_DP 与 USB_DM 通常分别连接到 PA12 和 PA11，对应 USB 连接器的 D+ 与 D-。
2. 对于部分 STM32F1 系列芯片，可能需要确认 D+ 上拉电阻相关硬件设计是否满足 USB 枚举要求。
3. USB 线必须是数据线，不能只用供电线，否则电脑无法识别虚拟串口。

4.6 电脑端驱动与串口助手准备

1. Windows 10/11 一般会自动识别 USB CDC 虚拟串口。
2. 较老系统如 Windows 7/8 可能需要手动安装 ST 官方驱动。
3. 打开设备管理器，在端口（COM 和 LPT）中查看新出现的虚拟串口号。
4. 打开串口调试助手，选择对应 COM 口即可进行测试。

建议在实验报告中把电脑端准备过程写得更具体，至少应包括以下内容：

1. 将开发板通过 USB 数据线连接到电脑后，打开设备管理器。
2. 展开端口（COM 和 LPT），找到新出现的设备名称，常见显示方式为 USB Serial Device (COMx) 或 ST 的虚拟串口设备。
3. 记录其中的 COM 口编号，例如 COM3、COM5、COM7，后续在串口调试助手中选择同名端口。
4. 如果插拔开发板前后端口列表发生变化，可以通过“插上设备后新增的那个端口”来判断哪一个是当前开发板对应的虚拟串口。

串口调试助手建议配置如下：

1. 端口号：选择设备管理器中识别到的 COM 口，例如 COM5。
2. 波特率：建议选择 115200。
3. 数据位：8 位。
4. 校验位：无校验（None）。
5. 停止位：1 位。
6. 流控：无（None）。
7. 发送行结束符：建议使用 CRLF，也就是回车换行 \r\n，这样更方便和本实验中的命令解析逻辑匹配。
8. 接收显示：可勾选“追加换行”或“自动换行显示”，便于观察返回结果。

说明：虽然串口调试助手通常要求设置波特率、数据位、校验位和停止位，但对于 USB CDC 虚拟串口，这些参数通常不会像传统 UART 那样真正决定底层 USB 传输速率。不过为了保证串口助手界面配置统一、实验报告书写规范，以及避免部分软件因参数异常而无法正常打开端口，仍建议统一使用 115200、8 位数据位、无校验、1 位停止位，即常见的 115200 8N1 配置。

如果设备管理器中没有看到新的 COM 口，应重点检查以下问题：

1. 使用的 USB 线是否为数据线，而不是仅供电线。
2. 开发板是否已经成功下载并运行了包含 USB CDC 功能的程序。

3. CubeMX 中 USB_DEVICE 的 CDC 类是否正确启用。
4. USB 时钟是否正确配置为 48MHz。
5. 当前系统是否缺少对应驱动。

4.7 本工程实现的实验内容

当前工程在代码中实现了一个简单的 USB 虚拟串口命令交互示例，功能如下：

1. 板子启动后输出欢迎信息。
2. 每隔一段时间输出一次 Heartbeat 信息，表示系统运行正常。
3. 上位机发送命令后，STM32 对命令进行解析并返回结果。
4. 支持的命令包括：help、ping、status、dice、echo 文本。

这样的设计比单纯的“发送 Hello”更适合作为课程实验，因为它同时覆盖了：

1. 主动发送。
2. 被动接收。
3. 命令解析。
4. 主循环与中断回调配合。

4.8 核心代码说明

这一部分把本次实验中真正修改过的源码放到文档中，并逐段解释其含义。对于课程作业来说，不能只写“调用了某个函数”，而应说明：

1. 代码写在什么文件中。
2. 这段代码解决了什么问题。
3. 为什么要这样写，而不是更简单地直接调用函数。

4.8.1 main.c 中新增的头文件、变量与函数声明

```c
#include "usbd_cdc_if.h"
#include <stdio.h>
#include <string.h>

static uint32_t s_lastHeartbeatTick = 0;
static uint32_t s_heartbeatCounter = 0;

static uint8_t USB_SendString(const char *text);
static void USB_ProcessCommand(char *cmdLine);
```

代码解析：

1. usbd_cdc_if.h 头文件提供了 CDC_Transmit_FS、接收缓冲区和接收标志位等接口声明。
2. stdio.h 用于 snprintf，把数值格式化成字符串后再发送到上位机。
3. string.h 用于 strlen、strcmp、memcpy 等字符串操作。
4. s_lastHeartbeatTick 用于记录上一次心跳发送的时间。
5. s_heartbeatCounter 用于统计系统已经输出了多少次心跳。
6. USB_SendString 是发送封装函数，用于提高发送稳定性。

7. USB_ProcessCommand 是命令处理函数，用于解析上位机发送的字符串命令。

编程技巧：

1. 将“发送逻辑”和“命令处理逻辑”拆分成独立函数，可以让 main 函数保持简洁。
2. 使用 static 修饰内部函数和内部变量，可以限制其作用域在当前文件内，减少命名冲突。

4.8.2 main.c 中的发送函数 USB_SendString

```
static uint8_t USB_SendString(const char *text)
{
    uint16_t len = (uint16_t)strlen(text);
    uint32_t startTick = HAL_GetTick();

    while (CDC_Transmit_FS((uint8_t *)text, len) == USBD_BUSY)
    {
        if ((HAL_GetTick() - startTick) > 100)
        {
            return USBD_BUSY;
        }
        HAL_Delay(1);
    }

    return USBD_OK;
}
```

代码解析：

1. 先通过 strlen 计算字符串长度。
2. startTick 记录当前时刻，用于后续超时判断。
3. CDC_Transmit_FS 是 USB CDC 的发送函数，但它不保证每次调用都成功。
4. 当 USB 底层仍在发送上一包数据时，CDC_Transmit_FS 会返回 USBD_BUSY。
5. 因此程序在 while 循环中等待发送资源空闲。
6. 如果等待超过 100ms，函数直接返回 USBD_BUSY，避免程序长时间卡住。
7. 如果发送成功，返回 USBD_OK。

为什么不能直接写成一行发送：

```
CDC_Transmit_FS((uint8_t *)text, len);
```

因为 USB CDC 是异步发送接口，如果上一次发送还没有结束，下一次立刻发送很可能失败。这类问题在实验早期最常见，现象通常是“有时能发，有时不能发”。

4.8.3 main.c 中的命令解析函数 USB_ProcessCommand

```
static void USB_ProcessCommand(char *cmdLine)
{
    size_t len = strlen(cmdLine);
    const char *echoText;
    char txBuf[160];

    while (len > 0 && (cmdLine[len - 1] == '\r' || cmdLine[len - 1] == '\n'))
    {
        cmdLine[len - 1] = '\0';
        len--;
```

```
    }

    if (len == 0)
    {
        return;
    }

    if (strcmp(cmdLine, "help") == 0)
    {
        USB_SendString("Commands: help, ping, status, dice, echo <text>\\r\\n");
    }
    else if (strcmp(cmdLine, "ping") == 0)
    {
        USB_SendString("pong\\r\\n");
    }
    else if (strcmp(cmdLine, "status") == 0)
    {
        int n = snprintf(txBuf, sizeof(txBuf),
                                    "Uptime: %lu ms, Heartbeat: %lu\\r\\n",
                                    HAL_GetTick(), s_heartbeatCounter);
        if (n > 0)
        {
            USB_SendString(txBuf);
        }
    }
    else if (strcmp(cmdLine, "dice") == 0)
    {
        uint32_t diceValue = (HAL_GetTick() % 6UL) + 1UL;
        int n = snprintf(txBuf, sizeof(txBuf), "Dice: %lu\\r\\n", diceValue);
        if (n > 0)
        {
            USB_SendString(txBuf);
        }
    }
    else if (strncmp(cmdLine, "echo ", 5) == 0)
    {
        echoText = &cmdLine[5];
        if (*echoText == '\0')
        {
            USB_SendString("Usage: echo <text>\\r\\n");
        }
        else
        {
            int n = snprintf(txBuf, sizeof(txBuf), "Echo: %s\\r\\n", echoText);
            if (n > 0)
            {
                USB_SendString(txBuf);
            }
        }
    }
    else
    {
        int n = snprintf(txBuf, sizeof(txBuf), "Unknown command: %s\\r\\n", cmdLine);
        if (n > 0)
        {
            USB_SendString(txBuf);
        }
    }
}
```

代码解析：

1. len 用于记录当前命令字符串长度。

2. txBuf 是发送缓冲区，用于保存格式化后的返回消息。
3. while 循环用于去掉命令末尾的回车和换行符。
4. 这一步很有必要，因为串口调试助手发送命令时，通常会附带回车或回车换行。
5. 如果去掉结尾字符后长度为 0，说明用户只按了回车，不需要处理。
6. strcmp 用于判断命令内容。
7. help 命令返回支持的命令列表。
8. ping 命令返回 pong，用于验证最基本的双向通信。
9. status 命令通过 snprintf 拼接运行时间和心跳计数。
10. dice 命令根据系统节拍生成 1 到 6 的点数，能让实验结果更有交互感。
11. echo 命令会把用户输入的文本原样返回，便于观察字符串接收与回传效果。
12. 对于未知命令，返回 Unknown command，便于用户定位输入错误。

编程技巧：

1. 先做输入清洗，再做命令判断，是常见的命令行处理方式。
2. 使用 snprintf 比 sprintf 更安全，因为它可以限制写入长度。
3. 使用 else if 链适合实验阶段的少量命令，后续命令多了可以改成查表或状态机。

4.8.4 main.c 中的初始化阶段代码

```
MX_GPIO_Init();
MX_USB_DEVICE_Init();

HAL_Delay(1200);
USB_SendString("\\r\\n===== STM32 USB CDC Demo =====\\r\\n");
USB_SendString("Type 'help' and press Enter.\\r\\n");
```

代码解析：

1. MX_GPIO_Init 初始化 GPIO 外设。
2. MX_USB_DEVICE_Init 初始化 USB Device 中间件与 CDC 类。
3. HAL_Delay(1200) 的目的是给 USB 枚举预留一点时间。
4. 如果设备刚初始化完成就立即发送数据，电脑端可能尚未完成枚举，导致首条信息发送失败。
5. 欢迎信息是实验调试中非常重要的“可观测信号”，只要一打开串口助手就能看到系统是否正常启动。

4.8.5 main.c 中的主循环核心代码

```
while (1)
{
    if ((HAL_GetTick() - s_lastHeartbeatTick) >= 1000)
    {
        char heartbeatMsg[64];
        s_lastHeartbeatTick = HAL_GetTick();
        s_heartbeatCounter++;

        if ((s_heartbeatCounter % 5UL) == 0UL)
        {
            int n = snprintf(heartbeatMsg, sizeof(heartbeatMsg),
                                        "Heartbeat %lu\\r\\n", s_heartbeatCounter);
            if (n > 0)
            {
                USB_SendString(heartbeatMsg);
            }
```

```
        }
    }

    if (g_usbRxFlag != 0U)
    {
        uint32_t rxLen;
        char cmdBuf[APP_RX_DATA_SIZE + 1U];

        __disable_irq();
        rxLen = g_usbRxLength;
        if (rxLen > APP_RX_DATA_SIZE)
        {
            rxLen = APP_RX_DATA_SIZE;
        }
        memcpy(cmdBuf, g_usbRxBuffer, rxLen);
        g_usbRxFlag = 0U;
        g_usbRxLength = 0U;
        __enable_irq();

        cmdBuf[rxLen] = '\0';
        USB_ProcessCommand(cmdBuf);
    }
}
```

代码解析：

1. 主循环第一部分负责周期任务，也就是心跳输出。
2. HAL_GetTick 返回系统运行毫秒数，可用于实现非阻塞定时。
3. 每过 1000ms，心跳计数加 1。
4. 每累计 5 次心跳，向上位机发送一次 Heartbeat 文本。
5. 这样做的目的不是频繁刷屏，而是让上位机定期看到系统仍在正常运行。
6. 主循环第二部分检查 g_usbRxFlag，这个标志由 USB 接收回调置位。
7. 当有新数据时，先进入临界区，读取长度并把共享缓冲区拷贝到本地变量中。
8. 随后清除接收标志和长度，再退出临界区。
9. 最后手动在字符串末尾补上 '\0'，使其成为合法 C 字符串。
10. 调用 USB_ProcessCommand 完成命令解析。

为什么这里要使用临界区：

1. g_usbRxFlag、g_usbRxLength、g_usbRxBuffer 都可能在中断回调中被修改。
2. 如果主循环读取到一半，USB 中断恰好又写入新数据，就可能出现数据不一致。
3. 因此这里使用 ___disable_irq 和 ___enable_irq 做最小范围保护。

4.8.6 usbd_cdc_if.h 中新增的共享变量声明

```
extern volatile uint8_t g_usbRxFlag;
extern volatile uint32_t g_usbRxLength;
extern uint8_t g_usbRxBuffer[APP_RX_DATA_SIZE];
```

代码解析：

1. g_usbRxFlag 表示是否收到一帧新的 USB 数据。
2. g_usbRxLength 记录当前接收到的数据长度。
3. g_usbRxBuffer 用于保存从上位机发来的原始数据。
4. extern 表示这些变量定义在别的 c 文件中，这里只是声明，供 main.c 使用。

5. volatile 的作用是告诉编译器，这些变量可能被中断或硬件异步修改，不能随意优化缓存。

这是嵌入式开发中非常基础但又非常容易忽略的一点。凡是由中断修改、主循环读取的状态变量，都应认真考虑是否需要 volatile。

4.8.7 usbd_cdc_if.c 中新增的共享变量定义

```
volatile uint8_t g_usbRxFlag = 0;
volatile uint32_t g_usbRxLength = 0;
uint8_t g_usbRxBuffer[APP_RX_DATA_SIZE];
```

代码解析：

1. 这里是变量真正分配存储空间的地方。
2. 初值都设置为 0，表示系统启动时还没有接收到任何数据。
3. 缓冲区大小使用 APP_RX_DATA_SIZE，和 USB CDC 配置保持一致。

4.8.8 usbd_cdc_if.c 中的接收回调函数

```
static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len)
{
    uint32_t rxLen = *Len;

    if (rxLen > APP_RX_DATA_SIZE)
    {
        rxLen = APP_RX_DATA_SIZE;
    }

    memcpy(g_usbRxBuffer, Buf, rxLen);
    g_usbRxLength = rxLen;
    g_usbRxFlag = 1;

    USBD_CDC_SetRxBuffer(&hUsbDeviceFS, &Buf[0]);
    USBD_CDC_ReceivePacket(&hUsbDeviceFS);
    return (USBD_OK);
}
```

代码解析：

1. Buf 指向 USB 底层收到的数据缓冲区。
2. Len 记录本次接收到的数据长度。
3. 先把长度取出来保存到 rxLen，便于后续处理。
4. 如果长度超过应用缓冲区大小，就截断到 APP_RX_DATA_SIZE，防止越界。
5. memcpy 把 USB 收到的数据复制到应用层自己的缓冲区 g_usbRxBuffer。
6. 保存长度到 g_usbRxLength。
7. 设置 g_usbRxFlag = 1，通知 main.c 主循环 “有新数据可以处理”。
8. 调用 USBD_CDC_SetRxBuffer 设置接收缓冲区。
9. 调用 USBD_CDC_ReceivePacket 重新开启下一次 USB 接收。

这里最重要的编程思想是 “中断里只搬运数据，不做复杂业务”。

如果在这个回调里直接做大量字符串比较、格式化输出甚至复杂协议解析，会带来以下风险：

1. 回调执行时间变长。

2. 影响 USB 协议层继续收包。
3. 程序结构混乱，不利于扩展。

4.8.9 代码结构的整体工作流程

本实验代码可以概括为下面 5 个步骤:

1. 系统启动后初始化 USB CDC。
2. STM32 向电脑发送欢迎信息。
3. 电脑通过串口调试助手发送命令。
4. USB 接收回调把数据存入缓冲区并置位标志。
5. 主循环读取数据并解析命令，然后返回处理结果。

这就是一个典型的“中断采集数据，主循环处理业务”的嵌入式软件结构。它简单、稳定，非常适合作为课程实验与后续扩展的基础框架。

4.9 当前示例支持的命令

4.9.1 help

显示支持的命令列表。

4.9.2 ping

返回 pong，用于验证双向通信正常。

4.9.3 status

返回系统运行时间与心跳计数，用于查看程序运行状态。

4.9.4 dice

返回一个 1 到 6 的随机点数，用于增强实验交互性。

4.9.5 echo

回显用户输入的文本，用于验证接收缓存与字符串处理是否正常。

4.10 编程技巧总结

1. 接收回调中少做事，把复杂处理放到主循环。
2. 与中断共享的变量应使用 volatile 修饰。
3. 读取共享缓冲区时要考虑临界区保护，防止数据被中断同时修改。
4. 发送函数要考虑忙状态，不能无条件连续发送。
5. 实验代码最好具备“可观测性”，例如欢迎信息、心跳包、状态命令，这样更便于调试。

4.11 实验操作步骤

4.11.1 编译工程

使用你当前的开发环境编译工程，确认无编译错误。

4.11.2 烧录程序

通过 ST-Link 将程序下载到 STM32 开发板。

4.11.3 连接电脑

使用 USB 数据线将开发板连接到电脑，等待系统识别虚拟串口。

4.11.4 查看 COM 口

打开设备管理器，确认端口列表中出现新的虚拟串口设备。

4.11.5 打开串口调试助手

在串口调试助手中：

1. 选择对应 COM 口。
2. 打开串口。
3. 观察是否收到欢迎信息。

4.11.6 输入测试命令

依次发送以下命令并观察返回结果：

1. help
2. ping
3. status
4. dice
5. echo hello stm32
6. 自定义未知命令，例如 abc

4.11.7 记录实验结果

建议在实验报告中保留以下截图：

1. CubeMX 配置截图。
2. 设备管理器中虚拟串口截图。
3. 串口调试助手中的欢迎信息截图。
4. help、ping、status、dice、echo 命令响应截图。
5. 未知命令测试截图。

4.12 实验现象与结果分析

如果实验成功，你应看到如下现象：

1. 电脑成功识别到一个新的 COM 口。
2. 打开串口助手后，STM32 自动输出欢迎信息。
3. 程序周期性输出 Heartbeat 信息。
4. 向 STM32 发送命令后，能够收到对应返回值。

更具体地说，实验过程中通常会观察到以下现象：

1. 串口打开后，窗口首先显示类似下面的欢迎信息：

```
===== STM32 USB CDC Demo =====
Type 'help' and press Enter.
```

2. 如果程序持续稳定运行，每隔约 5 秒会收到一次 Heartbeat 信息，例如 Heartbeat 5、Heartbeat 10、Heartbeat 15。这说明主循环没有卡死，USB 发送链路仍然可用。
3. 发送 help 命令后，串口助手会返回当前支持的命令列表。这说明命令解析分支已经进入正确路径。
4. 发送 ping 命令后，STM32 会立即返回 pong。这个现象最适合用来证明“上位机发送成功，开发板接收成功，开发板处理完成后又成功回传”。
5. 发送 status 命令后，会返回当前运行时间和心跳计数，例如 Uptime: 18342 ms, Heartbeat: 18。该现象说明程序能够在主循环中持续运行，并能把内部状态格式化后发送给上位机。
6. 发送 dice 命令后，会返回 Dice: 1 到 Dice: 6 之间的某个结果。虽然这里不是严格随机数，但对课程实验来说，它足以展示“同一个命令可产生不同返回结果”的交互特性。
7. 发送 echo hello stm32 后，会返回 Echo: hello stm32。这个现象说明接收缓冲区中的字符串被完整取出，并经过简单解析后重新发送出来。
8. 如果输入一个未定义命令，例如 abc，程序会返回 Unknown command: abc。这说明异常输入路径也得到了处理，程序不会因为非法命令而无响应。

从实验报告书写角度，可以把这些现象分别对应为以下结论：

1. 能识别 COM 口，说明 USB 枚举成功。
2. 能收到开机欢迎信息，说明 CDC 发送初始化基本正常。
3. 能稳定看到 Heartbeat，说明系统主循环在持续执行。
4. ping 和 echo 能正常返回，说明 CDC 接收链路与命令解析链路正常。
5. status 能返回运行时间和计数值，说明应用层状态变量工作正常。
6. dice 能产生变化结果，说明程序不仅能固定回复，还能根据当前运行状态生成不同输出。

这些现象说明:

1. USB 枚举成功。
2. CDC 发送功能正常。
3. CDC 接收功能正常。
4. 应用层命令解析逻辑正常。

4.13 常见问题排查

问题	可能原因与解决方案
电脑无法识别设备	检查 USB 时钟是否为 48MHz；检查 USB 线是否为数据线；检查硬件连线是否正确；检查程序是否已正常运行。
有 COM 口但没有输出	检查是否调用了 MX_USB_DEVICE_Init；检查上电后是否给了 USB 足够的枚举时间；检查发送函数是否因忙状态失败。
输入命令没有响应	检查串口助手是否发送了回车换行；检查接收标志位与缓冲区逻辑是否正常；检查回调中是否重新开启接收。
连续发送时偶发失败	说明 CDC 发送接口处于忙状态，应做重试、延时或发送队列处理。
clear 命令没有清屏	说明串口助手不支持 ANSI 控制序列，这不是 STM32 程序错误。

4.14 课后思考题

请结合本次实验，在报告中认真回答以下问题:

1. USB CDC 与传统 UART 串口通信在原理和使用方式上有什么区别?
2. 为什么 USB 外设时钟必须精确配置为 48MHz?
3. 为什么不建议在 CDC_Receive_FS 回调函数中直接做复杂业务处理?
4. 当前代码使用的是“单缓冲区 + 标志位”方式接收数据，这种方法有哪些优点和局限?
5. 如果要支持更长的数据帧或连续高速输入，应如何优化当前程序结构?
6. 如果要把本实验扩展为“通过上位机命令控制 LED 闪烁频率”，你会如何设计命令格式与程序流程?

4.15 实验报告提交要求

4.15.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述本实验需要实现的通信功能与验收指标。
3. 实验环境：开发板型号、USB 连接方式、调试器、软件版本、上位机工具。
4. CubeMX 配置：必须说明 SYS、RCC、USB、USB_DEVICE、Clock Configuration 的关键设置及其原因。

5. 程序设计：至少分析 main.c 中的初始化和主循环逻辑，以及 usbd_cdc_if.c 中的接收回调逻辑。
6. 程序流程图：必须展示“USB 接收 -> 缓冲区搬运 -> 主循环解析 -> 返回结果”的流程。
7. 实验步骤：包括编译、下载、连接电脑、识别 COM 口、串口助手测试命令等过程。
8. 实验结果：必须展示设备管理器识别结果、欢迎信息、命令交互结果和异常命令测试结果。
9. 问题分析与调试记录：说明遇到的枚举失败、发送忙、无回显等问题及排查方法。
10. 课后思考题答案：必须完整回答本 README 中 6 个思考题。
11. 总结：概括你对 USB CDC 通信、任务划分和调试方法的理解。
12. 附录：可附核心代码、更多截图、参考资料。

4.15.2 图片与流程图要求

1. 报告中至少包含 4 类图片：CubeMX 配置截图、设备管理器截图、串口调试助手截图、实验结果或调试截图。
2. 每张图片必须标注图号和图题，并在正文中说明该图用于证明什么现象。
3. 程序流程图必须单独成图，不能只用文字描述替代。
4. 若使用额外调试工具，如逻辑分析仪、串口日志或视频录屏，可作为附录或加分材料。

4.15.3 提交规范

1. 报告文件建议命名为：学号-姓名-作业 4-STM32USB 虚拟串口实验.pdf。
2. 若实验未完全成功，也必须提交完整报告，重点说明失败现象、原因分析和改进方向。
3. 报告中不能只贴代码或截图，必须结合现象进行解释。

4.15.4 最低验收标准

1. 电脑能够识别出虚拟串口设备。
2. STM32 能主动发送欢迎信息或心跳信息。
3. 上位机发送命令后能够收到正确响应。
4. 报告能够解释接收回调与主循环协作关系。

4.16 可进一步扩展的方向

1. 增加 LED 控制命令。
2. 增加 printf 重定向到 USB CDC。
3. 使用环形缓冲区替代当前单缓冲结构。
4. 增加简单通信协议，如帧头、长度、校验位。

4.17 总结

本实验不是单纯完成一次“USB 能发出字符串”的演示，而是要你掌握完整的嵌入式通信开发流程：先配置硬件与时钟，再生成底层框架，之后编写应用层逻辑，最后借助上位机工具完成验证与分析。只有把“配置步骤、代码结构、调试方法、实验结果”完整串起来，才算真正完成这次作业。

5 STM32 UART 串口通信实验

5.1 实验概述

本实验基于 **STM32F103C8T6** 微控制器，通过 **USART1** 实现 PC 与开发板之间的串口通信。用户通过串口助手发送命令控制板载 LED（PC13）的亮灭，同时 MCU 会周期性向串口发送心跳消息。

核心知识点：- STM32 UART/USART 外设的配置与使用 - HAL 库中 UART 轮询收发 API：`HAL_UART_Receive` / `HAL_UART_Transmit` - 开漏输出（Open-Drain）模式驱动 LED - STM32CubeMX 图形化配置引脚与外设

5.2 硬件需求

器件	说明
STM32F103C8T6 最小系统板（Blue Pill / 核心板）	主控 MCU
USB-TTL 串口模块（如 CH340 / CP2102）	与 PC 通信
ST-Link / J-Link 调试器	烧录程序
板载 LED（PC13）	被控指示灯

引脚连接：

STM32 引脚	功能	连接目标
PA9	USART1_TX	USB-TTL 模块 RX
PA10	USART1_RX	USB-TTL 模块 TX
PC13	GPIO_Output (LED)	板载 LED（低电平点亮）
PA13	SWDIO	ST-Link SWDIO
PA14	SWCLK	ST-Link SWCLK
GND	地	USB-TTL 模块 GND + ST-Link GND

5.3 STM32CubeMX 配置步骤

5.3.1 新建工程

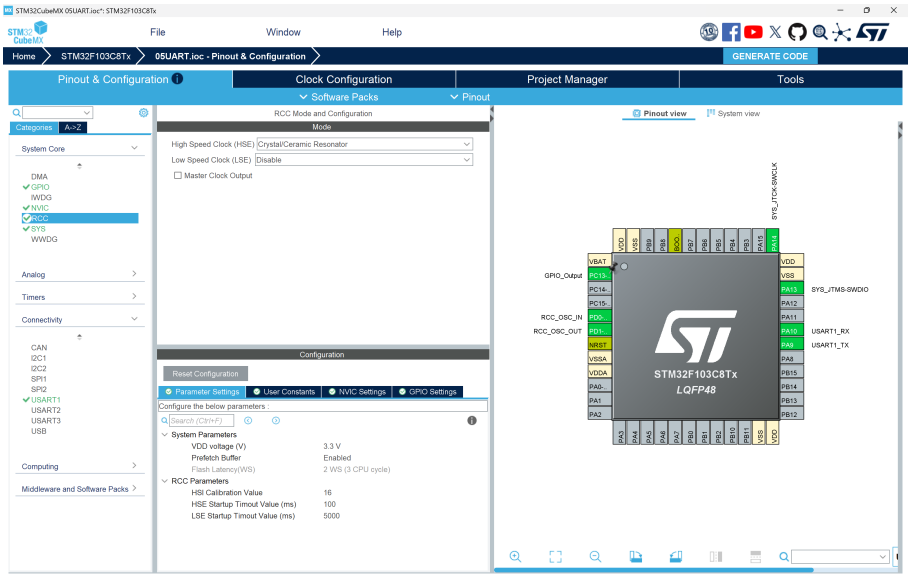
1. 打开 **STM32CubeMX**（本工程使用 6.17.0 版本）
2. 点击 **File → New Project**
3. 在 **MCU Selector** 中搜索 `STM32F103C8Tx`，选中后点击 **Start Project**

（如图：在 MCU Selector 搜索框中输入 “STM32F103C8”，在列表中选择对应型号。）

5.3.2 配置时钟源（RCC）

1. 点击左侧 **Pinout & Configuration** 标签
2. 在左侧 **Categories** 中找到 **System Core → RCC**
3. 将 **High Speed Clock (HSE)** 设为 **Crystal/Ceramic Resonator**（外部晶振）

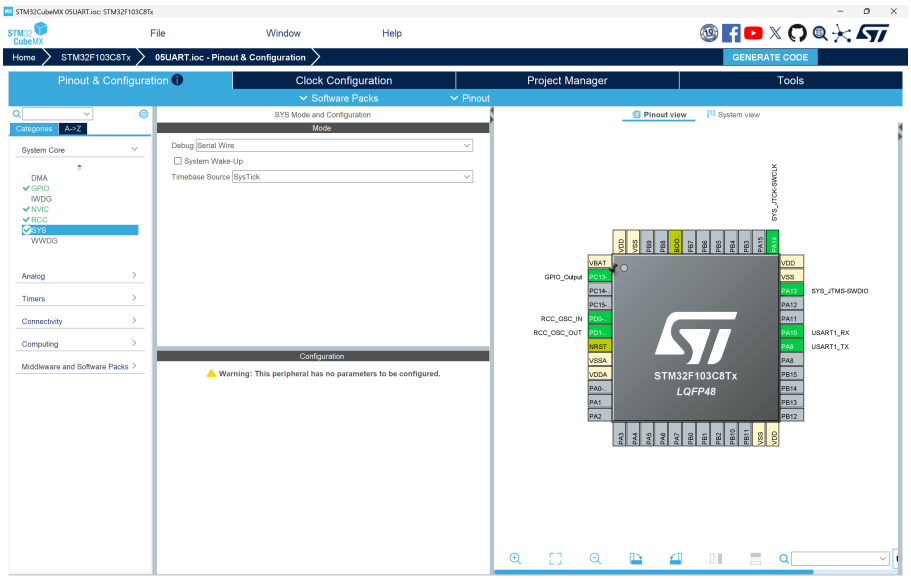
说明： STM32F103C8T6 核心板通常板载 8MHz 晶振，通过 HSE + PLL 倍频到 72MHz 作为系统主频。



5.3.3 配置调试接口（SYS）

1. 在 **System Core → SYS** 中
2. 将 **Debug** 设为 **Serial Wire**（关闭 JTAG，仅保留 SWD，释放 PA15/PB3/PB4 给 GPIO）

为什么要这样做？ STM32F103 默认同时启用 JTAG 和 SWD。关闭 JTAG 只保留 SWD 不仅节省引脚（PA15、PB3、PB4 可作普通 GPIO），SWD 本身也足以完成烧录和调试。

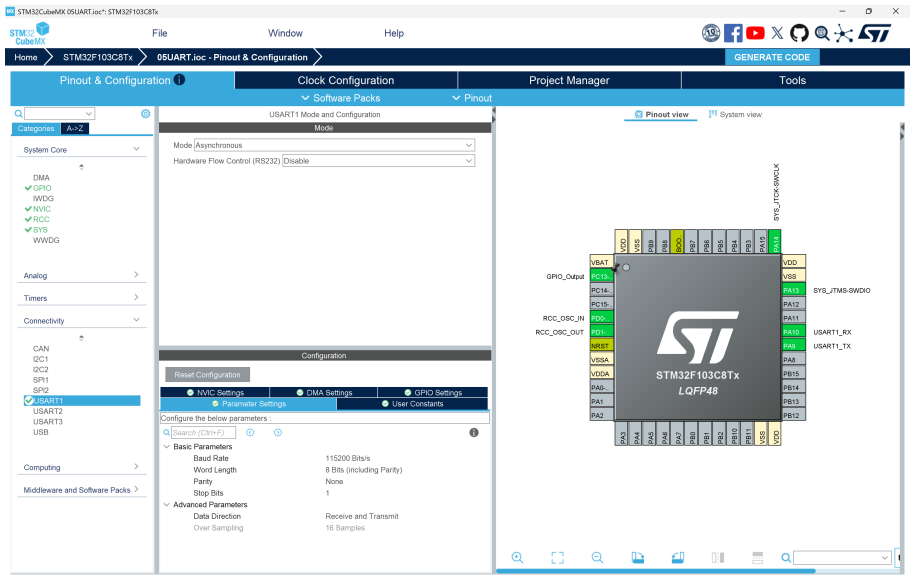


5.3.4 配置 USART1

1. 在 **Connectivity → USART1** 中
2. 将 **Mode** 设为 **Asynchronous**（异步模式）
3. 参数保持默认即可：

   - **Baud Rate:** 115200
   - **Word Length:** 8 Bits
   - **Parity:** None
   - **Stop Bits:** 1
   - **Hardware Flow Control:** None

关键概念：异步串口 **(Asynchronous UART)** 是最常见的串口模式，仅需 TX、RX 两根信号线。同步模式 (Synchronous USART) 还需 CLK 时钟线，本实验不涉及。

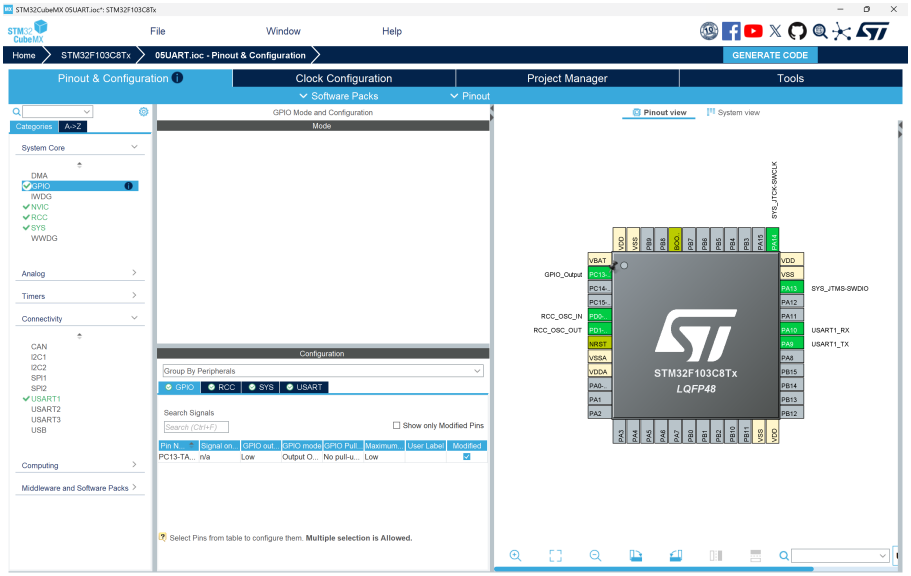


5.3.5 配置 GPIO（PC13 LED）

1. 在 **Pinout View** 中找到 **PC13** 引脚（通常在芯片图右侧）
2. 点击 PC13，在下拉菜单中将其设为 **GPIO_Output**
3. 在 **System Core → GPIO** 中找到 PC13，设置如下：

   - **GPIO output level:** Low
   - **GPIO mode:** Output Open Drain
   - **GPIO Pull-up/Pull-down:** No pull-up and no pull-down
   - **Maximum output speed:** Low

为什么要用开漏模式？ STM32F103C8T6 核心板的 LED 通常连接在 PC13 与 VDD 之间，因此 PC13 输出 低电平时 LED 点亮，输出 高阻态（开漏输出高）时 LED 熄灭。使用推挽输出（Push-Pull）也可以正常工作，但开漏模式在此场景下更安全，可避免因灌电流过大损坏引脚。

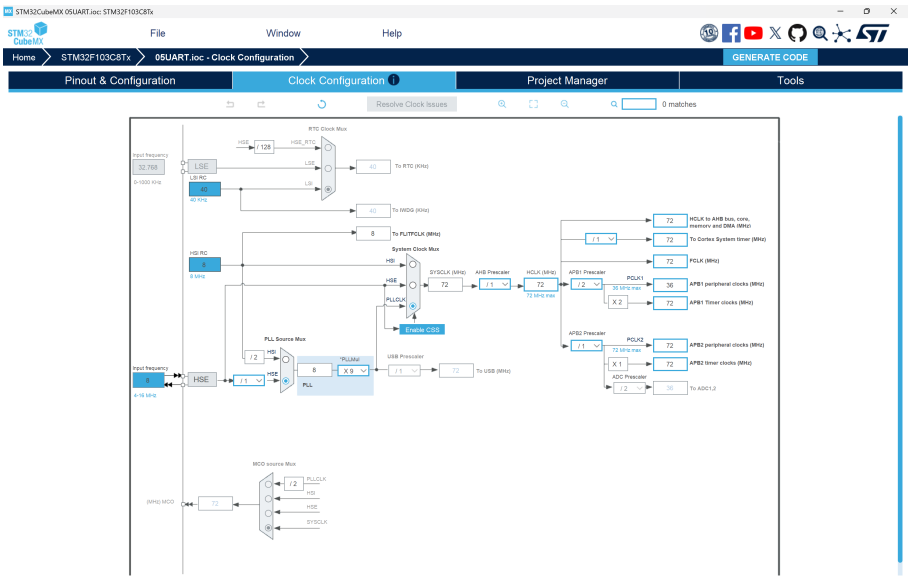


5.3.6 配置时钟树（Clock Configuration）

1. 点击顶部 **Clock Configuration** 标签
2. 按如下参数配置：

   - **HSE:** 8 MHz（外部晶振）
   - **PLL Source:** HSE
   - **PLL Mul:** x9 → 8 MHz × 9 = **72 MHz**
   - **System Clock Mux:** PLLCLK
   - **AHB Prescaler:** /1 → **HCLK = 72 MHz**
   - **APB1 Prescaler:** /2 → **APB1 = 36 MHz**
   - **APB2 Prescaler:** /1 → **APB2 = 72 MHz**

   为什么 **APB1** 要二分频？ STM32F103 的 APB1 总线最高频率为 36MHz，因此必须对 72MHz 的 AHB 进行二分频。而 APB2 总线最高可达 72MHz，所以无需分频。USART1 挂在 APB2 总线上。



5.3.7 配置工程输出

1. 点击 **Project Manager** 标签
2. **Project Name:** O5UART
3. **Project Location:** 选择你的工作目录
4. **Application Structure:** Basic
5. **Toolchain / IDE:** 选择 **STM32CubeIDE**

   注意：本工程采用 STM32CubeIDE 工具链，配合 VS Code 中的 **STM32CubeIDE for VS Code** 插件进行开发、编译和烧录。后文将介绍插件的安装与使用。

5.3.8 生成代码

1. 点击右上角 **GENERATE CODE** 按钮
2. 等待代码生成完成
3. 点击 **Open Project** 或直接进入工程目录

5.4 程序结构

```
O5UART/
├── O5UART.ioc                  # CubeMX 工程配置文件
├── CMakeLists.txt              # 顶层 CMake 构建文件
├── CMakePresets.json           # CMake 预设（Debug）
├── STM32F103XX_FLASH.ld        # 链接脚本（Flash/RAM 布局）
├── cmake/                      # CMake 工具链文件
│   ├── gcc-arm-none-eabi.cmake   #   ARM GCC 工具链配置
│   ├── starm-clang.cmake         #   Clang 工具链配置
│   └── stm32cubemx/
│       └── CMakeLists.txt        #   自动生成的子模块 CMake
├── Core/
│   ├── Inc/                    # 头文件
│   │   ├── main.h               #   主头文件（含 HAL 库引用）
│   │   ├── stm32f1xx_hal_conf.h  #   HAL 库配置文件
│   │   └── stm32f1xx_it.h        #   中断服务函数声明
│   └── Src/                    # 源文件
│       ├── main.c               #   * 主程序（用户代码在此）
│       ├── stm32f1xx_hal_msp.c   #   HAL 外设底层初始化（MSP）
│       ├── stm32f1xx_it.c        #   中断服务函数实现
│       ├── system_stm32f1xx.c    #   系统初始化
│       ├── sysmem.c              #   动态内存管理桩
│       └── syscalls.c            #   系统调用桩（_write 等）
├── Drivers/                    # HAL 驱动库
│   ├── CMSIS/                   #   ARM CMSIS 核心头文件
│   └── STM32F1xx_HAL_Driver/    #   STM32F1 HAL 库源码
```

```
└── startup_stm32f103xb.s          # 启动文件（汇编）
```

5.4.1 文件职责速览

文件	职责
main.c	用户程序入口，包含 main() 函数和业务逻辑
stm32f1xx_hal_msp.c	MSP（MCU Support Package）层：初始化外设对应的 GPIO 引脚和时钟
stm32f1xx_it.c	中断向量表实现，可在此编写中断回调
stm32f1xx_hal_conf.h	裁剪 HAL 库：启用/禁用各外设模块
system_stm32f1xx.c	SystemInit() 函数：上电后最早的时钟初始化
startup_stm32f103xb.s	启动汇编：初始化堆栈、跳转到 main()

5.5 核心代码详解

5.5.1 程序流程图

```
flowchart TD
    A[上电复位] --> B["HAL_Init()<br/> 初始化 HAL 库、SysTick"]
    B --> C["SystemClock_Config()<br/> 配置 HSE + PLL → 72MHz"]
    C --> D["MX_GPIO_Init()<br/> 初始化 PC13（LED 控制引脚）"]
    D --> E["MX_USART1_UART_Init()<br/> 初始化 USART1（115200-8-N-1）"]
    E --> F[接收一个字节 <br/> 超时 10ms]
    F -->| 收到 '0'| G["LED ON<br/> 发送 The light is on"]
    F -->| 收到 '1'| H["LED OFF<br/> 发送 The light is off"]
    F -->| 其他 / 超时 | I[忽略]
    G --> J[" 发送 HelloWorld!\\r\\n"]
    H --> J
    I --> J
    J --> K["HAL_Delay(1000ms)"]
    K --> F
```

5.5.2 代码逐段解析

5.5.2.1 HAL 库初始化与时钟配置

```
HAL_Init();              // 初始化 HAL 库，配置 SysTick 为 1ms 中断
SystemClock_Config();    // 配置系统时钟：HSE(8MHz) → PLL×9 → 72MHz
```

HAL_Init() 是所有 HAL 库程序的第一步，它会：- 设置 Flash 延迟周期（72MHz 需要 2 个等待周期）- 配置 SysTick 定时器产生 1ms 中断 - 初始化 NVIC 优先级分组

SystemClock_Config() 由 CubeMX 自动生成，依次配置 HSE、PLL、AHB/APB 总线的分频系数。

5.5.2.2 外设初始化

```
MX_GPIO_Init();          // 初始化 GPIO: PC13 设为开漏输出
MX_USART1_UART_Init();   // 初始化 USART1: 波特率 115200，8 数据位，无校验，1 停止位
```

这两个函数也是 CubeMX 自动生成的。MX_USART1_UART_Init() 内部调用 HAL_UART_Init()，后者会触发 HAL_UART_MspInit() 回调（位于 stm32f1xx_hal_msp.c），在回调中完成 GPIO 引脚复用和时钟使能。

5.5.2.3 用户变量声明

```
/* USER CODE BEGIN 2 */
uint8_t rx_buffer[1];  // 接收缓冲区，每次接收 1 字节
/* USER CODE END 2 */
```

为什么放在 **while** 循环外面？每次循环迭代都在栈上分配和释放 rx_buffer 虽然开销微小，但将其放在循环外部声明更加清晰高效。

5.5.2.4 UART 接收与命令处理

```
if (HAL_UART_Receive(&huart1, rx_buffer, 1, 10) == HAL_OK) {
    switch (rx_buffer[0]) {
        case '0':
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);  // LED ON
            HAL_UART_Transmit(&huart1, (uint8_t*)"The light is on\\r\\n", 17, 10);
            break;
        case '1':
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);    // LED OFF
            HAL_UART_Transmit(&huart1, (uint8_t*)"The light is off\\r\\n", 18, 10);
            break;
        default:
            break;
    }
}
```

关键细节：

- HAL_UART_Receive(&huart1, rx_buffer, 1, 10)：轮询模式接收，每次尝试读 1 个字节，超时时间 10ms。如果在 10ms 内收到数据，返回 HAL_OK，否则返回 HAL_TIMEOUT。
- HAL_UART_Transmit(&huart1, ..., 17, 10)：轮询模式发送，第三个参数是实际要发送的字节数。"The light is on\\r\\n" 正好 17 个字符，"The light is off\\r\\n" 正好 18 个字符。务必确保长度参数与实际字符串长度一致，否则会发送垃圾数据。
- **GPIO 控制 LED** 的逻辑：由于 LED 连接在 PC13 与 VDD 之间，且 PC13 配置为开漏输出——GPIO_PIN_RESET（低电平）点亮 LED，GPIO_PIN_SET（高阻态）熄灭 LED。

5.5.2.5 心跳消息与延时

```
HAL_UART_Transmit(&huart1, (uint8_t*)"HelloWorld!\\r\\n", 13, 1000);
HAL_Delay(1000);
```

每轮循环都会发送一条 "HelloWorld!\\r\\n" 心跳消息（13 字符），然后延时 1000ms。\\r\\n 是回车换行，确保串口助手中每条消息单独一行显示。

5.6 API 函数详解

5.6.1 HAL_UART_Receive

```
HAL_StatusTypeDef HAL_UART_Receive(
    UART_HandleTypeDef *huart,  // UART 句柄指针
    uint8_t           *pData,   // 接收数据缓冲区
    uint16_t           Size,    // 期望接收的字节数
    uint32_t           Timeout  // 超时时间（毫秒）
);
```

- 功能：在轮询模式下从 UART 接收指定数量的数据
- 返回值： HAL_OK（成功接收）、HAL_TIMEOUT（超时未收完）、HAL_ERROR（错误）
- 阻塞行为：在收到指定数量数据或超时前，CPU 将在此函数内等待
- 适用场景：简单、低速的通信；不适用于多任务或高实时性场景

5.6.2 HAL_UART_Transmit

```
HAL_StatusTypeDef HAL_UART_Transmit(
    UART_HandleTypeDef *huart,  // UART 句柄指针
    uint8_t           *pData,   // 待发送数据缓冲区
    uint16_t           Size,    // 发送字节数
    uint32_t           Timeout  // 超时时间（毫秒）
);
```

- 功能：在轮询模式下通过 UART 发送数据
- 注意： Size 参数必须与 pData 指向的数据的实际长度严格一致

5.6.3 HAL_GPIO_WritePin

```
void HAL_GPIO_WritePin(
    GPIO_TypeDef *GPIOx,        // GPIO 端口（如 GPIOC）
    uint16_t      GPIO_Pin,     // 引脚号（如 GPIO_PIN_13）
    GPIO_PinState PinState      // 输出状态：GPIO_PIN_RESET（低） / GPIO_PIN_SET（高）
);
```

5.6.4 HAL_Delay

```
void HAL_Delay(uint32_t Delay);  // 毫秒级延时（依赖 SysTick 中断）
```

5.7 使用说明

5.7.1 编译与烧录

本工程使用 **STM32CubeIDE for VS Code** 插件进行开发，无需手动配置 ARM GCC 工具链，插件会自动管理编译器和调试器依赖。

5.7.1.1 安装插件

1. 打开 VS Code，点击左侧 **Extensions** 图标（或按 `Ctrl+Shift+X`）
2. 搜索 **STM32CubeIDE**，找到 STMicroelectronics 官方发布的插件
3. 点击 **Install**，安装完成后重启 VS Code

5.7.1.2 导入工程

1. 启动 VS Code，打开本工程根目录（`05UART/` 文件夹）
2. 插件会自动识别 `.ioc` 文件和 `.cproject` 文件，加载 STM32CubeIDE 工程配置
3. 等待插件完成索引和构建配置（首次打开可能需要几分钟）

5.7.1.3 编译

点击 VS Code 底部状态栏的 **Build** 按钮（锤子图标），或使用快捷键 `Ctrl+Shift+B`。编译输出将显示在终端面板中，成功信息如下：

```
Finished building target: 05UART.elf
```

5.7.1.4 烧录（ST-Link）

1. 通过 ST-Link 调试器连接开发板与 PC
2. 点击底部状态栏的 **Run** 按钮（播放图标），或按下 `F5`
3. 插件将自动执行编译 → 烧录 → 运行，完成后程序开始在开发板上执行

   提示：如果烧录失败，检查 ST-Link 驱动是否已安装、连接是否正确。状态栏应可看到 ST-Link 设备被识别。

5.7.2 串口助手配置

1. 将 USB-TTL 模块连接到 PC
2. 打开串口助手（推荐使用 **SSCOM**、**PuTTY** 或 **MobaXterm**）
3. 配置参数：

参数	值
端口号	USB-TTL 对应的 COM 口
波特率	115200
数据位	8
校验位	None
停止位	1
流控	None

4. 点击 打开串口

5.7.3 预期现象

1. 串口助手中每隔约 1 秒收到一条 `HelloWorld!`
2. 在发送区输入字符 `0`，点击发送 → LED 点亮，同时收到 `The light is on`
3. 在发送区输入字符 `1`，点击发送 → LED 熄灭，同时收到 `The light is off`
4. 输入其他字符（如 `abc`、`2`）→ LED 状态不变，无额外响应

5.7.4 调试技巧

- 收不到数据? 检查 TX/RX 是否交叉连接 (STM32 TX → USB-TTL RX, STM32 RX ← USB-TTL TX)
- 收到乱码? 确认串口助手波特率设为 115200，与代码一致
- **LED** 不亮? 确认 PC13 配置为开漏输出（Open Drain），电平逻辑是反的（`RESET` = 亮）
- `HAL_UART_Receive` 一直超时? 这是正常的—-10ms 超时是设计行为。如果希望提高响应速度，可使用中断模式（`HAL_UART_Receive_IT`）或 **DMA** 模式

5.8 常见问题

5.8.1 Q1: 为什么不用中断或 DMA 接收 UART?

A: 本实验作为教学案例，首要目标是让初学者理解 HAL 库中 UART 最基本的轮询模式用法。轮询模式简单直观、易于调试。中断和 DMA 模式将在后续实验中引入。

5.8.2 Q2: 延时 1 秒会影响命令响应吗?

A: 会。当前设计下，串口接收仅在 10ms 超时窗口内生效，如果在这 10ms 内没有命令到达，就需要等待下一轮循环（约 1 秒后）。这意味着最坏情况下命令响应延迟为 ~1 秒。对于本教学实验这完全可接受，但在实际项目中应考虑中断或 DMA 方案。

5.8.3 Q3: PC13 开漏输出为什么这样控制 LED?

A: STM32F103C8T6 核心板（Blue Pill）的板载 LED 阳极接 VDD（3.3V），阴极通过限流电阻接 PC13。因此: - PC13 = 低电平 → LED 两端有电压差 → **LED** 亮 - PC13 = 高电平/高阻态 → 无电压差 → **LED** 灭

使用开漏输出（OD）而非推挽输出（PP）的好处是: 当 PC13 输出”高”时实际上是高阻态，不会向 LED 灌电流，对引脚更安全。

5.8.4 Q4: 如何使用虚拟串口（Virtual COM Port）?

A: 如果你的开发板自带 USB 转串口芯片（如 CH340G），只需一根 USB 线即可。否则需要外接 USB-TTL 模块。相关配置请参考课件中关于虚拟串口的章节。

5.9 扩展练习

1. 中断接收改造：将 `HAL_UART_Receive` 改为 `HAL_UART_Receive_IT`，在 `HAL_UART_RxCpltCallback` 回调中处理命令，观察响应速度的变化。
2. 多字符命令：支持更复杂的命令格式，如 `LED ON`、`LED OFF`（需要更大的接收缓冲区和字符串比较）。
3. 回显功能：将接收到的字符原样发回（echo），实现一个简易的串口回显终端。
4. **ADC** 数据上报：结合 ADC 外设，通过串口周期上报温度/电压等传感器数值。

5.10 参考资料

- STM32F103C8T6 数据手册
- STM32F1xx HAL 库用户手册 (UM1850)
- RM0008 – STM32F1xx 参考手册
- ARM GCC 工具链下载

6 STM32 UART 中断方式控制 LED

6.1 实验概述

本实验基于 **STM32F103C8T6** 微控制器，在上一实验（05UART 轮询模式）的基础上，将 USART1 接收方式从轮询（**Polling**）升级为中断（**Interrupt**）模式。用户通过串口助手发送命令控制板载 LED（PC13）的亮灭，MCU 无需在接收上阻塞等待，释放出来的 CPU 时间可用于心跳消息的周期性发送。

核心知识点： - STM32 UART 中断模式的配置与使用 - HAL 库中断接收 API: `HAL_UART_Receive_IT` / `HAL_UART_RxCpltCallback` - NVIC（嵌套向量中断控制器）的配置 - 中断方式相比轮询方式的优势

6.2 中断方式 vs 轮询方式

对比维度	轮询方式（Polling）	中断方式（Interrupt）
CPU 利用率	CPU 在 `HAL_UART_Receive()` 中阻塞等待，无法做其他事	CPU 启动接收后立即返回，可继续执行其他任务
响应延迟	取决于轮询周期（本实验中约 1 秒）	微秒级 – 数据到达立即触发中断处理
程序结构	单线程顺序执行，简单直观	主循环 + 中断回调，需要理解异步编程模型
适用场景	简单、低速、单任务场景	多任务、高实时性、需要 CPU 并行处理其他工作的场景
功耗	CPU 持续轮询，功耗较高	CPU 在无数据时可进入低功耗模式

为什么要从轮询升级到中断? 在轮询模式（05UART 实验）中，`HAL_UART_Receive()` 的超时等待会阻塞 CPU，命令响应延迟最坏可达 1 秒。中断模式让 CPU 在等待串口数据的同时继续执行心跳发送等任务 – 数据一到立即由硬件触发中断服务程序处理，响应速度大幅提升。

6.3 硬件需求

器件	说明
STM32F103C8T6 最小系统板（Blue Pill / 核心板）	主控 MCU
USB-TTL 串口模块（如 CH340 / CP2102）	与 PC 通信
ST-Link / J-Link 调试器	烧录程序
板载 LED（PC13）	被控指示灯

引脚连接:

STM32 引脚	功能	连接目标
PA9	USART1_TX	USB-TTL 模块 RX
PA10	USART1_RX	USB-TTL 模块 TX
PC13	GPIO_Output (LED)	板载 LED（低电平点亮）
PA13	SWDIO	ST-Link SWDIO
PA14	SWCLK	ST-Link SWCLK
GND	地	USB-TTL 模块 GND + ST-Link GND

6.4 STM32CubeMX 配置步骤

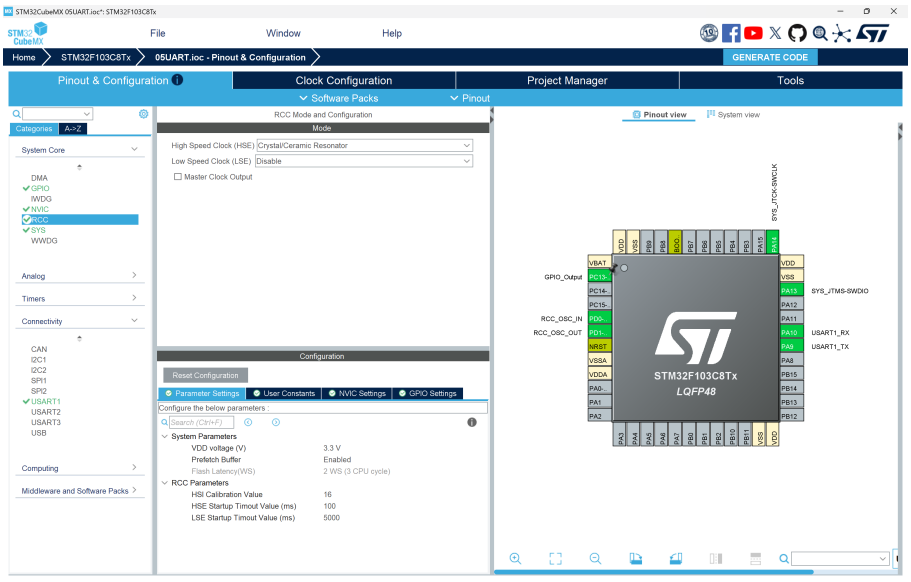
6.4.1 新建工程

1. 打开 **STM32CubeMX**
2. 点击 **File → New Project**
3. 在 **MCU Selector** 中搜索 `STM32F103C8Tx`，选中后点击 **Start Project**

6.4.2 配置时钟源（RCC）

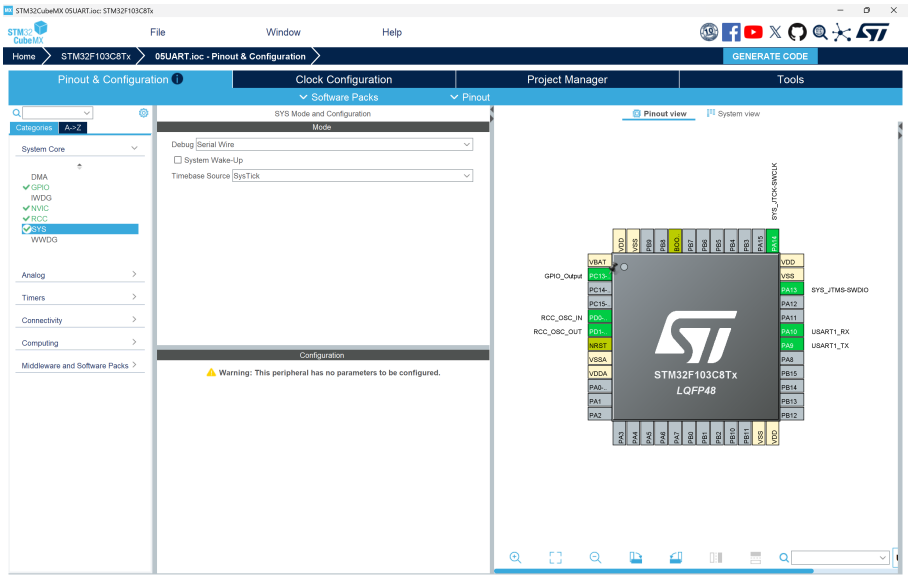
1. 在 **System Core → RCC** 中
2. 将 **High Speed Clock (HSE)** 设为 **Crystal/Ceramic Resonator**

STM32F103C8T6 核心板通常板载 8MHz 晶振，通过 HSE + PLL 倍频到 72MHz。



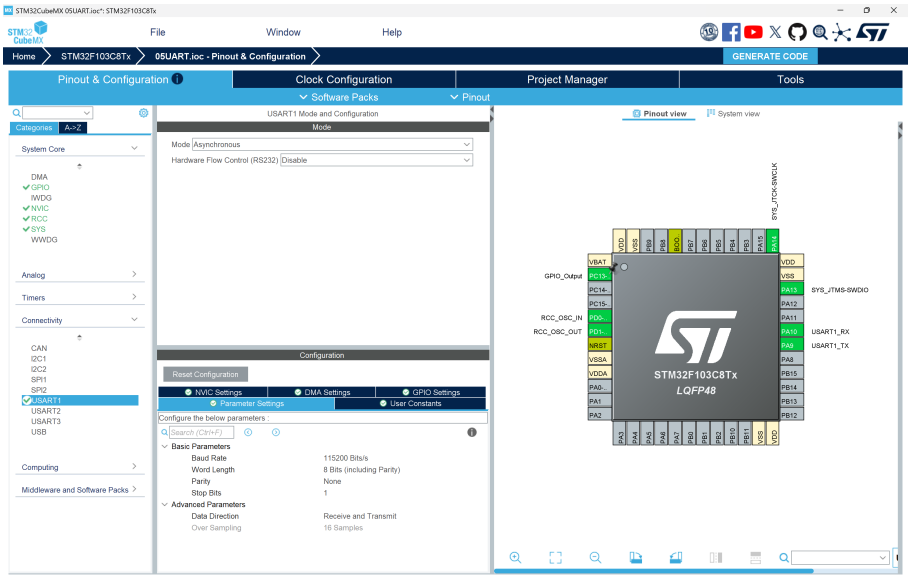
6.4.3 配置调试接口（SYS）

1. 在 **System Core → SYS** 中
2. 将 **Debug** 设为 **Serial Wire**（关闭 JTAG，仅保留 SWD）



6.4.4 配置 USART1

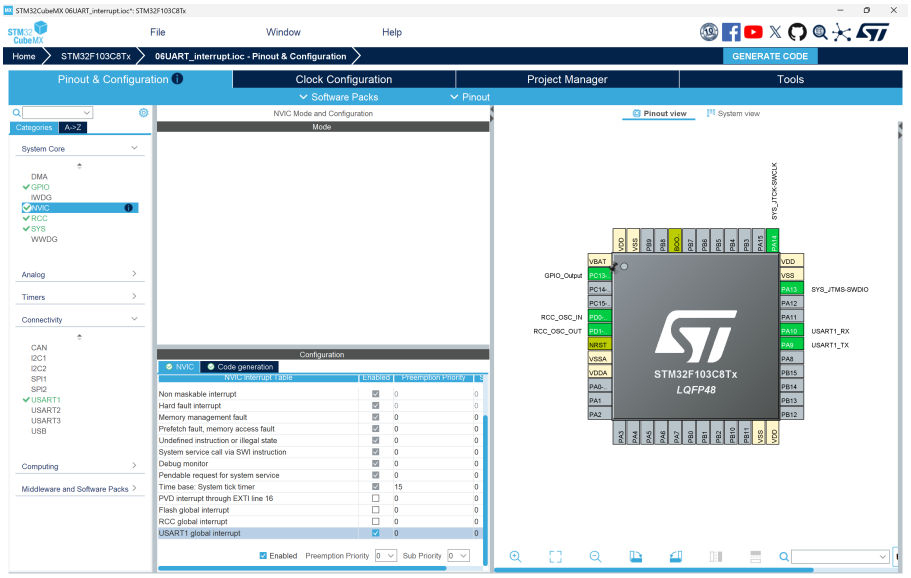
1. 在 **Connectivity → USART1** 中
2. 将 **Mode** 设为 **Asynchronous**（异步模式）
3. 参数保持默认：**Baud Rate: 115200, Word Length: 8, Parity: None, Stop Bits: 1**



6.4.5 使能 USART1 中断（NVIC）* 关键步骤

1. 在 **System Core → NVIC** 中
2. 找到 **USART1 global interrupt**，勾选 **Enabled** 列的复选框
3. 保持默认优先级（Preemption Priority: 0, Sub Priority: 0）

这是本次实验与轮询模式实验最关键的配置差异：必须使能 USART1 的全局中断，HAL 库的中断接收 API 才能正常工作。CubeMX 会自动生成 `USART1_IRQHandler()` 中断服务函数。

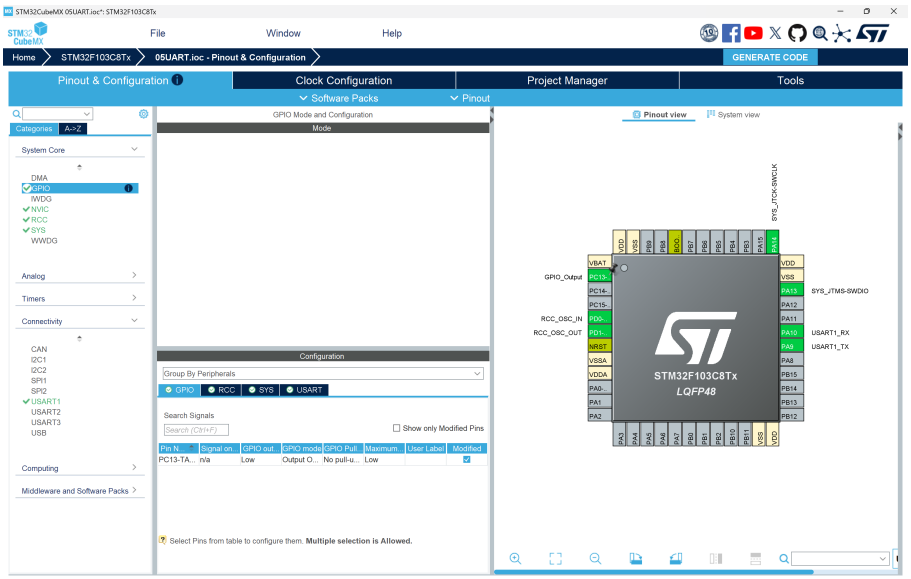


6.4.6 配置 GPIO（PC13 LED）

1. 将 **PC13** 设为 **GPIO_Output**
2. 在 GPIO 配置中设置：

   - **GPIO output level:** Low
   - **GPIO mode:** Output Open Drain
   - **GPIO Pull-up/Pull-down:** No pull-up and no pull-down
   - **Maximum output speed:** Low

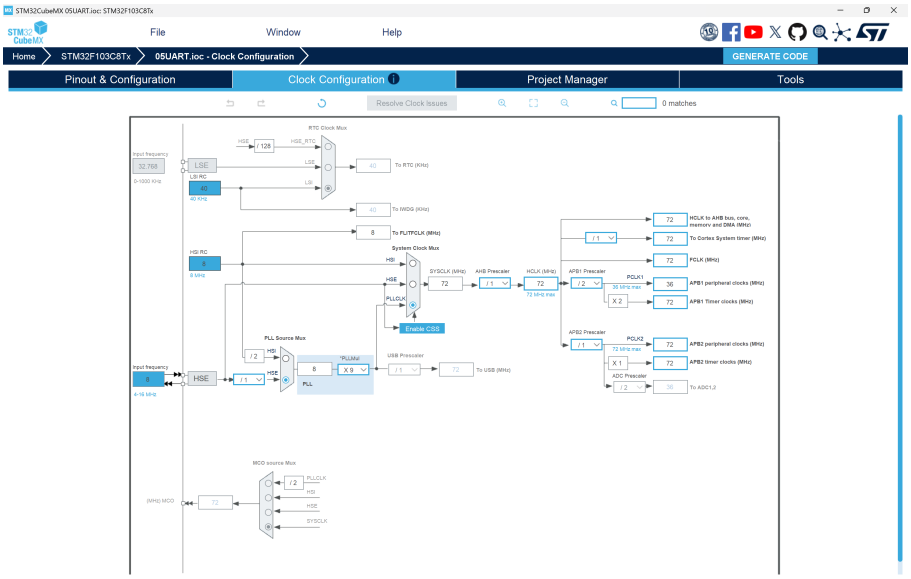
PC13 使用开漏输出：输出低电平时 LED 点亮，高阻态时 LED 熄灭。



6.4.7 配置时钟树（Clock Configuration）

1. 点击 **Clock Configuration** 标签
2. 配置参数：

   - **HSE:** 8 MHz
   - **PLL Source:** HSE，**PLL Mul:** x9 → 72 MHz
   - **System Clock Mux:** PLLCLK
   - **AHB Prescaler:** /1 → HCLK = 72 MHz
   - **APB1 Prescaler:** /2 → APB1 = 36 MHz
   - **APB2 Prescaler:** /1 → APB2 = 72 MHz（USART1 挂在此总线）



6.4.8 生成代码

1. 点击 **GENERATE CODE**
2. 确认 **Toolchain / IDE** 为 **STM32CubeIDE**

6.5 程序结构

```
06UART_interrupt/
├── 06UART_interrupt.ioc          # CubeMX 工程配置文件
├── CMakeLists.txt                 # 顶层 CMake 构建文件
├── CMakePresets.json              # CMake 预设
├── STM32F103XX_FLASH.ld           # 链接脚本
├── cmake/                         # CMake 工具链文件
├── Core/
│   ├── Inc/
│   │   ├── main.h                   # 主头文件
│   │   ├── stm32f1xx_hal_conf.h     # HAL 库配置
```

```
│   │   └── stm32f1xx_it.h          # 中断服务函数声明
│   └── Src/
│       ├── main.c                  # * 主程序（用户代码在此）
│       ├── stm32f1xx_hal_msp.c     # HAL 外设底层初始化
│       ├── stm32f1xx_it.c          # 中断服务函数实现（含 USART1_IRQHandler）
│       ├── system_stm32f1xx.c      # 系统初始化
│       ├── sysmem.c                # 动态内存管理桩
│       └── syscalls.c              # 系统调用桩
├── Drivers/                        # HAL 驱动库
│   ├── CMSIS/
│   └── STM32F1xx_HAL_Driver/
└── startup_stm32f103xb.s           # 启动文件
```

6.5.1 关键文件说明

文件	职责
main.c	主循环发送心跳 + 中断接收回调 HAL_UART_RxCpltCallback
stm32f1xx_it.c	USART1_IRQHandler() – CubeMX 自动生成，调用 HAL_UART_IRQHandler()
stm32f1xx_hal_msp.c	MSP 层：使能 USART1 时钟、配置 GPIO 复用、配置 NVIC

注意： stm32f1xx_it.c 中的 USART1_IRQHandler() 由 CubeMX 在使能 NVIC 后自动生成，不要手动修改其内部调用 HAL_UART_IRQHandler(&huart1) 的逻辑。

6.6 核心代码详解

6.6.1 程序流程图

```
flowchart TD
    A[上电复位] --> B["HAL_Init() / SystemClock_Config()"]
    B --> C["MX_GPIO_Init() / MX_USART1_UART_Init()"]
    C --> D["HAL_UART_Receive_IT()<br/> 启动中断接收（1 字节）"]
    D --> E[进入主循环]
    E --> F[" 发送心跳 HelloWorld!\\r\\n"]
    F --> G["HAL_Delay(1000ms)"]
    G --> E

    D -..-> H[" 硬件接收到 1 字节"]
    H --> I["USART1_IRQHandler()<br/>（stm32f1xx_it.c）"]
    I --> J["HAL_UART_IRQHandler()<br/>（HAL 库内部处理）"]
    J --> K["HAL_UART_RxCpltCallback()<br/>（main.c 用户回调）"]
    K --> L{rx_data?}
    L -->|'0'| M["LED ON + 回复 The light is on"]
    L -->|'1'| N["LED OFF + 回复 The light is off"]
    L -->| 其他 | O[忽略]
```

```
    M --> P["HAL_UART_Receive_IT()<br/> 重新启动中断接收"]
    N --> P
    O --> P
```

虚线路径表示中断触发路径，与主循环并行执行。主循环只负责心跳发送，命令处理完全由中断驱动。

6.6.2 代码逐段解析

6.6.2.1 全局变量声明

```c
/* USER CODE BEGIN PV */
uint8_t rx_data;
/* USER CODE END PV */
```

rx_data 是中断接收的目标缓冲区。因为中断方式单字节接收，只需要 1 字节即可。变量定义为全局变量，以便中断回调函数 HAL_UART_RxCpltCallback() 访问。

6.6.2.2 启动中断接收

```c
/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart1, &rx_data, 1);
/* USER CODE END 2 */
```

在外设初始化完成后调用 HAL_UART_Receive_IT() 启动第一次中断接收。此函数非阻塞 – 它配置好接收参数后立即返回，之后每当 USART1 收到 1 字节数据时，硬件自动触发中断，由 HAL 库在回调中通知用户。

6.6.2.3 主循环 – 心跳发送

```c
while (1)
{
    HAL_UART_Transmit(&huart1, (uint8_t *)"HelloWorld!\\r\\n", 13, 1000);
    HAL_Delay(1000);
}
```

主循环只做两件事：每隔 1 秒发送一条心跳消息，然后延时。与轮询模式不同，主循环中没有任何接收代码 – 接收工作完全由中断驱动，不占用主循环的 CPU 时间。

6.6.2.4 中断接收回调 * 核心

```c
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        switch (rx_data)
        {
            case '0':
                HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);  // LED ON
                HAL_UART_Transmit(&huart1, (uint8_t *)"The light is on\\r\\n", 17, 100);
                break;
            case '1':
                HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);    // LED OFF
                HAL_UART_Transmit(&huart1, (uint8_t *)"The light is off\\r\\n", 18, 100);
                break;
```

```
            default:
                break;
        }
        HAL_UART_Receive_IT(&huart1, &rx_data, 1);  // 重新启动下一次接收
    }
}
```

关键细节：

- HAL_UART_RxCpltCallback() 是 HAL 库定义的弱函数（__weak），用户在 main.c 中重写它以处理接收完成事件。当 USART1 成功接收指定字节数后，HAL 库自动调用此回调。
- huart->Instance == USART1 判断用于区分多个 UART 外设的场景（本实验仅使用 USART1）。
- switch (rx_data) 根据接收到的字符执行相应操作：'0' 点亮 LED，'1' 熄灭 LED。
- 重新启动接收：回调末尾必须再次调用 HAL_UART_Receive_IT()，否则下一次接收不会触发中断。这是中断接收模式的持续运行机制 – 每次接收完成后手动重启。

6.6.2.5 中断服务函数（CubeMX 自动生成）

```
// stm32f1xx_it.c
void USART1_IRQHandler(void)
{
    HAL_UART_IRQHandler(&huart1);
}
```

USART1_IRQHandler() 是硬件中断向量的入口。当 USART1 产生中断（如 RXNE – 接收寄存器非空）时，CPU 跳转至此函数，调用 HAL 库的 HAL_UART_IRQHandler() 进行中断状态判断和数据搬运。HAL 库内部处理完毕后，自动调用 HAL_UART_RxCpltCallback()。

6.7 API 函数详解

6.7.1 HAL_UART_Receive_IT

```
HAL_StatusTypeDef HAL_UART_Receive_IT(
    UART_HandleTypeDef *huart,  // UART 句柄指针
    uint8_t           *pData,   // 接收数据缓冲区
    uint16_t           Size     // 期望接收的字节数
);
```

- 功能：以中断模式启动 UART 接收
- 行为：函数立即返回（非阻塞），之后由硬件中断自动完成数据接收
- 完成通知：接收完成后 HAL 库调用 HAL_UART_RxCpltCallback() 回调
- 注意：每次接收完成后需重新调用此函数以继续接收

6.7.2 HAL_UART_RxCpltCallback

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
```

- 功能：中断接收完成的用户回调函数
- 触发时机： HAL 库内部处理完 RXNE 中断并收齐指定字节数后
- 使用方式：用户在自己的代码中重写此弱函数（__weak）
- 注意：此函数在中断上下文中执行，应尽量简短，避免耗时操作

6.7.3 中断数据流总结

```
硬件 RX 引脚收到数据
  → USART 外设产生 RXNE 中断
    → NVIC 向量表分发到 USART1_IRQHandler()    [stm32f1xx_it.c]
      → HAL_UART_IRQHandler(&huart1)            [HAL 库内部]
        → 数据从 DR 寄存器搬运到 pData 缓冲区
        → 收齐 Size 字节后调用 HAL_UART_RxCpltCallback()  [main.c 用户回调]
          → 处理命令 + HAL_UART_Receive_IT() 重新启动
```

6.8 中断方式的优势总结

6.8.1 响应速度

- 轮询模式：命令响应延迟 $= 0 \sim$ 轮询周期（本实验中 ~ 1 秒）
- 中断模式：命令响应延迟 $=$ 中断延迟（微秒级）

6.8.2 CPU 利用率

- 轮询模式：即使没有数据，CPU 也在 HAL_UART_Receive() 中阻塞等待（忙等）
- 中断模式： CPU 启动接收后立即返回，可并行处理心跳发送、传感器采集、数据处理等

6.8.3 程序扩展性

- 轮询模式：添加更多功能（如多个传感器、多路通信）时，主循环越来越复杂，响应变慢
- 中断模式：每个外设的中断独立处理，主循环保持简洁，系统易于扩展

6.8.4 功耗

- 轮询模式： CPU 持续运行，无法进入睡眠模式
- 中断模式：无数据时 CPU 可进入低功耗睡眠模式（WFI/WFE），外设中断唤醒 CPU 处理数据后继续睡眠 – 特别适合电池供电的嵌入式设备

6.9 使用说明

6.9.1 编译与烧录

使用 VS Code + **STM32CubeIDE for VS Code** 插件:

1. 打开工程根目录
2. 点击底部状态栏 **Build**（锤子图标）编译
3. 连接 ST-Link，点击 **Run**（播放图标）烧录

6.9.2 串口助手配置

参数	值
端口号	USB-TTL 对应 COM 口
波特率	115200
数据位	8
校验位	None
停止位	1
流控	None

6.9.3 预期现象

1. 串口助手中每隔约 1 秒收到一条 `HelloWorld!`
2. 发送 `0` → LED 立即点亮，同时收到 `The light is on`
3. 发送 `1` → LED 立即熄灭，同时收到 `The light is off`
4. 输入其他字符 → LED 状态不变
5. 与轮询模式的对比：无论心跳发送周期多长，命令响应都几乎即时，因为接收由中断驱动，不依赖主循环轮询

6.9.4 调试技巧

- 收不到数据? 检查 TX/RX 交叉连接、NVIC 中 USART1 中断是否勾选
- 只响应一次就不再响应? 检查 `HAL_UART_RxCpltCallback()` 末尾是否调用了 `HAL_UART_Receive_IT()` 重新启动
- 程序卡死? 检查 `HAL_UART_RxCpltCallback()` 中是否使用了阻塞发送函数且超时时间过长
- **LED** 不亮? 确认 PC13 配置为开漏输出，`GPIO_PIN_RESET`（低电平）= 点亮

6.10 常见问题

6.10.1 Q1: 为什么回调函数中还要用轮询发送 HAL_UART_Transmit，不也用中断发送吗?

A: 为了聚焦核心知识点，本实验仅在接收端引入中断模式。中断发送（`HAL_UART_Transmit_IT`）留作扩展练习。在实际项目中，对实时性要求较高的场景可以两路都用中断。

6.10.2 Q2: HAL_UART_Receive_IT 可以一次接收多个字节吗?

A: 可以。第三个参数 Size 指定期望接收的字节数，只有收齐后才触发 HAL_UART_RxCpltCallback。例如设为 4 可实现 4 字节命令帧的接收。

6.10.3 Q3: 多个 UART 同时使用中断模式，回调怎么区分?

A: 通过 huart->Instance 判断是哪个 UART 触发的回调:

```
if (huart->Instance == USART1) { /* USART1 */ }
else if (huart->Instance == USART2) { /* USART2 */ }
```

6.10.4 Q4: 中断接收会不会丢失数据?

A: 在波特率 115200 及以下，且回调函数执行时间极短的情况下，基本不会丢数据。如果需要在高速通信中保证可靠性，可考虑使用 DMA 模式（后续实验将涉及）。

6.11 扩展练习

1. 中断发送改造：将心跳发送改为 HAL_UART_Transmit_IT()，观察程序行为变化。
2. 多字节命令：修改接收长度为多个字节，实现 LED ON、LED OFF 等字符串命令的识别。
3. 环形缓冲区：实现一个环形接收缓冲区，支持更复杂的协议解析。
4. 空闲中断（**IDLE**）：使用 USART 的 IDLE 中断检测帧结束，实现不定长数据包的接收。

6.12 参考资料

- STM32F103C8T6 数据手册
- STM32F1xx HAL 库用户手册 (UM1850)
- RM0008 – STM32F1xx 参考手册

7 STM32 I2C OLED 显示实验

7.1 实验概述

本实验基于 **STM32F103C8T6** 微控制器，通过 **I2C1** 总线驱动 **SSD1306 OLED** 显示屏（**128×64**），实现字符串显示与滚动效果。

核心知识点： - I2C 总线协议基础（起始/停止条件、从机地址、ACK 应答）- STM32 I2C 外设的配置与使用 - HAL 库 I2C 主机发送 API：`HAL_I2C_Master_Transmit` / `HAL_I2C_Mem_Write` - SSD1306 OLED 控制器的工作原理与命令集 - 5×8 点阵字体的渲染方式 - 滚动显示的实现：硬件滚动 vs 软件逐帧滚动

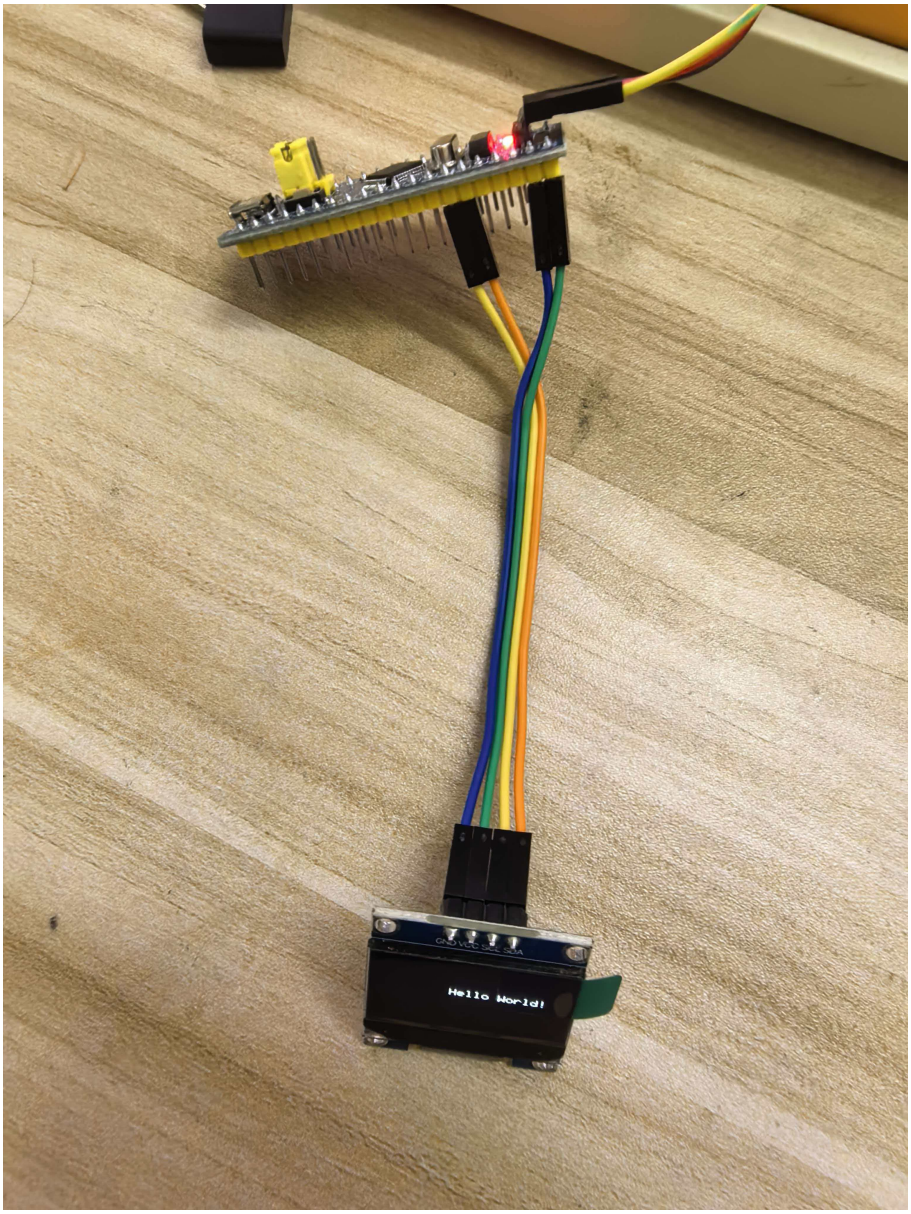
7.2 硬件需求

器件	说明
STM32F103C8T6 最小系统板	主控 MCU
0.96 寸 SSD1306 OLED 模块（I2C 接口，4 引脚）	显示设备
ST-Link / J-Link 调试器	烧录程序
面包板 + 杜邦线	连接电路

引脚连接：

STM32 引脚	OLED 模块引脚	功能
PB6	SCL	I2C1 时钟线
PB7	SDA	I2C1 数据线
3.3V	VCC	电源正极
GND	GND	电源地
PA13	–	SWDIO（ST-Link）
PA14	–	SWCLK（ST-Link）

> 注意：大多数 0.96 寸 SSD1306 OLED 模块已板载 I2C 上拉电阻（4.7kΩ），无需额外焊接。如果使用裸屏或通信不稳定，需要在 SCL 和 SDA 上各加 4.7kΩ 上拉电阻至 3.3V。



图示为 OLED 模块与开发板的实物连接参考。

7.3 STM32CubeMX 配置步骤

7.3.1 新建工程

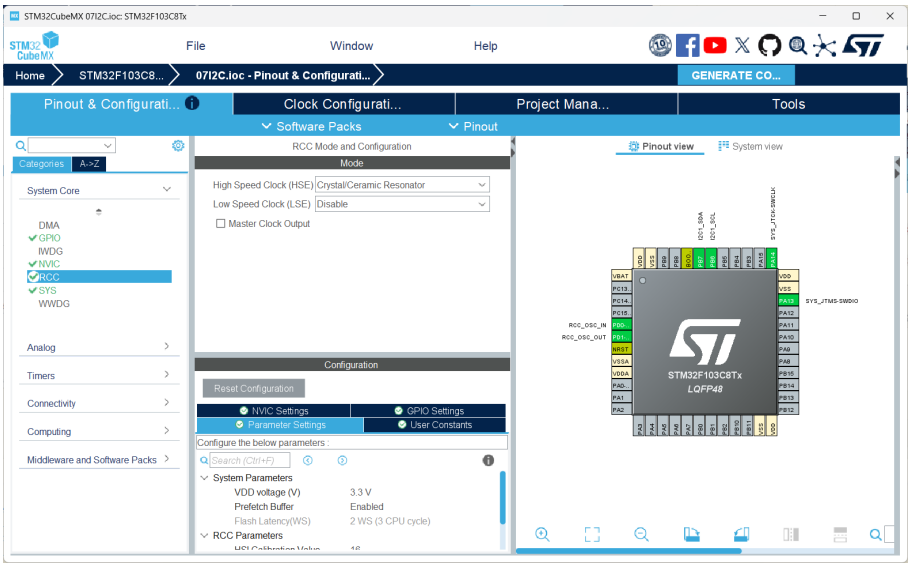
1. 打开 **STM32CubeMX**（本工程使用 6.17.0 版本）
2. 点击 **File → New Project**
3. 在 **MCU Selector** 中搜索 `STM32F103C8Tx`，选中后点击 **Start Project**

7.3.2 配置时钟源（RCC）

1. 点击左侧 **Pinout & Configuration** 标签

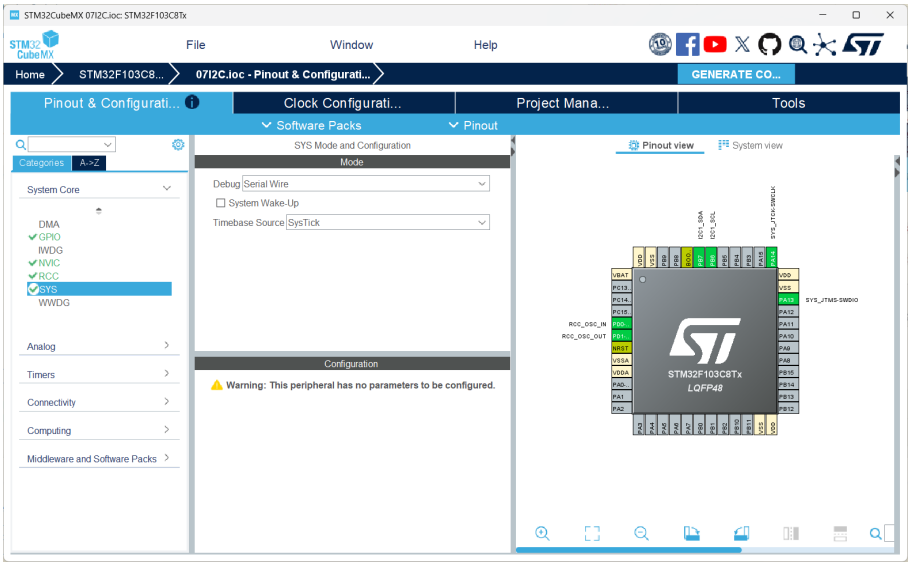
2. 在 **System Core → RCC** 中
3. 将 **High Speed Clock (HSE)** 设为 **Crystal/Ceramic Resonator**（外部晶振）

说明： STM32F103C8T6 核心板通常板载 8MHz 晶振，通过 HSE + PLL 倍频到 72MHz。



7.3.3 配置调试接口（SYS）

1. 在 **System Core → SYS** 中
2. 将 **Debug** 设为 **Serial Wire**（关闭 JTAG，仅保留 SWD）



> 为什么要这样做？ JTAG 默认占用 PA15、PB3、PB4 三个引脚。关闭 JTAG 仅保留 SWD，既可烧录调试，又不浪费 GPIO 资源。

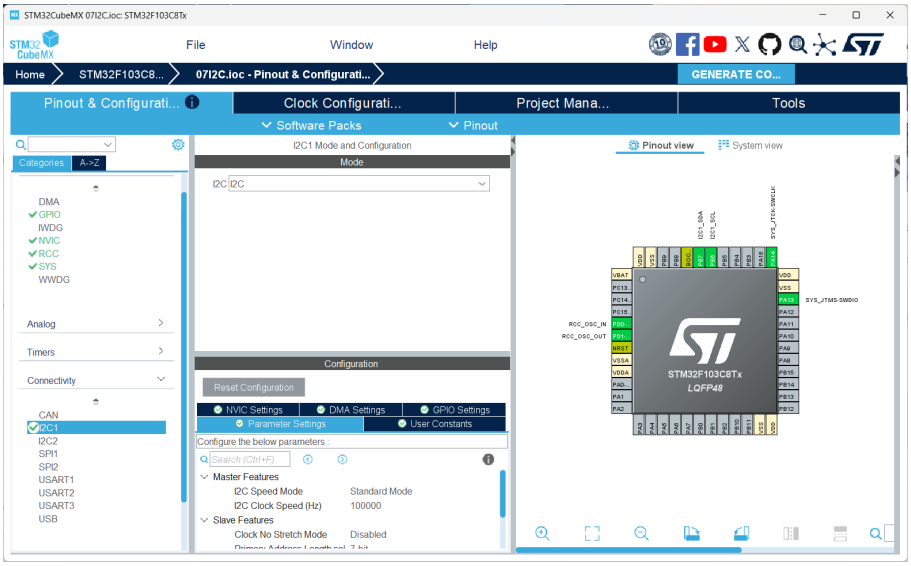
7.3.4 配置 I2C1

1. 在 **Connectivity → I2C1** 中
2. 将 **Mode** 设为 **I2C**
3. **I2C Speed Mode:** Standard Mode（100kHz）
4. 其他参数保持默认：

参数	值	说明
I2C Speed Mode	Standard Mode	100kHz 标准速率
Clock Speed	100000	100kHz 时钟
Duty Cycle	DutyCycle_2	标准模式忽略此参数
Addressing Mode	7-bit	7 位地址模式
Own Address1	0	本机不做从机，设为 0

关键概念：**I2C** 速率选择。 SSD1306 支持 100kHz（标准模式）和 400kHz（快速模式）。100kHz 兼容性最好，对布线要求低；400kHz 刷新更快但需要较短走线和更强的上拉。本实验使用 100kHz。

为什么 **OwnAddress1** 设为 **0**？ STM32 在本次实验中只作为 I2C 主机主动发送数据，不接收来自其他设备的 I2C 请求，因此不需要分配从机地址。

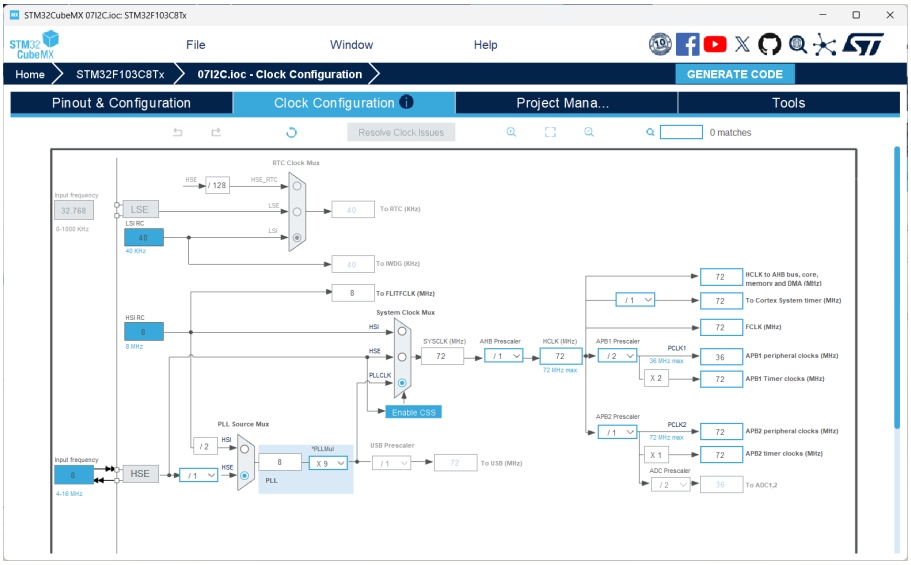


7.3.5 配置时钟树（Clock Configuration）

1. 点击顶部 **Clock Configuration** 标签
2. 按如下参数配置：

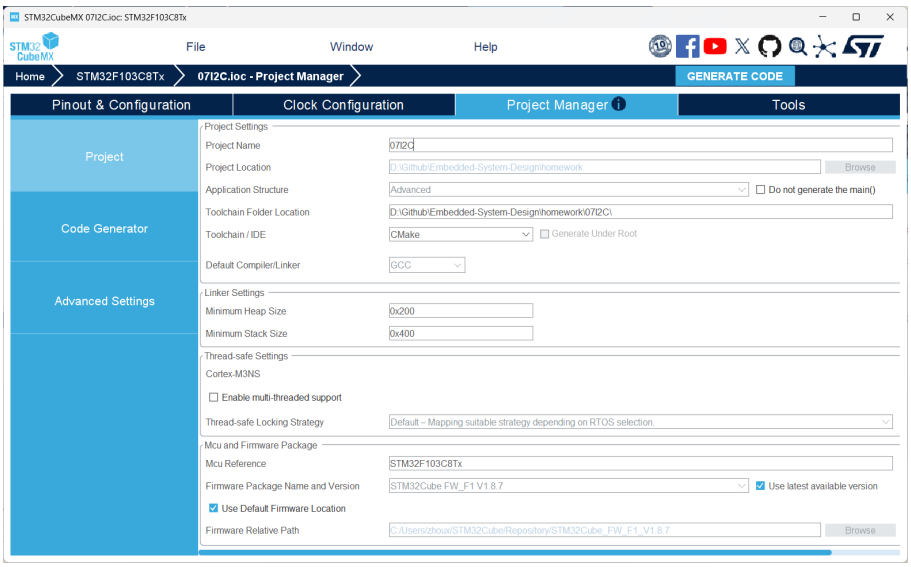
   - **HSE:** 8 MHz（外部晶振）
   - **PLL Source:** HSE
   - **PLL Mul:** x9 → 8 MHz × 9 = **72 MHz**
   - **System Clock Mux:** PLLCLK
   - **AHB Prescaler:** /1 → **HCLK = 72 MHz**
   - **APB1 Prescaler:** /2 → **APB1 = 36 MHz**
   - **APB2 Prescaler:** /1 → **APB2 = 72 MHz**

为什么 **APB1** 要二分频？ STM32F103 的 APB1 总线最高频率为 36MHz，必须对 72MHz 的 AHB 进行二分频。I2C1 挂在 APB1 上，其输入时钟为 36MHz，HAL 库会自动计算分频系数使 SCL 达到设定的 100kHz。



7.3.6 配置工程输出

1. 点击 **Project Manager** 标签
2. **Project Name:** 07I2C
3. **Project Location:** 选择你的工作目录
4. **Application Structure:** Basic
5. **Toolchain / IDE:** 选择 **CMake**（配合 VS Code + ARM GCC 使用）



7.3.7 生成代码

1. 点击右上角 **GENERATE CODE** 按钮
2. 等待代码生成完成
3. 点击 **Open Project** 或直接进入工程目录

7.4 程序结构

```
07I2C/
├── 07I2C.ioc                    # CubeMX 工程配置文件
├── CMakeLists.txt               # 顶层 CMake 构建文件
├── CMakePresets.json            # CMake 预设（Debug）
├── STM32F103XX_FLASH.ld         # 链接脚本（Flash/RAM 布局）
├── cmake/                       # CMake 工具链文件
│   ├── gcc-arm-none-eabi.cmake  #   ARM GCC 工具链配置
│   ├── starm-clang.cmake        #   Clang 工具链配置
│   └── stm32cubemx/
│       └── CMakeLists.txt       #   自动生成的子模块 CMake
├── Core/
│   ├── Inc/                     # 头文件
│   │   ├── main.h               #   主头文件（含 HAL 库引用）
│   │   ├── stm32f1xx_hal_conf.h #   HAL 库配置文件
│   │   └── stm32f1xx_it.h       #   中断服务函数声明
│   └── Src/                     # 源文件
│       ├── main.c               #   * 主程序（用户代码在此）
│       ├── stm32f1xx_hal_msp.c  #   HAL 外设底层初始化（MSP）
│       ├── stm32f1xx_it.c       #   中断服务函数实现
│       ├── system_stm32f1xx.c   #   系统初始化
│       ├── sysmem.c             #   动态内存管理桩
│       └── syscalls.c           #   系统调用桩（_write 等）
├── Drivers/                     # HAL 驱动库
│   ├── CMSIS/                   #   ARM CMSIS 核心头文件
│   └── STM32F1xx_HAL_Driver/    #   STM32F1 HAL 库源码
├── startup_stm32f103xb.s        # 启动文件（汇编）
└── img/                         # 文档截图
```

7.4.1 文件职责速览

文件	职责
main.c	用户程序入口，包含 OLED 驱动和业务逻辑
stm32f1xx_hal_msp.c	MSP 层：初始化 I2C1 对应的 GPIO（PB6/PB7 开漏复用）和时钟
stm32f1xx_it.c	中断向量表实现
stm32f1xx_hal_conf.h	裁剪 HAL 库：启用/禁用各外设模块
system_stm32f1xx.c	SystemInit() 函数：上电后最早的时钟初始化

7.5 核心代码详解

7.5.1 程序流程图

```
flowchart TD
    A[上电复位] --> B["HAL_Init()<br/> 初始化 HAL 库、SysTick 1ms 中断"]
    B --> C["SystemClock_Config()<br/>HSE(8MHz) → PLL×9 → 72MHz"]
    C --> D["MX_GPIO_Init()<br/> 使能 GPIOD / GPIOA / GPIOB 时钟"]
    D --> E["MX_I2C1_Init()<br/>I2C1: 100kHz, PB6(SCL) / PB7(SDA)"]
    E --> F["ssd1306_Init()<br/> 发送 SSD1306 初始化命令序列"]
    F --> G["ssd1306_Clear()<br/> 清空 GDDRAM（8 页 × 128 列）"]
    G --> H["ssd1306_ScrollMarquee()<br/> 逐帧绘制文字 → 整页写入 <br/> 偏移从 +128 递减到 -文字宽度"]
    H --> I["HAL_Delay(500ms)"]
    I --> H
```

7.5.2 I2C 协议基础

在阅读代码之前，先了解 I2C 总线的基本概念：

物理层： I2C 仅需两根线——SCL（时钟）和 SDA（数据）。两根线均通过上拉电阻接 VCC，设备通过开漏输出拉低总线来发送数据。

通信过程（主机发送）： 1. 主机发送起始条件（SCL=H 时 SDA: H→L）2. 主机发送 **7** 位从机地址 **+ 1** 位方向位（**0=** 写） 3. 从机回应 **ACK**（拉低 SDA 一个时钟周期）4. 主机发送数据字节，每个字节后从机返回 ACK 5. 主机发送停止条件（SCL=H 时 SDA: L→H）

SSD1306 的 I2C 帧格式：

SSD1306 的 I2C 通信使用控制字节区分命令和数据： - 控制字节 = `0x00` → 后续字节是命令 - 控制字节 = `0x40` → 后续字节是数据（写入 GDDRAM）

```
发送命令：START + [SlaveAddr + W] + 0x00 + [Command] + STOP
             └─── ACK ──┘     └ACK┘   └─── ACK ──┘

发送数据：START + [SlaveAddr + W] + 0x40 + [Data0] + [Data1] + ... + STOP
             └─── ACK ──┘     └ACK┘   └── ACK ──┘   └── ACK ──┘
```

7.5.3 I2C 地址说明

STM32 HAL 库要求将 **7** 位 **I2C** 地址左移 **1** 位后传入 API。常见 SSD1306 模块的 7 位地址为 `0x3C`（SA0 接地），对应 HAL 地址 `0x78`。

7 位地址	HAL 地址（« 1）	适用场景
0x3C	0x78	大部分 OLED 模块（SA0=0）
0x3D	0x7A	SA0 接 VCC 的模块

排查技巧：如果显示屏不响应，用 I2C 扫描程序确认实际地址，或直接在这两个值之间切换尝试。

7.5.4 代码逐段解析

7.5.4.1 HAL 库初始化与时钟配置

```
HAL_Init();              // 初始化 HAL 库，配置 SysTick 为 1ms 中断
SystemClock_Config();    // 配置系统时钟：HSE(8MHz) → PLL×9 → 72MHz
```

HAL_Init() 是所有 HAL 库程序的第一步，它会：- 设置 Flash 等待周期（72MHz 需要 2 个等待周期）- 配置 SysTick 定时器产生 1ms 中断 - 初始化 NVIC 优先级分组

7.5.4.2 外设初始化

```
MX_GPIO_Init();          // 使能 GPIO 端口时钟
MX_I2C1_Init();          // 初始化 I2C1：100kHz 标准模式
```

MX_I2C1_Init() 由 CubeMX 自动生成，调用 HAL_I2C_Init() 后会触发 HAL_I2C_MspInit() 回调（位于 stm32f1xx_hal_msp.c），在该回调中完成：- 使能 GPIOB 时钟 - 将 PB6、PB7 配置为复用开漏输出（GPIO_MODE_AF_OD）- 使能 I2C1 外设时钟

```
GPIO_InitStruct.Pin = GPIO_PIN_6|GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_AF_OD;   // 开漏复用----I2C 必须用开漏
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

> 为什么必须是开漏输出？ I2C 总线的”线与”特性要求所有设备只能拉低总线而不能主动拉高。开漏输出完美满足这一要求——输出低电平时拉低总线，输出高电平时引脚变为高阻态，由上拉电阻将总线拉高。如果使用推挽输出，当两个设备同时输出不同电平时会造成短路。

7.5.4.3 SSD1306 命令与数据发送

```
void ssd1306_WriteCommand(uint8_t cmd)
{
    uint8_t buf[2] = {SSD1306_CMD, cmd};
    HAL_I2C_Master_Transmit(&hi2c1, SSD1306_ADDR, buf, 2, HAL_MAX_DELAY);
}

void ssd1306_WriteData(uint8_t *data, uint16_t len)
{
    HAL_I2C_Mem_Write(&hi2c1, SSD1306_ADDR, SSD1306_DATA,
                      I2C_MEMADD_SIZE_8BIT, data, len, HAL_MAX_DELAY);
}
```

关键细节：

- 命令帧：[0x00][cmd] —- 控制字节 0x00 告知 SSD1306 下一个字节是命令
- 数据帧：[0x40][data...] —- 控制字节 0x40 告知 SSD1306 后续字节是显示数据
- HAL_I2C_Master_Transmit vs HAL_I2C_Mem_Write：
  - Master_Transmit：发送任意字节序列，适合简单的命令 + 数据场景
  - Mem_Write：自动在数据前插入一个”内存地址”字节（即控制字节 0x40），适合批量写 GDDRAM
- HAL_MAX_DELAY：无限等待直到传输完成，简单可靠

7.5.4.4 SSD1306 初始化序列

```
void ssd1306_Init(void)
{
    HAL_Delay(10);  // 等待 OLED 上电稳定

    ssd1306_WriteCommand(SSD1306_DISPLAYOFF);       // 0xAE: 关闭显示

    ssd1306_WriteCommand(SSD1306_SETDISPLAYCLOCKDIV);// 0xD5: 设置显示时钟
    ssd1306_WriteCommand(0x80);                      // 分频比 =1, 振荡频率 =0

    ssd1306_WriteCommand(SSD1306_SETMULTIPLEX);      // 0xA8: 设置复用比
    ssd1306_WriteCommand(0x3F);                      // 64 行 (0~63)

    ssd1306_WriteCommand(SSD1306_SETDISPLAYOFFSET);  // 0xD3: 显示偏移
    ssd1306_WriteCommand(0x00);                      // 无偏移

    ssd1306_WriteCommand(SSD1306_SETSTARTLINE | 0x00);// 0x40: 起始行 0

    ssd1306_WriteCommand(SSD1306_CHARGEPUMP);        // 0x8D: 电荷泵
    ssd1306_WriteCommand(0x14);                      // 使能电荷泵 (外部 3.3V 供电)

    ssd1306_WriteCommand(SSD1306_MEMORYMODE);        // 0x20: 内存寻址模式
    ssd1306_WriteCommand(0x00);                      // 水平模式

    ssd1306_WriteCommand(SSD1306_SEGREMAP);          // 0xA1: 列重映射 (左右镜像)
    ssd1306_WriteCommand(SSD1306_COMSCANDEC);        // 0xC8: COM 扫描方向 (上下翻转)

    ssd1306_WriteCommand(SSD1306_SETCOMPINS);        // 0xDA: COM 引脚配置
    ssd1306_WriteCommand(0x12);                      // 128x64 模式

    ssd1306_WriteCommand(SSD1306_SETCONTRAST);       // 0x81: 对比度
    ssd1306_WriteCommand(0x7F);                      // 中等对比度

    ssd1306_WriteCommand(SSD1306_SETPRECHARGE);      // 0xD9: 预充电周期
    ssd1306_WriteCommand(0x22);

    ssd1306_WriteCommand(SSD1306_SETVCOMDETECT);     // 0xDB: VCOMH 电压
    ssd1306_WriteCommand(0x20);

    ssd1306_WriteCommand(SSD1306_DISPLAYALLON_RESUME);// 0xA4: 正常显示模式
    ssd1306_WriteCommand(SSD1306_NORMALDISPLAY);      // 0xA6: 非反色
    ssd1306_WriteCommand(SSD1306_DEACTIVATE_SCROLL);  // 0x2E: 关闭滚动
    ssd1306_WriteCommand(SSD1306_DISPLAYON);          // 0xAF: 开启显示
}
```

初始化要点： `0x8D` 电荷泵命令对 3.3V 供电的模块至关重要—-如果跳过此步骤，OLED 完全不会显示任何内容。`0xA1`（SEGREMAP）和 `0xC8`（COMSCANDEC）决定了文字方向，删除或修改会导致显示翻转或镜像。

7.5.4.5 光标定位

```
void ssd1306_SetCursor(uint8_t page, uint8_t col)
{
    ssd1306_WriteCommand(SSD1306_PAGEADDR);  // 0x22: 页地址模式
    ssd1306_WriteCommand(page);              // 起始页 (0~7)
    ssd1306_WriteCommand(7);                 // 结束页

    ssd1306_WriteCommand(SSD1306_COLUMNADDR);// 0x21: 列地址模式
```

```
    ssd1306_WriteCommand(col);              // 起始列 (0~127)
    ssd1306_WriteCommand(127);              // 结束列
}
```

SSD1306 的 GDDRAM 组织结构： 128×64 像素被分为 8 个"页"(Page)，每页 8 行 × 128 列。每个字节控制一列的 8 个垂直像素（最低位为顶部像素）。

```
    列 0      列 1    ...    列 127
页0  Byte      Byte           Byte      ← 控制行 0~7
页1  Byte      Byte           Byte      ← 控制行 8~15
...
页7  Byte      Byte           Byte      ← 控制行 56~63
```

"Hello World!" 放在第 3 页（page=3），即显示在屏幕垂直方向的 24~31 行，大约在垂直中心。

7.5.4.6 字符渲染

```
void ssd1306_WriteChar(char ch)
{
    if (ch < ' ' || ch > '~') ch = ' ';  // 不可显示字符替换为空格
    uint8_t buf[6];
    buf[0] = 0x00;                       // 列间隔（1 像素空白）
    memcpy(&buf[1], font5x8[ch - ' '], 5); // 拷贝 5 列字模数据
    ssd1306_WriteData(buf, 6);           // 共 6 字节写入 GDDRAM
}
```

5×8 字体编码： - 每个字符占 6 列（5 列字模 + 1 列空白间隔）- 字模存储在 `font5x8[96][5]` 数组中（ASCII 32~126）- 字模数据的每个字节代表一列的 8 个像素（LSB= 顶，MSB= 底）

例如字符 `'A'` 的字模 `{0x7E,0x11,0x11,0x11,0x7E}` 对应像素：

```
列0:  01111110    ■
列1:  00010001    ■   ■
列2:  00010001    ■   ■
列3:  00010001    ■   ■
列4:  01111110    ■
列5:  00000000  (间隔)
```

7.5.4.7 软件逐帧滚动（跑马灯效果）

```
void ssd1306_ScrollMarquee(const char *str, uint8_t page, uint8_t delay_ms)
{
    int tw = (int)strlen(str) * 6;  // 文字总像素宽度
    uint8_t buf[128];

    // 文字从右侧 (off=128) 移入，从左侧 (off=-tw) 移出
    for (int off = 128; off > -tw; off--) {
        memset(buf, 0, sizeof(buf));          // 清零帧缓冲
        ssd1306_DrawTextBuf(buf, str, off);   // 在偏移处绘制文字
        ssd1306_SetCursor(page, 0);
        ssd1306_WriteData(buf, 128);          // 整页写入
        HAL_Delay(delay_ms);                  // 控制滚动速度
    }
```

```
}
```

滚动原理: 维护一个 128 字节的页缓冲区, 每次循环在其中以不同偏移量绘制文字, 然后将整页数据写入 GDDRAM。偏移量从 +128 递减到 -(文字宽度)，文字便从右向左平滑移过屏幕。delay_ms 控制每帧间隔，值越小滚动越快。

7.5.4.8 SSD1306 硬件滚动（备选方案）

SSD1306 内置水平滚动功能，无需 CPU 持续刷新:

```
void ssd1306_ScrollRight(uint8_t start, uint8_t end, uint8_t speed)
{
    ssd1306_WriteCommand(SSD1306_DEACTIVATE_SCROLL);      // 先关闭已有滚动
    ssd1306_WriteCommand(SSD1306_RIGHT_HORIZONTAL_SCROLL); // 0x26: 右滚
    ssd1306_WriteCommand(0x00);         // dummy
    ssd1306_WriteCommand(start);        // 起始页
    ssd1306_WriteCommand(speed & 0x07);// 滚动速度 (0= 最快, 7= 最慢)
    ssd1306_WriteCommand(end);          // 结束页
    ssd1306_WriteCommand(0x00);         // dummy
    ssd1306_WriteCommand(0xFF);         // dummy
    ssd1306_WriteCommand(SSD1306_ACTIVATE_SCROLL); // 0x2F: 启动
}
```

硬件滚动 **vs** 软件滚动对比:

特性	硬件滚动	软件滚动
CPU 占用	零（后台自动）	每帧都需刷新
滚动范围	整页滚动	可精确到像素
内容变化	无法动态改变	每帧可变
适用场景	静态文字持续滚动	动态内容、跑马灯

> 坑点: 硬件滚动在某些 SSD1306 克隆芯片上行为异常（不动、闪烁、范围偏移）。本实验改用软件滚动，兼容性最好，且能精确控制滚动行为。

7.6 API 函数详解

7.6.1 HAL_I2C_Master_Transmit

```
HAL_StatusTypeDef HAL_I2C_Master_Transmit(
    I2C_HandleTypeDef *hi2c,    // I2C 句柄
    uint16_t           DevAddress, // 从机地址 (7 位地址 << 1)
    uint8_t           *pData,     // 发送数据缓冲区
    uint16_t           Size,      // 发送字节数
    uint32_t           Timeout    // 超时时间 (ms)
);
```

- 功能： I2C 主机轮询模式发送。发送 START → 地址 → 数据... → STOP
- DevAddress 格式: 必须是 7 位地址 << 1。例如 7 位地址 0x3C，传入 0x78
- 返回值： HAL_OK（成功）、HAL_TIMEOUT（从机无应答超时）、HAL_ERROR（总线错误）
- 阻塞行为: 在传输完成或超时前 CPU 在此等待

7.6.2 HAL_I2C_Mem_Write

```
HAL_StatusTypeDef HAL_I2C_Mem_Write(
    I2C_HandleTypeDef *hi2c,    // I2C 句柄
    uint16_t           DevAddress, // 从机地址（7 位地址 << 1）
    uint16_t           MemAddress, // " 内存地址"（SSD1306 的控制字节）
    uint16_t           MemAddSize, // 内存地址大小：I2C_MEMADD_SIZE_8BIT
    uint8_t           *pData,     // 数据缓冲区
    uint16_t           Size,      // 数据字节数
    uint32_t           Timeout    // 超时时间（ms）
);
```

- 功能： I2C 主机向从机”寄存器”写入数据
- 帧格式： START + DevAddr + MemAddress + Data[] + STOP
- **SSD1306** 中的妙用： MemAddress 设为 0x40（数据模式），MemAddSize 设为一字节，这样 HAL 会自动在数据前插入控制字节。比手动拼接字节更清晰
- 对比： Mem_Write 适合”地址 + 数据”格式的设备；Master_Transmit 适合自由格式的帧

7.6.3 HAL_I2C_Init

```
HAL_StatusTypeDef HAL_I2C_Init(I2C_HandleTypeDef *hi2c);
```

- 根据 hi2c->Init 结构体中的参数配置 I2C 外设：速率、地址模式、占空比等
- 内部调用 HAL_I2C_MspInit() 完成 GPIO 和时钟的底层初始化

7.7 使用说明

7.7.1 编译与烧录

本工程使用 **CMake + ARM GCC** 工具链，配合 VS Code 插件进行开发。

7.7.1.1 编译

```
cmake --preset Debug
cmake --build build/Debug
```

或在 VS Code 中按 Ctrl+Shift+B 选择 CMake 构建任务。

7.7.1.2 烧录（ST-Link / OpenOCD）

连接 ST-Link 到开发板，执行：

```
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
        -c "program build/Debug/07I2C.elf verify reset exit"
```

7.7.2 预期现象

1. 上电后 OLED 点亮，屏幕清空
2. “Hello World!” 从屏幕右侧滑入，向左平滑滚动
3. 文字完整移出屏幕左侧后，停顿约 0.5 秒
4. 循环往复

7.7.3 调试技巧

问题	可能原因	排查方法
OLED 完全不亮	地址错误 / 接线错误	尝试 `0x78` 和 `0x7A`；检查 VCC/GND 连接
OLED 亮但不显示	未配置电荷泵	确认 `0x8D, 0x14` 初始化命令存在
文字翻转/镜像	硬件版本不同	修改 `SEGREMAP`(0xA0/0xA1) 和 `COMSCANDEC`(0xC0/0xC8)
通信不稳定/花屏	上拉电阻缺失	在 SCL/SDA 各加 4.7kΩ 上拉至 3.3V
滚动太慢/太快	delay_ms 参数	调整 `ssd1306_ScrollMarquee` 第三个参数

7.8 常见问题

7.8.1 Q1: 为什么我的 OLED 地址是 0x7B 但代码用的是 0x78?

A: 0x7B 是 8-bit 读地址（`0x3D << 1 | 1`）。STM32 HAL 需要的是 7 位地址左移 1 位后的值，即 8-bit 写地址。对于 7 位地址 0x3C，HAL 地址为 0x78；对于 7 位地址 0x3D，HAL 地址为 0x7A。先用 I2C 扫描程序确认你的 OLED 实际地址。

7.8.2 Q2: 为什么不能使用硬件滚动?

A: SSD1306 克隆芯片（如 SH1106 兼容模式）对硬件滚动命令的支持参差不齐。部分芯片忽略滚动命令，部分芯片滚动范围偏移。软件滚动虽然占用 CPU，但兼容性 100%，且能实现更灵活的效果。

7.8.3 Q3: 字体如何修改?

A: 修改 `font5x8` 数组即可。每个字符由 5 个字节定义，可替换为自己设计的字模。常用工具如 **LCD Assistant**、**PCtoLCD2002** 可以生成任意字符的字模数据。

7.8.4 Q4: 如何显示中文?

A: 英文只需 5×8 点阵，但中文至少需要 16×16 点阵。需要：1. 准备中文字模库（GB2312/GBK 编码）2. 修改渲染函数支持 16×16 块 3. 每次写一个汉字使用 2 页（16 行高）

7.8.5 Q5: I2C 速率能否提高到 400kHz?

A: 可以。在 CubeMX 中将 I2C Speed Mode 改为 Fast Mode，同时建议：- 上拉电阻从 4.7kΩ 降为 2.2kΩ~3.3kΩ - 缩短杜邦线长度 - 避免在面包板上走长跳线

7.9 扩展练习

1. 反色与闪烁效果：调用 `0xA7`（反色显示）命令，实现文字闪烁提示。
2. 多页内容显示：在不同页分别绘制不同内容（如第 0 页标题、第 2~5 页正文），实现多行文本显示。
3. 简单动画：利用软件逐帧刷新实现进度条、跳动字符、或乒乓球动画。
4. 传感器数据展示：结合 ADC 或温湿度传感器，在 OLED 上实时刷新显示传感器数值。
5. **I2C** 从机扫描：编写程序扫描 I2C 总线上的所有设备，打印出每个从机的 7 位地址。

7.10 作业要求

7.10.1 基础要求（必做）

7.10.1.1 CubeMX 配置与 I2C 通信验证

1. 在 STM32CubeMX 中按本教程完成 **I2C1** 的配置（PB6=SCL, PB7=SDA, 100kHz 标准模式）
2. 生成代码，编译烧录到开发板
3. 使用逻辑分析仪或示波器抓取 I2C 总线波形，验证：
   - SCL 频率 ~ 100kHz
   - START / STOP 条件时序正确
   - 从机地址为 0x78（7-bit 0x3C « 1）

验收标准：提供 I2C 波形截图，标注 START、地址字节、ACK、数据字节、STOP。

7.10.1.2 OLED 初始化与静态显示

1. 将 SSD1306 初始化代码添加到 `main.c` 的 USER CODE 区域
2. 在 OLED 上显示 学号 + 姓名拼音（或自定义英文短句）
3. 验证 SSD1306 的所有关键初始化命令（电荷泵、显示开、寻址模式等）

验收标准： OLED 正常点亮，显示内容清晰无乱码，无雪花点。

7.10.1.3 滚动显示

实现以下任意一种滚动效果：

- 方案一（硬件滚动）：使用 SSD1306 内置命令 `0x26/0x27` + `0x2F` 驱动滚动
- 方案二（软件滚动）：编写跑马灯函数，每帧清屏后在新位置绘制文字，循环往复

验收标准：文字持续平滑滚动，无闪烁、无断裂、无残影。

7.10.2 进阶要求（选做，加分）

7.10.2.1 多行显示与页面布局

在不滚动的情况下，将 OLED 分为 3 个区域：- 第 **0~1** 页：标题行（大号效果，如用 2 页重复写同一文本模拟粗体）- 第 **3~4** 页：正文内容（多行文本）- 第 **7** 页：状态栏（如计数器、时间戳）

验收标准：三个区域内容独立，布局清晰。

7.10.2.2 交互动画

实现一个由按键（或定时器）触发的动画效果，例如：- 按下按键后文字翻转/镜像 - 定时切换显示内容（幻灯片效果）- 逐帧动画（如弹跳的小球）

验收标准：动画流畅自然，触发逻辑正确。

7.10.3 实验报告要求

提交的实验报告应包含以下内容：

章节	内容
1. 实验目的	简述本实验的学习目标
2. 硬件连接	画出 STM32 与 OLED 的引脚连接图（含实物照片）
3. CubeMX 配置	截图说明 I2C1、RCC、SYS 的配置参数
4. 核心代码	粘贴并解释 OLED 初始化序列、命令/数据发送函数、滚动实现（不要全部粘贴 main.c）
5. I2C 波形分析	逻辑分析仪截图，标注各字段含义
6. 实验结果	OLED 显示效果照片（含静态和滚动两张）
7. 问题与思考	记录实验中遇到的问题及解决方案；回答：I2C 为什么需要开漏输出和上拉电阻?
8. 附录	完整的 `main.c` 代码（可选）

7.11 参考资料

- STM32F103C8T6 数据手册
- SSD1306 数据手册 (Solomon Systech)
- RM0008 – STM32F1xx 参考手册 (I2C 章节)
- STM32F1xx HAL 库用户手册 (UM1850)
- I2C-bus specification (NXP UM10204)

8 STM32 SPI 主从回环通信实验

8.1 实验概述

本实验基于 **STM32F103C8T6** 微控制器，通过 **SPI1**（主机）与 **SPI2**（从机）在同一芯片上实现主从回环通信，并通过 VS Code 调试窗口观测收发数据验证通信正确性。

核心知识点：- SPI 总线协议基础（四线制：NSS / SCK / MOSI / MISO）- STM32 SPI 外设的主机与从机配置 - 时钟极性（CPOL）与时钟相位（CPHA）的匹配原则 - HAL 库 SPI 阻塞式收发 API：`HAL_SPI_TransmitReceive` - HAL 库 SPI 中断式收发 API：`HAL_SPI_TransmitReceive_IT` - SPI 回调函数 `HAL_SPI_TxRxCpltCallback` 的使用 - 硬件 NSS 控制（主机输出 / 从机输入）- 主从同步：主机驱动 SCK，从机被动响应

8.2 硬件需求

器件	说明
STM32F103C8T6 最小系统板	主控 MCU
ST-Link / J-Link 调试器	烧录与调试
面包板 + 杜邦线 × 4	连接 SPI1 与 SPI2 引脚

引脚连接（**SPI1 <-> SPI2** 回环）：

SPI1 主机引脚	SPI2 从机引脚	功能
PA4 (NSS)	PB12 (NSS)	片选信号
PA5 (SCK)	PB13 (SCK)	串行时钟
PA6 (MISO)	PB14 (MISO)	主机输入 / 从机输出
PA7 (MOSI)	PB15 (MOSI)	主机输出 / 从机输入

> 注意：**MISO** 交叉连接。 SPI 的 MISO 含义是 Master In Slave Out。连接时，主机的 MOSI（PA7）对从机的 MOSI（PB15），主机的 MISO（PA6）对从机的 MISO（PB14）—-但这里”从机 MISO”是从机输出，所以交叉后正好实现主机发送 → 从机接收，从机发送 → 主机接收。
>
> 速度选择。本实验 SPI1 波特率预分频器设为 64，即 72MHz / 64 = **1.125 Mbps**。回环测试中信号完整性问题较少，可使用较高速率。实际外部 SPI 设备（如 Flash、传感器）通常支持 1~10 Mbps。

8.3 SPI 协议基础

在阅读代码之前，先了解 SPI 总线的基本概念：

8.3.1 四线制信号

信号	方向（主机视角）	功能
NSS (Slave Select)	输出	拉低选中从机，拉高释放
SCK (Serial Clock)	输出	主机产生时钟，驱动数据传输
MOSI (Master Out Slave In)	输出	主机发送数据到从机
MISO (Master In Slave Out)	输入	从机发送数据到主机

STM32 数据手册中有时将 MOSI 标注为” 主出从入”，MISO 标注为” 主入从出”，但实际引脚标签直接以主机视角命名。本实验中 SPI2 是从机，其 PB15 虽然在 STM32 文档中是”SPI2_MOSI”，但在从机模式下它接收数据。

8.3.2 时钟极性与相位（SPI Mode）

SPI 有 4 种工作模式，由 CPOL（时钟极性）和 CPHA（时钟相位）组合决定：

模式	CPOL	CPHA	SCK 空闲电平	数据采样沿
Mode 0	0	0	低电平	第 1 个边沿（上升沿）
Mode 1	0	1	低电平	第 2 个边沿（下降沿）
Mode 2	1	0	高电平	第 1 个边沿（下降沿）
Mode 3	1	1	高电平	第 2 个边沿（上升沿）

本实验使用 **Mode 0**（CPOL=0, CPHA=0），这是最常见的 SPI 模式。

关键原则：主机和从机的 CPOL/CPHA 必须完全一致，否则数据采样点错位，通信失败。

8.3.3 通信过程（全双工）

SPI 是全双工总线：在 SCK 的驱动下，主机和从机同时通过 MOSI 和 MISO 各发送一个 bit。发送完一个字节后，双方各得到一个字节。

```
主机:  发送 0xA5  →  MOSI →  从机收到 0xA5
从机:  发送 0x5A  →  MISO →  主机收到 0x5A
```

一次 `HAL_SPI_TransmitReceive` 调用同时完成发送和接收，这正是本实验 loopback 验证的核心机制。

8.3.4 SPI 与 I2C 的对比

特性	SPI	I2C
信号线数	4 线（NSS+SCK+MOSI+MISO）	2 线（SCL+SDA）
通信模式	全双工	半双工
地址机制	NSS 硬件片选	软件地址字节 + ACK
速率	1~50 Mbps	100kHz~3.4MHz

特性	SPI	I2C
拓扑	点对点为主（一主多从需多 NSS）	多设备共享总线
流控	无（主机控制节奏）	ACK 应答

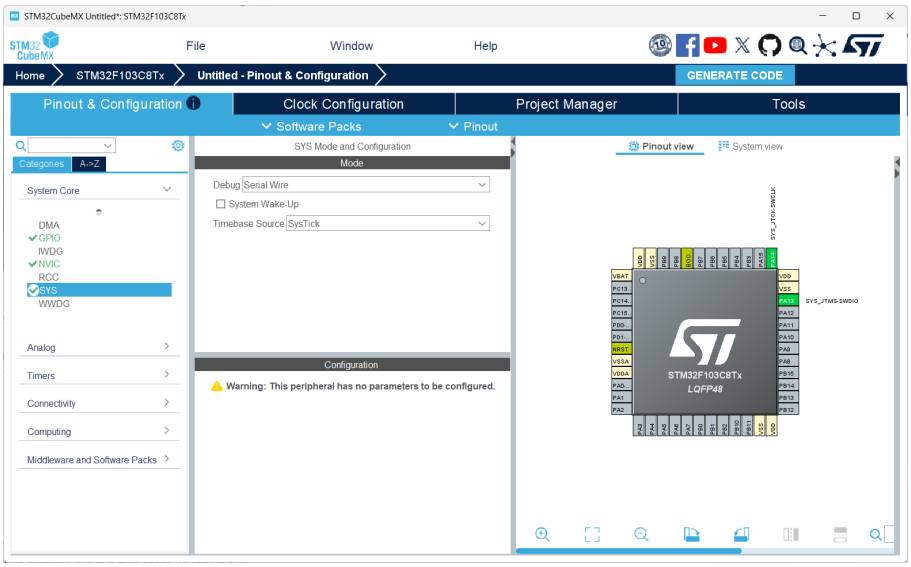
8.4 STM32CubeMX 配置步骤

8.4.1 新建工程

1. 打开 **STM32CubeMX**（本工程使用 6.17.0 版本）
2. 点击 **File → New Project**
3. 在 **MCU Selector** 中搜索 `STM32F103C8Tx`，选中后点击 **Start Project**

8.4.2 配置调试接口（SYS）

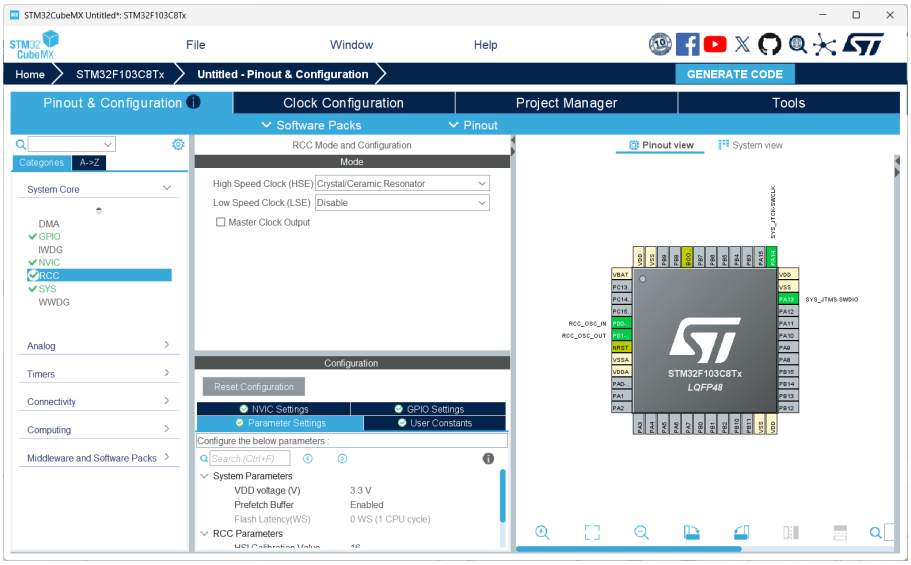
1. 点击左侧 **Pinout & Configuration** 标签
2. 在 **System Core → SYS** 中
3. 将 **Debug** 设为 **Serial Wire**（关闭 JTAG，仅保留 SWD）



> 为什么要这样做？ JTAG 默认占用 PA15、PB3、PB4 三个引脚。关闭 JTAG 仅保留 SWD，既可烧录调试，又不浪费 GPIO 资源。本实验中 SPI2 使用 PB12~PB15，虽然不与 JTAG 冲突，但这是 STM32 项目的标准做法。

8.4.3 配置时钟源（RCC）

1. 在 **System Core → RCC** 中
2. 将 **High Speed Clock (HSE)** 设为 **Crystal/Ceramic Resonator**（外部晶振）

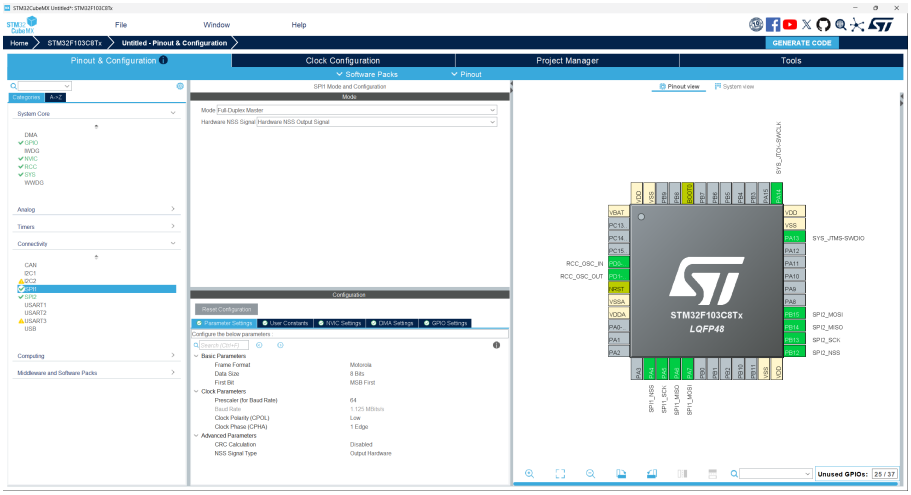


> 说明：STM32F103C8T6 核心板通常板载 8MHz 晶振，通过 HSE + PLL 倍频到 72MHz。

8.4.4 配置 SPI1（主机）

1. 在 **Connectivity → SPI1** 中
2. 将 **Mode** 设为 **Full-Duplex Master**
3. **NSS Signal** 设为 **Hardware NSS Output Signal**
4. 其他参数保持默认：

参数	值	说明
Mode	Full-Duplex Master	全双工主机模式
NSS Signal	Hardware NSS Output	硬件自动控制 NSS 输出
Data Size	8 Bits	每帧 8 位数据
Clock Polarity (CPOL)	Low	空闲时 SCK= 低电平
Clock Phase (CPHA)	1 Edge	第一个边沿采样
Baud Rate Prescaler	64	72MHz/64 = 1.125 Mbps
First Bit	MSB	高位先发

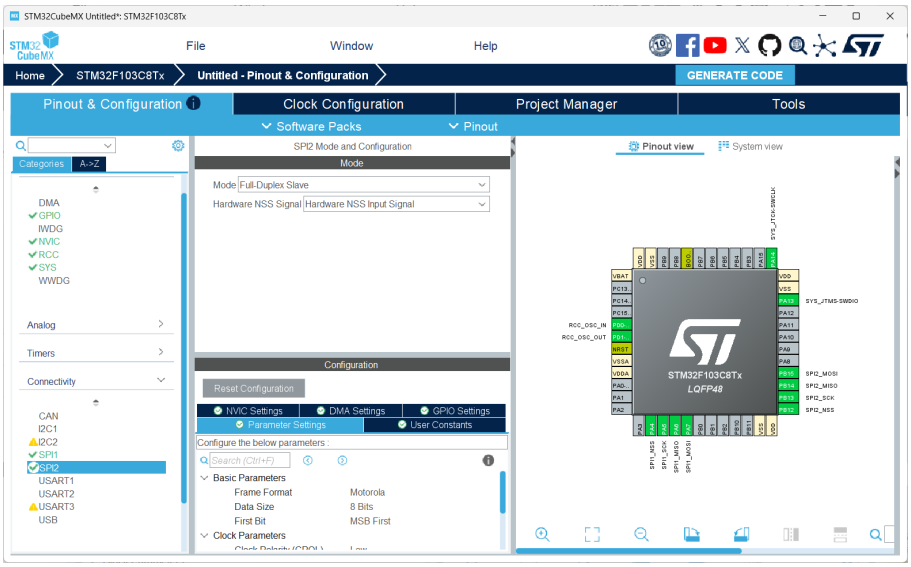


> 为什么用硬件 **NSS**？SPI1 作为主机，硬件 NSS 输出模式会在传输开始前自动拉低 NSS 引脚，传输结束后自动拉高。这样无需软件手动控制，且时序精准。

8.4.5 配置 SPI2（从机）

1. 在 **Connectivity → SPI2** 中
2. 将 **Mode** 设为 **Full-Duplex Slave**
3. **NSS Signal** 设为 **Hardware NSS Input Signal**
4. **CPOL / CPHA** 必须与 SPI1 主机完全一致

参数	值	说明
Mode	Full-Duplex Slave	全双工从机模式
NSS Signal	Hardware NSS Input	硬件检测 NSS 片选
Data Size	8 Bits	与主机一致
Clock Polarity (CPOL)	Low	与主机一致
Clock Phase (CPHA)	1 Edge	与主机一致
First Bit	MSB	与主机一致



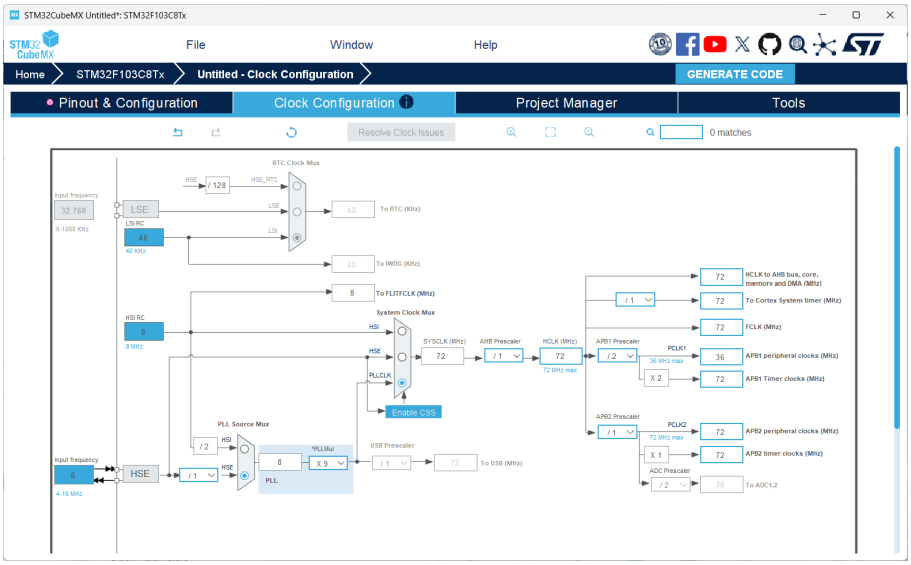
> 关键配置：SPI2 除 Mode 和 NSS 外，其余参数（CPOL/CPHA/DataSize/FirstBit）必须与 SPI1 完全一致，否则通信失败。

> 为什么从机不需要设置 **BaudRate**？从机使用主机提供的 SCK 时钟，自己不产生时钟，因此 CubeMX 不会显示波特率配置项。

8.4.6 配置时钟树（Clock Configuration）

1. 点击顶部 **Clock Configuration** 标签
2. 按如下参数配置：

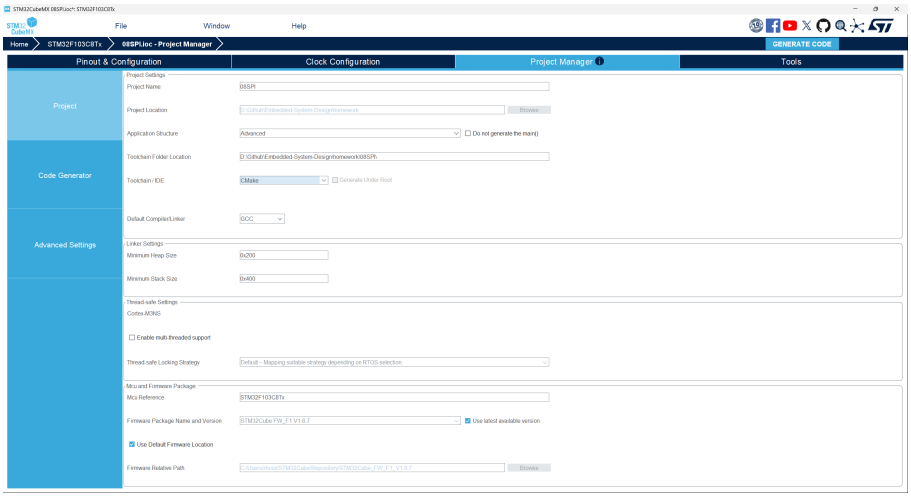
- **HSE:** 8 MHz（外部晶振）
- **PLL Source:** HSE
- **PLL Mul:** x9 → 8 MHz × 9 = **72 MHz**
- **System Clock Mux:** PLLCLK
- **AHB Prescaler:** /1 → **HCLK = 72 MHz**
- **APB1 Prescaler:** /2 → **APB1 = 36 MHz**
- **APB2 Prescaler:** /1 → **APB2 = 72 MHz**



> **SPI** 时钟来源：SPI1 挂在 APB2 总线（72MHz），SPI2 挂在 APB1 总线（36MHz）。SPI1 的波特率由此计算：72MHz / 64 = 1.125 Mbps。SPI2 作为从机不产生时钟，其 APB1 频率只影响数字逻辑的最小响应延迟。

8.4.7 配置工程输出

1. 点击 **Project Manager** 标签
2. **Project Name:** `08SPI`
3. **Project Location:** 选择你的工作目录
4. **Application Structure:** Basic
5. **Toolchain / IDE:** 选择 **CMake**（配合 VS Code + ARM GCC 使用）



8.4.8 生成代码

1. 点击右上角 **GENERATE CODE** 按钮
2. 等待代码生成完成
3. 点击 **Open Project** 或直接进入工程目录

8.5 程序结构

```
08SPI/
├── 08SPI.ioc                    # CubeMX 工程配置文件
├── CMakeLists.txt               # 顶层 CMake 构建文件
├── CMakePresets.json            # CMake 预设（Debug）
├── STM32F103XX_FLASH.ld         # 链接脚本（Flash/RAM 布局）
├── cmake/                       # CMake 工具链文件
│   ├── gcc-arm-none-eabi.cmake  #   ARM GCC 工具链配置
│   ├── starm-clang.cmake        #   Clang 工具链配置
│   └── stm32cubemx/
│       └── CMakeLists.txt       #   自动生成的子模块 CMake
├── Core/
│   ├── Inc/                     # 头文件
│   │   ├── main.h               #   主头文件（含 HAL 库引用）
│   │   ├── stm32f1xx_hal_conf.h #   HAL 库配置文件
│   │   └── stm32f1xx_it.h       #   中断服务函数声明
│   └── Src/                     # 源文件
│       ├── main.c               #   * 主程序（回环测试逻辑）
│       ├── stm32f1xx_hal_msp.c  #   HAL 外设底层初始化（MSP）
│       ├── stm32f1xx_it.c       #   中断服务函数实现
│       ├── system_stm32f1xx.c   #   系统初始化
│       ├── sysmem.c             #   动态内存管理桩
│       └── syscalls.c           #   系统调用桩（_write 等）
├── Drivers/                     # HAL 驱动库
│   ├── CMSIS/                   #   ARM CMSIS 核心头文件
│   └── STM32F1xx_HAL_Driver/    #   STM32F1 HAL 库源码
├── startup_stm32f103xb.s        # 启动文件（汇编）
└── img/                         # 文档截图
```

8.5.1 文件职责速览

文件	职责
main.c	用户程序入口，包含 SPI 回环测试逻辑和回调函数
stm32f1xx_hal_msp.c	MSP 层：初始化 SPI1（PA4~PA7）和 SPI2（PB12~PB15）的 GPIO 及时钟
stm32f1xx_it.c	中断向量表实现，含 SPI2_IRQHandler
stm32f1xx_hal_conf.h	裁剪 HAL 库：启用/禁用各外设模块
system_stm32f1xx.c	SystemInit() 函数：上电后最早的时钟初始化

8.6 核心代码详解

8.6.1 程序流程图

```
flowchart TD
    A[上电复位] --> B["HAL_Init()<br/> 初始化 HAL 库、SysTick 1ms 中断"]
    B --> C["SystemClock_Config()<br/>HSE(8MHz) → PLL×9 → 72MHz"]
    C --> D["MX_GPIO_Init()<br/> 使能 GPIOA / GPIOB 时钟"]
    D --> E["MX_SPI1_Init()<br/>SPI1: 主机, CPOL=0/CPHA=0, 1.125Mbps"]
    E --> F["MX_SPI2_Init()<br/>SPI2: 从机, CPOL=0/CPHA=0"]
    F --> G[" 使能 SPI2 中断 <br/> 优先级 1"]
    G --> H[" 初始化: master_tx=0xA5, slave_tx=0x5A"]
    H --> I[" 清除 spi2_done 标志"]
    I --> J["HAL_SPI_TransmitReceive_IT<br/> 从机: 中断模式准备收发"]
    J --> K["HAL_SPI_TransmitReceive<br/> 主机: 阻塞模式收发 (产生 SCK)"]
    K --> L[" 等待 spi2_done 标志 <br/> 超时 10ms 则报错"]
    L --> M[" 数据取反: 0xA5<->0x5A"]
    M --> N["HAL_Delay(500ms)<br/>* 调试断点位置"]
    N --> I
```

8.6.2 全局变量与回调函数

```c
/* 用户定义的私有变量 */
volatile uint8_t spi2_done = 0;  // 从机传输完成标志

/* SPI 传输完成回调函数 */
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
{
  if (hspi->Instance == SPI2) {
    spi2_done = 1;  // 从机中断传输完成，置位标志
  }
}
```

设计要点：

- spi2_done 声明为 volatile，确保编译器每次都从内存读取，不会将变量缓存在寄存器中导致死循环
- 回调函数是弱定义（__weak）的重写：HAL 库默认提供空实现，用户在 main.c 中重写后，链接器自动覆盖库中的弱定义
- 回调在中断上下文中执行（SPI2 中断服务例程 → HAL_SPI_IRQHandler → 回调），因此必须尽量简短

  什么是弱定义（**Weak Symbol**）？ HAL 库中所有回调函数都使用 __weak 属性声明。链接时，如果用户代码中定义了同名函数，用户的强定义自动覆盖库中的弱定义。这使得 HAL 库可以提供默认空实现，用户只需重写需要的回调即可。

8.6.3 中断配置

```c
/* main() 中，外设初始化之后 */
HAL_NVIC_SetPriority(SPI2_IRQn, 1, 0);   // 抢占优先级 1，子优先级 0
HAL_NVIC_EnableIRQ(SPI2_IRQn);           // 使能 SPI2 中断
```

中断服务函数位于 stm32f1xx_it.c：

```c
void SPI2_IRQHandler(void)
{
  HAL_SPI_IRQHandler(&hspi2);  // 委托 HAL 库处理中断事件
```

```
}
```

为什么只给 **SPI2** 使能中断？主机 SPI1 使用阻塞模式（轮询等待），无需中断。从机 SPI2 使用中断模式，可以在主机发起传输时由硬件自动触发中断处理，避免 CPU 空等。

8.6.4 主循环回环测试逻辑

```c
uint8_t master_tx = 0xA5;   // 主机发送数据（初始值）
uint8_t master_rx = 0;      // 主机接收缓冲
uint8_t slave_tx  = 0x5A;   // 从机发送数据（初始值）
uint8_t slave_rx  = 0;      // 从机接收缓冲

while (1)
{
    // 步骤 1: 清除完成标志
    spi2_done = 0;

    // 步骤 2: 从机准备（中断模式，非阻塞）
    //         slave_tx → 从机将在传输中发送此数据
    //         slave_rx ← 从机将在传输中接收数据存于此
    if (HAL_SPI_TransmitReceive_IT(&hspi2, &slave_tx, &slave_rx, 1) != HAL_OK) {
        Error_Handler();
    }

    // 步骤 3: 主机发起传输（阻塞模式，驱动 SCK）
    //         master_tx → 主机发送到从机
    //         master_rx ← 主机接收来自从机的数据
    result = HAL_SPI_TransmitReceive(&hspi1, &master_tx, &master_rx, 1, 100);
    if (result != HAL_OK) {
        Error_Handler();
    }

    // 步骤 4: 等待从机中断完成（超时 10ms）
    uint32_t tickstart = HAL_GetTick();
    while (!spi2_done && (HAL_GetTick() - tickstart < 10)) {}

    if (!spi2_done) {
        Error_Handler();  // 超时：从机未响应
    }

    // 步骤 5: 翻转测试数据，实现交替发送
    master_tx ^= 0xFF;   // 0xA5 → 0x5A, 0x5A → 0xA5
    slave_tx  ^= 0xFF;   // 同上

    HAL_Delay(500);  // * 在此设置断点，观察变量
}
```

8.6.5 数据传输详解

第一轮循环（初始值）：

```
主机 SPI1: 发送 0xA5 ——MOSI——→ 从机 SPI2: 收到 0xA5 → 存入 slave_rx
从机 SPI2: 发送 0x5A ——MISO——→ 主机 SPI1: 收到 0x5A → 存入 master_rx
```

第一轮结束后变量状态：

变量	值	说明
master_tx	0xA5	主机发出的数据
master_rx	**0x5A**	主机收到来自从机的数据
slave_tx	0x5A	从机发出的数据
slave_rx	**0xA5**	从机收到来自主机的数据

取反后进入第二轮：

主机 SPI1：发送 0x5A ——MOSI——→ 从机 SPI2：收到 0x5A → 存入 slave_rx
从机 SPI2：发送 0xA5 ——MISO——→ 主机 SPI1：收到 0xA5 → 存入 master_rx

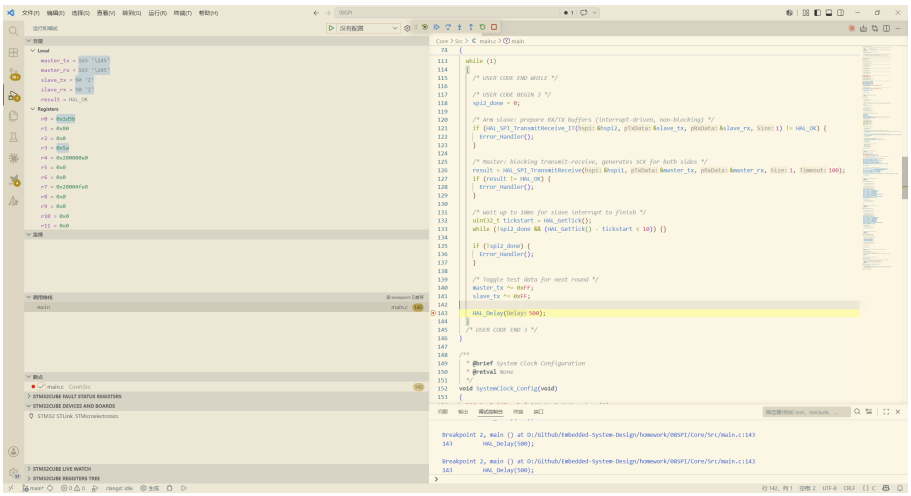
第二轮结束后变量状态：

变量	值	说明
master_tx	0x5A	主机发出的数据（已翻转）
master_rx	**0xA5**	主机收到来自从机的数据
slave_tx	0xA5	从机发出的数据（已翻转）
slave_rx	**0x5A**	从机收到来自主机的数据

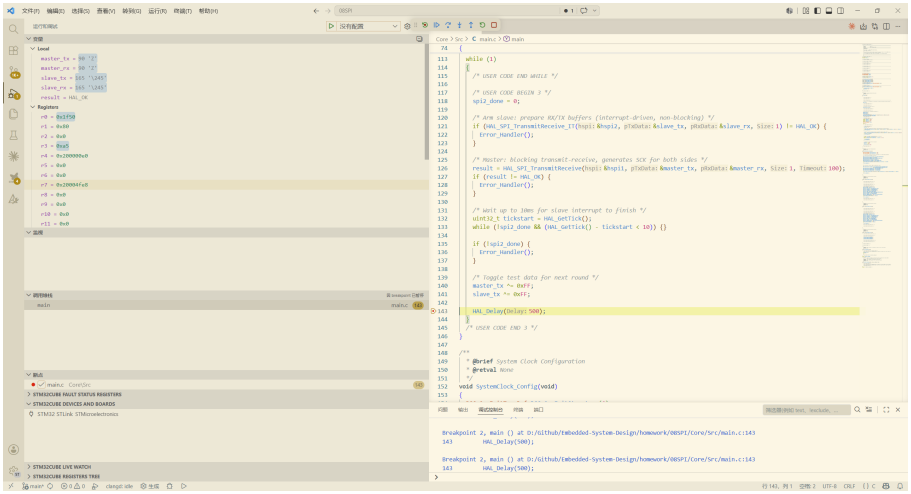
> 验证方法：在 HAL_Delay(500) 处设置断点，每运行一次在调试窗口观察四个变量的值。如果 master_rx == slave_tx 且 slave_rx == master_tx，则 SPI 回环通信完全正确。

调试截图：

- 第一次运行到断点：master_tx=0xA5, master_rx=0x5A, slave_tx=0x5A, slave_rx=0xA5



- 第二次运行到断点：master_tx=0x5A, master_rx=0xA5, slave_tx=0xA5, slave_rx=0x5A



8.6.6 SPI1 初始化（主机）

```
hspi1.Instance = SPI1;
hspi1.Init.Mode              = SPI_MODE_MASTER;           // 主机模式
hspi1.Init.Direction         = SPI_DIRECTION_2LINES;      // 双线全双工
hspi1.Init.DataSize          = SPI_DATASIZE_8BIT;         // 8 位数据帧
hspi1.Init.CLKPolarity       = SPI_POLARITY_LOW;          // CPOL=0
hspi1.Init.CLKPhase          = SPI_PHASE_1EDGE;           // CPHA=0 (Mode 0)
hspi1.Init.NSS               = SPI_NSS_HARD_OUTPUT;       // 硬件 NSS 输出
hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_64;  // 72M/64=1.125Mbps
hspi1.Init.FirstBit          = SPI_FIRSTBIT_MSB;          // 高位先发
hspi1.Init.TIMode            = SPI_TIMODE_DISABLE;        // 禁用 TI 模式
hspi1.Init.CRCCalculation    = SPI_CRCCALCULATION_DISABLE; // 禁用 CRC
HAL_SPI_Init(&hspi1);
```

8.6.7 SPI2 初始化（从机）

```
hspi2.Instance = SPI2;
hspi2.Init.Mode           = SPI_MODE_SLAVE;             // 从机模式
hspi2.Init.Direction      = SPI_DIRECTION_2LINES;       // 双线全双工
hspi2.Init.DataSize       = SPI_DATASIZE_8BIT;          // 与主机一致
hspi2.Init.CLKPolarity    = SPI_POLARITY_LOW;           // 与主机一致
hspi2.Init.CLKPhase       = SPI_PHASE_1EDGE;            // 与主机一致
hspi2.Init.NSS            = SPI_NSS_HARD_INPUT;          // 硬件 NSS 输入
hspi2.Init.FirstBit       = SPI_FIRSTBIT_MSB;            // 与主机一致
hspi2.Init.TIMode         = SPI_TIMODE_DISABLE;
hspi2.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
HAL_SPI_Init(&hspi2);
```

从机与主机的关键区别： 1. Mode 设为 SPI_MODE_SLAVE 2. NSS 设为 SPI_NSS_HARD_INPUT（从机被动接受片选）3. 没有 **BaudRatePrescaler**（从机时钟由主机 SCK 引脚提供）4. CPOL/CPHA/DataSize/FirstBit 必须与主机完全一致

8.6.8 HAL_SPI_MspInit – GPIO 引脚配置

HAL_SPI_MspInit() 由 HAL_SPI_Init() 内部自动调用，完成 GPIO 和时钟的底层初始化。

SPI1 主机引脚（**PA4~PA7**）：

```
/* PA4(NSS), PA5(SCK), PA7(MOSI): 复用推挽输出 */
GPIO_InitStruct.Pin   = GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_7;
GPIO_InitStruct.Mode  = GPIO_MODE_AF_PP;       // 复用推挽
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/* PA6(MISO): 浮空输入（主机接收来自从机的数据） */
GPIO_InitStruct.Pin  = GPIO_PIN_6;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

SPI2 从机引脚（**PB12~PB15**）：

```
/* PB12(NSS), PB13(SCK), PB15(MOSI): 浮空输入（从机被动接收） */
GPIO_InitStruct.Pin   = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_15;
GPIO_InitStruct.Mode  = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull  = GPIO_NOPULL;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/* PB14(MISO): 复用推挽输出（从机通过此引脚向主机发送数据） */
GPIO_InitStruct.Pin   = GPIO_PIN_14;
GPIO_InitStruct.Mode  = GPIO_MODE_AF_PP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

GPIO 方向总结：

引脚	功能	方向	原因
PA4 (SPI1_NSS)	主机片选输出	AF_PP	主机主动驱动 NSS
PA5 (SPI1_SCK)	主机时钟输出	AF_PP	主机产生时钟
PA6 (SPI1_MISO)	主机数据输入	Input	主机从此引脚读数据
PA7 (SPI1_MOSI)	主机数据输出	AF_PP	主机向此引脚写数据
PB12 (SPI2_NSS)	从机片选输入	Input	从机被动检测 NSS
PB13 (SPI2_SCK)	从机时钟输入	Input	从机接收主机时钟
PB14 (SPI2_MISO)	从机数据输出	AF_PP	从机向此引脚写数据
PB15 (SPI2_MOSI)	从机数据输入	Input	从机从此引脚读数据

核心规律：谁驱动信号线，谁就配置为输出（AF_PP）；谁接收信号，谁就配置为输入。主机驱动 SCK/MOSI/NSS，从机只驱动 MISO。

8.7 API 函数详解

8.7.1 HAL_SPI_TransmitReceive（阻塞模式）

```
HAL_StatusTypeDef HAL_SPI_TransmitReceive(
    SPI_HandleTypeDef *hspi,   // SPI 句柄
    uint8_t           *pTxData, // 发送数据缓冲区
    uint8_t           *pRxData, // 接收数据缓冲区
    uint16_t           Size,    // 数据长度（字节数）
    uint32_t           Timeout  // 超时时间（ms）
);
```

- 功能： SPI 全双工阻塞式收发。发送 Size 字节，同时接收 Size 字节
- 主机行为：拉低 NSS → 产生 SCK 时钟 → 同时收发数据 → 拉高 NSS
- 从机行为：等待 NSS 拉低 → 跟随 SCK 同时收发数据
- 阻塞方式： CPU 在此等待直到传输完成或超时
- 返回值： HAL_OK（成功）、HAL_TIMEOUT（超时）、HAL_ERROR（总线错误）

8.7.2 HAL_SPI_TransmitReceive_IT（中断模式）

```
HAL_StatusTypeDef HAL_SPI_TransmitReceive_IT(
    SPI_HandleTypeDef *hspi,   // SPI 句柄
    uint8_t           *pTxData, // 发送数据缓冲区
    uint8_t           *pRxData, // 接收数据缓冲区
    uint16_t           Size     // 数据长度
);
```

- 功能： SPI 全双工中断式收发。函数立即返回，传输在后台进行，完成后触发回调
- 非阻塞： CPU 可以继续执行其他任务
- 完成通知：传输完成后 HAL 库调用 HAL_SPI_TxRxCpltCallback()
- 重要规则：调用此函数前要确保上一次 IT 传输已完成（检查 hspi->State == HAL_SPI_STATE_READY）
- 本实验用途：从机使用中断模式，在主机发起传输时自动完成收发

8.7.3 HAL_SPI_TxRxCpltCallback（回调函数）

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi);
```

- 调用时机：中断模式（或 DMA 模式）传输完成时，由 HAL 库在中断上下文中调用
- 参数说明： hspi 指向触发回调的 SPI 句柄（可用来区分是 SPI1 还是 SPI2 完成）
- 用户任务：在此设置完成标志、读取数据、启动下一轮传输等
- 注意事项：在中断上下文中执行，不宜做耗时操作

8.7.4 HAL_SPI_Init

```
HAL_StatusTypeDef HAL_SPI_Init(SPI_HandleTypeDef *hspi);
```

- 根据 hspi->Init 结构体中的参数配置 SPI 外设
- 内部自动调用 HAL_SPI_MspInit() 完成 GPIO 和时钟的底层初始化

8.8 使用说明

8.8.1 编译与烧录

本工程使用 **CMake + ARM GCC** 工具链，配合 VS Code 插件进行开发。

8.8.1.1 编译

```
cmake --preset Debug
cmake --build build/Debug
```

或在 VS Code 中按 Ctrl+Shift+B 选择 CMake 构建任务。

8.8.1.2 烧录（ST-Link / OpenOCD）

连接 ST-Link 到开发板，执行：

```
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
        -c "program build/Debug/08SPI.elf verify reset exit"
```

8.8.2 调试与验证

1. 在 main.c 中 HAL_Delay(500) 行设置断点
2. 按 **F5** 启动调试（Debug 预设已配置好）
3. 程序运行到断点时，在 监视窗口中添加变量：
   - master_tx
   - master_rx
   - slave_tx
   - slave_rx
4. 每次按 **F5**（继续运行），四个变量的值会交替变化

预期结果：

断点次数	master_tx	master_rx	slave_tx	slave_rx
第 1 次	0xA5	0x5A	0x5A	0xA5
第 2 次	0x5A	0xA5	0xA5	0x5A
第 3 次	0xA5	0x5A	0x5A	0xA5
...	(交替)	(交替)	(交替)	(交替)

> 验证通过标准：master_rx == slave_tx 且 slave_rx == master_tx，且两轮数据交替翻转，说明 SPI1→SPI2 和 SPI2→SPI1 两条数据通路均正常工作。

8.8.3 调试技巧

问题	可能原因	排查方法
从机接收数据为 0x00 或 0xFF	SPI 引脚未连接/接触不良	用万用表通断档检查 4 条杜邦线
master_rx 始终为 0x00	SPI2 MISO 引脚配置错误	检查 PB14 是否为 AF_PP
slave_rx 始终为 0x00	SPI2 MOSI 引脚配置错误	检查 PB15 是否为输入模式
从机中断一直不触发	NSS 信号异常	检查 PA4→PB12 连接；确认 NSS 硬件模式

问题	可能原因	排查方法
程序停在 Error_Handler	SPI2 中断超时（10ms）	检查 SPI2_IRQn 是否使能；优先级配置
数据偶尔错误	杜邦线过长/接触不良	缩短走线；更换杜邦线；降低 SPI 速率

8.9 常见问题

8.9.1 Q1: 为什么从机也要准备发送数据？它不是在”接收”吗？

A: SPI 是全双工总线。每次传输时，主机和从机同时发送一个字节。即使从机只关心接收，它的移位寄存器也会随着 SCK 时钟移出数据。如果不事先准备好 `slave_tx`，从机会发出 0x00（或其他随机值），但这不影响它正确接收主机发来的数据。

8.9.2 Q2: 为什么要用硬件 NSS 而不是软件 NSS？

A: 硬件 NSS 有以下优势：- 时序精准：硬件在传输开始前自动拉低，传输结束后自动拉高，无需软件干预 - 无延迟：没有 GPIO 翻转的软件指令延迟 - 从机必需：从机使用硬件 NSS 输入时，NSS 的上升沿会触发内部状态机复位，确保每次传输从干净状态开始

软件 NSS 适合一主多从的场景（需要多个独立的 NSS GPIO 引脚）。

8.9.3 Q3: 为什么 SPI1 用阻塞模式，SPI2 用中断模式？

A: 这是回环测试的一种经典设计：- 主机 **SPI1**（阻塞）：主机主动发起传输，需要确保传输完成后才继续执行，阻塞等待是最简单的实现 - 从机 **SPI2**（中断）：从机被动等待主机发起传输，使用中断可以在传输完成后自动得到通知

也可以用两种中断模式或 DMA 模式，但阻塞 + 中断的组合最便于教学演示。

8.9.4 Q4: 如果我把 CPOL/CPHA 设反了会怎样？

A: 数据采样点偏移半拍。例如主机在上升沿采样，从机在下降沿采样，则从机读到的数据会产生 1 bit 的位移或完全错误。症状通常是数据能收到但值是错的（不是 0x00 或 0xFF）。

8.9.5 Q5: SPI1 和 SPI2 可以放在不同芯片上吗？

A: 当然可以。本实验将 SPI1 和 SPI2 放在同一芯片上是为了简化硬件连接（仅需 4 条杜邦线跳接）。如果使用两块开发板，连接方式完全相同，且各自独立供电—-只需确保共地（GND 相连）。

8.10 扩展练习

1. 多字节传输：将单字节测试扩展为多字节数组传输（如 `uint8_t test_data[16]`），验证批量数据的收发正确性。
2. 不同 **SPI** 模式测试：修改 CPOL/CPHA 配置为 Mode 1/2/3，观察不匹配时数据如何出错，验证 SPI 模式匹配的必要性。
3. 速度极限测试：将 SPI1 的 BaudRatePrescaler 从 64 改为 4/8/16 等更小值，测试最高可稳定通信的速率。
4. **DMA** 模式回环：将 SPI1 和 SPI2 都改为 DMA 模式收发，实现完全不占用 CPU 的后台传输。
5. 外接真实 **SPI** 设备：将 SPI1 连接到外部 SPI 设备（如 W25Q64 Flash、NRF24L01 无线模块），在回环测试基础上学习对外通信。

8.11 作业要求

8.11.1 基础要求（必做）

8.11.1.1 CubeMX 配置与工程搭建

1. 在 STM32CubeMX 中按本教程完成 **SPI1**（主机）和 **SPI2**（从机）的配置
2. 确保两个 SPI 外设的 CPOL/CPHA 参数完全一致
3. 生成 CMake 工程，编译成功（0 错误 0 警告）

验收标准：提供 SPI1 和 SPI2 的 CubeMX 配置截图各一张。

8.11.1.2 SPI 回环通信验证

1. 将本实验提供的 `main.c` 代码烧录到开发板（或自行编写等价的回环测试代码）
2. 用 4 条杜邦线连接 SPI1 和 SPI2 对应引脚
3. 在 `HAL_Delay(500)` 处设置断点，使用 VS Code 调试窗口观察变量

验收标准：提供两张调试截图（第一次和第二次运行到断点的变量窗口），证明 `master_rx == slave_tx` 且 `slave_rx == master_tx`，且数据交替翻转。

8.11.1.3 SPI 波形分析

使用逻辑分析仪或示波器同时抓取 **SCK**（**PA5**）、**MOSI**（**PA7**）、**MISO**（**PA6**）、**NSS**（**PA4**）四路信号，完成以下分析：

验收标准： - 标注 SCK 频率，验证 ~ 1.125 MHz - 在 MOSI 波形上标注出一个完整字节的 8 个 bit（高位 → 低位），验证其值与 `master_tx` 一致 - 在 MISO 波形上标注出对应字节，验证其值与 `slave_tx` 一致 - 标注 NSS 的使能区间（低电平有效），说明一次完整 SPI 传输的时序

8.11.2 进阶要求（选做，加分）

8.11.2.1 多字节数组传输

将单字节回环改为 16 字节数组回环测试。准备两组测试数据：- 主机发送递增序列：`0x00, 0x01, 0x02, ..., 0x0F` - 从机发送递减序列：`0xFF, 0xFE, 0xFD, ..., 0xF0`

用逻辑分析仪抓取完整时序，或在调试器中逐字节验证收发正确性。

验收标准： 16 字节全部收发正确，提供调试窗口截图或逻辑分析仪截图。

8.11.2.2 SPI 模式切换验证

将 SPI1 和 SPI2 的 CPOL/CPHA 分别配置为以下组合，用 LED 闪烁次数指示结果：- 组合 **1**：主机 Mode 0, 从机 Mode 0（正常通信 → LED 快闪）- 组合 **2**：主机 Mode 0, 从机 Mode 3（通信失败 → LED 慢闪）- 组合 **3**：主机 Mode 0, 从机 Mode 1（通信失败 → LED 熄灭）

验收标准：证明 SPI 的模式匹配是通信成功的前提，分析为何不匹配时数据出错。

8.11.3 实验报告要求

提交的实验报告应包含以下内容：

章节	内容
1. 实验目的	简述本实验的学习目标
2. 硬件连接	画出 STM32 SPI1 与 SPI2 的引脚连接图（实物照片 + 示意连线）
3. CubeMX 配置	截图说明 SYS、RCC、SPI1、SPI2、Clock 的配置参数
4. 核心代码	粘贴并解释回环测试主循环逻辑、回调函数、中断配置（不要全部粘贴 main.c）
5. SPI 波形分析	逻辑分析仪截图，标注 SCK/MOSI/MISO/NSS 各字段含义
6. 实验结果	VS Code 调试窗口截图（两次断点的变量对比）
7. 问题与思考	记录实验中遇到的问题及解决方案；思考题 **1**： SPI 的从机能否主动发起通信？为什么？思考题 **2**：如果主机和从机的 CPOL 不一致，具体会发生什么？思考题 **3**：为什么本实验中 SPI2 的 MOSI 引脚配置为输入模式？
8. 附录	完整的 `main.c` 代码（可选）

8.12 参考资料

- STM32F103C8T6 数据手册
- RM0008 – STM32F1xx 参考手册 (SPI 章节)
- STM32F1xx HAL 库用户手册 (UM1850)
- SPI Block Guide (Motorola/Freescale)
- STM32CubeMX 用户手册 (UM1718)

9 STM32 ADC 内置温度传感器采集与虚拟串口传输实验

9.1 统一作业说明

9.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成 ADC1（内置温度传感器通道 + VREFINT 内部参考电压通道）、USB CDC、时钟树、调试接口等配置，并保留 `.ioc` 文件。
2. 基于 HAL 库实现 ADC 轮询采集，先后读取温度传感器和 VREFINT 两个通道的数据。
3. 利用 VREFINT（内部参考电压 1.20V）反推实际 VDDA，对温度传感器数据进行校正，同时输出校正与未校正两组温度值。
4. 将 ADC 原始值、VDDA、校正/未校正温度通过 USB CDC 虚拟串口每 1 秒发送一次到电脑端串口调试助手。
5. 成功编译、下载并在电脑端串口调试助手中验证温度数据的正确输出。
6. 在实验报告中说明 ADC 时钟配置、VREFINT 校正原理、温度计算原理、代码结构、实验现象、问题排查和课后思考。
7. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

9.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6 或课程指定的带 USB 开发板
采集方式	ADC1 轮询采集温度传感器（通道 16）+ VREFINT（通道 17）双通道
通信方式	USB CDC 虚拟串口（VCP）
必做功能	每 1 秒采集双通道数据，用 VREFINT 校正温度并通过 VCP 发送
理论要求	能解释 ADC 采样时间、VREFINT 校正原理、温度传感器转换公式、12 位 ADC 量化原理
验收方式	现场演示或结果截图，能展示串口助手中每秒输出的校正/未校正温度数据

9.1.3 本次必须提交的内容

1. 一份 PDF 格式实验报告。
2. STM32CubeMX 配置截图（ADC1、时钟树、USB CDC 等）各至少 1 张。
3. 设备管理器识别截图、串口调试助手输出截图各至少 1 张。
4. 课后思考题的书面回答。

9.1.4 报告必须回答的问题

1. 说明 VREFINT（内部参考电压）通道的作用，以及它如何用于校正 ADC 测量结果。
2. 为什么 ADC 采样时间设置较长（如 239.5 Cycles）对温度传感器和 VREFINT 采集很重要。
3. STM32 内置温度传感器的 V25 和 Avg_Slope 参数有什么意义？为什么即使经过 VREFINT 校正，温度测量值仍可能与真实结温有偏差？
4. 本实验使用 `HAL_ADC_ConfigChannel` 动态切换通道来分时采集两个 ADC 通道，与在 CubeMX 中配置扫描模式一次性采集两个通道相比，各有什么优缺点？

9.2 实验目的

本实验基于 STM32 的 ADC1 采集芯片内置温度传感器和 VREFINT 内部参考电压数据，利用 VREFINT 对 ADC 结果进行校正，并通过 USB CDC 虚拟串口实时传输到电脑端显示。通过本实验，你应掌握以下内容：

1. 理解 STM32 ADC 的基本工作原理，包括采样时间、转换精度和量化方法。
2. 理解 STM32 内置温度传感器的电气特性和温度计算原理。
3. 理解 VREFINT（内部参考电压）的作用及其对 ADC 测量的校正原理。
4. 掌握 STM32CubeMX 中多个 ADC 内部通道（温度传感器 + VREFINT）的配置方法。
5. 学会使用 HAL 库进行多通道动态切换采集，以及 `HAL_ADC_ConfigChannel` 的运行时重配置。
6. 学会将 ADC 原始值转换为物理量（电压、温度），并理解整型运算在嵌入式系统中的优势。
7. 能够结合 USB CDC 虚拟串口与 ADC 采集，完成一个带有校正功能的完整数据采集与传输系统。

9.3 实验原理

9.1.1 STM32F103 内置温度传感器

STM32F103C8T6 芯片内部集成了一个温度传感器，连接到 **ADC1** 的通道 **16**。该温度传感器的电气特性如下（来自数据手册）：

参数	符号	典型值	范围	单位
25°C 时的输出电压	V25	1.43	1.34 ~ 1.52	V
温度变化斜率	Avg_Slope	4.3	4.0 ~ 4.6	mV/°C
ADC 采样时间	–	–	>= 17.1 us	–

温度计算公式：

$$\text{Temp}(^\circ C) = \frac{V_{25} - V_{sense}}{Avg_Slope} + 25$$

其中：- V_{25}：25°C 时传感器的输出电压（典型值 1.43V）- V_{sense}：ADC 实测的传感器输出电压 - Avg_Slope：温度每变化 1°C 输出电压的变化量（典型值 4.3 mV/°C）

温度传感器不能直接测量芯片外部环境温度，它测量的是芯片的结温（**Junction Temperature**）。由于芯片在运行时自身会发热，因此 ADC 采集到的温度通常高于环境温度。

9.3.2 VREFINT 内部参考电压与 ADC 校正原理

STM32F103 芯片内部集成了一个内部参考电压源（**VREFINT**），连接到 **ADC1** 的通道 **17**。其电气特性如下（来自数据手册）：

参数	典型值	说明
输出电压	**1.20 V**	内部基准电压，与 VDDA 无关
ADC 采样时间	>= 5.1 us	推荐较长的采样时间保证稳定

VREFINT 的核心用途—-校正 **VDDA** 波动：

ADC 的转换结果依赖于参考电压 VDDA（通常设计为 3.3V）。但实际电路中 VDDA 可能偏离 3.3V（例如 USB 供电时 VDDA 可能为 3.1V~3.5V），导致 ADC 读数产生比例误差。

VREFINT 的输出电压固定为 1.20V，不随 VDDA 变化。因此可以通过读取 VREFINT 通道的 ADC 值反推出实际 **VDDA**：

$$V_{DDA_actual} = \frac{1.20\text{V} \times 4096}{ADC_{VREFINT}}$$

然后使用实际的 VDDA 对其它通道的 ADC 读数进行校正：

$$V_{sensor_calibrated} = \frac{ADC_{sensor} \times V_{DDA_actual}}{4096}$$

示例：

- 读得 `ADC_VREFINT = 1429` → VDDA_actual = 1.20 × 4096 / 1429 = **3.439 V**（非标称的 3.3V）
- 读得 `ADC_TS = 1634` → V_ts_cal = 1634 × 3.439 / 4096 = **1.372 V** → 温度 38.5°C
- 而假设 VDDA = 3.3V 时 V_ts = 1634 × 3.3 / 4096 = **1.316 V** → 温度 51.5°C

两者相差 **13°C**，可见 VREFINT 校正对测温精度有显著影响。

9.3.3 ADC 量化原理

STM32F103 的 ADC 分辨率为 12 位，参考电压为 VDDA（通常 3.3V），量化关系如下：

$$V_{sense} = \frac{ADC_Value}{4096} \times V_{DDA}$$

其中：- ADC_Value 为读取到的 12 位原始值（0 ~ 4095）- V_{DDA} 为 ADC 参考电压（通常为 3.3V，但实际可能有偏差）- 4096 为 12 位 ADC 的量化级数（2^{12}）

9.3.4 USB CDC 虚拟串口原理

USB CDC（Communication Device Class）是 USB 协议中定义的一种设备类，它可以在 USB 总线上模拟传统的串行通信接口。当 STM32 通过 USB 连接电脑后，操作系统会将其识别为一个标准 COM 端口，应用程序（如串口调试助手）可以像操作普通串口一样进行读写。

与物理 UART 串口不同，USB CDC 的波特率、数据位、停止位等参数不影响实际传输速率——数据始终以 USB 全速（12 Mbps）在底层传输。

9.4 实验环境

9.4.1 硬件环境

1. 一块带 USB 接口的 STM32F103C8T6 开发板。
2. 一根可正常传输数据的 USB 数据线。
3. ST-Link 下载器或板载调试器。

9.4.2 软件环境

1. STM32CubeMX（v6.17.0 或更高版本）。
2. VS Code / STM32CubeIDE 或其他支持该工程的 STM32 开发环境。
3. ARM GCC / CMake 工具链或等效编译环境。
4. 串口调试助手。
5. ST-Link 驱动。

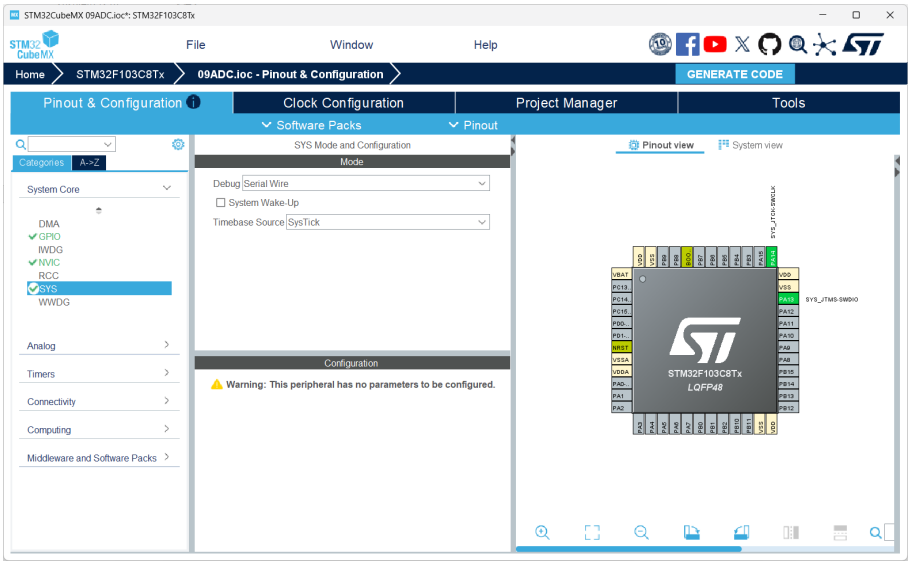
9.5 STM32CubeMX 配置步骤

9.5.1 新建工程并选择芯片

打开 STM32CubeMX，点击 ACCESS TO MCU SELECTOR，根据你使用的开发板或芯片型号选择 **STM32F103C8Tx**（LQFP48 封装）。

9.5.2 配置调试接口（SYS）

在 Pinout & Configuration 页面中，找到 **SYS**，将 **Debug** 设置为 **Serial Wire**。

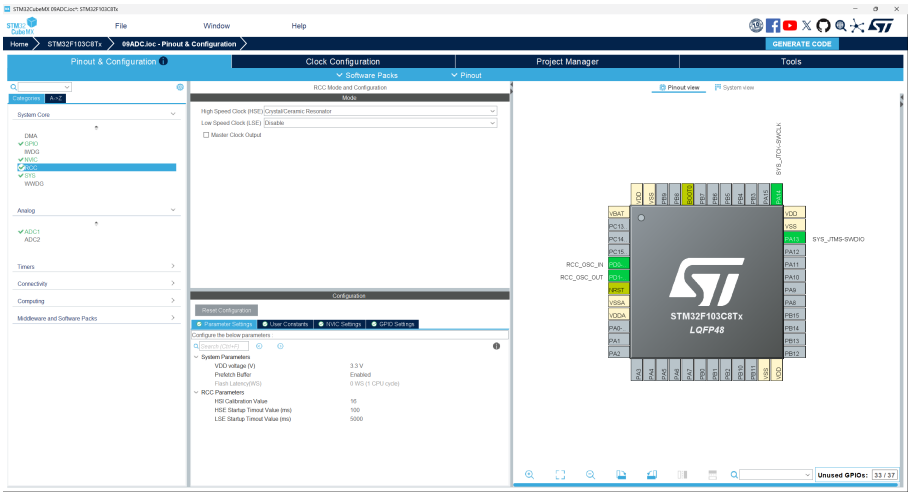


这样做的原因是：

1. 便于后续通过 ST-Link / DAP-Link 下载和调试程序。
2. 仅使用 SWD（SWDIO + SWCLK），不占用 JTAG 的其他引脚，释放 PA15、PB3、PB4。
3. 如果未正确配置调试接口，芯片可能被锁死无法调试。

9.5.3 配置高速外部时钟（RCC）

在 **RCC** 配置项中，将 **High Speed Clock (HSE)** 设置为 **Crystal/Ceramic Resonator**，即外部 8 MHz 晶振。

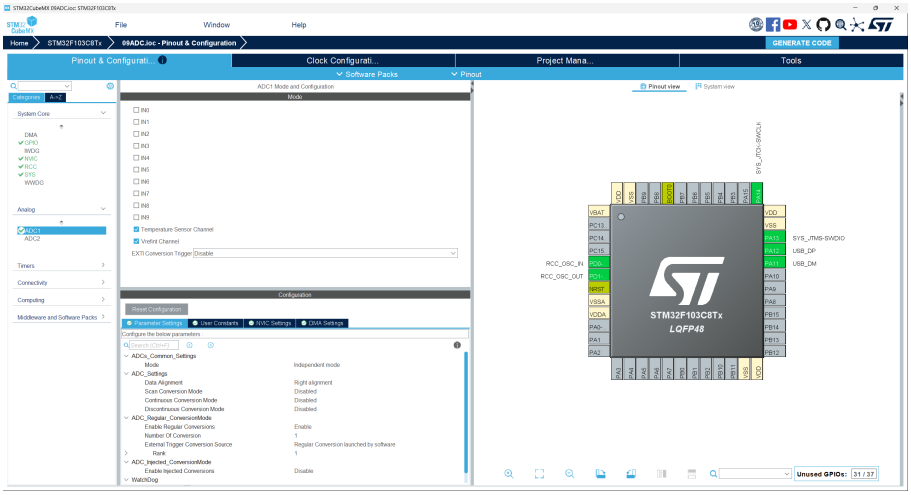


这样做的目的是为 PLL 和 USB 提供稳定、精确的时钟来源。内部 HSI 的精度（±1%）不足以满足 USB 通信对时钟精度的要求。

9.5.4 配置 ADC1（温度传感器 + VREFINT 双通道）

在 **Analog** → **ADC1** 中：

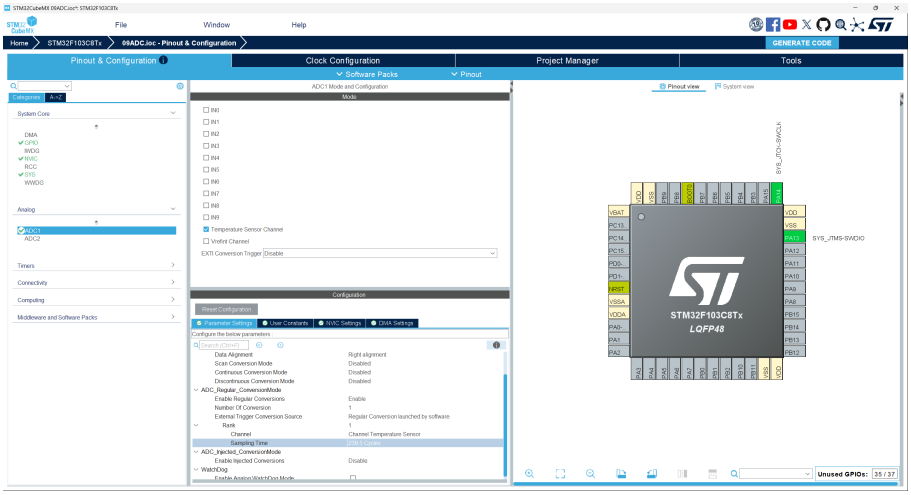
1. 勾选 **IN0** 区域的 **Temperature Sensor Channel**，使能内部温度传感器通道（ADC 通道 16）。
2. 勾选 **IN0** 区域的 **Vrefint Channel**，使能内部参考电压通道（ADC 通道 17）。
3. **Mode** 保持为 **Independent mode**（独立模式）。



ADC 参数配置说明：

参数	设置值	说明
ScanConvMode	DISABLE	代码中通过动态切换通道来分时采集两个通道
ContinuousConvMode	DISABLE	单次转换，由软件触发每次采集
ExternalTrigConv	ADC_SOFTWARE_START	软件触发，在代码中调用启动函数
DataAlign	ADC_DATAALIGN_RIGHT	数据右对齐，12 位结果在低 12 位
NbrOfConversion	1	CubeMX 初始化只配温度传感器通道，VREFINT 在代码中动态切换

9.5.4.1 配置 ADC 采样时间



采样时间选择 **239.5 Cycles**（`ADC_SAMPLETIME_239CYCLES_5`）。

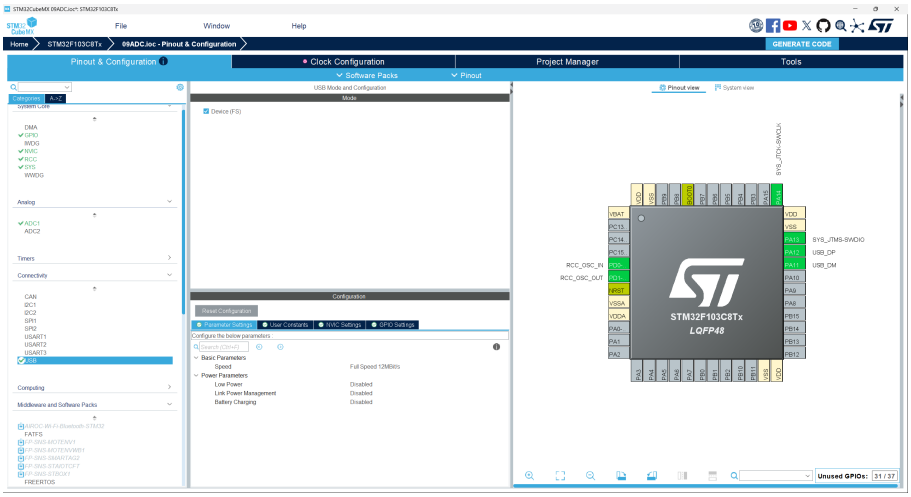
为什么需要较长的采样时间：

ADC 内部有一个采样保持电容，在采样阶段需要对该电容充电。温度传感器是内部模拟信号源，其输出阻抗较高。如果采样时间太短，采样电容上的电压不足以达到与信号源相同的电平，导致 ADC 读数偏低且不稳定。数据手册要求温度传感器的采样时间至少为 17.1 us。

在 12 MHz ADC 时钟下：- 采样时间 = (239.5 + 12.5) / 12 MHz ~ 21 us（其中 12.5 为固定的逐次逼近转换周期）- 20.0 us > 17.1 us，满足数据手册要求

9.5.5 启用 USB 外设

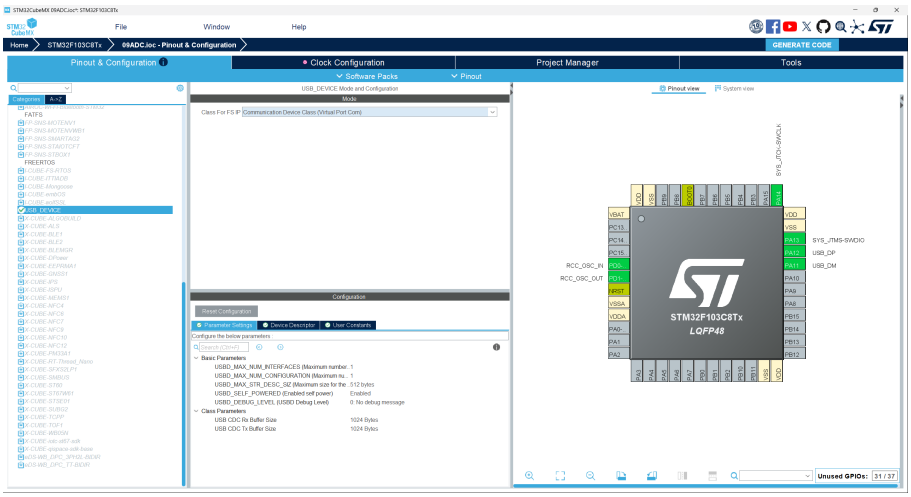
在左侧 **Connectivity** 菜单下找到 **USB**，将其配置为 **Device (FS)**。



此步骤启用芯片的 USB 全速设备功能，PA11（USB_DM）和 PA12（USB_DP）引脚将自动配置为 USB 差分数据线。

9.5.6 配置 USB CDC 中间件

在 **Middleware and Software Packs** 中打开 **USB_DEVICE**，将 **Class for FS IP** 设置为 **Communication Device Class (Virtual Port Com)**。



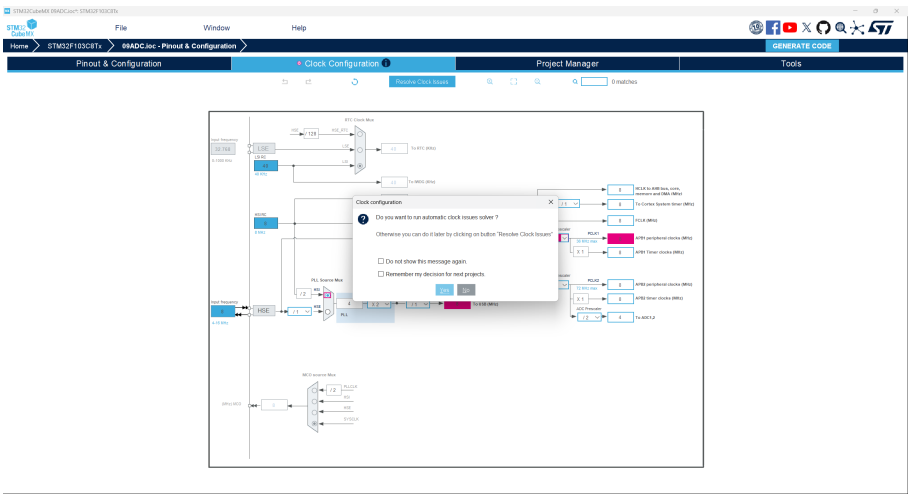
这一步的意义在于：

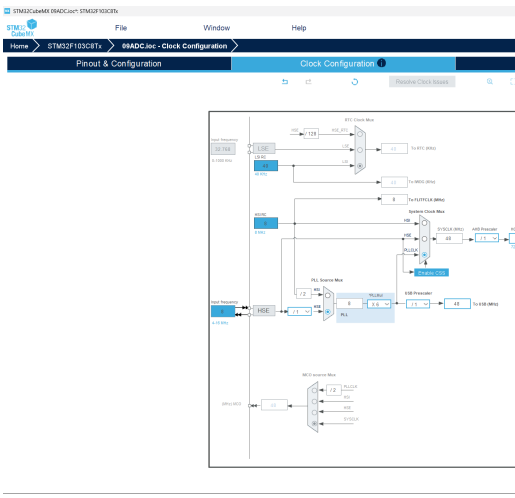
1. 告诉 CubeMX 当前 USB 设备工作在 CDC 类（通信设备类）。
2. 自动生成 USB 虚拟串口所需的设备描述符、接口文件和 CDC 中间件代码。
3. 电脑端将识别此设备为标准 COM 端口。

9.5.7 配置时钟树

这是实验最关键的一步。进入 **Clock Configuration** 页面：

1. **HSE**：8 MHz（外部晶振）。
2. **PLL Source**：选择 HSE。
3. **PLL Mul**：设置为 ×6，PLL 输出 = 8 MHz × 6 = 48 MHz。
4. **SYSCLK**：选择 PLLCLK = 48 MHz。
5. **AHB Prescaler**：÷1，HCLK = 48 MHz。
6. **APB1 Prescaler**：÷2，APB1 = 24 MHz（USB 挂载在 APB1 上）。
7. **APB2 Prescaler**：÷1，APB2 = 48 MHz（ADC1 挂载在 APB2 上）。
8. **ADC Prescaler**：÷4，ADC 时钟 = 48 MHz / 4 = 12 MHz。
9. **USB Prescaler**：选择 PLLCLK，USB 时钟 = 48 MHz。





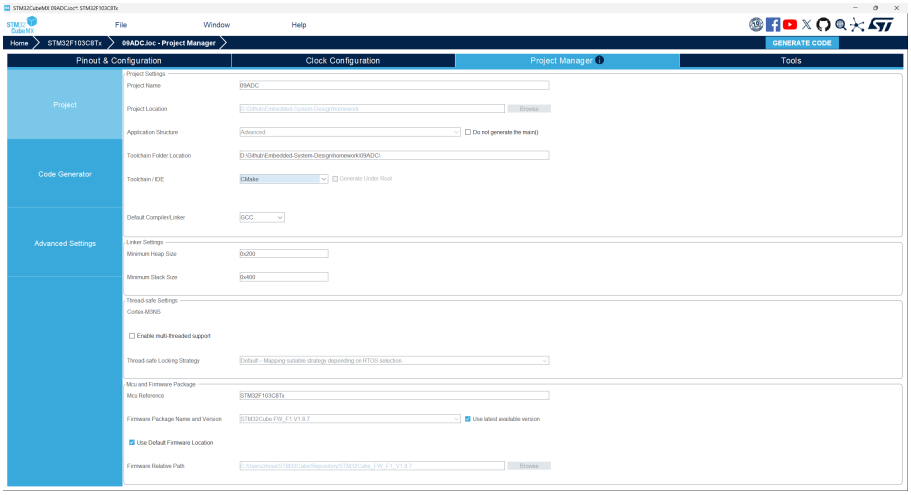
为什么必须精确配置：

- USB 全速设备要求精确的 **48 MHz** 时钟。误差超过 ±0.25%（±120 kHz）可能导致设备无法枚举或通信出错。
- ADC 时钟最大允许 14 MHz，12 MHz 在安全范围内，保证了逐次逼近转换的正确性。
- APB2 为 48 MHz，保证了 GPIO 和其他高速外设的运行效率。

9.5.8 配置工程并生成代码

在 **Project Manager** 页面中：

1. **Project Name**：设置为 `09ADC`。
2. **Project Location**：选择合适的保存路径。
3. **Toolchain / IDE**：选择 **CMake**（与课程仓库结构保持一致）。
4. **Stack Size**：0x400；**Heap Size**：0x200。
5. 点击 **GENERATE CODE** 生成工程。



以上界面确认无误后即可继续后续硬件连接与代码说明。

9.6 硬件连接要点

1. USB_DP（PA12）与 USB_DM（PA11）分别连接 USB 连接器的 D+ 与 D-。
2. 对于 STM32F1 系列，确保开发板的 USB D+ 线路上有 1.5kΩ 上拉电阻到 3.3V（这是 USB 全速设备枚举的硬件要求）。
3. USB 线必须是数据线（内部有四根线：VCC、GND、D+、D-），不能只用供电线，否则电脑无法识别。
4. 温度传感器在芯片内部，无需任何外部接线。

9.7 核心代码说明

这一部分把本次实验中真正修改过的源码放到文档中，并逐段解释其含义。

9.7.1 main.c 中新增的头文件

```
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "usbd_cdc_if.h"
/* USER CODE END Includes */
```

代码解析：

1. `stdio.h`：提供 `snprintf` 函数，用于将温度数值格式化到字符串缓冲区。
2. `usbd_cdc_if.h`：声明 `CDC_Transmit_FS()` 函数，用于通过 USB 虚拟串口向电脑发送数据。

9.7.2 main.c 中的初始化阶段代码

```
/* USER CODE BEGIN 2 */
/* USB 枚举需要时间，上电后等待片刻 */
HAL_Delay(800);

CDC_Transmit_FS((uint8_t *)"\\r\\n--- ADC Temp Demo ---\\r\\n\\r\\n", 27);
/* USER CODE END 2 */
```

代码解析：

1. USB 初始化完成后延时 800ms，等待电脑端完成 USB 枚举和驱动加载。
2. 发送一条启动信息，方便确认系统已正常启动且 VCP 通信正常。
3. 如果设备刚初始化完成就立即发送数据，电脑端可能尚未完成枚举，导致首条信息发送失败。

9.7.3 main.c 中的主循环核心代码

```
while (1)
{
    /* USER CODE BEGIN 3 */
    uint32_t adc_ts = 0, adc_vref = 0;

    /* ---- ① 采集温度传感器通道 ---- */
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK)
    {
        adc_ts = HAL_ADC_GetValue(&hadc1);
    }
    HAL_ADC_Stop(&hadc1);

    /* ---- ② 切换到 VREFINT 通道再采集 ---- */
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = ADC_CHANNEL_VREFINT;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);

    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK)
    {
        adc_vref = HAL_ADC_GetValue(&hadc1);
    }
    HAL_ADC_Stop(&hadc1);

    /* ---- ③ 切回温度传感器通道，为下一次采集做准备 ---- */
    sConfig.Channel = ADC_CHANNEL_TEMPSENSOR;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);

    if (adc_ts == 0 || adc_vref == 0)
    {
        HAL_Delay(1000);
        continue;
    }

    /* ---- ④ 未校正：直接假设 VDDA = 3.3V ---- */
    int v_ts_raw_mv = (int)(adc_ts * 3300 / 4096);
    int temp_raw_x10 = ((1430 - v_ts_raw_mv) * 100) / 43 + 250;

    /* ---- ⑤ VREFINT 校正：VREFINT 固定为 1.20V（数据手册典型值）---- */
    /* 反推实际 VDDA = 1.20V × 4096 / adc_vref（单位：mV） */
```

```
    int vdda_mv = 1200 * 4096 / (int)adc_vref;

    /* 校正后的温度传感器电压 (mV)  */
    int v_ts_cal_mv = (int)((uint32_t)adc_ts * vdda_mv / 4096);

    /* 校正后的温度 */
    int temp_cal_x10 = ((1430 - v_ts_cal_mv) * 100) / 43 + 250;

    /* ---- ⑥ 格式化并发送 ---- */
    char msg[128];
    int len = snprintf(msg, sizeof(msg),
                       "TS=%lu VREF=%lu VDDA=%d.%03dV | "
                       "Uncal: V=%d.%03dV T=%d.%dC | "
                       "Cal:   V=%d.%03dV T=%d.%dC\\r\\n",
                       adc_ts, adc_vref,
                       vdda_mv / 1000, vdda_mv % 1000,
                       v_ts_raw_mv / 1000, v_ts_raw_mv % 1000,
                       temp_raw_x10 / 10, (temp_raw_x10 < 0 (1) -temp_raw_x10 : temp_raw_x10) % 10,
                       v_ts_cal_mv / 1000, v_ts_cal_mv % 1000,
                       temp_cal_x10 / 10, (temp_cal_x10 < 0 (1) -temp_cal_x10 : temp_cal_x10) % 10);
    CDC_Transmit_FS((uint8_t *)msg, len);

    HAL_Delay(1000);
    /* USER CODE END 3 */
}
```

流程说明:

步骤	函数/操作	说明
	采集温度传感器	CubeMX 初始化为 TEMPSENSOR 通道，直接 Start → Poll → Get → Stop
	切换采集 VREFINT	运行时调用 `HAL_ADC_ConfigChannel` 动态切换到 VREFINT 通道，再重复采集流程
	切回温度传感器	为下一轮循环恢复温度传感器通道配置
	未校正计算	假设 VDDA = 3.3V，直接用 V = ADC × 3300 / 4096 计算电压和温度
	VREFINT 校正计算	用 VREFINT = 1.20V 反推实际 VDDA，重新计算校正后的电压和温度
	`CDC_Transmit_FS()`	将所有数据格式化后通过 USB CDC 发送到 PC

9.7.4 温度计算原理详解

9.7.4.1 浮点公式

STM32F103 数据手册中给出温度传感器公式:

$$Temp(°C) = \frac{V_{25} - V_{sense}}{Avg_Slope} + 25$$

代入典型值 V25 = 1.43V, Avg_Slope = 4.3 mV/°C:

$$Temp = \frac{1.43 - V_{sense}}{0.0043} + 25$$

9.7.4.2 整型优化

直接使用浮点运算有两个问题：1. STM32F103 没有硬件 FPU，浮点运算由软件模拟，速度较慢。2. 本工程使用 `--specs=nano.specs`（newlib-nano），默认不支持 `%f` 浮点格式化。如需支持需添加 `-u _printf_float` 链接选项，会显著增加代码体积。

因此本实验采用整型运算，所有计算在 mV 量纲下完成：

```
voltage_mv = ADC值 × 3300 / 4096                    （电压，mV）
temp_times_10 = ((1430 - voltage_mv) × 100) / 43 + 250    （温度 × 10）
```

推导过程：

$$Temp \times 10 = \frac{(1430 - V_{sense_mV}) \times 10}{4.3} + 250 = \frac{(1430 - V_{sense_mV}) \times 100}{43} + 250$$

最终温度输出时：- 整数部分 = `temp_times_10 / 10` - 小数部分 = `|temp_times_10| % 10`

9.7.4.3 计算示例

假设 ADC 原始值 = 1631：

1. 电压：1631 × 3300 / 4096 = 1314 mV = 1.314 V
2. 温度 × 10：((1430 - 1314) × 100) / 43 + 250 = 11600 / 43 + 250 = 269 + 250 = 519
3. 显示温度：51.9°C

9.7.5 CDC_Transmit_FS 发送函数

`CDC_Transmit_FS()` 定义在 `usbd_cdc_if.c` 中，由 STM32CubeMX 自动生成：

```c
uint8_t CDC_Transmit_FS(uint8_t* Buf, uint16_t Len)
{
    uint8_t result = USBD_OK;
    USBD_CDC_HandleTypeDef *hcdc = (USBD_CDC_HandleTypeDef*)hUsbDeviceFS.pClassData;
    if (hcdc->TxState != 0){
        return USBD_BUSY;  // 上次发送未完成，返回忙
    }
    USBD_CDC_SetTxBuffer(&hUsbDeviceFS, Buf, Len);
    result = USBD_CDC_TransmitPacket(&hUsbDeviceFS);
    return result;
}
```

注意要点：- 函数会先检查发送状态，若上次发送未完成则返回 USBD_BUSY。- 本实验每秒只发送一次短数据（约 40 字节），基本不会遇到忙状态。- 如果需要连续高频发送，应像作业 4（USB CDC 实验）那样进行忙状态重试处理。

9.7.6 ADC1 初始化代码（由 STM32CubeMX 生成）

```
static void MX_ADC1_Init(void)
{
    ADC_ChannelConfTypeDef sConfig = {0};

    hadc1.Instance = ADC1;
    hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;         // 单通道，不扫描
    hadc1.Init.ContinuousConvMode = DISABLE;            // 单次转换模式
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;   // 软件触发
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;         // 数据右对齐
    hadc1.Init.NbrOfConversion = 1;                     // 每次转换 1 个通道
    HAL_ADC_Init(&hadc1);

    sConfig.Channel = ADC_CHANNEL_TEMPSENSOR;           // 内部温度传感器通道
    sConfig.Rank = ADC_REGULAR_RANK_1;                  // 规则组排名第 1
    sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5; // 采样时间 239.5 周期
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
}
```

关键参数说明：

- ADC_CHANNEL_TEMPSENSOR：STM32 内部温度传感器通道（通道 16），无需外部引脚。
- ADC_SOFTWARE_START：由软件调用 HAL_ADC_Start() 触发转换，无需外部触发源。
- ADC_DATAALIGN_RIGHT：数据右对齐，12 位转换结果放在数据寄存器的低 12 位（bit 0-11）。
- ADC_SAMPLETIME_239CYCLES_5：采样周期 239.5 个 ADC 时钟周期，保证温度传感器信号充分建立。
- CubeMX 初始化时只配置了温度传感器通道，**VREFINT** 通道（**ADC_CHANNEL_VREFINT**，通道 **17**）在代码运行时通过 HAL_ADC_ConfigChannel 动态切换，无需修改 CubeMX 生成的初始化代码。

9.7.7 系统时钟配置代码（由 STM32CubeMX 生成）

```
static void SystemClock_Config(void)
{
    // HSE 8MHz → PLL ×6 → SYSCLK 48MHz
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL6;

    // APB1 ÷2 = 24MHz, APB2 ÷1 = 48MHz
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    // ADC 时钟 = APB2 / 4 = 12 MHz
    // USB 时钟来源 = PLL = 48 MHz
    PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_ADC | RCC_PERIPHCLK_USB;
    PeriphClkInit.AdcClockSelection = RCC_ADCPCLK2_DIV4;
    PeriphClkInit.UsbClockSelection = RCC_USBCLKSOURCE_PLL;
}
```

9.7.8 代码结构的整体工作流程

本实验代码可以概括为以下步骤：

1. 系统启动后初始化 HAL 库、系统时钟（48 MHz）。
2. 初始化 GPIO、ADC1（温度传感器通道）、USB CDC 虚拟串口。
3. 等待 800ms 让 USB 完成枚举，发送启动提示信息。
4. 进入主循环，每轮执行：
    - 采集温度传感器通道 → Start → Poll → Get → Stop
    - 动态切换到 VREFINT 通道 → ConfigChannel → Start → Poll → Get → Stop
    - 切回温度传感器通道，为下一轮做准备
    - 计算未校正温度（VDDA = 3.3V 假设）
    - 用 VREFINT = 1.20V 反推实际 VDDA，计算校正后温度
    - 格式化所有数据并发送
    - 延时 1000 ms

这就是一个典型的”双通道分时采集 → 校正计算 → 数据上传”的嵌入式数据采集与校正系统结构。

为什么要动态切换通道而非使用扫描模式：

本实验选择在代码中用 `HAL_ADC_ConfigChannel` 运行时重配置通道，而不是在 CubeMX 中配置扫描模式（ScanConvMode = ENABLE, NbrOfConversion = 2），原因是：

1. CubeMX 重新生成代码不会覆盖主循环中动态切换通道的逻辑。
2. 分时采集两个通道避免了扫描模式下连续转换两次后需要区分哪个值对应哪个通道的问题。
3. 对于 1 Hz 的低频采集，两次分时采集之间的微小时间差完全可以忽略。

9.8 编程技巧总结

1. 使用整型运算替代浮点运算：在无 FPU 的 Cortex-M3 上避免浮点计算，同时规避 newlib-nano 下浮点 printf 不支持的问题。
2. **ADC** 单次转换 + 软件触发：对于每秒一次的低频采集，比连续转换模式更节能，且代码逻辑更清晰。
3. 先 **Start** → 等待完成 → 读取 → **Stop**：这是 HAL 轮询方式的标准 ADC 操作流程，顺序不能颠倒。
4. 运行时动态切换 **ADC** 通道：使用 `HAL_ADC_ConfigChannel` 在运行中切换 ADC 通道，比 CubeMX 中配置扫描模式更灵活，且不依赖 CubeMX 代码生成。
5. **VREFINT** 校正无需读取 **Flash** 校准值：直接使用数据手册中 VREFINT = 1.20V 的典型值即可完成校正，不依赖芯片是否烧录了出厂校准数据。
6. 所有用户代码放在 `USER CODE` 块内：确保 STM32CubeMX 重新生成代码时不会覆盖自定义逻辑。
7. **USB** 上电后需要等待枚举：如果代码中需要在启动阶段发送数据，应在 USB 初始化后加数百毫秒延时。
8. 温度传感器测量的是结温而非环境温度：实验报告中不应将测量值等同于室温，应说明芯片自发热的影响。

9.9 实验操作步骤

9.9.1 编译工程

使用你当前的开发环境（STM32CubeIDE / VS Code / CLion）编译工程，确认无编译错误。

9.9.2 烧录程序

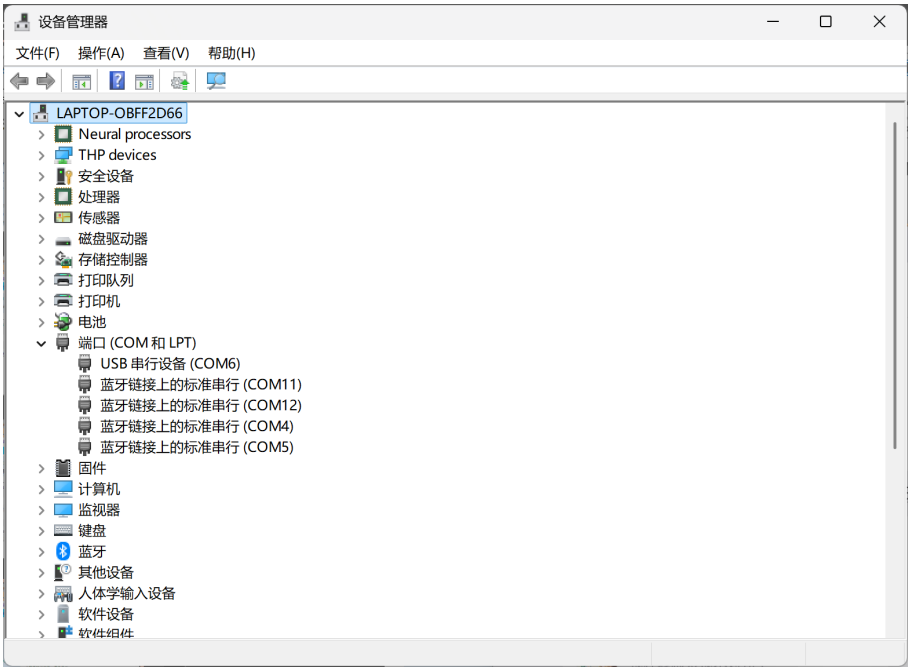
通过 ST-Link 将编译生成的 `.elf` 或 `.bin` 文件下载到 STM32 开发板。

9.9.3 连接电脑

使用 USB 数据线将开发板的 **USB** 口（PA11/PA12 对应的接口）连接到电脑，等待系统识别虚拟串口。

9.9.4 查看 COM 口

1. 打开设备管理器。
2. 展开端口（**COM** 和 **LPT**）。
3. 确认出现新设备（如 `STMicroelectronics Virtual COM Port (COMx)`），并记录 COM 口编号。



9.9.5 打开串口调试助手

在串口调试助手中：

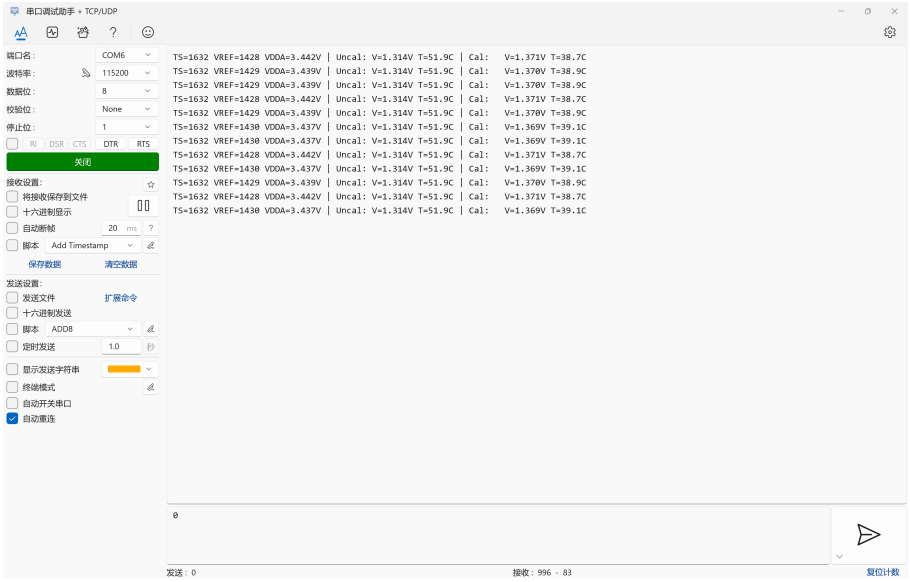
1. 选择对应 COM 口。
2. 波特率、数据位等参数可保持默认（USB CDC 无视这些设置，但建议统一使用 115200 8N1）。
3. 打开串口。

9.9.6 观察温度输出

打开串口后，每 1 秒应收到一条温度数据，格式如下：

```
TS=1634 VREF=1429 VDDA=3.439V | Uncal: V=1.316V T=51.5C | Cal: V=1.372V T=38.5C
```

其中：- `TS` / `VREF`：温度传感器和 VREFINT 的 ADC 原始值 - `VDDA`：根据 VREFINT = 1.20V 反推的实际 VDDA - `Uncal`：未校正（假设 VDDA = 3.3V）的电压和温度 - `Cal`：用实际 VDDA 校正后的电压和温度



9.9.7 记录实验结果

建议在实验报告中保留以下截图：

1. CubeMX 配置截图（ADC1 双通道、时钟树、USB CDC 各至少 1 张）。
2. 设备管理器中虚拟串口截图 1 张。
3. 串口调试助手中连续多行的完整温度输出截图 1 张（含 TS、VREF、VDDA、校正/未校正温度）。

9.10 实验现象与结果分析

如果实验成功，你应看到如下现象：

1. 电脑成功识别到 STM32 虚拟串口设备。
2. 打开串口调试助手后，显示启动提示 `--- ADC Temp Demo ---`。
3. 每秒收到一条包含 TS、VREF、VDDA、校正/未校正温度的完整数据。
4. TS 值在 1630~1650 左右，VREF 值在 1420~1440 左右（以实际为准）。
5. 未校正温度约 45°C~55°C，校正后温度约 35°C~45°C（校正后约低 8~13°C）。

校正前后的数据对比分析（实测数据示例）：

参数	未校正 (VDDA=3.3V)	校正后 (VDDA~3.44V)
VDDA	3.300 V（固定假设）	3.439 V（VREFINT 反推）
温度传感器电压	$1634 \times 3.3 \,/\, 4096 = 1.316$ V	$1634 \times 3.439 \,/\, 4096 = 1.372$ V
计算温度	(1.43-1.316)/0.0043+25 = **51.5°C**	(1.43-1.372)/0.0043+25 = **38.5°C**

结论分析：

1. **VREFINT** 校正有效：实际 VDDA（3.44V）明显高于标称值 3.3V，说明开发板的 3.3V 供电偏高。修正后温度下降约 13°C，说明了 VREFINT 校正的重要性。
2. **USB** 枚举成功：CDC 虚拟串口驱动正常，48 MHz 时钟配置正确。
3. 双通道 **ADC** 采集正常：温度传感器和 VREFINT 通道读数均稳定，采样时间充足。
4. 温度计算正确：整型运算公式无误，数值在合理范围内。
5. 芯片自发热现象：即使校正后温度（38.5°C）仍可能高于室温，因为 STM32F103 在 48 MHz 全速运行且 USB 外设工作时有约 10~20°C 的自发热。

关于精度的讨论：

STM32F103 内置温度传感器主要用于芯片过热保护（如检测芯片是否超过 125°C 阈值），而非精确的环境温度测量。其典型绝对精度约为 ±10°C，主要原因有：

1. V25 参数存在 ±90 mV 的制造偏差（对应约 ±21°C 的初始测量误差）。
2. Avg_Slope 参数存在 ±0.3 mV/°C 的偏差。
3. 芯片自发热与工作频率、外设使用情况、PCB 散热设计密切相关。
4. VREFINT 本身的误差（典型值 1.20V，范围 1.16V~1.24V）也会影响校正精度。

VREFINT 校正能消除 VDDA 偏差引入的误差，但无法消除温度传感器 V25 和 Avg_Slope 本身的制造偏差。如需更高精度的测温，应结合出厂校准值（TS_CAL1）或使用外部温度传感器。

9.11 常见问题排查

问题	可能原因与解决方案
串口助手中没有输出	检查 USB 时钟是否为 48MHz；确认 ADC 转换是否成功（可在调试器中打断点查看）；检查 `CDC_Transmit_FS` 返回值。
温度显示为乱码或空值	确认 `snprintf` 格式化字符串正确；检查栈大小是否足够（建议 0x400 以上）。
温度值明显异常（如 >100°C 或 <0°C）	检查 ADC 参考电压是否正确（应为 3.3V）；确认 V25 和 Avg_Slope 参数在代码中是否正确使用；检查整型计算公式有无溢出。
VREFINT 校正后温度反而不合理	检查 `VDDA = 1200 × 4096 / VREF` 的计算是否正确；确认 VREFINT ADC 值是否正常（通常 1400~1500 左右）；检查 `HAL_ADC_ConfigChannel` 是否成功切换通道。

问题	可能原因与解决方案
ADC 值读数不稳定、跳动大	增加采样时间（如从 239.5 改为 71.5 试下看是否更差）；检查电源是否干净；可采用多次采样取平均值的滤波方法。
电脑无法识别设备	检查 USB 时钟是否为 48MHz；检查 USB 线是否为数据线；确认开发板 D+ 上拉电阻正常。
串口助手打开后无数据显示	确认电脑已识别 COM 口；检查串口助手 COM 号是否正确；尝试按一下开发板复位按钮。

9.12 课后思考题

请结合本次实验，在报告中认真回答以下问题：

1. 说明 **VREFINT**（内部参考电压）通道的作用，以及它如何用于校正 **ADC** 测量结果。（提示：从 VDDA 波动和 ADC 量化的角度分析。）
2. 为什么 **ADC** 采样时间设置较长（如 **239.5 Cycles**）对温度传感器和 **VREFINT** 采集很重要？（提示：从内部模拟信号源的输出阻抗和 ADC 采样保持电容的角度分析。）
3. **STM32** 内置温度传感器的 **V25** 和 **Avg_Slope** 参数有什么意义？为什么即使经过 **VREFINT** 校正，温度测量值仍可能与真实结温有偏差？（提示：从制造工艺偏差和芯片自发热两个角度分析。）
4. 本实验使用 `HAL_ADC_ConfigChannel` 动态切换通道来分时采集两个 **ADC** 通道，与在 **CubeMX** 中配置扫描模式（**ScanConvMode = ENABLE**）一次性采集两个通道相比，各有什么优缺点？

9.13 实验报告提交要求

9.13.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述本实验需要实现的 ADC 采集与 VCP 传输功能。
3. 实验原理：简述内置温度传感器原理、ADC 量化公式、温度计算公式。
4. 实验环境：开发板型号、USB 连接方式、调试器、软件版本、上位机工具。
5. **CubeMX** 配置：必须说明 SYS、RCC、ADC1、USB、USB_DEVICE、Clock Configuration 的关键设置及其原因。
6. 程序设计：至少分析 `main.c` 中的主循环逻辑（含每个步骤的说明）、ADC 初始化代码和温度计算原理。
7. 程序流程图：必须展示”ADC 启动 → 等待转换 → 读取数据 → 电压转换 → 温度计算 → 格式化 → VCP 发送 → 延时 1s” 的完整流程。
8. 实验步骤：包括编译、下载、连接电脑、识别 COM 口、串口助手观察数据等过程。
9. 实验结果：必须展示设备管理器识别结果和串口调试助手温度输出结果。对温度数据进行分析（是否合理、误差来源等）。
10. 问题分析与调试记录：说明遇到的任何问题及排查方法。
11. 课后思考题答案：必须完整回答本 README 中 4 个思考题。

12. 总结：概括你对 STM32 ADC、内置温度传感器和 USB CDC 数据采集系统的理解。

9.13.2 图片与流程图要求

1. 报告中至少包含 4 类图片：CubeMX 配置截图、时钟树截图、设备管理器截图、串口调试助手输出截图。
2. 每张图片必须标注图号和图题（如” 图 1 ADC1 温度传感器通道配置”），并在正文中引用说明。
3. 程序流程图必须单独成图，不能只用文字描述替代。

9.13.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 9-ADC 温度传感器实验.pdf`。
2. 若实验未完全成功，也必须提交完整报告，重点说明失败现象、原因分析和改进方向。
3. 报告中不能只贴代码或截图，必须结合实验现象进行解释和分析。

9.13.4 最低验收标准

1. 电脑能够识别出虚拟串口设备。
2. 串口调试助手能每 1 秒收到一条完整数据，包含 TS、VREF、VDDA、校正温度、未校正温度。
3. 输出格式为 `TS=< 值 > VREF=< 值 > VDDA=< 电压 >V | Uncal: V=... Cal: V=...`。
4. 温度计算值在合理范围内（校正后约 35°C ~ 50°C），且校正后温度与未校正温度有明显差异。
5. 报告能够解释 VREFINT 校正原理和 ADC 采集流程。

9.14 可进一步扩展的方向

1. 多点校准：用外部精确温度计在两点（如 0°C 冰水混合物和 25°C 室温）标定实际 V25 和 Avg_Slope，结合 VREFINT 校正后精度可显著提高。
2. 多次采样取平均：连续采集 8~16 次 VREFINT 和 TS 取平均值，降低随机噪声影响。
3. 添加中断/**DMA** 模式：将 ADC 改为定时器触发 + DMA 传输配合扫描模式，实现后台自动采集双通道。
4. 结合作业 **4** 的命令交互：添加 `temp`、`vdda`、`vref` 等命令，在收到命令时才返回数据，而非定时发送。
5. 添加 **OLED/LCD** 显示：将校正后的温度数据同时显示在本地屏幕上。
6. 过温报警：当校正后温度超过阈值时，点亮 LED 或发出蜂鸣器告警（模拟芯片过热保护功能）。

9.15 总结

本实验不是简单的”读一个 ADC 值、发一个字符串”，而是要你掌握从 CubeMX 配置（ADC 双通道 + 采样时间 + 时钟树）、到 HAL 库编程（运行时通道切换 + 轮询采集流程）、到数据处理（物理量转换、VREFINT 校正、整型运算优化）和上位机验证的完整嵌入式开发流程。

实验的核心价值在于让你理解 VREFINT 校正的实际意义—-电源电压的微小偏差会导致 ADC 测量结果出现显著误差，而利用芯片内置的 1.20V 参考电压可以低成本地提升测量精度。这种”利用已知基准反推实际工作条件”的思想，不仅适用于 ADC，也是嵌入式系统中传感器校准的通用方法之一。同时，通过对温度传感器精度和自发热的分析，加深对”数据手册参数 vs 实际测量值”之间差异的理解—-这是嵌入式工程师在实际项目中最重要的工程判断力之一。

9.16 参考

- STM32F103x8/xB Datasheet（第 5.3.22 节: Temperature sensor characteristics）
- RM0008 Reference Manual（第 11 章: Analog-to-digital converter）
- STM32CubeMX User Manual (UM1718)
- 本课程作业 4（USB CDC 虚拟串口实验）相关代码和说明

10 STM32 ADC 温度传感器 DMA 采集与虚拟串口高速传输实验

10.1 统一作业说明

10.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成 ADC1（内置温度传感器通道）、DMA（循环模式）、USB CDC、时钟树、调试接口等配置，并保留 `.ioc` 文件。
2. 基于 HAL 库实现 ADC + DMA 连续采集温度传感器数据，通过 DMA 传输完成回调自动获取每次转换结果。
3. 通过 USB CDC 虚拟串口将 ADC 原始值发送到电脑端。
4. 用 Python 编写客户端程序，实时解析 ADC 数据、计算温度并绘制温度曲线。
5. 成功编译、下载并在 Python 客户端中验证温度波形和实时更新。
6. 在实验报告中说明 ADC 连续转换模式、DMA 循环模式、`HAL_ADC_ConvCpltCallback` 回调机制、newlib-nano 浮点格式化限制、Python 客户端设计。
7. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

10.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6 或课程指定的带 USB 开发板
采集方式	ADC1 连续转换 + DMA1 循环传输，后台自动采集温度传感器（通道 16）
通信方式	USB CDC 虚拟串口（VCP）
必做功能	ADC 连续转换 → DMA 搬运 → 回调中通过 VCP 发送原始 ADC 值
理论要求	能解释 DMA 循环模式、ADC 连续转换模式、HAL 回调机制、newlib-nano 浮点限制
验收方式	Python 客户端实时显示温度曲线，波形连续无断点

10.1.3 本次必须提交的内容

1. 一份 PDF 格式实验报告。
2. STM32CubeMX 配置截图（ADC1、DMA、时钟树、USB CDC 等）各至少 1 张。
3. Python 客户端运行截图至少 1 张。
4. 课后思考题的书面回答。

10.1.4 报告必须回答的问题

1. 说明 DMA 循环模式（Circular Mode）与普通模式（Normal Mode）的区别，以及为什么本实验需要循环模式。
2. 为什么本实验中 ADC 必须配置为连续转换模式（ContinuousConvMode = ENABLE），若配置为单次转换模式结合 DMA 循环模式会出现什么问题?
3. STM32F103 内置温度传感器在 HAL 初始化后需要手动置位 TSVREFE 位才能工作。为什么 CubeMX 生成的 `MX_ADC1_Init()` 中不自动使能该位?
4. 本实验中 `HAL_ADC_ConvCpltCallback` 回调函数在什么上下文中被调用（中断 or 主循环)? 在高频数据采集（约 1000~2000 sps）场景下，回调中直接调用 `CDC_Transmit_FS` 可能存在什么问题? 如何优化?

10.2 实验目的

本实验基于 STM32 的 ADC1 配合 DMA 循环模式连续采集芯片内置温度传感器数据，并通过 USB CDC 虚拟串口高速发送到电脑端，使用 Python 客户端实时解析并绘制温度曲线。通过本实验，你应掌握以下内容:

1. 理解 STM32 DMA（直接存储器访问）的工作原理，特别是循环模式与 ADC 配合时的数据传输流程。
2. 理解 ADC 连续转换模式与单次转换模式的区别，以及如何与 DMA 协同工作。
3. 掌握 STM32CubeMX 中 ADC + DMA 的联合配置方法。
4. 学会使用 HAL 库的 DMA 传输完成回调 `HAL_ADC_ConvCpltCallback` 进行数据处理。
5. 理解 newlib-nano 库对浮点格式化的限制及其对嵌入式串口输出的影响。
6. 学会编写 Python 上位机程序，通过 pyserial + matplotlib 实现实时数据采集与可视化。
7. 能够评估 ADC + DMA + USB CDC 系统的数据吞吐能力。

10.3 实验原理

10.3.1 STM32F103 内置温度传感器

STM32F103C8T6 芯片内部集成了一个温度传感器，连接到 **ADC1** 的通道 **16**。该温度传感器的电气特性如下(来自数据手册):

参数	符号	典型值	范围	单位
25°C 时的输出电压	V25	1.43	1.34 ~ 1.52	V
温度变化斜率	Avg_Slope	4.3	4.0 ~ 4.6	mV/°C
ADC 采样时间	–	–	>= 17.1 us	–

温度计算公式:

$$\text{Temp}(^\circ C) = \frac{V_{25} - V_{sense}}{Avg_Slope} + 25$$

其中: - V_{25}: 25°C 时传感器的输出电压（典型值 1.43V）- V_{sense}: ADC 实测的传感器输出电压 - Avg_Slope: 温度每变化 1°C 输出电压的变化量（典型值 4.3 mV/°C）

注意：内部温度传感器测量的是芯片的结温（**Junction Temperature**）。芯片运行时自身会发热（约 10~20°C 的温升），而且 V25 和 Avg_Slope 存在制造偏差，因此测量值可能与室温有显著差异。

10.3.2 ADC 连续转换模式

STM32F103 的 ADC1 支持两种工作模式：

模式	ContinuousConvMode	行为
单次转换	DISABLE	每次软件触发只进行一次转换，转换完成后 ADC 停止
连续转换	ENABLE	一次转换完成后自动启动下一次转换，无需重新触发

本实验选择连续转换模式，原因是：DMA 配置为循环模式后会持续等待 ADC 数据，但 ADC 本身如果是单次转换模式，每次软件触发只产生一个数据后就停止，DMA 之后拿不到新数据。只有 ADC 连续转换模式才能不断产生数据流，使 DMA 循环传输形成闭环。

10.3.3 DMA 循环模式原理

DMA（Direct Memory Access）可以在不占用 CPU 的情况下，将数据从外设（如 ADC 数据寄存器）搬运到内存。STM32F103 的 DMA1 通道 1 专用于 ADC1。

DMA 有两种主要工作模式：

模式	Mode	行为
普通模式	DMA_NORMAL	传输指定数量后停止，需重新配置才能再次启动
循环模式	DMA_CIRCULAR	传输完成后自动重载计数器，从缓冲区首地址重新开始

本实验选择循环模式，因为：1. ADC 在连续转换模式下不断产生新数据。2. DMA 循环模式保证每个 ADC 转换结果都被自动搬运到内存，无需 CPU 干预。3. 每次 DMA 传输完成都会触发中断，进入 `HAL_ADC_ConvCpltCallback` 回调。

DMA 通道映射关系（**STM32F103**）：

外设	DMA 通道
ADC1	DMA1 Channel 1
SPI1_RX	DMA1 Channel 2
USART1_TX	DMA1 Channel 4
…	…

10.3.4 HAL ADC DMA 回调机制

当使用 `HAL_ADC_Start_DMA()` 启动 ADC 后，HAL 库会自动注册 DMA 传输完成回调。DMA 每搬运完成一次 ADC 数据，中断处理流程如下：

```
DMA1_Channel1_IRQHandler()
  → HAL_DMA_IRQHandler(&hdma_adc1)
    → ADC_DMAConvCplt()      [HAL 内部]
      → HAL_ADC_ConvCpltCallback()   [用户可重写的弱回调]
```

用户只需在 `main.c` 中实现 `void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)`，每次 ADC 转换完成时该函数会自动被调用，且在中断上下文中执行。

10.3.5 USB CDC 虚拟串口原理

USB CDC（Communication Device Class）是 USB 协议中定义的一种设备类，它可以在 USB 总线上模拟传统的串行通信接口。当 STM32 通过 USB 连接电脑后，操作系统会将其识别为一个标准 COM 端口。

与物理 UART 串口不同，USB CDC 的波特率、数据位、停止位等参数不影响实际传输速率——数据始终以 USB 全速（12 Mbps）在底层传输。但对于 pyserial 等库，仍需要指定一个波特率值来打开端口（该值会被忽略）。

10.3.6 newlib-nano 与浮点格式化限制

嵌入式 GCC 工具链默认使用 `--specs=nano.specs`，即 newlib-nano 标准库。newlib-nano 为了减小代码体积，默认不支持 `printf/sprintf` 系列函数的浮点格式化（`%f`、`%e`、`%g` 等）。

当代码中执行 `sprintf(buf, "%.1f", temp)` 时，浮点值不会被正确格式化，输出为空或乱码。

解决方案有两种：

方案	操作	代价
添加链接器标志	`-u _printf_float`	增大约 12 KB 代码
使用整型运算	在整数域完成所有计算，用 `%d.%d` 拼小数	零开销

本实验采用整型运算 + 发送原始 **ADC** 值的方案——STM32 端只发送整型原始 ADC 值，温度换算与浮点显示完全放到 Python 客户端进行。这样同时避免了代码体积膨胀和格式化问题。

10.3.7 数据吞吐能力估算

ADC 转换一次的时间：

$$T_{conv} = (239.5 + 12.5)/12\text{MHz} \approx 21\mu s$$

ADC 理论最大采样率约 **47,000 sps**。

但实际瓶颈在 USB CDC 传输端：- USB 全速设备每 1ms 一帧，每帧最大 64 字节（CDC bulk 端点）。- 每个样本约 6 字节（如 `1732\\r\\n`），每帧约可传 10 个样本。- USB CDC 理论上限约 **10,000 sps**。

实测稳定速率约 **1,000 ~ 2,000 sps**，受 HAL 回调开销、USB 主机调度等因素影响。

10.4 实验环境

10.4.1 硬件环境

1. 一块带 USB 接口的 STM32F103C8T6 开发板。
2. 一根可正常传输数据的 USB 数据线。
3. ST-Link 下载器或板载调试器。

10.4.2 软件环境

1. STM32CubeMX（v6.17.0 或更高版本）。
2. VS Code / STM32CubeIDE 或其他支持 CMake 的 STM32 开发环境。
3. ARM GCC / CMake 工具链。
4. Python 3.x + pyserial + matplotlib（用于客户端可视化）。
5. ST-Link 驱动。

10.4.3 Python 环境配置

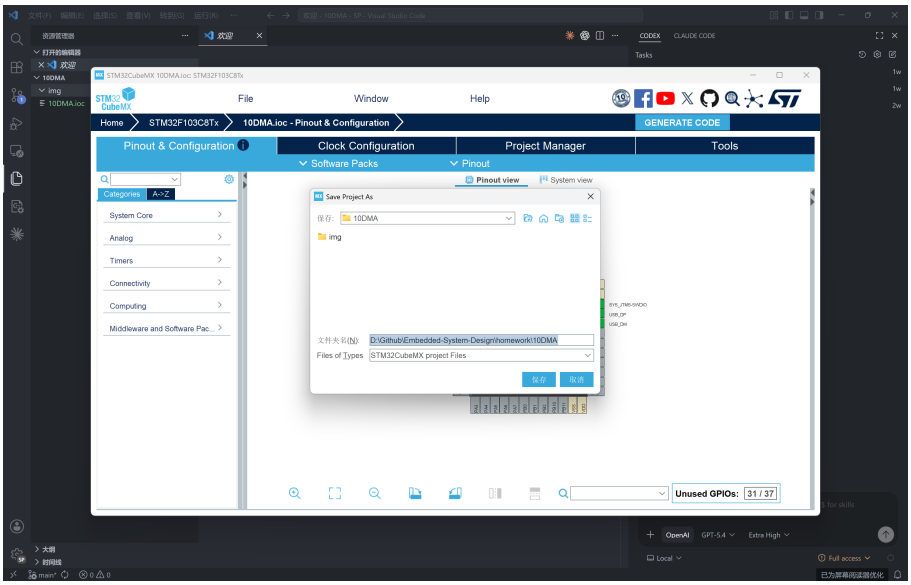
```
pip install pyserial matplotlib
```

10.5 STM32CubeMX 配置步骤

10.5.1 从作业 9 工程另存为新工程

本实验在作业 9 的基础上修改，无需从头新建。

1. 打开 STM32CubeMX，点击 **File → Load Project**，选择作业 9 的 `09ADC.ioc` 工程文件。
2. 点击 **File → Save As**，将工程另存为 `10DMA.ioc`（存放在 `10DMA` 文件夹中）。
3. 后续配置步骤在此基础上进行修改。



10.5.2 检查调试接口（SYS）

在 Pinout & Configuration 页面中，打开 **SYS**，确认 **Debug** 已设置为 **Serial Wire**（从 09ADC 继承，无需重新配置）。

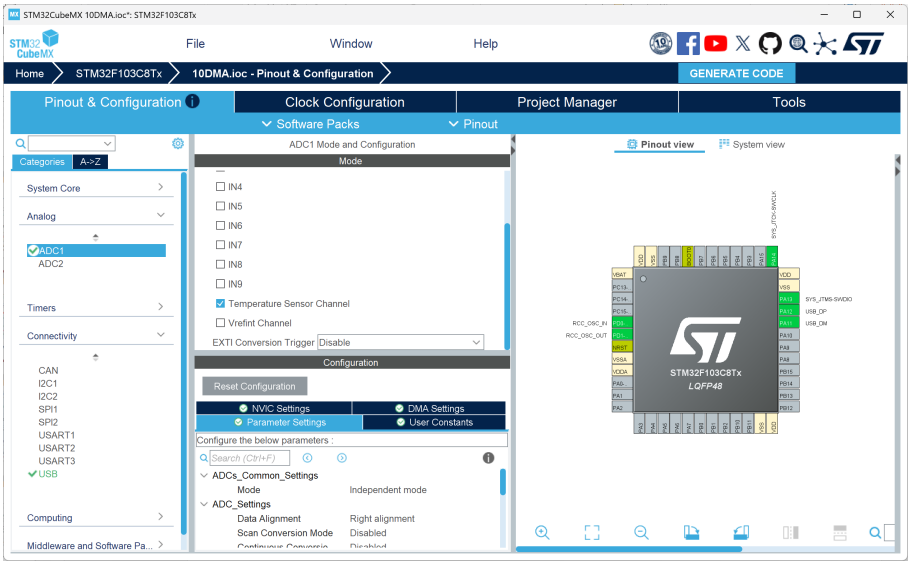
10.5.3 检查高速外部时钟（RCC）

打开 **RCC** 配置项，确认 **High Speed Clock (HSE)** 已设置为 **Crystal/Ceramic Resonator**（外部 8 MHz 晶振），从 09ADC 继承。

10.5.4 配置 ADC1（温度传感器通道）

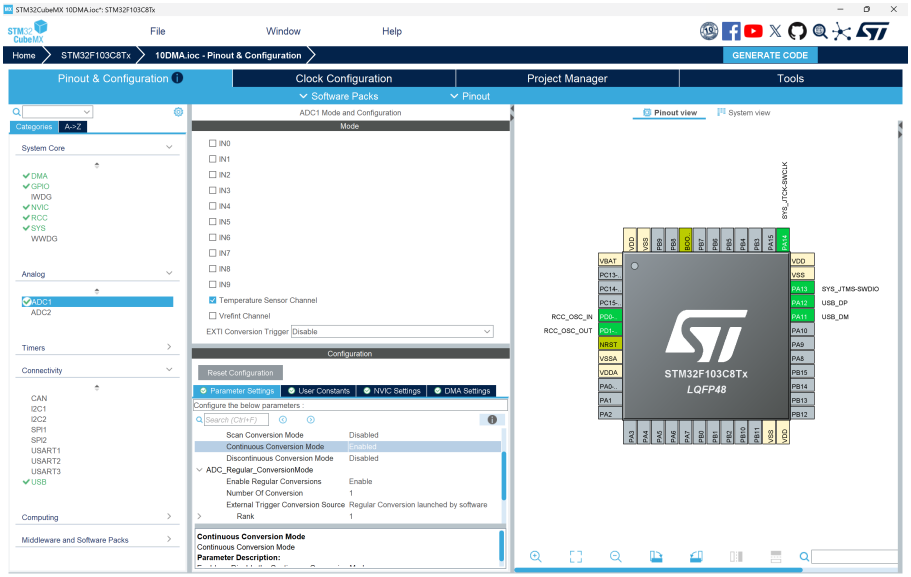
在 **Analog → ADC1** 中：

1. 勾选 **IN0** 区域的 **Temperature Sensor Channel**，使能内部温度传感器通道（ADC 通道 16）。
2. **Mode** 保持为 **Independent mode**（独立模式）。



ADC 参数配置说明：

参数	设置值	说明
ScanConvMode	DISABLE	仅采集 1 个通道，无需扫描
ContinuousConvMode	**ENABLE**	关键配置：连续转换，持续输出数据流给 **DMA**
DiscontinuousConvMode	DISABLE	–
ExternalTrigConv	ADC_SOFTWARE_START	由 `HAL_ADC_Start_DMA()` 软件触发
DataAlign	ADC_DATAALIGN_RIGHT	数据右对齐，12 位结果在低 12 位
NbrOfConversion	1	每次转换 1 个通道



10.5.4.1 配置 ADC 采样时间

采样时间选择 **239.5 Cycles**（`ADC_SAMPLETIME_239CYCLES_5`）。

为什么需要较长的采样时间：

ADC 内部有一个采样保持电容，在采样阶段需要对该电容充电。温度传感器是内部模拟信号源，其输出阻抗较高。如果采样时间太短，采样电容上的电压不足以达到与信号源相同的电平，导致 ADC 读数偏低且不稳定。数据手册要求温度传感器的采样时间至少为 17.1 us。

在 12 MHz ADC 时钟下: - 总转换时间 = (239.5 + 12.5) / 12 MHz ~ 21 us - 21 us > 17.1 us，满足数据手册要求

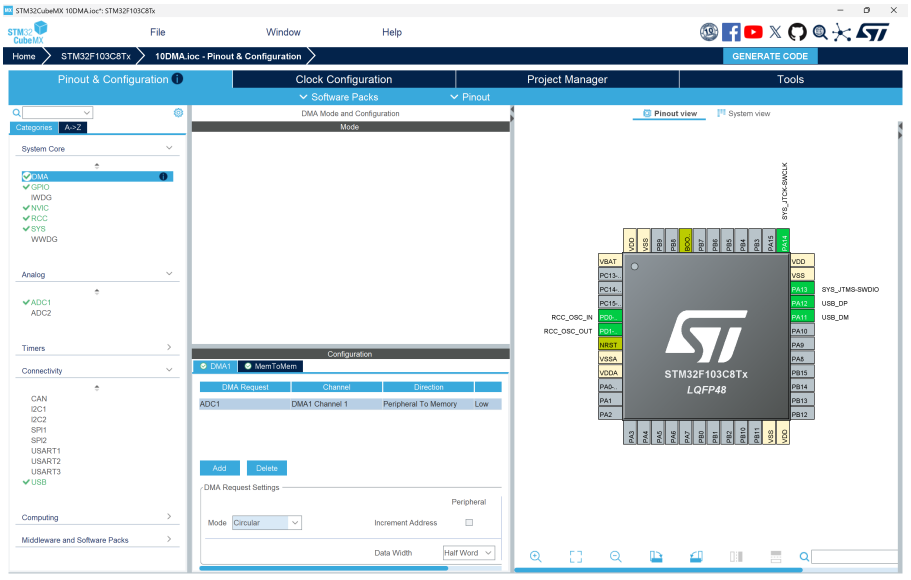
10.5.5 配置 DMA（ADC1 数据传输）

这是本实验区别于作业 9 的核心配置。在 **System Core → DMA** 中:

1. 点击 **Add**，选择 **ADC1**。
2. DMA Request 选项卡中配置:

参数	设置值	说明
Channel	DMA1 Channel 1	ADC1 专用 DMA 通道
Direction	Peripheral To Memory	从 ADC 数据寄存器搬运到内存
Mode	**Circular**	循环模式，持续搬运，本实验核心
Priority	Low	ADC 采集优先级无需最高
Peripheral (Data Width)	Half Word (16-bit)	ADC 为 12 位，16-bit 对齐
Memory (Data Width)	Half Word (16-bit)	与 ADC 数据宽度一致
Peripheral (Increment)	Disable	外设地址固定（ADC_DR 寄存器）
Memory (Increment)	Enable	内存地址自增（写满缓冲区）

3. NVIC Settings 选项卡中，确认 **DMA1 Channel 1 global interrupt** 已使能（勾选 Enabled）。



DMA 配置原理说明:

- **Direction = Peripheral To Memory**: DMA 从 ADC 的数据寄存器（ADC_DR）读取数据，写入内存变量。
- **Mode = Circular**: 每次传输完成后，DMA 的存储器地址指针自动重置到缓冲区首地址。由于本实验只传 1 个半字（缓冲区大小 = 1），每次 ADC 转换完成后 DMA 都将新数据覆盖写入同一个变量 `adc_value`。
- **Peripheral Increment = Disable**: 外设地址是 ADC_DR 寄存器，固定不变。

- **Memory Increment = Enable**：当缓冲区大小 > 1 时，DMA 会依次填充数组元素。
- **Data Width = Half Word**：ADC 转换结果为 12 位，用 16 位存储刚好。

10.5.6 启用 USB 外设并配置 CDC 中间件

在左侧 **Connectivity** 菜单下找到 **USB**，将其配置为 **Device (FS)**。

在 **Middleware and Software Packs** 中打开 **USB_DEVICE**，将 **Class for FS IP** 设置为 **Communication Device Class (Virtual Port Com)**。

这一步的意义在于：1. 告诉 CubeMX 当前 USB 设备工作在 CDC 类（通信设备类）。2. 自动生成 USB 虚拟串口所需的设备描述符、接口文件和 CDC 中间件代码。3. 电脑端将识别此设备为标准 COM 端口。

10.5.7 配置时钟树

进入 **Clock Configuration** 页面：

1. **HSE**：8 MHz（外部晶振）。
2. **PLL Source**：选择 HSE。
3. **PLL Mul**：设置为 ×6，PLL 输出 = 8 MHz × 6 = 48 MHz。
4. **SYSCLK**：选择 PLLCLK = 48 MHz。
5. **AHB Prescaler**：÷1，HCLK = 48 MHz。
6. **APB1 Prescaler**：÷2，APB1 = 24 MHz（USB 挂载在 APB1 上）。
7. **APB2 Prescaler**：÷1，APB2 = 48 MHz（ADC1 挂载在 APB2 上）。
8. **ADC Prescaler**：÷4，ADC 时钟 = 48 MHz / 4 = 12 MHz。
9. **USB Prescaler**：选择 PLLCLK，USB 时钟 = 48 MHz。

为什么必须精确配置：

- USB 全速设备要求精确的 **48 MHz** 时钟。误差超过 ±0.25%（±120 kHz）可能导致设备无法枚举或通信出错。
- ADC 时钟最大允许 14 MHz，12 MHz 在安全范围内。
- APB2 为 48 MHz，保证了 GPIO 和 ADC 外设的运行效率。
- DMA 属于 AHB 总线设备，HCLK = 48 MHz，确保 DMA 传输不成为瓶颈。

10.5.8 配置工程并生成代码

在 **Project Manager** 页面中：

1. **Project Name**：设置为 `10DMA`。
2. **Project Location**：选择合适的保存路径。
3. **Toolchain / IDE**：选择 **CMake**。
4. **Stack Size**：0x400；**Heap Size**：0x200。
5. 点击 **GENERATE CODE** 生成工程。

10.6 硬件连接要点

1. USB_DP（PA12）与 USB_DM（PA11）分别连接 USB 连接器的 D+ 与 D-。
2. 确保开发板的 USB D+ 线路上有 1.5kΩ 上拉电阻到 3.3V（USB 全速设备枚举的硬件要求）。
3. USB 线必须是数据线（内部有四根线：VCC、GND、D+、D-），不能只用供电线。
4. 温度传感器在芯片内部，无需任何外部接线。
5. SWD 接口（PA13/PA14）保持连接用于下载和调试。

10.7 核心代码说明

10.7.1 代码整体结构

```
// Private variables
uint16_t adc_value;   // DMA 目标缓冲区

// Private user code (回调实现)
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    if (hadc->Instance == ADC1) {
        char buf[50];
        int len = sprintf(buf, "%u\\r\\n", adc_value);
        CDC_Transmit_FS((uint8_t*)buf, len);
    }
}

// main() 初始化后
SET_BIT(ADC1->CR2, ADC_CR2_TSVREFE);            // ① 手动使能温度传感器
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)&adc_value, 1);  // ② 启动 ADC + DMA

// while(1) 为空，数据采集和发送全在回调中完成
```

核心流程：

```
ADC 连续转换 → DMA 搬运到 adc_value → DMA TC 中断
  → HAL_DMA_IRQHandler → ADC_DMAConvCplt
    → HAL_ADC_ConvCpltCallback → sprintf → CDC_Transmit_FS
```

10.7.2 main.c 中新增的头文件和变量声明

```
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */

/* Private variables ---------------------------------------------------------*/
/* USER CODE BEGIN PV */
uint16_t adc_value;
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
/* USER CODE BEGIN PFP */
void CDC_Transmit_FS(uint8_t* Buf, uint16_t Len);
```

```
/* USER CODE END PFP */
```

代码解析：1. stdio.h：提供 sprintf 函数，用于将 ADC 值格式化为字符串。2. uint16_t adc_value：DMA 的目标缓冲区。由于 BufferSize = 1 且 DMA 为循环模式，每次 ADC 转换完成后 DMA 都会将新值写入此变量。3. CDC_Transmit_FS 的前向声明：该函数定义在 usbd_cdc_if.c 中，需要在 main.c 中引用。

10.7.3 DMA 传输完成回调函数

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if (hadc->Instance == ADC1)
    {
        char buf[50];
        int len = sprintf(buf, "%u\\r\\n", adc_value);
        CDC_Transmit_FS((uint8_t*)buf, len);
    }
}
/* USER CODE END 0 */
```

代码解析：1. HAL_ADC_ConvCpltCallback 是 HAL 库定义的弱回调函数（__weak），用户重写后会自动覆盖默认空实现。2. 该回调在 **DMA** 中断上下文中执行。每次 ADC 转换完成 → DMA 搬运完成 → DMA TC 中断 → HAL 处理 → 回调被调用。3. if (hadc->Instance == ADC1)：在多个 ADC 实例时区分来源，确保只处理 ADC1 的事件。4. sprintf(buf, "%u\\r\\n", adc_value)：将 ADC 原始值格式化为十进制字符串。使用 \\r\\n 换行，兼容大多数串口工具。5. CDC_Transmit_FS((uint8_t*)buf, len)：通过 USB CDC 发送数据到电脑。

为什么发送原始 **ADC** 值而非温度： - 单片机端避免浮点运算（Cortex-M3 无 FPU）。- 避免 newlib-nano 的 %f 格式化问题（需 -u _printf_float 增大 12KB 代码）。- 温度换算放在 Python 客户端完成，更灵活（可更换校正算法、做数据平滑等）。- 原始值只有 0~4095，占用串口带宽更小。

10.7.4 main() 中的初始化与启动代码

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    /* USER CODE BEGIN SysInit */
    SET_BIT(ADC1->CR2, ADC_CR2_TSVREFE);   // ①
    /* USER CODE END SysInit */

    MX_GPIO_Init();
    MX_DMA_Init();       // ②
    MX_ADC1_Init();      // ③
    MX_USB_DEVICE_Init();

    /* USER CODE BEGIN 2 */
    HAL_ADC_Start_DMA(&hadc1, (uint32_t*)&adc_value, 1);  // ④
    /* USER CODE END 2 */

    while (1) { }  // ⑤ 主循环为空，一切由中断驱动
}
```

流程说明：

步骤	代码	说明
	`SET_BIT(ADC1->CR2, ADC_CR2_TSVREFE)`	手动使能温度传感器。CubeMX 不会自动生成此代码，需在初始化 ADC1 之前设置 TSVREFE 位，否则温度传感器不工作。
	`MX_DMA_Init()`	使能 DMA1 时钟，配置 DMA1_Channel1 中断优先级并使能。
	`MX_ADC1_Init()`	配置 ADC1 为连续转换模式，校准并初始化温度传感器通道。
	`HAL_ADC_Start_DMA()`	一次性启动 ADC + DMA。参数 `(uint32_t*)&adc_value` 指定 DMA 目标地址，`1` 指定缓冲区大小（1 个半字）。启动后 ADC 和 DMA 自动持续运行，无需再次调用。
	`while (1) { }`	主循环为空——ADC 采集、DMA 传输、数据发送全部在中断回调中完成，CPU 负载极低。

为什么 **TSVREFE** 需要手动设置：

STM32F103 参考手册规定，使用内部温度传感器或 VREFINT 前必须置位 ADC_CR2 寄存器的 TSVREFE 位。CubeMX 在代码生成过程中并不会自动添加此位操作——这是 STM32F1 系列 HAL 库的历史遗留设计，ST 没有在 `HAL_ADC_Init()` 中集成此功能。因此必须在代码中手动 `SET_BIT(ADC1->CR2, ADC_CR2_TSVREFE)`，且在 `MX_ADC1_Init()` 之前执行以确保 ADC 校准时传感器已上电稳定。

10.7.5 ADC1 初始化代码（由 STM32CubeMX 生成）

```c
static void MX_ADC1_Init(void)
{
    ADC_ChannelConfTypeDef sConfig = {0};

    hadc1.Instance = ADC1;
    hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;        // 单通道，不扫描
    hadc1.Init.ContinuousConvMode = ENABLE;            // * 连续转换模式
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;  // 软件触发
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;        // 数据右对齐
    hadc1.Init.NbrOfConversion = 1;                    // 每次转换 1 个通道
    HAL_ADC_Init(&hadc1);

    sConfig.Channel = ADC_CHANNEL_TEMPSENSOR;          // 内部温度传感器通道
    sConfig.Rank = ADC_REGULAR_RANK_1;                 // 规则组排名第 1
    sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5; // 采样时间 239.5 周期
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
}
```

关键参数说明：- `ContinuousConvMode = ENABLE`：本实验最关键配置。ADC 在完成一次转换后自动开始下一次转换，持续产生数据流。这是 DMA 循环模式能够持续工作的前提。- `ADC_SOFTWARE_START`：由 `HAL_ADC_Start_DMA()` 中的软件触发启动首次转换。- `ADC_SAMPLETIME_239CYCLES_5`：采样周期 239.5 个 ADC 时钟周期（约 20 us），满足温度传感器 >= 17.1 us 的建立时间要求。

10.7.6 DMA 初始化代码（由 STM32CubeMX 生成）

```
static void MX_DMA_Init(void)
{
    __HAL_RCC_DMA1_CLK_ENABLE();

    HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);
}
```

DMA 的具体传输参数（方向、数据宽度、循环模式等）在 stm32f1xx_hal_msp.c 的 HAL_ADC_MspInit() 中配置：

```
void HAL_ADC_MspInit(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance==ADC1)
    {
        __HAL_RCC_ADC1_CLK_ENABLE();

        hdma_adc1.Instance = DMA1_Channel1;
        hdma_adc1.Init.Direction = DMA_PERIPH_TO_MEMORY;
        hdma_adc1.Init.PeriphInc = DMA_PINC_DISABLE;
        hdma_adc1.Init.MemInc = DMA_MINC_ENABLE;
        hdma_adc1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
        hdma_adc1.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
        hdma_adc1.Init.Mode = DMA_CIRCULAR;     // * 循环模式
        hdma_adc1.Init.Priority = DMA_PRIORITY_LOW;
        HAL_DMA_Init(&hdma_adc1);

        __HAL_LINKDMA(hadc, DMA_Handle, hdma_adc1);  // 关联 DMA 句柄到 ADC 句柄
    }
}
```

10.7.7 DMA 中断处理函数

```
void DMA1_Channel1_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_adc1);
}
```

该函数在 stm32f1xx_it.c 中，由 CubeMX 自动生成。DMA 的传输完成（TC）、半传输（HT）、传输错误（TE）等中断事件均由 HAL_DMA_IRQHandler 统一处理。TC 事件最终触发 HAL_ADC_ConvCpltCallback。

10.7.8 Python 客户端代码

客户端使用 tkinter + matplotlib 构建实时温度曲线界面。

核心架构：

```
主线程（GUI）                          后台线程（串口读取）
──────────────────────────────────    ──────────────────────────────────
tkinter event loop                    serial.readline() 循环
    │                                       │
    │  root.after(100ms)                    │
    └─→ _anim():                            │  adc → mv → temp
      读取共享数据队列                       │  写入 self.times / self.temps
```

```
二分查找时间窗口                    |
更新 matplotlib 曲线               |
更新温度数值显示                    |
```

ADC 原值到温度的转换（Python 端）：

```
mv = adc * 3300 / 4095          # ADC 值 → 电压 (mV)
temp = (1430 - mv) / 4.3 + 25   # 电压 → 温度 (℃)
```

时间窗口控制：

横坐标显示已运行时间（秒），可通过输入框自由调整时间量程，支持 0.001s（1ms）到 60s。数据存储使用 `collections.deque`（最大 200,000 点），时间窗口边界通过 `bisect.bisect_left` 二分查找定位。

关键设计决策：

决策	原因
串口读取使用独立线程	避免 `readline()` 阻塞 GUI
数据共享使用带锁的 deque	读写线程安全，无需额外加锁
`bisect` 二分查找时间窗口	O(log n) vs 线性扫描 O(n)
`canvas.draw_idle()` 而非 `draw()`	降低 GUI 刷新对 CPU 的占用
读线程失败时自动关闭串口	避免 `Stop` 按钮与 `readline()` 之间的竞态条件

10.7.9 代码结构的整体工作流程

本实验代码可以概括为以下步骤：

1. 系统启动后初始化 HAL 库、系统时钟（48 MHz）。
2. 手动使能温度传感器（TSVREFE 位）。
3. 初始化 GPIO、DMA1（中断使能 + 优先级配置）、ADC1（连续转换模式）、USB CDC。
4. 调用 `HAL_ADC_Start_DMA()` 一次性启动 ADC 连续转换 + DMA 循环传输。
5. 此后主循环空闲—-每约 21 us，ADC 完成一次转换，DMA 自动搬运到 `adc_value`，触发 TC 中断，回调中 `sprintf` + `CDC_Transmit_FS` 发送数据。
6. 电脑端 Python 客户端打开 COM 口，后台线程持续读取原始 ADC 值，主线程每 100ms 更新一次温度曲线。

10.8 编程技巧总结

1. **ADC 连续转换 + DMA 循环模式是黄金搭档**：单次转换 + 循环 DMA 无法形成连续数据流，两者必须同时使能。
2. **TSVREFE 必须手动使能**：`SET_BIT(ADC1->CR2, ADC_CR2_TSVREFE)` 需要在 `MX_ADC1_Init()` 之前执行，确保 ADC 校准时传感器已稳定。
3. **发送原始 ADC 值而非温度**：避免嵌入式端浮点运算和 newlib-nano 浮点格式化问题，将数据解释交给更灵活的上位机。

4. 中断回调中操作要轻量：`HAL_ADC_ConvCpltCallback` 在中断上下文执行。发送原始值（`sprintf` + `CDC_Transmit_FS`）已经有一定开销，不宜在其中做浮点计算或复杂逻辑。
5. 主循环可为空：ADC + DMA 初始化后即可全自动运行，充分体现 DMA 解放 CPU 的核心优势。
6. 所有用户代码放在 `USER CODE` 块内：确保 STM32CubeMX 重新生成代码时不会覆盖自定义逻辑。
7. **Python** 端做数据转换和可视化：利用 Python 丰富的科学计算和绘图生态，避免嵌入式端的诸多限制。

10.9 实验操作步骤

10.9.1 编译工程

使用当前开发环境编译工程，确认无编译错误。

10.9.2 烧录程序

通过 ST-Link 将编译生成的 `.elf` 或 `.bin` 文件下载到 STM32 开发板。

10.9.3 连接电脑

使用 USB 数据线将开发板的 USB 口（PA11/PA12 对应的接口）连接到电脑。

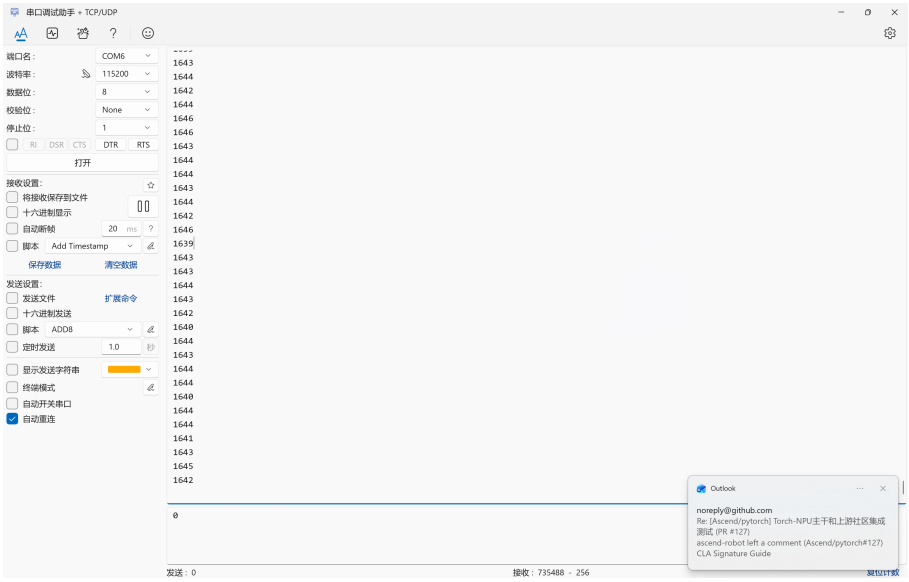
10.9.4 查看 COM 口

打开设备管理器，展开”端口（COM 和 LPT）“，确认出现 `STMicroelectronics Virtual COM Port (COMx)`，记录 COM 口编号。

10.9.5 用串口调试助手验证单片机输出

打开串口调试助手（如 SSCOM、PuTTY 等），选择对应 COM 口，打开串口。如果一切正常，应看到单片机持续发送的 ADC 原始值：

```
1732
1734
1730
1735
...
```



这一步的目的是先验证单片机端软硬件工作正常（TSVREFE 使能、ADC 连续转换、DMA 循环传输、USB CDC 发送均正确），再进入 Python 客户端环节。

10.9.6 安装 Python 依赖

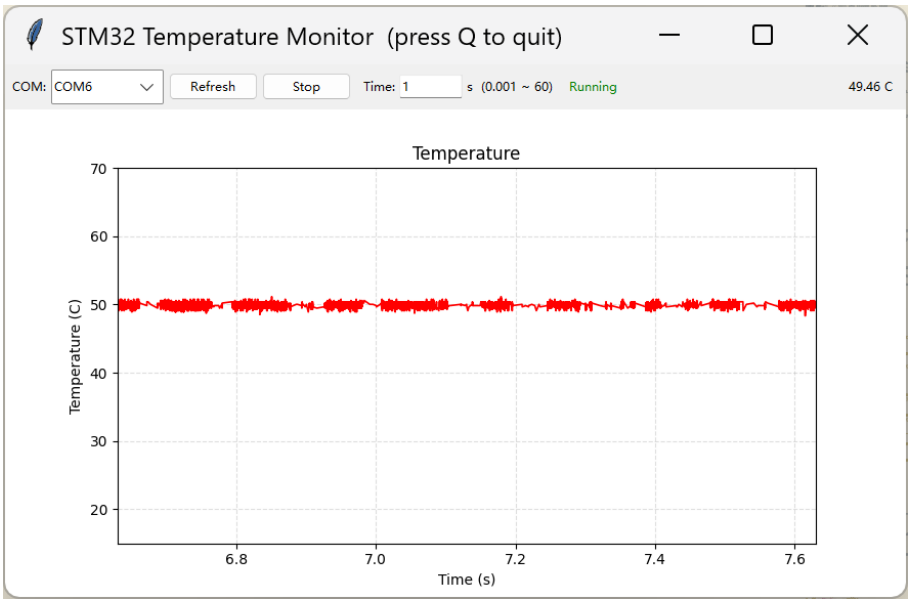
```
pip install pyserial matplotlib
```

10.9.7 运行 Python 客户端

```
python client/temperature_monitor.py
```

1. 在 COM 下拉框中选择对应的串口。
2. 点 **Start** 开始采集。
3. 调整时间量程（输入秒数，如 `10`、`0.1`、`60` 等）观察不同时间尺度下的温度波形。
4. 点 **Stop** 或按 **Q** / **Esc** 停止。

客户端应实时显示温度曲线，当前温度值显示在右上角。波形应连续无断点，温度值在合理范围内波动。



10.9.8 记录实验结果

建议在实验报告中保留以下截图：1. CubeMX 配置截图（ADC1、DMA、时钟树、USB CDC 各至少 1 张）。2. 设备管理器中虚拟串口截图 1 张。3. 串口调试助手验证输出截图 1 张。4. Python 客户端运行截图 1 张（显示温度曲线和实时数值）。

10.10 实验现象与结果分析

10.10.1 预期现象

1. 电脑成功识别到 STM32 虚拟串口设备。
2. Python 客户端启动后，点击 Start，温度曲线立即开始滚动。
3. 曲线平滑连续，温度值约 40~60°C（内部温度传感器 + 芯片自发热）。
4. 调整时间量程，可观察到不同时间尺度下的温度波动情况。
5. 长时间运行（数分钟）可观察到芯片从冷启动到热平衡的温升过程。

10.10.2 结果分析

1. **DMA** 循环传输正常：曲线连续无断点，说明 ADC → DMA → 回调 → VCP 的数据管道工作正常。
2. 时钟配置正确：USB 设备能在设备管理器中正确枚举，说明 48 MHz USB 时钟精度满足要求。
3. 采样时间充足：ADC 读数稳定，无显著离散跳动，说明 239.5 Cycles 的采样时间满足了温度传感器的建立时间要求。
4. 温度值偏高：测量值约 40~60°C（远高于室温），原因有二：
   - 芯片在 48 MHz 全速运行且 USB 外设工作时有约 10~20°C 的自发热。
   - V25 和 Avg_Slope 的制造偏差（没有使用 VREFINT 校正和出厂校准值）。
5. 数据吞吐：Python 端每秒收到约 1000~2000 个数据点，连续无丢帧。

10.11 常见问题排查

问题	可能原因与解决方案
Python 客户端无数据显示	检查 COM 口选择是否正确；确认 STM32 程序已下载并运行；尝试按复位按钮后重新 Start。
温度曲线不连续、有断点	检查 USB 数据线质量；确认 DMA 中断优先级未被 USB 中断长时间阻塞；检查回调中代码是否耗时过长。
温度值明显异常（如 -100°C 或 >150°C）	检查 TSVREFE 位是否使能（未使能时 ADC 读数为随机值）；确认 V25 和 Avg_Slope 参数正确。
ADC 读数恒为 0	检查 `HAL_ADC_Start_DMA` 是否被调用；确认 DMA 目标地址参数是否正确。

问题	可能原因与解决方案
电脑无法识别设备	检查 USB 时钟是否为 48MHz；检查 USB 线是否为数据线；确认开发板 D+ 上拉电阻正常。
程序编译时 Flash/Heap 溢出	检查栈大小（建议 0x400）；如使用 `-u _printf_float` 注意代码体积增加约 12KB。
Python 端 `shape mismatch` 错误	读线程和动画线程同时读写 deque 导致的竞态，客户端已通过 `min(len(xs), len(ys))` 对齐处理。
Python 端 `TypeError: byref()` 错误	点 Stop 时主线程关闭串口与读线程 `readline()` 的竞态，客户端已通过读线程自行关闭串口解决。

10.12 课后思考题

请结合本次实验，在报告中认真回答以下问题：

1. 说明 **DMA** 循环模式（**Circular Mode**）与普通模式（**Normal Mode**）的区别，以及为什么本实验需要循环模式。（提示：从 ADC 连续产生数据流的角度分析，普通模式下 DMA 传输完成后会停止。）
2. 为什么本实验中 **ADC** 必须配置为连续转换模式（**ContinuousConvMode = ENABLE**），若配置为单次转换模式结合 **DMA** 循环模式会出现什么问题？（提示：分析 ADC 数据产生的时机与 DMA 数据搬运的时机之间的依赖关系。）
3. **STM32F103** 内置温度传感器在 **HAL** 初始化后需要手动置位 **TSVREFE** 位才能工作。为什么 **CubeMX** 生成的 `MX_ADC1_Init()` 中不自动使能该位？（提示：从 HAL 库的设计原则和不同 STM32 系列的差异角度分析。）
4. 本实验中 `HAL_ADC_ConvCpltCallback` 回调函数在什么上下文中被调用（中断 **or** 主循环）？在高频数据采集（约 **1000~2000 sps**）场景下，回调中直接调用 `CDC_Transmit_FS` 可能存在什么问题？如何优化？（提示：从 ISR 执行时间、USB 发送缓冲、批量发送策略等角度分析。）

10.13 实验报告提交要求

10.13.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述本实验需要实现的 ADC + DMA + VCP 连续数据采集功能。
3. 实验原理：简述 ADC 连续转换模式、DMA 循环模式、HAL 回调机制、温度传感器原理。
4. 实验环境：开发板型号、USB 连接方式、调试器、软件版本、Python 环境。
5. **CubeMX** 配置：必须说明 SYS、RCC、ADC1（含连续转换模式）、DMA（含循环模式）、USB、USB_DEVICE、Clock Configuration 的关键设置及其原因。
6. 程序设计：至少分析 `main.c` 中 TSVREFE 使能、DMA 启动、`HAL_ADC_ConvCpltCallback` 回调的完整逻辑。
7. 程序流程图：必须展示”TSVREFE 使能 → ADC Init → DMA Init → ADC_Start_DMA → ADC 连续转换 → DMA 搬运 → TC 中断 → ConvCpltCallback → sprintf → CDC_Transmit → 下一轮” 的完整数据流。

8. **Python** 客户端设计：说明串口读取线程、动画更新、时间窗口控制、温度计算公式。
9. 实验步骤：包括编译、下载、连接电脑、运行 Python 客户端等过程。
10. 实验结果：必须展示 Python 客户端运行截图。对温度数据进行分析（是否合理、数据速率、稳定性等）。
11. 问题分析与调试记录：说明遇到的任何问题及排查方法（如 TSVREFE 未使能、newlib-nano 浮点格式化、连续转换模式未使能等）。
12. 课后思考题答案：必须完整回答本 README 中 4 个思考题。
13. 总结：概括你对 STM32 ADC + DMA + USB CDC 数据采集系统的理解。

10.13.2 图片与流程图要求

1. 报告中至少包含 3 类图片：CubeMX 配置截图（含 DMA 配置）、时钟树截图、Python 客户端运行截图。
2. 每张图片必须标注图号和图题，并在正文中引用说明。
3. 程序流程图必须单独成图，不能只用文字描述替代。

10.13.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 10-DMA 温度传感器实验.pdf`。
2. 若实验未完全成功，也必须提交完整报告，重点说明失败现象、原因分析和改进方向。
3. 报告中不能只贴代码或截图，必须结合实验现象进行解释和分析。

10.13.4 最低验收标准

1. 电脑能够识别出虚拟串口设备。
2. Python 客户端能够接收到连续的数据流。
3. 温度曲线连续平滑无断点。
4. 温度计算值在合理范围内（约 40°C ~ 60°C）。
5. 报告能够解释 DMA 循环模式和 ADC 连续转换模式的协同工作原理。

10.14 可进一步扩展的方向

1. **VREFINT** 校正：参考作业 9，增加 VREFINT 通道的动态切换采集，用内部参考电压校正 VDDA 偏差，提高温度测量精度。
2. 出厂校准值校正：STM32F103 在 Flash 中存储了 TS_CAL1（30°C 时的 ADC 值），可利用此值结合 VREFINT 进行更高精度的温度校准。
3. 批量缓冲发送：在回调中将数据放入环形缓冲区，主循环中攒够 64 字节（填满一个 USB 包）再发送，可突破 10,000 sps 的 USB 吞吐上限。
4. 双缓冲 **DMA**：使用 DMA 半传输完成中断（Half Transfer Complete），实现乒乓缓冲，在采集的同时处理上一半数据。
5. 多个传感器通道：在 CubeMX 中配置 ScanConvMode = ENABLE，配合 DMA 缓冲区（BufferSize = 通道数），一次采集多个通道的数据。
6. 添加数据导出功能：在 Python 客户端中添加” 保存为 CSV” 按钮，方便后续用 Excel 或其他工具做离线分析。

7. 添加统计显示：在 Python 客户端界面显示平均值、最小值、最大值、标准差等统计量。

10.15 总结

本实验不同于作业 9 的”轮询 + 每秒一次”模式，而是使用 **ADC** 连续转换 **+ DMA** 循环传输实现了全自动高频数据采集。这个架构的关键理解点在于：

1. **ADC** 连续转换模式和 **DMA** 循环模式必须配对使用—-少了任何一个，都无法形成自动化的连续数据流。
2. **DMA** 解放 **CPU**—-主循环 `while(1)` 完全空闲，所有数据搬运由 DMA 硬件完成，CPU 仅需在中断回调中做轻量的格式化和发送操作。
3. 嵌入式端只发原始值—-将数据解释和可视化交给上位机，突破了 MCU 的算力和库限制，获得了更好的灵活性。
4. **Python** 客户端是完整的上位机方案—-从串口读取 → 数据解析 → 物理量转换 → 实时可视化 → 交互控制，构成一个完整的嵌入式 + 上位机协同系统。

这种”DMA 后台采集 + 中断回调处理 + 上位机可视化”的架构，是嵌入式数据采集系统中最常用、最高效的模式之一。

10.16 参考

- STM32F103x8/xB Datasheet（第 5.3.22 节：Temperature sensor characteristics）
- RM0008 Reference Manual（第 11 章：ADC，第 10 章：DMA）
- STM32CubeMX User Manual (UM1718)
- RM0008：ADC 外部触发转换与 DMA 请求的交互时序
- 本课程作业 4（USB CDC 虚拟串口实验）相关代码和说明
- 本课程作业 9（ADC 轮询采集实验）相关代码和说明

11 STM32 通用定时器 – LED 呼吸灯实验

11.1 统一作业说明

11.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成 TIM3 定时器、时钟树、调试接口等配置，并保留 `.ioc` 文件。
2. 基于 HAL 库与通用定时器中断实现软件 PWM，驱动 PC13 共阳极 LED 产生呼吸灯效果。
3. 成功编译、下载并在硬件上观察到 LED 渐亮渐灭的呼吸效果。
4. 在实验报告中说明定时器配置参数、代码结构、实验现象、问题排查和课后思考。
5. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

11.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6
外设	TIM3 通用定时器 + PC13 GPIO 输出
必做功能	LED 呼吸灯效果（渐亮渐灭循环）
理论要求	能解释定时器中断周期计算、软件 PWM 原理、中断与主循环协同
验收方式	提交 PDF 格式实验报告，包含配置截图、代码分析、现象描述和课后思考题书面回答

11.1.3 本次必须提交的内容

1. 一份 PDF 格式实验报告（含 CubeMX 各步骤配置截图、关键代码截图、实验现象描述）。
2. 课后思考题的书面回答（写在报告中）。

11.1.4 报告必须回答的问题

1. 为什么 PC13 不能直接使用定时器硬件 PWM 输出通道? 软件 PWM 与硬件 PWM 各自的优缺点是什么?
2. 修改 `PWM_PERIOD` 或主循环中占空比更新频率会对呼吸速度产生什么影响?
3. 定时器中断周期配置为 100 us，如果改为 50 us 或 200 us 分别会对呼吸效果产生什么影响?
4. 为什么 `s_pwmCounter` 和 `s_pwmDuty` 要使用 `volatile` 修饰?
5. 如果要在不改变硬件的前提下让 LED 在最亮状态停留 1 秒再开始变暗，应该如何修改代码?

11.2 关键说明

PC13 在 STM32F103C8T6 上没有定时器 PWM 输出复用功能，不能直接使用硬件 PWM 输出通道。本实验采用定时器中断 + 软件 **PWM** 方案，利用定时器中断周期性更新 GPIO 输出，实现呼吸灯效果。

11.3 实验目的

本实验基于 STM32 的通用定时器 TIM3 实现软件 PWM 呼吸灯，要求学生完成从 CubeMX 配置、代码生成、程序编写到硬件验证的完整实验流程。通过本实验，你应掌握以下内容:

1. 理解通用定时器的工作原理与中断机制。
2. 掌握 STM32CubeMX 中通用定时器的配置方法。
3. 理解软件 PWM 的实现原理，以及与硬件 PWM 的区别。
4. 学会在定时器中断回调中编写 GPIO 控制逻辑。
5. 学会在主循环与中断回调之间协调共享变量的读写。

11.4 实验环境

11.4.1 硬件环境

1. STM32F103C8T6 核心板一块。
2. 板载 PC13 共阳极 LED。
3. ST-Link 下载器。

11.4.2 软件环境

1. STM32CubeMX。
2. VS Code 或其他支持该工程的 STM32 开发环境。
3. ARM GCC / CMake 工具链或等效编译环境。
4. ST-Link 驱动。

11.5 STM32CubeMX 配置步骤

下面每一步的截图必须保留在实验报告中，因为这些配置是实验成功的前提。你不仅要会”照着做”，还要理解每一步为什么这样设置。

11.5.1 新建工程并选择芯片

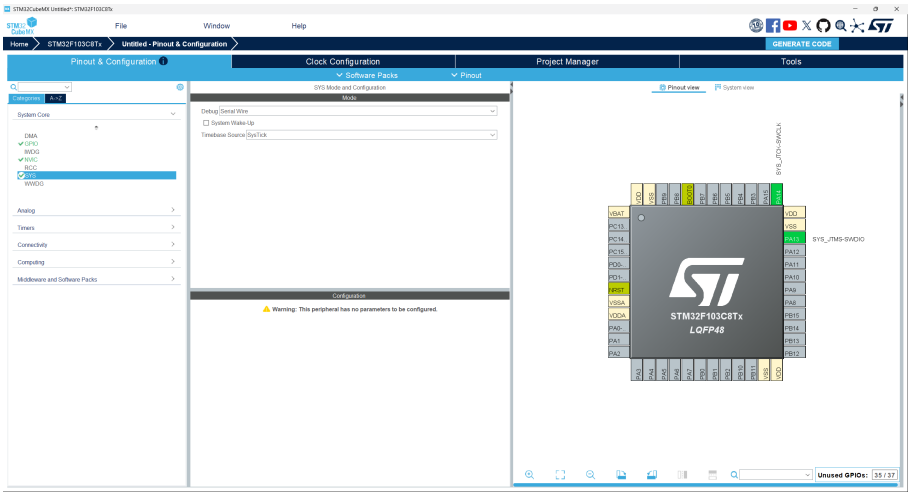
打开 STM32CubeMX，点击 **ACCESS TO MCU SELECTOR**，选择 **STM32F103C8T6**。

11.5.2 配置调试接口（SYS）

在 **Pinout & Configuration → System Core → SYS** 中，将 **Debug** 设置为 **Serial Wire**。

这样做的原因是：

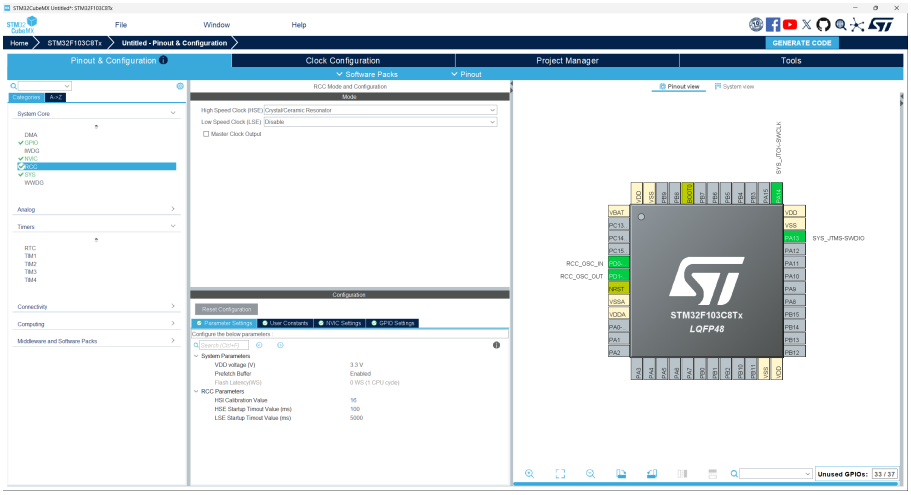
1. 便于后续通过 ST-Link 下载和调试程序。
2. 如果未正确配置调试接口，可能出现芯片无法正常调试或下载的问题。



11.5.3 配置高速外部时钟（RCC）

在 **Pinout & Configuration → System Core → RCC** 中，将 **High Speed Clock (HSE)** 设置为 **Crystal/Ceramic Resonator**。

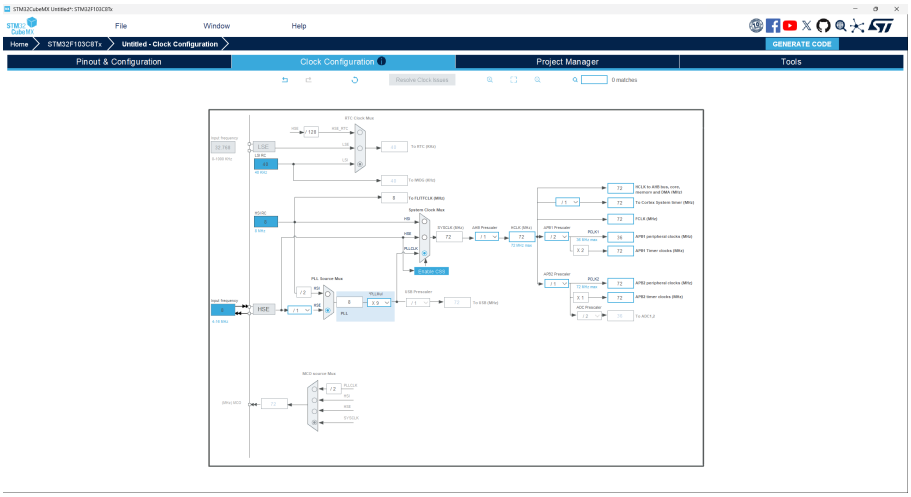
这样做的目的是为 PLL 和系统时钟提供稳定的外部时钟来源（8 MHz 晶振）。



11.5.4 配置时钟树（Clock Configuration）

进入 **Clock Configuration** 页面：

1. 将系统时钟源选择为 **PLLCLK**。
2. 设置 **HSE** 为 8 MHz。
3. 设置 **PLLMul** 为 x9，使 **PLLCLK** = 72 MHz。
4. 确保 **HCLK** = 72 MHz，**APB1 Timer Clocks** = 72 MHz，**APB2 Timer Clocks** = 72 MHz。
5. 确认 **APB1 Prescaler** = /2，这样 APB1 外设时钟为 36 MHz，而 APB1 定时器时钟为 72 MHz（自动 ×2）。



11.5.5 配置 PC13 为 GPIO 输出

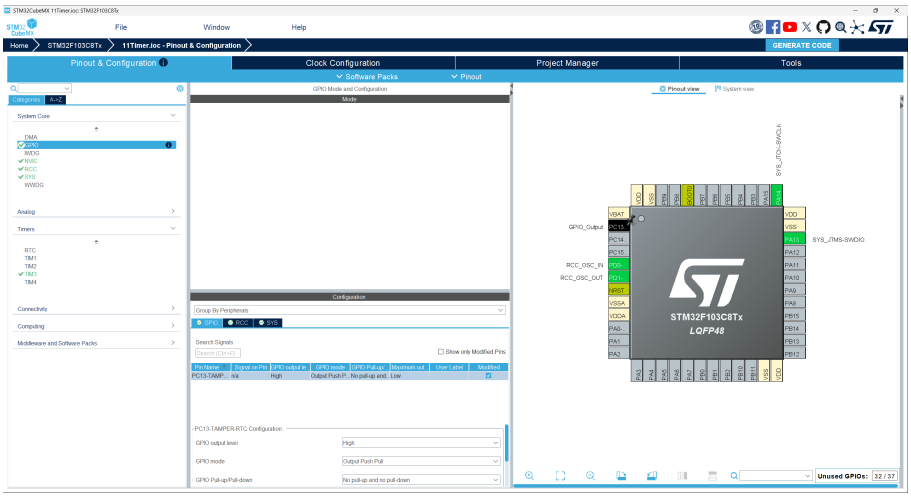
在右侧 **Pinout view** 中，点击 **PC13** 引脚，选择 **GPIO_Output**。

在 **System Core** → **GPIO** 中找到 PC13，进行以下设置：

1. **GPIO output level**: **High**（共阳极 LED，高电平 = 熄灭，初始状态为熄灭）
2. 可修改 **User Label**，但不修改也不影响功能。

这样做的原因是：

1. PC13 连接板上共阳极 LED 的阴极，LED 阳极接 VCC。
2. PC13 输出低电平时 LED 点亮，输出高电平时 LED 熄灭。
3. 初始设为高电平，上电后 LED 默认熄灭。



11.5.6 配置通用定时器参数

在 **Pinout & Configuration → Timers** 中，选择 **TIM3**：

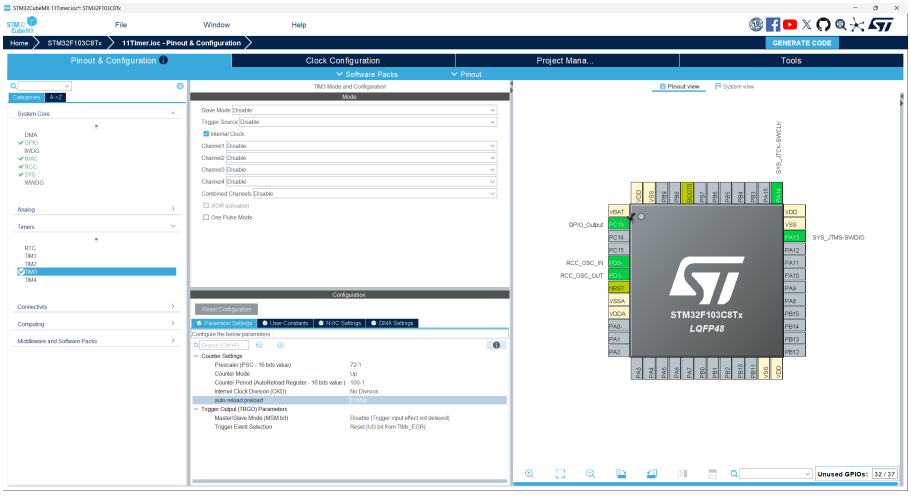
1. **Clock Source**: **Internal Clock**（使用内部时钟源）
2. 在下方参数设置区域：

   - **Prescaler**: 72 - 1（即 71），定时器计数时钟 = 72 MHz / 72 = 1 MHz，即每 1 us 计数一次
   - **Counter Mode**: **Up**（向上计数模式）
   - **Counter Period (AutoReload - 16 bits value)**: 100 - 1（即 99），即每 100 us 溢出一次并触发中断
   - **auto-reload preload**: **Enable**

   参数说明：定时器溢出时间计算公式为——

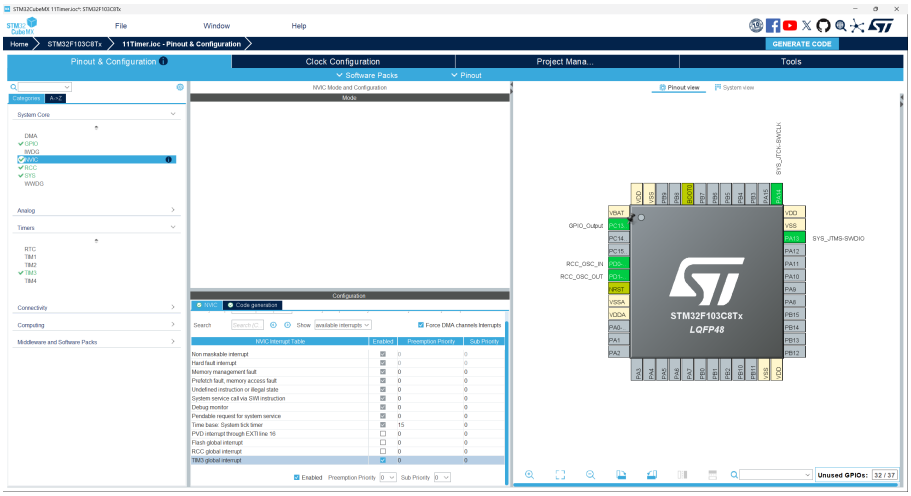
   ```
   T_overflow = (Prescaler + 1) × (Counter Period + 1) / Tclk
              = 72 × 100 / 72 MHz
              = 100 us
   ```

   100 us 的中断周期配合软件 PWM，可提供 100 级的占空比精度（PWM 频率 = 100 Hz），呼吸效果足够平滑。



11.5.7 使能定时器中断

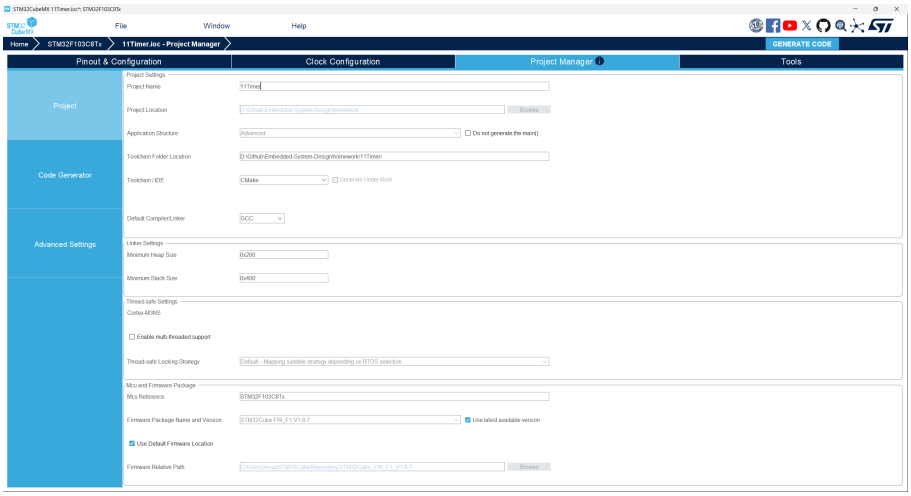
切换到 **NVIC Settings** 选项卡，勾选 **TIM3 global interrupt**，使能定时器全局中断。这样当 TIM3 计数值溢出时，硬件会自动触发中断，HAL 库会调用 `HAL_TIM_PeriodElapsedCallback` 回调函数。



11.5.8 工程设置与生成代码

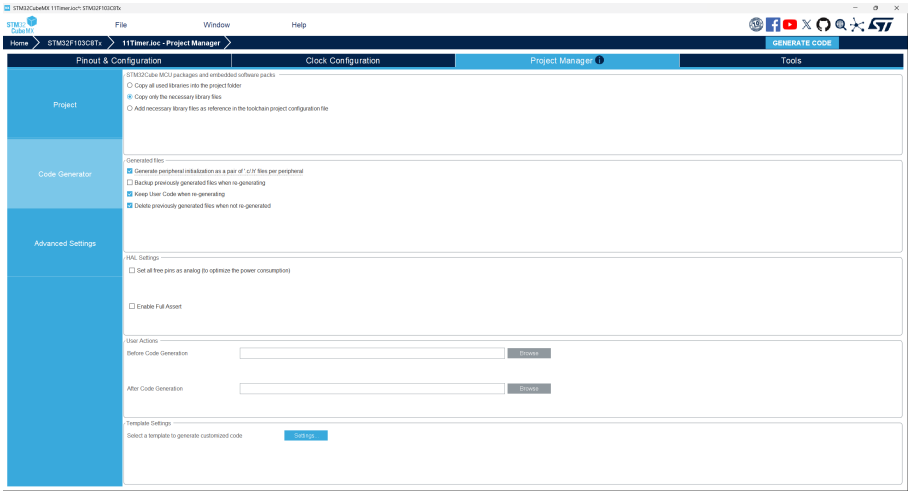
在 **Project Manager** 页面：

1. **Project Name**：`11Timer`
2. **Project Location**：选择当前 `11Timer` 目录
3. **Toolchain / IDE**：选择 **CMake**



切换到 **Code Generator** 选项卡：

1. 勾选 **Generate peripheral initialization as a pair of ‘.c/.h’ files per peripheral**，这样 TIM3 初始化代码会生成到独立的 `tim.c` / `tim.h` 文件中，便于代码管理。



最后点击 **GENERATE CODE** 生成工程。

11.6 硬件连接要点

1. PC13 通过限流电阻连接共阳极 LED 的阴极，LED 阳极接 VCC。
2. PC13 输出低电平时 LED 点亮（电流从 VCC 经 LED 流向 PC13），输出高电平时 LED 熄灭。
3. 本实验使用板载 LED，无需额外外部接线。

11.7 本工程实现的实验内容

当前工程在代码中实现了基于定时器中断的软件 PWM 呼吸灯，功能如下：

1. 系统启动后自动开始呼吸灯效果。
2. 利用 TIM3 每 100 us 触发一次中断，在中断中完成软件 PWM 输出。
3. 主循环中按固定步进更新占空比，实现 LED 渐亮渐灭。

11.8 核心代码说明

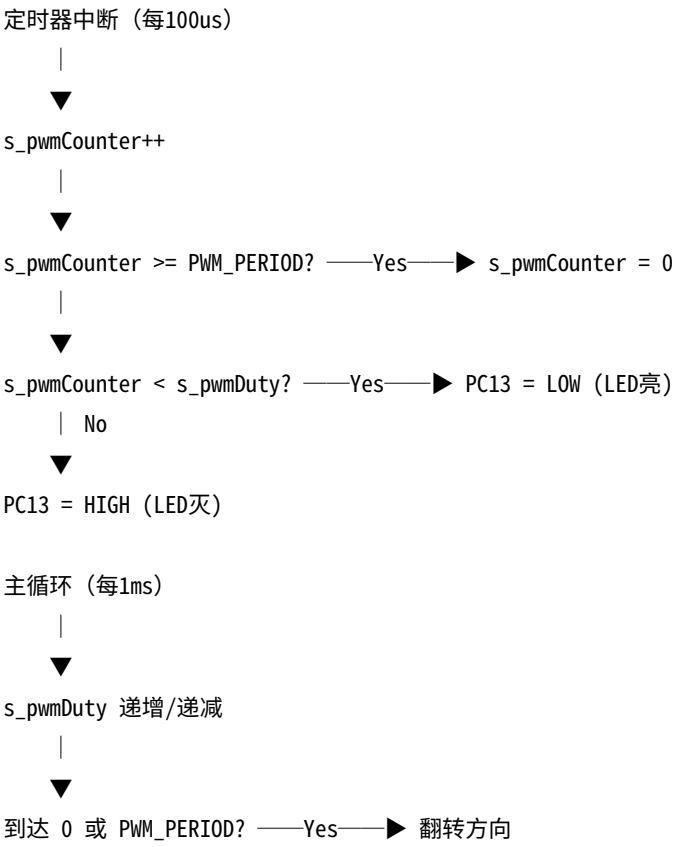
这一部分把本次实验中需要手动添加的代码放到文档中，并逐段解释其含义。

11.8.1 软件 PWM 呼吸灯原理

软件 PWM 通过定时器中断实现：

1. 定时器以固定周期（100 us）进入中断。
2. 在中断服务函数中维护一个计数器 s_pwmCounter，从 0 累加到 PWM 周期值 PWM_PERIOD。
3. 同时维护一个 s_pwmDuty 变量表示当前占空比。
4. 每次中断比较 s_pwmCounter 与 s_pwmDuty：
   - s_pwmCounter < s_pwmDuty 时 PC13 输出低电平（LED 亮）
   - s_pwmCounter >= s_pwmDuty 时 PC13 输出高电平（LED 灭）
5. 在主循环中按呼吸规律逐步改变 s_pwmDuty，实现渐亮渐灭。

下图展示了软件 PWM 的工作流程：



11.8.2 main.c 中新增的变量定义

> CubeMX 生成的 main.c 已自动包含 #include "tim.h"（提供 MX_TIM3_Init() 和 htim3 声明），无需额外添加头文件。

在 Core/Src/main.c 中找到 /* USER CODE BEGIN PV */，添加以下代码：

```
/* USER CODE BEGIN PV */
#define PWM_PERIOD  100U    // PWM 周期 = 100 级（100us × 100 = 10ms，即 100Hz）

static volatile uint32_t s_pwmCounter = 0;  // 当前 PWM 计数值（0 ~ PWM_PERIOD-1）
static volatile uint32_t s_pwmDuty    = 0;  // 当前占空比（0 ~ PWM_PERIOD）
static          uint32_t s_breathStep = 0;  // 主循环呼吸步进计数
```

```
static          int8_t   s_breathDir  = 1;  // 呼吸方向：1 = 变亮，-1 = 变暗
/* USER CODE END PV */
```

代码解析：

1. PWM_PERIOD 定义为 100，与定时器 AutoReload 值（99 + 1）一致，表示 PWM 有 100 个时隙。
2. s_pwmCounter 在中断中累加，每 100 us +1，在 0~99 之间循环，对应 PWM 的 100 个时隙。
3. s_pwmDuty 表示当前占空比，取值范围 0 ~ PWM_PERIOD。0 表示全暗（LED 始终灭），PWM_PERIOD 表示全亮（LED 始终亮）。
4. s_breathStep 和 s_breathDir 在主循环中控制占空比的变化速率和方向。

编程技巧：

1. s_pwmCounter 和 s_pwmDuty 使用 volatile 修饰，因为它们同时在中断和主循环中被访问。
2. s_breathStep 和 s_breathDir 只在主循环中使用，不需要 volatile。
3. 所有变量使用 static 限制作用域在当前文件内，减少命名冲突。

11.8.3 main.c 中的定时器中断回调函数

```
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3)
    {
        s_pwmCounter++;
        if (s_pwmCounter >= PWM_PERIOD)
        {
            s_pwmCounter = 0;
        }

        if (s_pwmCounter < s_pwmDuty)
        {
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
        }
        else
        {
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
        }
    }
}
/* USER CODE END 0 */
```

代码解析：

1. HAL_TIM_PeriodElapsedCallback 是 HAL 库定义的弱函数（__weak），当定时器溢出（计数值到达 AutoReload 值）时由 HAL 库的中断处理函数调用。
2. 回调函数的参数 htim 指向触发中断的定时器句柄。由于系统中可能有多个定时器，通过 htim->Instance == TIM3 判断是否是 TIM3 触发的中断。
3. s_pwmCounter 每次中断加 1，到达 PWM_PERIOD 时归零，形成一个循环计数器。
4. 比较 s_pwmCounter 与 s_pwmDuty 决定 GPIO 输出电平。这就是软件 PWM 的核心——用一个计数器和一个比较值模拟硬件 PWM 的输出比较功能。
5. GPIO_PIN_RESET 即低电平，使共阳极 LED 点亮。

为什么不在中断中做复杂业务：

1. 这个回调每 100 us 调用一次，频率较高（10 kHz）。
2. 回调执行时间过长会影响下一次中断的响应。
3. 当前回调只做了计数、比较和 GPIO 翻转，执行时间极短（几个时钟周期），是合理的中断设计。

11.8.4 main.c 中的主函数

```c
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM3_Init();

    /* USER CODE BEGIN 2 */
    HAL_TIM_Base_Start_IT(&htim3);
    /* USER CODE END 2 */

    while (1)
    {
        /* USER CODE BEGIN 3 */
        s_breathStep++;
        if (s_breathStep >= 10)  // 每 10ms 更新一次占空比，控制呼吸速度
        {
            s_breathStep = 0;
            s_pwmDuty += s_breathDir;

            if (s_pwmDuty >= PWM_PERIOD)
            {
                s_pwmDuty = PWM_PERIOD;
                s_breathDir = -1;  // 到达最亮，开始变暗
            }
            else if (s_pwmDuty == 0)
            {
                s_breathDir = 1;   // 到达最暗，开始变亮
            }
        }
        HAL_Delay(1);
        /* USER CODE END 3 */
    }
}
```

代码解析：

1. `MX_TIM3_Init()` 是由 CubeMX 生成在 `tim.c` 中的定时器初始化函数，配置了预分频器、自动重载值等寄存器。
2. `HAL_TIM_Base_Start_IT(&htim3)` 是关键的启动函数。CubeMX 生成的初始化函数只配置寄存器，不会启动定时器。必须手动调用此函数来启动 TIM3 并使能定时器中断。
3. 主循环中 `HAL_Delay(1)` 每次延时 1 ms，因此主循环每 1 ms 执行一次。
4. `s_breathStep` 每 1 ms 加 1，当累积到 10 时才更新一次占空比，即每 10 ms 更新一次。
5. `s_pwmDuty` 每次增加或减少 1（由 `s_breathDir` 控制方向），在 0 ~ PWM_PERIOD（100）之间变化。
6. 当 `s_pwmDuty` 到达 PWM_PERIOD（最亮）时，方向翻转为 -1（变暗）；当到达 0（最暗）时，方向翻转为 +1（变亮）。

呼吸周期计算：

单向变化：100 级 × 10 ms/级 = 1000 ms = 1 s
一个完整呼吸周期（暗→亮→暗）：1 s × 2 = 2 s

11.8.5 代码结构整体工作流程

1. 系统启动后初始化时钟、GPIO、TIM3。
2. HAL_TIM_Base_Start_IT 启动 TIM3，定时器开始计数。
3. TIM3 每 100 us 溢出一次，触发中断，HAL 库调用 HAL_TIM_PeriodElapsedCallback。
4. 中断回调中执行软件 PWM 逻辑：更新计数值并比较占空比，控制 LED 亮灭。
5. 主循环每 1 ms 运行一次，每 2 ms 更新一次占空比。
6. 占空比在 0~100 之间往复变化，LED 呈现渐亮渐灭的呼吸效果。

11.9 编程技巧总结

1. 定时器中断回调中尽量少做事，只做必要的计数和 GPIO 翻转。
2. 与中断共享的变量应使用 volatile 修饰，防止编译器优化导致数据不一致。
3. CubeMX 生成的 MX_TIMx_Init() 只初始化配置，启动定时器需要额外调用 HAL_TIM_Base_Start_IT。
4. 软件 PWM 的核心思想是用定时器 + 计数器 + 比较值模拟硬件 PWM 的输出比较。
5. 使用 HAL_Delay 控制呼吸速度简单可靠，适合实验场景；更复杂的需求可改用非阻塞定时。

11.10 实验操作步骤

11.10.1 配置 CubeMX 并生成代码

按第 3 节步骤完成配置，点击 GENERATE CODE。

11.10.2 添加应用代码

在 Core/Src/main.c 中按第 6 节代码说明，在对应 USER CODE BEGIN/END 区域添加代码。

11.10.3 编译工程

使用当前开发环境编译工程，确认无编译错误。

11.10.4 烧录程序

通过 ST-Link 将程序下载到 STM32 开发板。

11.10.5 观察现象

程序下载完成后，PC13 LED 应自动开始呼吸效果—-从暗到亮、从亮到暗循环往复。

11.11 实验现象与结果分析

如果实验成功，你应看到如下现象：

1. 上电或下载完成后，PC13 LED 自动开始呼吸效果。
2. LED 从完全熄灭逐渐变亮，到达最亮后逐渐变暗，到达最暗后再变亮，循环往复。
3. 呼吸周期约 2 s（暗 $\rightarrow$ 亮约 1 s + 亮 $\rightarrow$ 暗约 1 s），肉眼观察为流畅的呼吸效果。
4. LED 亮度变化平滑，无明显阶梯感。

这些现象说明：

1. TIM3 定时器及其中断配置正确，定时器正常计数并在 100 us 周期触发中断。
2. 软件 PWM 逻辑正确，`s_pwmCounter` 和 `s_pwmDuty` 协作完成了 GPIO 的 PWM 波形输出。
3. 主循环中占空比更新逻辑正确，`s_pwmDuty` 在 0~100 之间往复变化。
4. 中断回调与主循环之间的共享变量访问正常，未出现数据不一致。

11.12 常见问题排查

问题	可能原因与解决方案
LED 不亮也不闪	检查是否调用了 `HAL_TIM_Base_Start_IT` 启动定时器；检查定时器中断是否在 NVIC 中使能。
LED 常亮不灭	检查 GPIO 初始输出电平是否设为 High；检查 `s_pwmDuty` 是否正确更新。
LED 闪烁而非呼吸	可能是占空比更新过快或 PWM 周期太小；检查 `s_breathStep` 判断阈值和 `PWM_PERIOD` 值。
LED 亮度变化不平滑	可减小定时器周期（如改为 50 us）或增大 PWM 周期值以获得更多灰度等级。
程序下载后无反应	检查 SYS Debug 是否配置为 Serial Wire；检查 HSE 和时钟树配置。

11.13 课后思考题

请结合本次实验，在报告中认真回答以下问题：

1. 为什么 PC13 不能直接使用定时器硬件 PWM 输出通道? 软件 PWM 与硬件 PWM 各自的优缺点是什么?
2. 修改 `PWM_PERIOD` 或主循环中 `s_breathStep >= 2` 的判断阈值会对呼吸速度和效果产生什么影响?
3. 定时器中断周期配置为 100 us，如果改为 50 us 或 200 us 分别会对呼吸效果产生什么影响?
4. 为什么 `s_pwmCounter` 和 `s_pwmDuty` 要使用 `volatile` 修饰? 如果不加 `volatile` 可能出现什么问题?
5. 如果要在不改变硬件的前提下让 LED 在最亮状态停留 1 秒再开始变暗，应该如何修改代码?

11.14 实验报告提交要求

11.14.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述本实验需要实现的功能与验收指标。
3. 实验环境：开发板型号、调试器、软件版本。
4. CubeMX 配置：必须说明 SYS、RCC、Clock Configuration、GPIO、TIM3 参数、NVIC 的关键设置及其原因，并附上每一步的配置截图。
5. 程序设计：分析 `main.c` 中的定时器中断回调和主循环逻辑。
6. 程序流程图：必须展示”定时器中断 → 软件 PWM 计数比较 → GPIO 输出 → 主循环更新占空比”的流程。
7. 实验步骤：包括编译、下载、观察现象等过程。
8. 实验结果：必须展示呼吸灯的亮灭效果（视频或动图截图）。
9. 问题分析与调试记录：说明遇到的任何问题及排查方法。
10. 课后思考题答案：必须完整回答本 README 中 5 个思考题。
11. 总结：概括你对通用定时器、软件 PWM 和中断编程的理解。
12. 附录：可附核心代码、更多截图、参考资料。

11.14.2 图片与流程图要求

1. 报告中至少包含 8 张 CubeMX 配置截图（SYS、RCC、时钟树、GPIO、TIM3 参数、NVIC、工程设置、代码生成选项）。
2. 每张图片必须标注图号和图题，并在正文中说明该图用于证明什么。
3. 程序流程图必须单独成图，不能只用文字描述替代。

11.14.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 11-定时器呼吸灯实验.pdf`。
2. 若实验未完全成功，也必须提交完整报告，重点说明失败现象、原因分析和改进方向。
3. 报告中不能只贴代码或截图，必须结合现象进行解释。

11.14.4 最低验收标准

1. LED 能够实现渐亮渐灭的呼吸效果。
2. 能够说明定时器中断周期的计算方法。
3. 报告能够解释软件 PWM 的原理和中断回调与主循环的协作关系。

11.15 可进一步扩展的方向

1. 改用正弦波查表法更新占空比，使呼吸效果更自然。
2. 使用多个定时器通道驱动多个 LED 实现不同节奏的呼吸效果。
3. 增加按键切换呼吸速度的功能。
4. 将软件 PWM 抽象为一个独立模块，方便其他项目复用。

11.16 总结

本实验通过通用定时器中断实现软件 PWM，驱动 PC13 共阳极 LED 产生呼吸灯效果。实验涵盖定时器配置、中断回调、GPIO 控制和主循环协同工作等核心内容。相比于直接使用硬件 PWM，软件 PWM 方案更考验对定时器中断和主循环协同的理解，是嵌入式定时器应用的典型入门实验。

12 STM32 通用定时器 – 三通道硬件 PWM 交替呼吸灯实验

12.1 统一作业说明

12.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成 TIM3 定时器、时钟树、调试接口等配置，并保留 `.ioc` 文件。
2. 基于 HAL 库与硬件 PWM 实现三路 LED 交替呼吸灯效果。
3. 成功编译、下载并在硬件上观察到三路 LED 轮流渐亮渐灭的交替呼吸效果。
4. 在实验报告中说明定时器配置参数、代码结构、实验现象、问题排查和课后思考。
5. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

12.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6
外设	TIM3 通用定时器 PWM 输出
引脚	PA7 (TIM3_CH2)、PB0 (TIM3_CH3)、PB1 (TIM3_CH4)
必做功能	三路 LED 交替呼吸灯效果（轮流渐亮渐灭）
理论要求	能解释定时器 PWM 周期计算、硬件 PWM 原理、主循环与 PWM 占空比更新的协同
验收方式	提交 PDF 格式实验报告，包含配置截图、代码分析、现象描述和课后思考题书面回答

12.1.3 本次必须提交的内容

1. 一份 PDF 格式实验报告（含 CubeMX 各步骤配置截图、关键代码截图、实验现象描述）。
2. 课后思考题的书面回答（写在报告中）。

12.1.4 报告必须回答的问题

1. 本实验使用硬件 PWM 输出通道，与软件 PWM 方案相比各自的优缺点是什么?
2. 修改 `HAL_Delay(10)` 中的延时值或定时器的 Prescaler/Period 会对呼吸速度和效果产生什么影响?
3. 将 PWM 周期从 100 改为 200（Period = 200-1），占空比分辨率提高但 PWM 频率降低，这对 LED 视觉效果有何影响?
4. 全局变量 `Led` 在主循环中递增并控制通道切换，如果改用定时器中断来控制通道切换，需要如何修改代码?
5. 如果要在不改变硬件的前提下让三路 LED 同时以不同亮度呼吸（而非交替），应该如何修改代码?

12.2 关键说明

本实验使用 **TIM3** 的硬件 **PWM** 输出通道（CH2: PA7, CH3: PB0, CH4: PB1），通过主循环动态修改 CCR 寄存器（`__HAL_TIM_SET_COMPARE`）来改变各通道占空比，实现三路 LED 交替呼吸灯效果。与软件 PWM 不同，硬件 PWM 的波形由定时器硬件自动生成，主循环仅需更新占空比值即可。

12.3 实验目的

本实验基于 STM32 的通用定时器 TIM3 实现三通道硬件 PWM 交替呼吸灯，要求学生完成从 CubeMX 配置、代码生成、程序编写到硬件验证的完整实验流程。通过本实验，你应掌握以下内容:

1. 理解通用定时器的硬件 PWM 输出原理。
2. 掌握 STM32CubeMX 中多通道 PWM 输出的配置方法。
3. 理解硬件 PWM 与软件 PWM 的区别及各自适用场景。
4. 学会通过动态修改 CCR 寄存器控制 PWM 占空比。
5. 学会在主循环中协调多通道 PWM 的交替工作。

12.4 实验环境

12.4.1 硬件环境

1. STM32F103C8T6 核心板一块。
2. 三路 LED（分别连接 PA7、PB0、PB1 引脚，需外接或使用板载 LED 扩展）。
3. ST-Link 下载器。
4. 逻辑分析仪（可选，用于抓取 PWM 波形）。

12.4.2 软件环境

1. STM32CubeMX。
2. VS Code 或其他支持该工程的 STM32 开发环境。
3. ARM GCC / CMake 工具链或等效编译环境。
4. ST-Link 驱动。

12.5 STM32CubeMX 配置步骤

下面每一步的截图必须保留在实验报告中，因为这些配置是实验成功的前提。

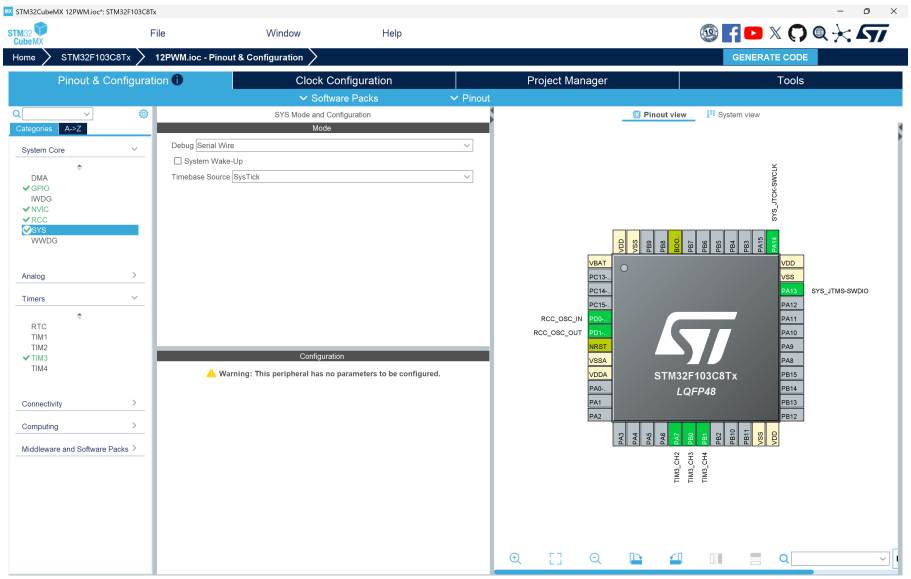
12.5.1 新建工程并选择芯片

打开 STM32CubeMX，点击 **ACCESS TO MCU SELECTOR**，选择 **STM32F103C8T6**。

12.5.2 配置调试接口（SYS）

在 **Pinout & Configuration → System Core → SYS** 中，将 **Debug** 设置为 **Serial Wire**。

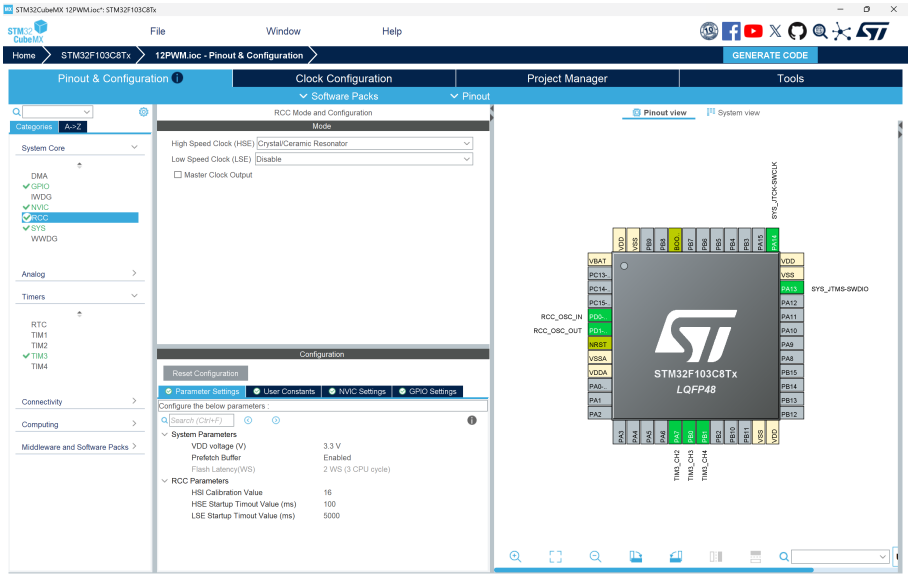
这样便于后续通过 ST-Link 下载和调试程序。如果未正确配置调试接口，可能出现芯片无法正常调试或下载的问题。



12.5.3 配置高速外部时钟（RCC）

在 **Pinout & Configuration → System Core → RCC** 中，将 **High Speed Clock (HSE)** 设置为 **Crystal/Ceramic Resonator**。

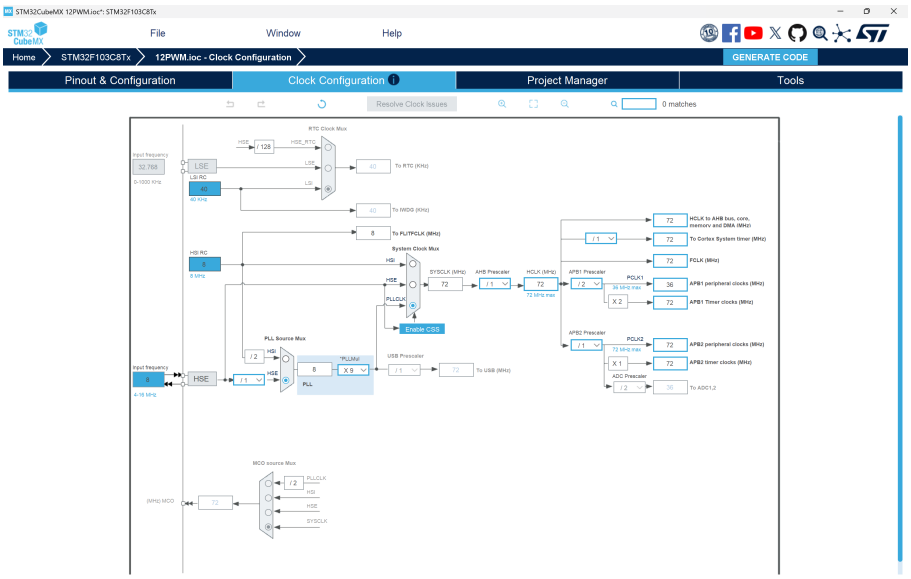
这样做的目的是为 PLL 和系统时钟提供稳定的外部时钟来源（8 MHz 晶振）。



12.5.4 配置时钟树（Clock Configuration）

进入 **Clock Configuration** 页面：

1. 将系统时钟源选择为 **PLLCLK**。
2. 设置 **HSE** 为 8 MHz。
3. 设置 **PLLMul** 为 x9，使 **PLLCLK** = 72 MHz。
4. 确保 **HCLK** = 72 MHz，**APB1 Timer Clocks** = 72 MHz，**APB2 Timer Clocks** = 72 MHz。
5. 确认 **APB1 Prescaler** = /2，这样 APB1 外设时钟为 36 MHz，而 APB1 定时器时钟为 72 MHz（自动 ×2）。



12.5.5 配置 TIM3 PWM 输出通道

在 **Pinout & Configuration** → **Timers** 中，选择 **TIM3**：

1. **Clock Source**：**Internal Clock**（使用内部时钟源）
2. **Channel 2/3/4**：均设置为 **PWM Generation CHx**
3. 在下方参数设置区域：

   - **Prescaler**：720 - 1（即 719），定时器计数时钟 = 72 MHz / 720 = 100 kHz，即每 10 us 计数一次
   - **Counter Mode**：**Up**（向上计数模式）
   - **Counter Period (AutoReload - 16 bits value)**：100 - 1（即 99），PWM 周期 = 100 × 10 us = 1 ms，PWM 频率 = 1 kHz
   - **Auto-reload preload**：**Disable**
   - **Pulse**（各通道）：均设为 50（初始占空比 50%）

   参数说明：PWM 频率计算公式为—-

```
F_pwm = Tclk / ((Prescaler + 1) × (Counter Period + 1))
      = 72 MHz / (720 × 100)
      = 1 kHz
```

PWM 周期 = 1 ms。Period = 100-1 提供 100 级（0~99）的占空比精度，呼吸效果足够平滑。1 kHz 的 PWM 频率远超人眼闪烁感知阈值（约 50 Hz），视觉上完全平滑。

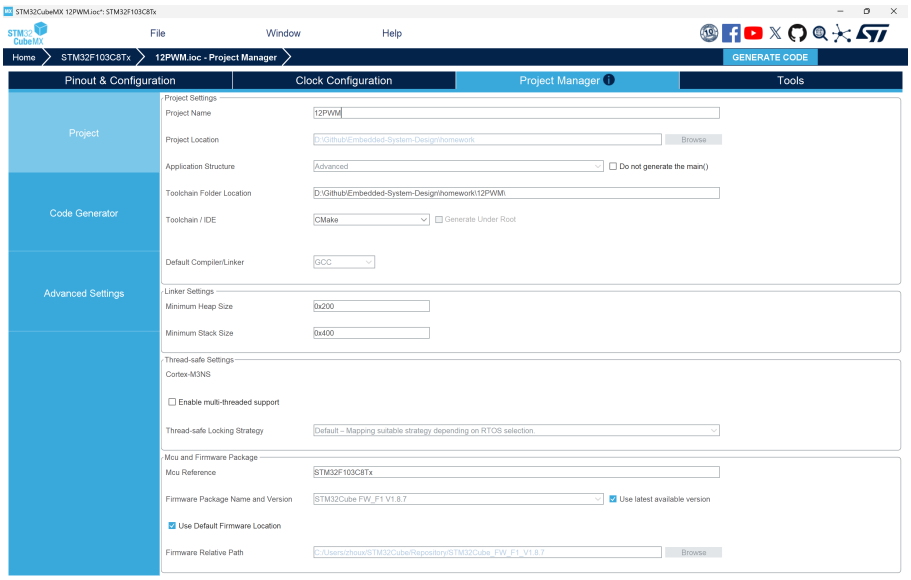
4. 确认 PA7、PB0、PB1 引脚分别被分配为 **TIM3_CH2**、**TIM3_CH3**、**TIM3_CH4**。

> 注意：本实验无需在 NVIC Settings 中使能 TIM3 全局中断。硬件 PWM 的波形由定时器硬件自动生成，占空比通过主循环直接修改 CCR 寄存器更新，不需要中断参与。

12.5.6 工程设置与生成代码

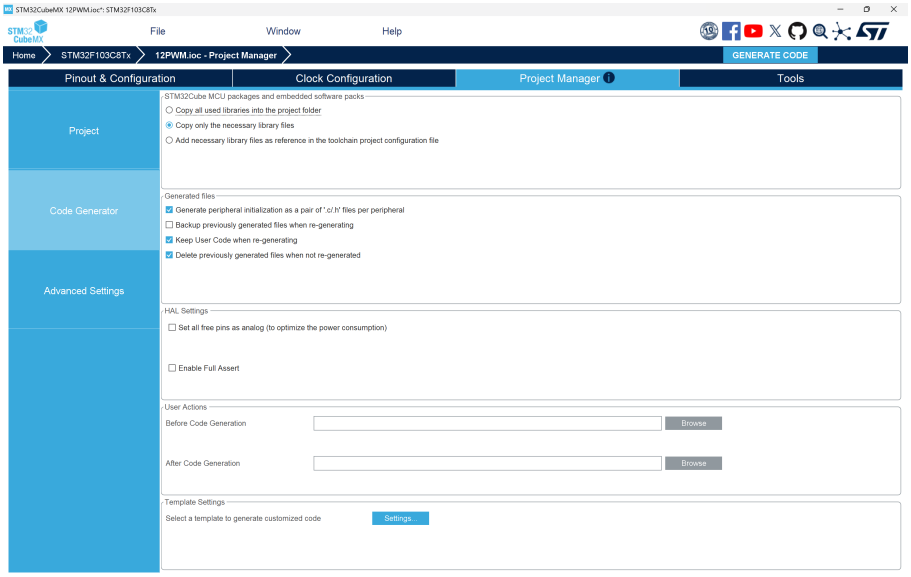
在 **Project Manager** 页面：

1. **Project Name**：`12PWM`
2. **Project Location**：选择当前 `12PWM` 目录
3. **Toolchain / IDE**：选择 **CMake**



切换到 **Code Generator** 选项卡：

1. 勾选 **Generate peripheral initialization as a pair of '.c/.h' files per peripheral**，这样 TIM3 初始化代码会生成到独立的 `tim.c` / `tim.h` 文件中，便于代码管理。



最后点击 **GENERATE CODE** 生成工程。

12.6 硬件连接要点

1. **PA7 (TIM3_CH2)**：连接 LED1 的阴极，LED 阳极通过限流电阻接 VCC。
2. **PB0 (TIM3_CH3)**：连接 LED2 的阴极，LED 阳极通过限流电阻接 VCC。
3. **PB1 (TIM3_CH4)**：连接 LED3 的阴极，LED 阳极通过限流电阻接 VCC。
4. 各 GPIO 配置为复用推挽输出（AF_PP），由 TIM3 硬件 PWM 直接驱动。
5. PWM 极性为高电平有效（OCPolarity = High），占空比越高 LED 越亮（需要根据实际 LED 接法确认）。

12.7 本工程实现的实验内容

当前工程实现了基于硬件 PWM 的三通道交替呼吸灯，功能如下：

1. 系统启动后自动开始交替呼吸灯效果。
2. TIM3 以 1 kHz 频率自动输出 PWM 波形，无需软件干预。
3. 主循环中按固定步进更新当前活跃通道的 CCR 值（0~99 渐变），不活跃通道 CCR 置 0。
4. 三路 LED 轮流呼吸：CH3（PB0）→ CH4（PB1）→ CH2（PA7）→ CH3（PB0）……循环往复。
5. 每路 LED 完成一次完整呼吸（暗 → 亮 → 暗）约 2 秒。

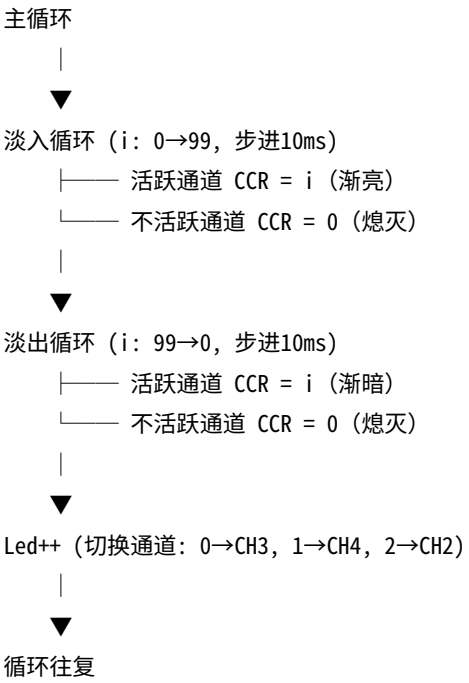
12.8 核心代码说明

12.8.1 硬件 PWM 交替呼吸灯原理

硬件 PWM 交替呼吸灯通过以下方式实现：

1. TIM3 被配置为三通道 PWM 输出模式，定时器自动产生 PWM 波形。
2. 每个通道的占空比由其 CCR（Capture/Compare Register）寄存器值决定：CCR / Period = 占空比。
3. 主循环中动态修改 CCR 值（__HAL_TIM_SET_COMPARE）来改变占空比。
4. 全局变量 Led 记录当前呼吸轮次，Led % 3 决定本轮哪个通道呼吸。
5. 活跃通道的 CCR 按呼吸曲线变化（0→99→0），不活跃通道的 CCR 置 0（LED 熄灭）。
6. HAL_Delay(10) 每步延时 10 ms，控制呼吸速度。

下图展示了交替呼吸的工作流程：



12.8.2 main.c 中新增的变量定义

在 Core/Src/main.c 中找到 /* USER CODE BEGIN PV */，添加以下代码：

```
/* USER CODE BEGIN PV */
uint32_t Led = 0;   // 呼吸轮次计数器，Led % 3 决定当前活跃通道
/* USER CODE END PV */
```

代码解析：

1. Led 是一个全局变量，每完成一轮呼吸（一次淡入 + 一次淡出）自增 1。
2. Led % 3 的值为 0 时活跃通道为 CH3（PB0），1 时活跃 CH4（PB1），2 时活跃 CH2（PA7）。
3. 使用 uint32_t 类型，取值范围足够覆盖整个运行周期。

12.8.3 main.c 中的 PWM 启动代码

在 Core/Src/main.c 中找到 /* USER CODE BEGIN 2 */，添加以下代码：

```
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_4);
/* USER CODE END 2 */
```

代码解析：

1. CubeMX 生成的 MX_TIM3_Init() 只配置寄存器，不启动 PWM 输出。
2. 必须手动调用 HAL_TIM_PWM_Start() 来使能每个通道的 PWM 输出并启动定时器计数器。
3. 三个通道全部启动后，TIM3 开始计数并自动输出 PWM 波形。
4. 不需要使用 HAL_TIM_PWM_Start_IT，因为本实验不使用定时器中断。

12.8.4 main.c 中的主循环

在 Core/Src/main.c 的主循环中添加以下代码：

```
while (1)
{
    /* USER CODE END WHILE */
    for (int i = 0; i < 100; i++)
    {
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, (Led % 3 == 2) ? i : 0);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, (Led % 3 == 0) ? i : 0);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, (Led % 3 == 1) ? i : 0);
        HAL_Delay(10);
    }
    for (int i = 99; i >= 0; i--)
    {
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, (Led % 3 == 2) ? i : 0);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, (Led % 3 == 0) ? i : 0);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, (Led % 3 == 1) ? i : 0);
        HAL_Delay(10);
    }
    Led++;
    /* USER CODE BEGIN 3 */
}
```

代码解析：

1. __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_x, value)：这是 HAL 库提供的宏，用于直接向指定通道的 CCR 寄存器写入比较值。CCR 值决定 PWM 占空比（CCR / Period）。
2. 三个通道同一时刻只有一个通道的 **CCR** 为非零值：活跃通道的 CCR 随 i 变化实现呼吸效果，其余两个通道 CCR = 0（LED 熄灭）。
3. 通道分配逻辑：
   - Led % 3 == 0：CH3 活跃，CH2/CH4 的 CCR 置 0
   - Led % 3 == 1：CH4 活跃，CH2/CH3 的 CCR 置 0
   - Led % 3 == 2：CH2 活跃，CH3/CH4 的 CCR 置 0
4. 淡入阶段：i 从 0 递增到 99，每步延时 10 ms，活跃通道 CCR 从 0→99（LED 渐亮）。
5. 淡出阶段：i 从 99 递减到 0，每步延时 10 ms，活跃通道 CCR 从 99→0（LED 渐暗）。
6. HAL_Delay(10)：每步延时 10 ms，一个完整呼吸周期 = $100 \times 10\ \text{ms} \times 2 = 2000\ \text{ms} = 2$ 秒。

7. Led++：一轮呼吸完成后递增，切换到下一个通道。

呼吸周期计算：

单向变化时间：100 级 × 10 ms/级 = 1000 ms = 1 s
单通道完整呼吸周期：1 s（亮）+ 1 s（暗）= 2 s
三通道完整循环：2 s × 3 = 6 s

12.8.5 为什么不需要中断

本实验与 11Timer（软件 PWM 呼吸灯）不同，不需要使用定时器中断。原因如下：

1. 硬件 **PWM** 自动产生波形：TIM3 被配置为 PWM 输出模式后，定时器硬件根据 CCR 寄存器的值自动控制 GPIO 输出电平。当计数器 CNT < CCR 时输出有效电平，CNT >= CCR 时输出无效电平，全程由硬件完成。
2. 主循环只修改占空比：呼吸灯只需要定期更新 CCR 值来改变占空比。这个更新频率很低（每 10 ms 一次），完全可以在主循环中通过 HAL_Delay 控制节奏。
3. 无需中断回调：因为没有需要在中断中处理的逻辑（如软件 PWM 的计数比较），所以不需要使能 TIM3 中断。

12.9 编程技巧总结

1. 硬件 PWM 的核心优势：波形由定时器硬件自动生成，CPU 只需在需要改变占空比时更新 CCR 寄存器。
2. __HAL_TIM_SET_COMPARE 宏可直接修改指定通道的 CCR 值，无需停止或重新配置定时器。
3. 通过将不活跃通道的 CCR 置 0 来实现通道的开关切换，无需操作 CCER 寄存器或调用 Start/Stop。
4. HAL_Delay 控制呼吸速度简单可靠，适合实验场景；更复杂的需求可改用非阻塞定时。
5. 三个通道共享同一个定时器，PWM 频率相同，只是占空比各自独立可调。

12.10 实验操作步骤

12.10.1 配置 CubeMX 并生成代码

按第 3 节步骤完成配置，点击 GENERATE CODE。

12.10.2 添加应用代码

在 Core/Src/main.c 中按第 6 节代码说明，在对应 USER CODE BEGIN/END 区域添加代码。

12.10.3 编译工程

使用当前开发环境编译工程，确认无编译错误。

12.10.4 烧录程序

通过 ST-Link 将程序下载到 STM32 开发板。

12.10.5 观察现象

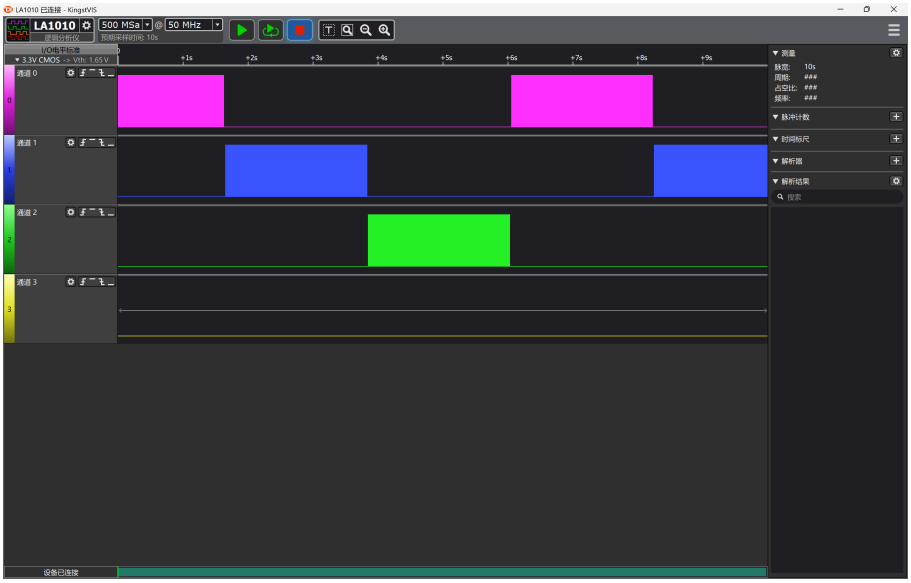
程序下载完成后，三路 LED 应自动开始交替呼吸效果——-LED1 渐亮渐灭 → LED2 渐亮渐灭 → LED3 渐亮渐灭，循环往复。

12.11 实验现象与结果分析

如果实验成功，你应看到如下现象：

1. 上电或下载完成后，三路 LED 自动开始交替呼吸。
2. 每路 LED 从完全熄灭逐渐变亮，到达最亮后逐渐变暗，完成一轮后切换到下一路 LED。
3. 每路 LED 呼吸周期约 2 s（暗 → 亮约 1 s + 亮 → 暗约 1 s）。
4. LED 亮度变化平滑，无明显阶梯感（1 kHz PWM 频率 + 100 级占空比精度）。
5. 三路 LED 之间切换流畅，无明显停顿。

通过逻辑分析仪抓取的 PWM 波形如下：



这些现象说明：

1. TIM3 定时器及 PWM 配置正确，三路 PWM 输出正常工作。
2. 主循环中 CCR 更新逻辑正确，活跃通道按呼吸曲线变化，不活跃通道保持 0。
3. `Led` 变量正确控制通道切换顺序。
4. 硬件 PWM 波形稳定，无抖动。

12.12 常见问题排查

问题	可能原因与解决方案
LED 不亮也不闪	检查是否调用了 `HAL_TIM_PWM_Start` 启动各通道；检查 GPIO 是否配置为 AF_PP 复用推挽输出。
三路 LED 同时亮灭	检查主循环中 CCR 更新逻辑是否使用了 `Led % 3` 条件判断；确认不活跃通道的 CCR 是否置 0。
LED 亮度变化不平滑	可增大 Period 值以增加占空比分辨率；或减小 `HAL_Delay` 延时使过渡更细腻。
LED 闪烁	可能是 PWM 频率过低；检查 Prescaler 和 Period 配置是否正确。
某一通道不亮	检查该通道对应的 GPIO 引脚连接和配置；确认 `HAL_TIM_PWM_Start` 已包含该通道。

12.13 课后思考题

请结合本次实验，在报告中认真回答以下问题：

1. 本实验使用硬件 PWM 输出通道，与软件 PWM 方案相比各自的优缺点是什么?
2. 修改 `HAL_Delay(10)` 中的延时值或定时器的 Prescaler/Period 会对呼吸速度和效果产生什么影响?
3. 将 PWM 周期从 100 改为 200（Period = 200-1），占空比分辨率提高但 PWM 频率降低，这对 LED 视觉效果有何影响?
4. 全局变量 `Led` 在主循环中递增并控制通道切换，如果改用定时器中断来控制通道切换，需要如何修改代码?
5. 如果要在不改变硬件的前提下让三路 LED 同时以不同亮度呼吸（而非交替），应该如何修改代码?

12.14 实验报告提交要求

12.14.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述本实验需要实现的功能与验收指标。
3. 实验环境：开发板型号、调试器、软件版本。
4. CubeMX 配置：必须说明 SYS、RCC、Clock Configuration、TIM3 PWM 通道的参数配置及其原因，并附上每一步的配置截图。
5. 程序设计：分析 `main.c` 中的 PWM 启动逻辑和三通道交替呼吸的主循环逻辑。
6. 程序流程图：必须展示” 主循环更新 CCR → 活跃通道渐亮/渐暗 → 不活跃通道熄灭 → 切换通道” 的流程。

7. 实验步骤：包括编译、下载、观察现象等过程。
8. 实验结果：必须展示三路 LED 交替呼吸的效果（逻辑分析仪波形截图或实拍照片）。
9. 问题分析与调试记录：说明遇到的任何问题及排查方法。
10. 课后思考题答案：必须完整回答本 README 中 5 个思考题。
11. 总结：概括你对通用定时器硬件 PWM 的理解。
12. 附录：可附核心代码、更多截图、参考资料。

12.14.2 图片与流程图要求

1. 报告中至少包含 6 张 CubeMX 配置截图（SYS、RCC、时钟树、TIM3 参数、工程设置、代码生成选项）。
2. 每张图片必须标注图号和图题，并在正文中说明该图用于证明什么。
3. 程序流程图必须单独成图，不能只用文字描述替代。
4. 实验现象部分需附逻辑分析仪波形图或 LED 实拍照片。

12.14.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 12-硬件 PWM 呼吸灯实验.pdf`。
2. 若实验未完全成功，也必须提交完整报告，重点说明失败现象、原因分析和改进方向。
3. 报告中不能只贴代码或截图，必须结合现象进行解释。

12.14.4 最低验收标准

1. 三路 LED 能够实现轮流渐亮渐灭的交替呼吸效果。
2. 能够说明 PWM 频率和占空比分辨率与定时器参数的关系。
3. 报告能够解释硬件 PWM 的工作原理和主循环更新机制。

12.15 可进一步扩展的方向

1. 改用正弦波查表法更新占空比，使呼吸效果更自然。
2. 使三路 LED 以不同相位同时呼吸（如 RGB 色彩混合效果）。
3. 增加按键切换呼吸速度或呼吸模式的功能。
4. 使用定时器中断实现更精确的 CCR 更新时序。

12.16 总结

本实验通过 TIM3 的硬件 PWM 输出通道驱动三路 LED，在主循环中动态修改 CCR 寄存器实现交替呼吸灯效果。相比于 11Timer 的软件 PWM 方案，硬件 PWM 将波形生成交给定时器硬件，CPU 仅需更新占空比，代码更简洁、CPU 占用更低。实验涵盖定时器 PWM 配置、多通道协调、主循环时序控制等核心内容，是嵌入式定时器 PWM 应用的典型实验。

13 STM32 高级定时器 – 互补 PWM 输出实验

13.1 统一作业说明

13.1.1 学生需要完成的核心任务

1. 使用 STM32CubeMX 完成 TIM1 高级定时器、互补 PWM 通道、死区时间、刹车输入、时钟树、调试接口等配置，并保留 `.ioc` 文件。
2. 基于 HAL 库与高级定时器实现硬件互补 PWM 输出，CH2 (PA9) 与 CH2N (PB14) 输出 50% 占空比的互补波形，带死区插入。
3. 成功编译、下载并在硬件上用逻辑分析仪验证互补 PWM 波形及死区时间。
4. 在实验报告中说明定时器配置参数、互补 PWM 原理、死区时间计算、代码结构、实验现象、问题排查和课后思考。
5. 按 00Template/README.md 中提供的 LaTeX 模板撰写中文实验报告并提交 PDF。

13.1.2 本次作业验收目标

项目	要求
处理器平台	STM32F103C8T6
外设	TIM1 高级定时器 + CH2 (PA9) / CH2N (PB14) 互补输出 + BKIN (PB12) 刹车输入
必做功能	50% 占空比互补 PWM 输出，带死区时间
理论要求	能解释互补 PWM 原理、死区时间计算、高级定时器与通用定时器的区别、刹车功能的作用
验收方式	提交 PDF 格式实验报告，包含配置截图、代码分析、逻辑分析仪波形截图和课后思考题书面回答

13.1.3 本次必须提交的内容

1. 一份 PDF 格式实验报告（含 CubeMX 各步骤配置截图、关键代码截图、逻辑分析仪波形截图）。
2. 课后思考题的书面回答（写在报告中）。

13.1.4 报告必须回答的问题

1. 什么是互补 PWM? 为什么电机驱动、开关电源等应用中需要互补 PWM 和死区时间?
2. 死区时间 DTG=43 对应的实际时间是多少? 如何通过定时器时钟频率计算死区时间?
3. 高级定时器 TIM1 与通用定时器（如 TIM3）在功能上有哪些主要区别?
4. 刹车功能（Break）在互补 PWM 应用中的作用是什么? 本实验中 BKIN 引脚为何要配置为 Pull-down?
5. 如果将 OCPolarity 和 OCNPolarity 分别改为不同极性（一个 HIGH 一个 LOW），输出波形会发生什么变化? 为什么?

13.2 关键说明

TIM1 是 STM32F103 的高级定时器，自带死区发生器和互补 PWM 输出功能，可直接输出带死区保护的 CHx 和 CHxN 互补信号。互补 PWM 广泛用于电机驱动半桥 / 全桥电路、开关电源等需要上下桥臂交替导通的场景，死区时间用于防止上下桥臂同时导通造成短路。

13.3 实验目的

本实验基于 STM32 的高级定时器 TIM1 实现硬件互补 PWM 输出，要求学生完成从 CubeMX 配置、代码生成、程序编写到硬件验证的完整实验流程。通过本实验，你应掌握以下内容：

1. 理解高级定时器与通用定时器的功能区别。
2. 掌握 STM32CubeMX 中高级定时器互补 PWM 的配置方法。
3. 理解互补 PWM 的工作原理以及死区时间的作用和计算方法。
4. 理解刹车功能在电机驱动中的保护作用。
5. 学会使用逻辑分析仪验证互补 PWM 波形和死区时间。

13.4 实验环境

13.4.1 硬件环境

1. STM32F103C8T6 核心板一块。
2. ST-Link 下载器。
3. 逻辑分析仪（用于观察互补 PWM 波形和死区时间）。

13.4.2 软件环境

1. STM32CubeMX。
2. VS Code 或其他支持该工程的 STM32 开发环境。
3. ARM GCC / CMake 工具链或等效编译环境。
4. ST-Link 驱动。

13.5 STM32CubeMX 配置步骤

下面每一步的截图必须保留在实验报告中，因为这些配置是实验成功的前提。你不仅要会”照着做”，还要理解每一步为什么这样设置。

13.5.1 新建工程并选择芯片

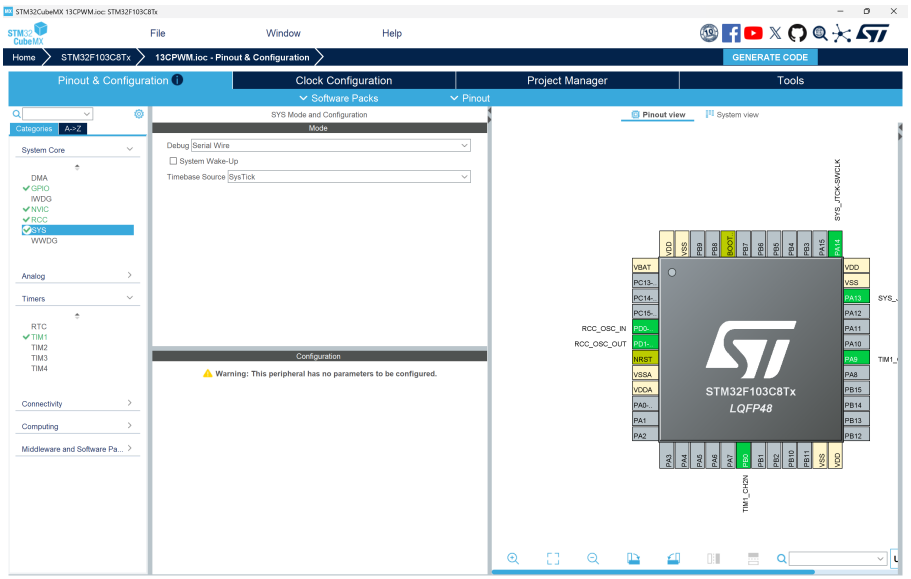
打开 STM32CubeMX，点击 **ACCESS TO MCU SELECTOR**，选择 **STM32F103C8T6**。

13.5.2 配置调试接口（SYS）

在 **Pinout & Configuration** → **System Core** → **SYS** 中，将 **Debug** 设置为 **Serial Wire**。

这样做的原因是：

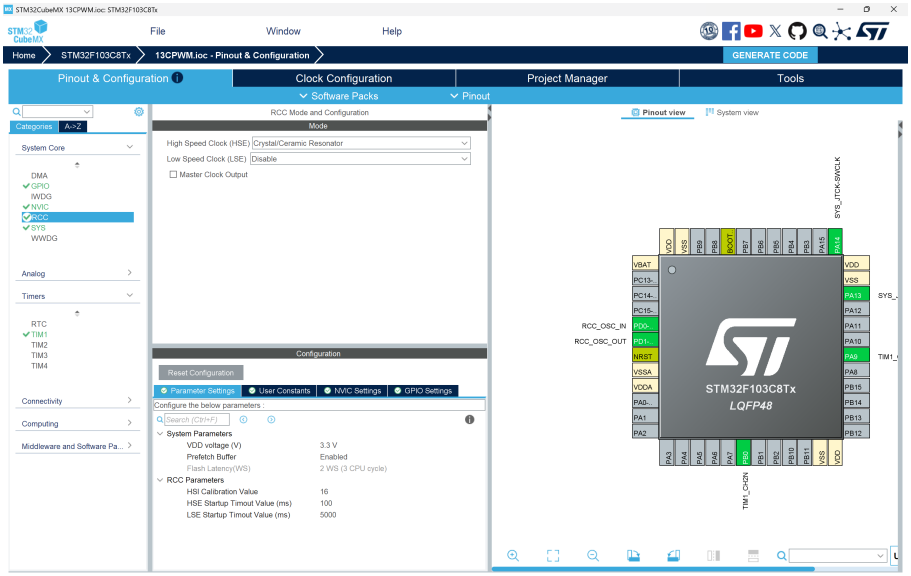
1. 便于后续通过 ST-Link 下载和调试程序。
2. 如果未正确配置调试接口，可能出现芯片无法正常调试或下载的问题。



13.5.3 配置高速外部时钟（RCC）

在 **Pinout & Configuration** → **System Core** → **RCC** 中，将 **High Speed Clock (HSE)** 设置为 **Crystal/Ceramic Resonator**。

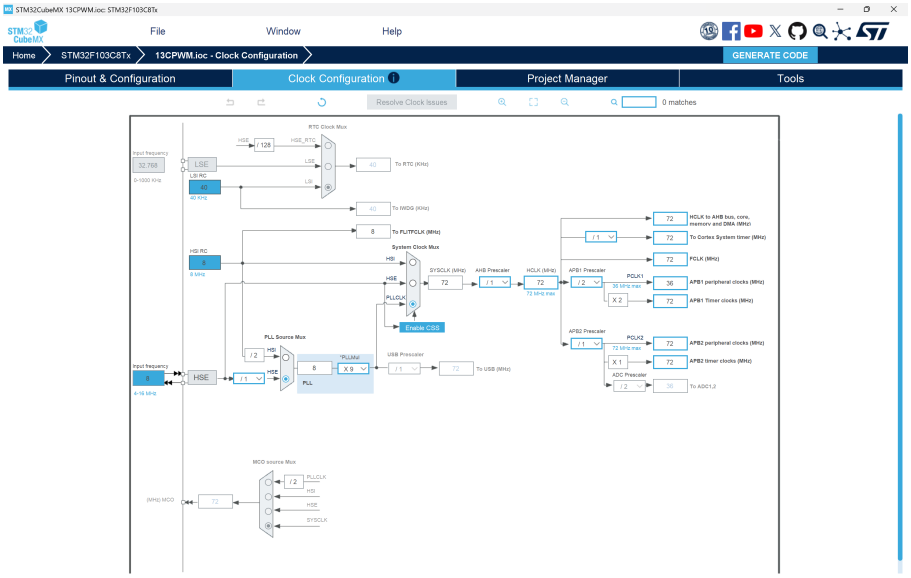
这样做的目的是为 PLL 和系统时钟提供稳定的外部时钟来源（8 MHz 晶振）。



13.5.4 配置时钟树（Clock Configuration）

进入 **Clock Configuration** 页面：

1. 将系统时钟源选择为 **PLLCLK**。
2. 设置 **HSE** 为 8 MHz。
3. 设置 **PLLMul** 为 x9，使 **PLLCLK** = 72 MHz。
4. 确保 **HCLK** = 72 MHz，**APB2 Timer Clocks** = 72 MHz（TIM1 挂在 APB2 总线上）。
5. 确认 **APB1 Prescaler** = /2，**APB2 Prescaler** = /1。



13.5.5 配置 TIM1 基本参数

在 **Pinout & Configuration** → **Timers** 中，选择 **TIM1**：

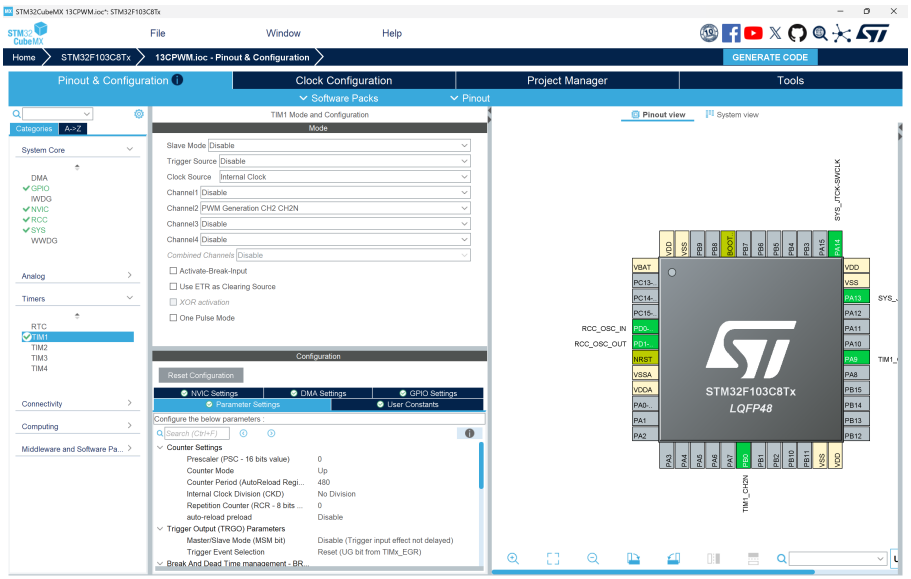
1. **Clock Source**：**Internal Clock**（使用内部时钟源）
2. 在下方参数设置区域：

- **Prescaler**：0（不分频，定时器计数时钟 = 72 MHz）
- **Counter Mode**：**Up**（向上计数模式）
- **Counter Period (AutoReload - 16 bits value)**：480
- **Auto-reload preload**：**Enable**

参数说明：定时器 PWM 频率计算公式为—-

```
F_pwm = Tclk / ((Prescaler + 1) × (Counter Period + 1))
      = 72 MHz / (1 × 481)
      ~ 149.7 kHz
```

占空比分辨率 = 481 级（0 ~ 480），设置 CCR = 240 即可获得约 50% 占空比。



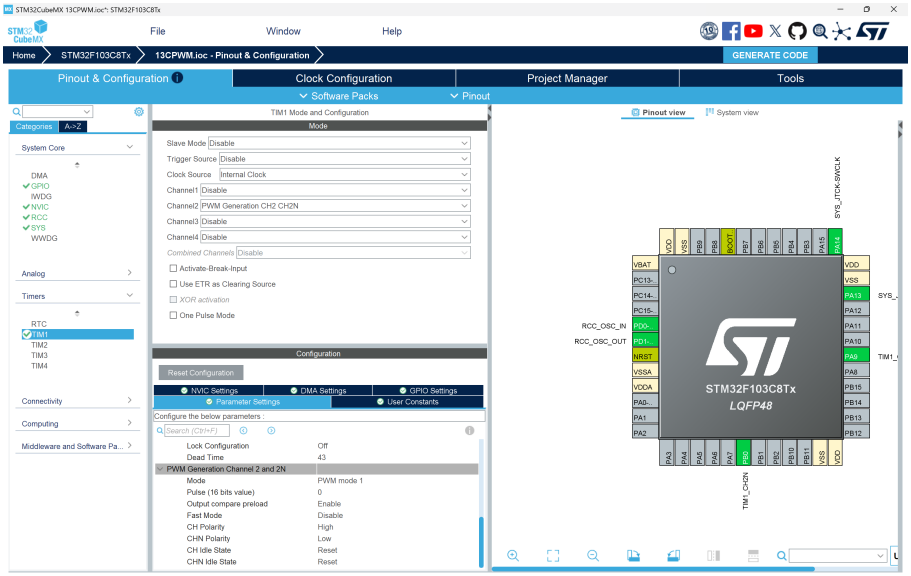
13.5.6 配置互补 PWM 输出通道（CH2 + CH2N）

在 **TIM1 Mode and Configuration** 面板中：

1. 将 **Channel2** 设置为 **PWM Generation CH2 CH2N**（互补 PWM 输出模式）。
2. 在下方 Channel2 参数设置区域：

- **Mode**：**PWM mode 1**（CNT < CCR 时输出有效电平）
- **Pulse**：0（初始占空比，程序中设为 240）
- **CH Polarity**：**High**（CH2 输出极性为高有效）
- **CHN Polarity**：**High**（CH2N 输出极性为高有效，与 CH 相同极性才能得到互补输出）

互补输出原理：TIM1 高级定时器的死区发生器内部已对 CHxN 做了反相处理。因此 CHx 和 CHxN 的极性必须设为相同，才能得到真正的互补波形。若设为不同极性，内部反相与极性反相抵消，两路输出将变为同相。



13.5.7 配置死区时间与刹车功能

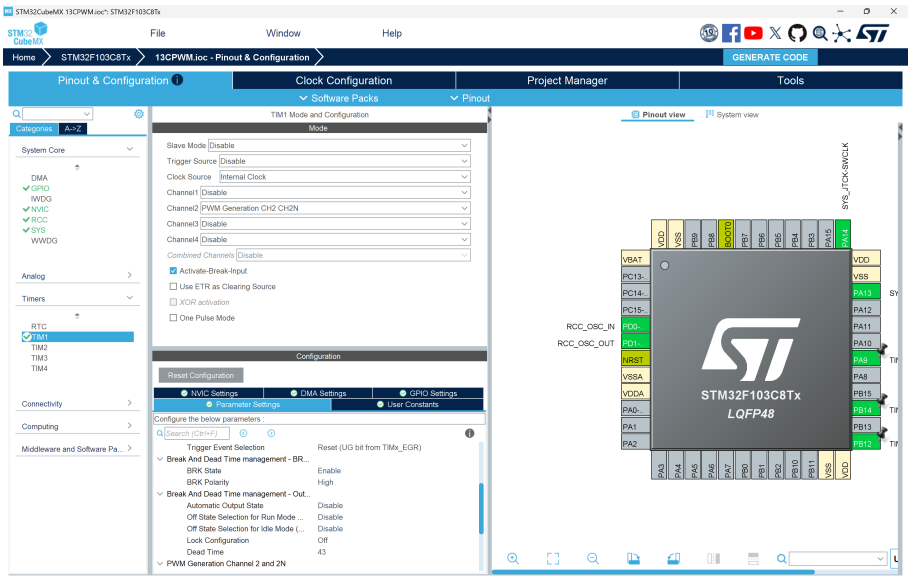
在 TIM1 配置面板中找到 **Break and Dead-Time settings** 区域：

1. **Dead Time**：43（约 0.6 us 死区时间）
2. **BRK**：勾选启用刹车功能
3. **BRK Polarity**：**High**（刹车输入高有效）

死区时间计算：

```
Tdts = 1 / 72 MHz ~ 13.89 ns
Dead Time = DTG × Tdts = 43 × 13.89 ns ~ 0.6 us
```

刹车功能说明：当 BKIN 引脚（PB12）检测到有效电平时，硬件自动关闭 CHx 和 CHxN 输出，用于电机驱动中的过流、过压等故障保护。PB12 配置为 Pull-down 确保正常工作时刹车不被误触发。



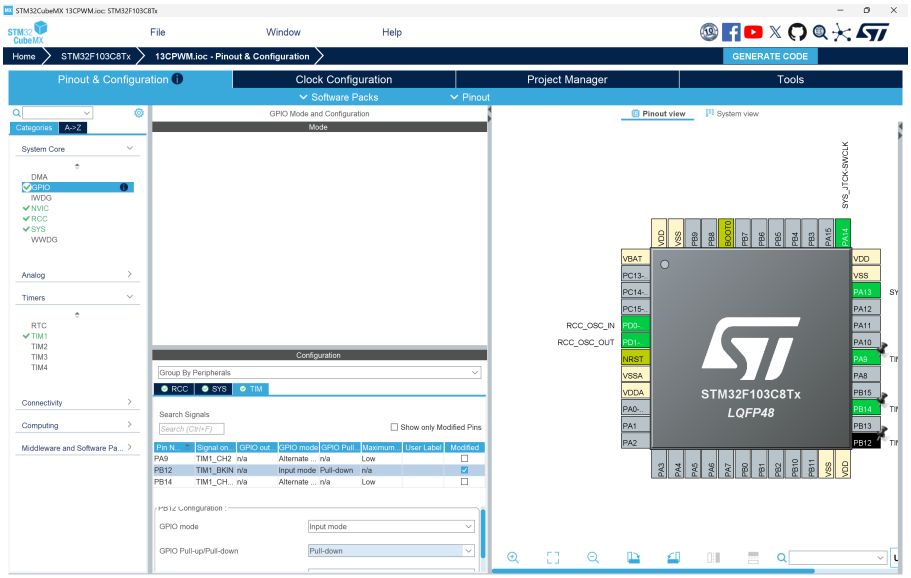
13.5.8 配置 BKIN 刹车输入引脚（GPIO）

在右侧 **Pinout view** 中，点击 **PB12** 引脚，选择 **TIM1_BKIN**。

在 **System Core** → **GPIO** 中找到 PB12，进行以下设置：

1. **Mode**：**Input**（刹车输入）
2. **GPIO Pull-up/Pull-down**：**Pull-down**（下拉，确保正常工作时刹车不被触发）

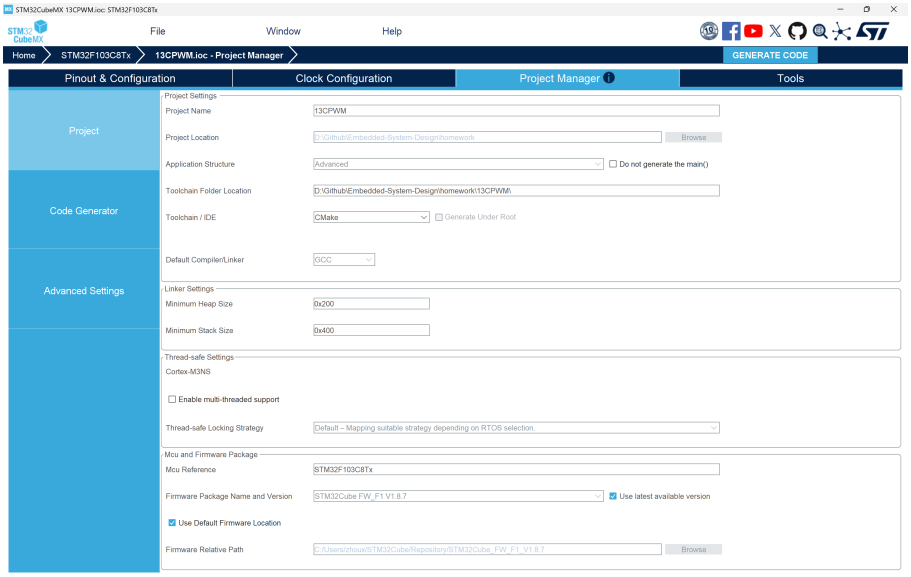
这样做的原因是：刹车极性设为 High，若 PB12 浮空或为高电平，会误触发刹车导致 PWM 无输出。配置为 Pull-down 可确保默认不触发刹车。



13.5.9 工程设置与生成代码

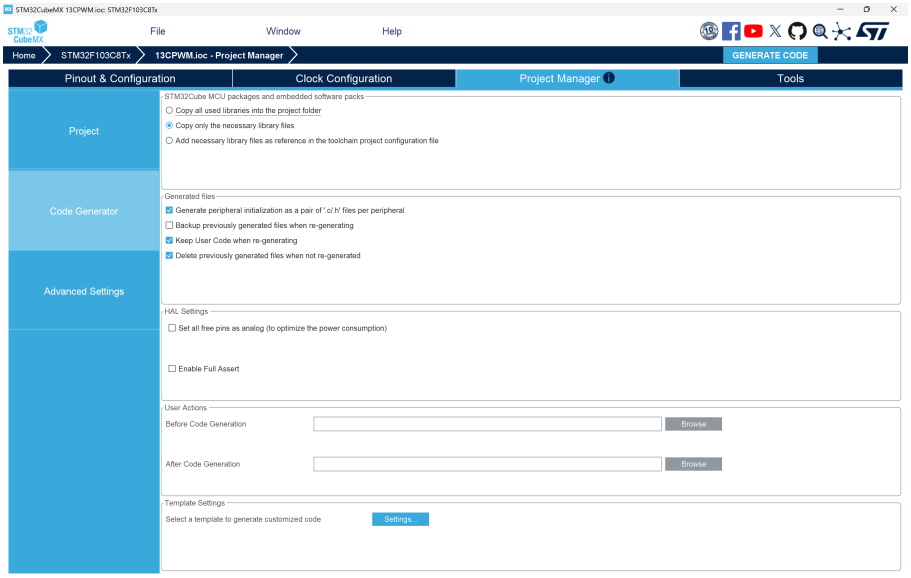
在 **Project Manager** 页面：

1. **Project Name**：`13CPWM`
2. **Project Location**：选择当前 `13CPWM` 目录
3. **Toolchain / IDE**：选择 **CMake**



切换到 **Code Generator** 选项卡：

1. 勾选 **Generate peripheral initialization as a pair of '.c/.h' files per peripheral**，这样 TIM1 初始化代码会生成到独立的 `tim.c` / `tim.h` 文件中，便于代码管理。



最后点击 **GENERATE CODE** 生成工程。

13.6 硬件连接要点

引脚	功能	连接
PA9	TIM1_CH2（主输出）	逻辑分析仪通道 0
PB14	TIM1_CH2N（互补输出）	逻辑分析仪通道 1
PB12	TIM1_BKIN（刹车输入）	板载 Pull-down，无需外部连接
GND	地	逻辑分析仪 GND

注意：逻辑分析仪的 GND 必须与 STM32 核心板的 GND 共地，否则可能无法正确捕获波形。

13.7 本工程实现的实验内容

当前工程实现了基于 TIM1 高级定时器的硬件互补 PWM 输出，功能如下：

1. 系统启动后自动输出固定 50% 占空比的互补 PWM 波形。
2. CH2 (PA9) 与 CH2N (PB14) 输出极性相反的互补波形，带约 0.6 us 死区时间。
3. PWM 频率约 149.7 kHz。
4. 刹车功能已启用，PB12 默认下拉确保正常输出。

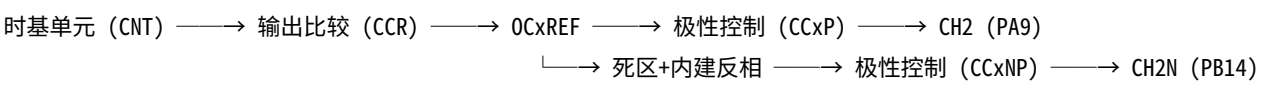
13.8 核心代码说明

13.8.1 互补 PWM 硬件原理

硬件互补 PWM 通过 TIM1 高级定时器的以下硬件模块实现：

1. 时基单元（Prescaler + Counter + ARR）：产生 PWM 载波。
2. 输出比较通道（CCR）：根据比较值产生 OCxREF 参考信号。
3. 死区发生器（Dead-Time Generator）：在 OCxREF 基础上插入死区，并产生互补的 CHx 和 CHxN 输出。
4. 刹车模块（Break）：检测 BKIN 引脚，故障时硬件关闭输出。

下图展示了硬件互补 PWM 的信号流程：



与软件 PWM 不同，硬件互补 PWM 的波形生成完全由定时器硬件完成，无需 CPU 中断参与，波形精度高、时序稳定、CPU 占用为零。

13.8.2 main.c 中启动互补 PWM 的代码

在 Core/Src/main.c 中找到 /* USER CODE BEGIN 2 */，添加以下代码：

```
/* USER CODE BEGIN 2 */
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 240);  // 设置 CCR = 240，占空比 50%
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);           // 启动 CH2 主输出（PA9）
HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_2);        // 启动 CH2N 互补输出（PB14）
/* USER CODE END 2 */
```

代码解析：

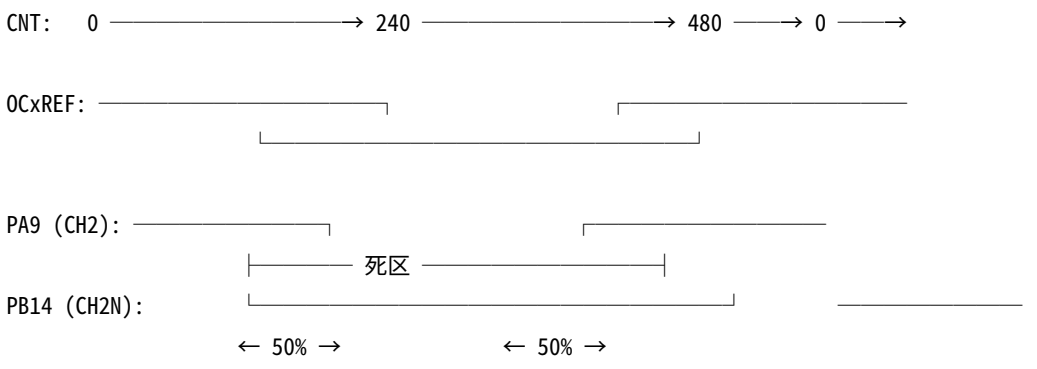
1. `__HAL_TIM_SET_COMPARE` 宏用于设置捕获 / 比较寄存器 CCR 的值。ARR = 480，CCR = 240 表示占空比 = 240 / 480 = 50%。修改此值即可改变占空比。
2. `HAL_TIM_PWM_Start` 启动 CH2 主输出，使能 TIM1 计数器并使 PA9 输出 PWM。
3. `HAL_TIMEx_PWMN_Start` 启动 CH2N 互补输出，使 PB14 输出与 CH2 互补的 PWM 波形。

为什么 CubeMX 生成了 `MX_TIM1_Init()` 仍需手动启动？CubeMX 生成的初始化函数只配置寄存器（预分频器、ARR、CCR、输出比较模式等），不会启动定时器计数和 PWM 输出。必须手动调用 `HAL_TIM_PWM_Start` 和 `HAL_TIMEx_PWMN_Start` 来使能计数器并连接输出。

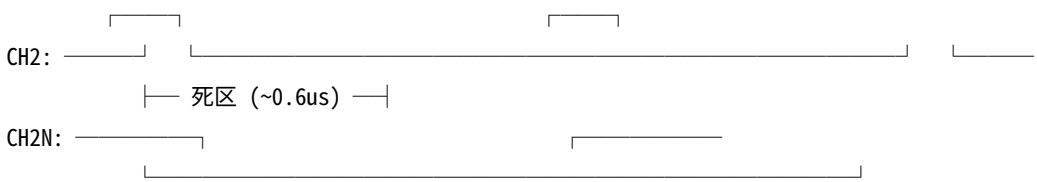
13.8.3 CubeMX 生成的 TIM1 初始化代码

```
htim1.Instance = TIM1;
htim1.Init.Prescaler = 0;                          // 不分频，72 MHz
htim1.Init.CounterMode = TIM_COUNTERMODE_UP;       // 向上计数
htim1.Init.Period = 480;                           // ARR = 480
htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;  // 预装载使能

sConfigOC.OCMode = TIM_OCMODE_PWM1;                // PWM 模式 1
sConfigOC.Pulse = 0;                               // 初始 CCR = 0
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;        // CH2 极性：高有效
sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;      // CH2N 极性：高有效（相同 = 互补）

sBreakDeadTimeConfig.DeadTime = 43;                // 死区 ~ 0.6 us
sBreakDeadTimeConfig.BreakState = TIM_BREAK_ENABLE;  // 刹车使能
sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;  // 刹车高有效
```

13.8.4 互补 PWM 波形示意图



死区细节（切换瞬间放大）：



13.9 编程技巧总结

1. 高级定时器的互补 PWM 由硬件自动生成，不需要中断参与，CPU 可完全专注于其他任务。
2. CHx 和 CHxN 的极性必须设为相同，因为死区发生器内部已对互补通道做了反相。
3. 刹车输入引脚应配置为 Pull-down（刹车高有效时）或 Pull-up（刹车低有效时），避免浮空误触发。
4. CubeMX 生成的 `MX_TIMx_Init()` 只初始化配置，启动 PWM 需额外调用 `HAL_TIM_PWM_Start` 和 `HAL_TIMEx_PWMN_Start`。
5. 使能 Auto-reload preload 可保证修改 ARR 值时在新周期开始时生效，避免周期毛刺。

13.10 实验操作步骤

13.10.1 配置 CubeMX 并生成代码

按第 3 节步骤完成配置，点击 GENERATE CODE。

13.10.2 添加应用代码

在 `Core/Src/main.c` 中按第 6.2 节代码说明，在 `/* USER CODE BEGIN 2 */` 区域添加 PWM 启动代码。

13.10.3 编译工程

使用当前开发环境编译工程，确认无编译错误。

13.10.4 烧录程序

通过 ST-Link 将程序下载到 STM32 开发板。

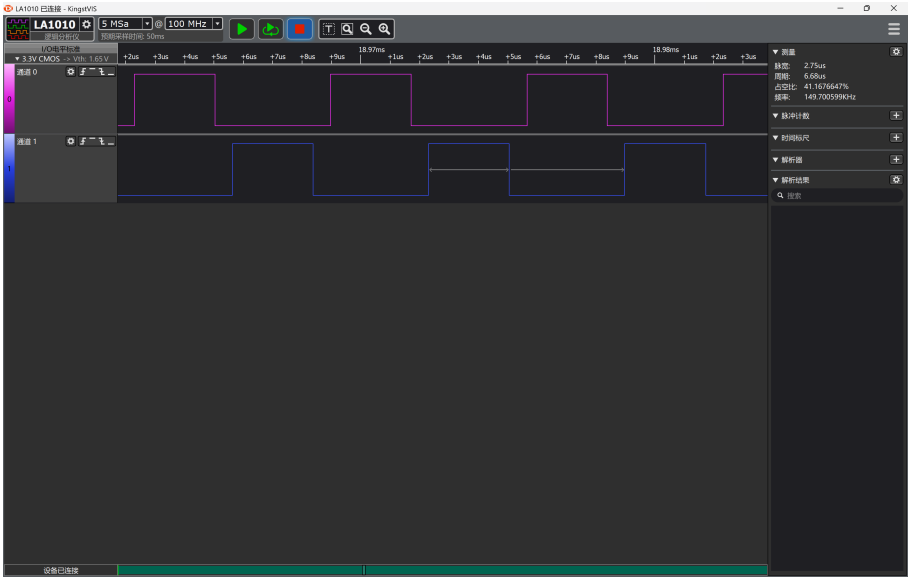
13.10.5 逻辑分析仪验证

1. 将逻辑分析仪通道 0 接 PA9，通道 1 接 PB14，GND 接板子 GND。
2. 设置采样率 >= 10 MHz（PWM 频率约 150 kHz，至少 10 倍以上采样率）。
3. 观察两路波形：CH2 与 CH2N 应为互补关系，且切换瞬间存在约 0.6 us 的死区（两路同时为低）。

13.11 实验现象与结果分析

如果实验成功，你应在逻辑分析仪上看到如下现象：

1. PA9 (CH2) 输出方波，频率约 149.7 kHz，占空比约 50%。
2. PB14 (CH2N) 输出方波，与 CH2 互补（CH2 为高时 CH2N 为低，反之亦然）。
3. 放大观察 PWM 切换边沿，可见两路信号同时为低的死区区域，持续时间约 0.6 us。



这些现象说明：

1. TIM1 定时器及其 PWM 配置正确，时基正常计数并在 481 个时钟周期处溢出。
2. 互补 PWM 逻辑正确，CH2 和 CH2N 输出真正的互补波形。
3. 死区发生器工作正常，DTG=43 对应的约 0.6 us 死区时间符合预期。
4. 刹车功能已启用但未触发（PB12 下拉），PWM 正常输出。

13.12 常见问题排查

问题	可能原因与解决方案
两路均无波形输出	检查是否调用了 `HAL_TIM_PWM_Start` 和 `HAL_TIMEx_PWMN_Start`；检查 PB12 是否为低电平（刹车是否误触发）。
CH2 与 CH2N 波形完全相同	检查 OCPolarity 与 OCNPolarity 是否设为相同。TIM1 死区发生器内部已反相，极性必须相同才互补。
波形正常但看不到死区	逻辑分析仪采样率太低，需提高至 10 MHz 以上并放大边沿观察。
程序下载后无反应	检查 SYS Debug 是否配置为 Serial Wire；检查 HSE 和时钟树配置。
刹车功能误触发	检查 BKIN 引脚是否有上拉 / 下拉，确认与 Break Polarity 设置匹配。

13.13 课后思考题

请结合本次实验，在报告中认真回答以下问题：

1. 什么是互补 PWM? 为什么电机驱动、开关电源等应用中需要互补 PWM 和死区时间?
2. 死区时间 DTG=43 对应的实际时间是多少？如何通过定时器时钟频率计算死区时间?
3. 高级定时器 TIM1 与通用定时器（如 TIM3）在功能上有哪些主要区别?
4. 刹车功能（Break）在互补 PWM 应用中的作用是什么？本实验中 BKIN 引脚为何要配置为 Pull-down?
5. 如果将 OCPolarity 和 OCNPolarity 分别改为不同极性（一个 HIGH 一个 LOW），输出波形会发生什么变化？为什么?

13.14 实验报告提交要求

13.14.1 报告建议结构

1. 封面：课程名称、作业名称、姓名、学号、班级、日期。
2. 作业目标：简述本实验需要实现的功能与验收指标。
3. 实验环境：开发板型号、调试器、逻辑分析仪、软件版本。
4. CubeMX 配置：必须说明 SYS、RCC、Clock Configuration、TIM1 基本参数、互补 PWM 通道、死区与刹车、BKIN GPIO、工程设置、代码生成选项的关键设置及其原因，并附上每一步的配置截图。
5. 程序设计：分析 `main.c` 中的 PWM 启动代码和 `tim.c` 中 CubeMX 生成的初始化代码。
6. 程序流程图：必须展示” 时基计数 → 输出比较 → 死区发生器 → 极性控制 → 互补 PWM 输出” 的硬件信号流程。
7. 实验步骤：包括编译、下载、逻辑分析仪测量的过程。
8. 实验结果：必须展示逻辑分析仪捕获的互补 PWM 波形和死区放大图。
9. 问题分析与调试记录：说明遇到的任何问题及排查方法。
10. 课后思考题答案：必须完整回答本 README 中 5 个思考题。
11. 总结：概括你对高级定时器、互补 PWM、死区时间和刹车功能的理解。
12. 附录：可附核心代码、更多截图、参考资料。

13.14.2 图片与流程图要求

1. 报告中至少包含 10 张截图（SYS、RCC、时钟树、TIM1 基本参数、互补 PWM 通道、死区与刹车、BKIN GPIO、工程设置、代码生成选项、逻辑分析仪波形）。
2. 每张图片必须标注图号和图题，并在正文中说明该图用于证明什么。
3. 程序流程图必须单独成图，不能只用文字描述替代。

13.14.3 提交规范

1. 报告文件建议命名为：`学号-姓名-作业 13-互补 PWM 实验.pdf`。
2. 若实验未完全成功，也必须提交完整报告，重点说明失败现象、原因分析和改进方向。
3. 报告中不能只贴代码或截图，必须结合现象进行解释。

13.14.4 最低验收标准

1. 逻辑分析仪能够捕获到 PA9 和 PB14 的互补 PWM 波形。
2. 能够说明互补 PWM 的原理和死区时间的计算方法。
3. 报告能够解释高级定时器与通用定时器的区别，以及刹车功能的作用。

13.15 可进一步扩展的方向

1. 通过按键调整占空比，观察互补波形变化。
2. 实际驱动半桥电路 + 电机，验证互补 PWM 的驱动效果。
3. 使能刹车中断，在刹车触发时执行故障处理逻辑。
4. 使用多个通道输出多组互补 PWM，驱动多相电机。
5. 结合 ADC 采样实现闭环控制（如电流环、速度环）。

13.16 总结

本实验通过 STM32F103 的高级定时器 TIM1 实现硬件互补 PWM 输出，涵盖高级定时器时基配置、互补 PWM 通道配置、死区时间设定、刹车功能配置等核心内容。相比于通用定时器，高级定时器专为电机控制和电力电子应用设计，其内建的互补输出、死区发生器和刹车保护功能使驱动半桥 / 全桥电路变得简单可靠。本实验是入门电机驱动控制的基础。